

Universitatea Politehnica Timișoara  
Facultatea de Automatică și Calculatoare

# Programare .NET

Suport de curs



ș.l. dr. ing. Valer BOCAN, CSSLP

E-mail: [valer.bocan@upt.ro](mailto:valer.bocan@upt.ro)

Web: [www.bocan.ro](http://www.bocan.ro)



## Cuprins

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 1. Introducere în platforma .NET .....                          | 9  |
| 1.1. Obiectivele cursului .....                                 | 9  |
| 1.2. Audiență .....                                             | 10 |
| 1.3. Istoric și arhitectura .NET .....                          | 11 |
| 1.3.1. Common Language Runtime (CLR).....                       | 11 |
| 1.3.2. Bibliotecile de clasă .NET .....                         | 11 |
| 1.3.3. Limbajele de programare compatibile .NET .....           | 11 |
| 1.3.4. Arhitecturi hardware suportate .....                     | 12 |
| 1.4. Limbaje .NET suportate oficial.....                        | 12 |
| 1.4.1. C#.....                                                  | 12 |
| 1.4.2. F# .....                                                 | 13 |
| 1.4.3. Interoperabilitatea între C# și F# .....                 | 13 |
| 1.5. Limbaje .NET neoficiale .....                              | 13 |
| 1.6. Componentele platformei .NET .....                         | 14 |
| 1.6.1. Common Language Runtime (CLR).....                       | 14 |
| 1.6.2. Compilarea Just-in Time (JIT).....                       | 15 |
| 1.6.3. Garbage Collector (GC) .....                             | 15 |
| 1.7. Medii de dezvoltare .NET .....                             | 16 |
| 1.7.1. Visual Studio .....                                      | 16 |
| 1.7.2. Visual Studio Code.....                                  | 17 |
| 1.7.3. .NET CLI .....                                           | 17 |
| 2. Bazele limbajului C#.....                                    | 19 |
| 2.1. Fundamente și sintaxă de bază .....                        | 19 |
| 2.1.1. Structura unui program C#.....                           | 20 |
| 2.1.2. Namespace-uri și directiva using.....                    | 21 |
| 2.1.3. Clasa Program și metoda Main .....                       | 22 |
| 2.1.4. Comentarii și documentație în cod .....                  | 23 |
| 2.1.5. Convenții de denumire în C# .....                        | 24 |
| 2.2. Tipuri de date și variabile.....                           | 25 |
| 2.2.1. Tipuri de date primitive (int, double, bool, char) ..... | 26 |
| 2.2.2. Tipul string și operațiuni cu șiruri de caractere .....  | 27 |
| 2.2.3. Tipuri nullable și operatorul '?' .....                  | 29 |

---

|        |                                                                   |    |
|--------|-------------------------------------------------------------------|----|
| 2.2.4. | Conversii între tipuri de date (casting) .....                    | 30 |
| 2.2.5. | Variabile constante (const) și readonly .....                     | 32 |
| 2.2.6. | Inferența tipurilor cu var.....                                   | 33 |
| 2.3.   | Operatori și expresii.....                                        | 34 |
| 2.3.1. | Operatori aritmetici .....                                        | 35 |
| 2.3.2. | Operatori de comparare.....                                       | 36 |
| 2.3.3. | Operatori logici .....                                            | 38 |
| 2.3.4. | Operatori de atribuire .....                                      | 39 |
| 2.3.5. | Operatori la nivel de biți .....                                  | 40 |
| 2.3.6. | Precedența operatorilor .....                                     | 42 |
| 2.4.   | Structuri de control .....                                        | 43 |
| 2.4.1. | Instrucțiuni condiționale .....                                   | 44 |
| 2.4.2. | Bucle .....                                                       | 47 |
| 2.4.3. | Instrucțiuni de salt .....                                        | 50 |
| 2.5.   | Tablouri și colecții .....                                        | 51 |
| 2.5.1. | Tablouri unidimensionale și multidimensionale.....                | 52 |
| 2.5.2. | Declararea și inițializarea tablourilor .....                     | 53 |
| 2.5.3. | Operațiuni cu tablouri .....                                      | 55 |
| 2.5.4. | Tablouri zimțate (jagged).....                                    | 56 |
| 2.5.5. | Introducere în colecții generice (List<T>, Dictionary<K,V>) ..... | 58 |
| 2.6.   | Metode și parametri .....                                         | 59 |
| 2.6.1. | Declararea și apelarea metodelor .....                            | 60 |
| 2.6.2. | Parametri și argumente .....                                      | 61 |
| 2.6.3. | Parametri opționali și parametri numiți .....                     | 63 |
| 2.6.4. | Supraîncărcarea metodelor .....                                   | 64 |
| 2.6.5. | Parametri ref, out și in.....                                     | 66 |
| 2.6.6. | Recursivitate .....                                               | 67 |
| 2.7.   | Gestionarea erorilor .....                                        | 69 |
| 2.7.1. | Blocuri try-catch-finally .....                                   | 69 |
| 2.7.2. | Tipuri de excepții comune .....                                   | 71 |
| 2.7.3. | Aruncarea excepțiilor (throw) .....                               | 73 |
| 2.7.4. | Crearea excepțiilor personalizate .....                           | 74 |

|        |                                                          |     |
|--------|----------------------------------------------------------|-----|
| 2.7.5. | Bune practici în gestionarea erorilor .....              | 76  |
| 2.8.   | Introducere în programarea orientată pe obiecte .....    | 77  |
| 2.8.1. | Conceptul de clasă și obiect .....                       | 78  |
| 2.8.2. | Membri ai clasei (câmpuri, proprietăți, metode) .....    | 80  |
| 2.8.3. | Constructorii și destructori .....                       | 81  |
| 2.8.4. | Modificatori de acces (public, private, protected) ..... | 82  |
| 2.8.5. | Proprietăți auto-implementate .....                      | 84  |
| 2.8.6. | Membri statici .....                                     | 85  |
| 3.     | LINQ (Language Integrated Query) .....                   | 87  |
| 3.1.   | Fundamentele LINQ .....                                  | 87  |
| 3.1.1. | Arhitectura LINQ .....                                   | 88  |
| 3.1.2. | Furnizori și extensibilitate .....                       | 89  |
| 3.1.3. | Expresii și operatori de bază .....                      | 90  |
| 3.1.4. | Sintaxa de interogare vs. sintaxa metodă .....           | 91  |
| 3.2.   | LINQ to Objects .....                                    | 93  |
| 3.2.1. | Lucrul cu colecții în memorie .....                      | 93  |
| 3.2.2. | Proiecția .....                                          | 94  |
| 3.2.3. | Ordonare și grupare .....                                | 95  |
| 3.2.4. | Operații de agregare .....                               | 96  |
| 3.3.   | Expresii Lambda și Delegates .....                       | 98  |
| 3.3.1. | Conceptul de delegate în .NET .....                      | 99  |
| 3.3.2. | Sintaxa și utilizarea expresiilor lambda .....           | 100 |
| 3.3.3. | Closure și variabile capturate .....                     | 101 |
| 3.3.4. | Funcții anonime .....                                    | 102 |
| 3.4.   | Operatori LINQ avansați .....                            | 103 |
| 3.4.1. | Join și GroupJoin .....                                  | 103 |
| 3.4.2. | SelectMany și aplatizare .....                           | 105 |
| 3.4.3. | Operatori de mulțimi .....                               | 107 |
| 3.4.4. | Operatori de cuantificare .....                          | 108 |
| 3.5.   | LINQ to XML și JSON .....                                | 109 |
| 3.5.1. | Manipularea documentelor XML .....                       | 109 |
| 3.5.2. | Interogări pe structuri XML .....                        | 110 |

---

|        |                                                         |     |
|--------|---------------------------------------------------------|-----|
| 3.5.3. | Parsare și generare JSON .....                          | 112 |
| 3.5.4. | Transformări între formate .....                        | 112 |
| 4.     | Bazele limbajului F# .....                              | 114 |
| 4.1.   | Introducere în programarea funcțională .....            | 114 |
| 4.1.1. | Paradigma funcțională vs. paradigma imperativă.....     | 115 |
| 4.1.2. | Principii fundamentale ale programării funcționale..... | 115 |
| 4.1.3. | Imutabilitatea și efectele secundare.....               | 116 |
| 4.1.4. | Evaluarea expresiilor .....                             | 116 |
| 4.1.5. | Structura unui program F# .....                         | 117 |
| 4.2.   | Tipuri de date și valori .....                          | 118 |
| 4.2.1. | Tipuri de date primitive .....                          | 119 |
| 4.2.2. | Inferența tipurilor .....                               | 119 |
| 4.2.3. | Tipuri parametrizate și generice .....                  | 120 |
| 4.2.4. | Tipul unit .....                                        | 121 |
| 4.2.5. | Tipul Option.....                                       | 122 |
| 4.2.6. | Tipul Tuple și deconstrucție .....                      | 123 |
| 4.3.   | Funcții și aplicarea funcțiilor .....                   | 125 |
| 4.3.1. | Definirea funcțiilor .....                              | 125 |
| 4.3.2. | Funcții anonime (lambda).....                           | 126 |
| 4.3.3. | Aplicare parțială și currying.....                      | 127 |
| 4.3.4. | Compunerea funcțiilor .....                             | 129 |
| 4.3.5. | Funcții de ordin superior.....                          | 130 |
| 4.3.6. | Recursivitate și recursivitate coadă .....              | 131 |
| 4.4.   | Pattern matching și expresii .....                      | 133 |
| 4.4.1. | Sintaxa de bază a pattern matching .....                | 133 |
| 4.4.2. | Pattern matching pe tipuri de date.....                 | 135 |
| 4.4.3. | Guards în pattern matching .....                        | 136 |
| 4.4.4. | Active patterns.....                                    | 137 |
| 4.4.5. | Match expressions în funcții.....                       | 139 |
| 4.4.6. | Pattern matching complet .....                          | 140 |
| 4.5.   | Colecții și prelucrarea datelor .....                   | 141 |
| 4.5.1. | Liste (List).....                                       | 142 |

|        |                                                   |     |
|--------|---------------------------------------------------|-----|
| 4.5.2. | Tablouri .....                                    | 143 |
| 4.5.3. | Secvențe (Seq).....                               | 144 |
| 4.5.4. | Map și Set.....                                   | 145 |
| 4.5.5. | Operații comune pe colecții .....                 | 147 |
| 4.5.6. | List comprehension .....                          | 149 |
| 4.6.   | Operatori și pipe.....                            | 150 |
| 4.6.1. | Operatorul pipe-forward ( >) .....                | 151 |
| 4.6.2. | Operatorul pipe-backward (< ) .....               | 152 |
| 4.6.3. | Operatorul de compunere (>>) .....                | 153 |
| 4.6.4. | Operatorul de compunere inversă (<<).....         | 154 |
| 4.6.5. | Operatori personalizați .....                     | 155 |
| 4.6.6. | Precedența operatorilor.....                      | 157 |
| 4.7.   | Tipuri de date algebrice.....                     | 158 |
| 4.7.1. | Tipuri înregistrare (record).....                 | 159 |
| 4.7.2. | Mulțimi discriminate (discriminated unions) ..... | 160 |
| 4.7.3. | Single-case discriminated unions .....            | 161 |
| 4.7.4. | Tipuri recursive .....                            | 162 |
| 4.7.5. | Unități de măsură.....                            | 165 |
| 4.7.6. | Abrevieri de tipuri .....                         | 166 |
| 4.8.   | Module și organizarea codului.....                | 167 |
| 4.8.1. | Definirea modulelor .....                         | 168 |
| 4.8.2. | Namespace-uri în F# .....                         | 170 |
| 4.8.3. | Importarea și exportarea funcționalității .....   | 171 |
| 4.8.4. | Signature files.....                              | 173 |
| 4.8.5. | Modularizarea codului.....                        | 175 |
| 4.9.   | Gestionarea erorilor în stil funcțional .....     | 177 |
| 4.9.1. | Tipul Result .....                                | 178 |
| 4.9.2. | Tipul Option pentru valori absente.....           | 179 |
| 4.9.3. | Railway-oriented programming .....                | 180 |
| 4.9.4. | Computation expressions.....                      | 182 |
| 4.9.5. | Exception handling.....                           | 183 |
| 4.9.6. | Validarea datelor.....                            | 184 |

---

|         |                                       |     |
|---------|---------------------------------------|-----|
| 4.10.   | Interoperabilitatea cu C# .....       | 187 |
| 4.10.1. | Utilizarea tipurilor din C# .....     | 187 |
| 4.10.2. | Expunerea tipurilor F# către C# ..... | 188 |
| 4.10.3. | Folosirea bibliotecilor .NET .....    | 190 |
| 4.10.4. | Conversii între tipuri F# și C# ..... | 191 |



# 1. Introducere în platforma .NET

## 1.1. Obiectivele cursului

Cursul de programare .NET are ca scop principal oferirea unei înțelegeri aprofundate a ecosistemului .NET și dezvoltarea abilităților practice de programare utilizând limbajele C# și F#. Pe parcursul acestui curs, studenții vor dobândi cunoștințele și competențele necesare pentru a crea aplicații robuste, scalabile și ușor de întreținut, folosind cele mai recente tehnologii și practici din domeniu.

Cursul este structurat într-un mod progresiv, pornind de la conceptele de bază ale platformei .NET și ale limbajelor de programare C# și F#, și avansând treptat către subiecte mai complexe, precum dezvoltarea de aplicații web cu ASP.NET Core, interacțiunea cu baze de date folosind Entity Framework Core și programarea asincronă cu Task Parallel Library.

Pe măsură ce studenții parcurg cursul, aceștia vor fi expuși la o gamă largă de paradigme de programare, inclusiv programarea orientată pe obiecte (OOP) în C# și programarea funcțională în F#. Această abordare le va permite să înțeleagă diferențele și similitudinile dintre aceste paradigme și să aleagă cea mai potrivită abordare pentru fiecare proiect în parte.

Un alt obiectiv important al cursului este familiarizarea studenților cu instrumentele și practicile moderne de dezvoltare software, cum ar fi utilizarea unui sistem de control al versiunilor (e.g., Git), scrierea de teste automate și depanarea eficientă a codului. Aceste abilități sunt esențiale pentru orice programator profesionist și vor fi dezvoltate pe parcursul laboratoarelor și a proiectelor practice.

La finalul cursului, studenții vor fi capabili să:

1. Înțeleagă arhitectura și componentele cheie ale platformei .NET, incluzând Common Language Runtime (CLR), bibliotecile de clasă .NET și limbajele de programare compatibile.
2. Scrie cod C# curat, eficient și ușor de întreținut, aplicând conceptele de programare orientată pe obiecte, precum clase, moștenire, polimorfism și încapsulare.
3. Utilizeze constructorii limbajului C#, cum ar fi instrucțiunile condiționale, buclele, tratarea excepțiilor și colecțiile generice, pentru a implementa logica și fluxul de control al programelor.
4. Aplice tehnici avansate de interogare și manipulare a datelor folosind Language Integrated Query (LINQ), inclusiv operații de filtrare, proiecție, grupare și agregare.
5. Aplice paradigma de programare funcțională în F#, utilizând expresii, funcții pure, tipuri de date imutabile și tehnici de potrivire a modelelor.

6. Scrie cod F# concis, expresiv și robust, profitând de caracteristici precum inferența de tip, pipe-uri, compoziția funcțiilor și expresii de calcul.
7. Utilizeze tipurile de date algebrice din F#, cum ar fi înregistrări, discriminated unions și tipuri recursive, pentru a modela domenii complexe și a împinge verificarea corectitudinii la compile-time.
8. Gestioneze erorile și cazurile excepționale în F# într-un stil funcțional, folosind tipurile Option și Result.
9. Organizeze și structureze codul F# în module, namespace-uri și componente reutilizabile, aplicând principii de design modular și favorizând compoziția.
10. Profite de ecosistemul bogat al platformei .NET, integrând și utilizând biblioteci și framework-uri existente în C# și F# pentru a accelera dezvoltarea software.

Prin atingerea acestor obiective, studenții vor fi pregătiți să își înceapă cariera ca dezvoltatori .NET sau să își aplice cunoștințele dobândite în proiecte personale sau academice. Cursul le oferă o bază solidă pe care să își construiască viitoarea expertiză în domeniul dezvoltării software folosind platforma .NET.

## 1.2. Audiență

Cursul de programare .NET se adresează unei game largi de studenți și profesioniști interesați de dezvoltarea software utilizând platforma .NET. Audiența principală include studenți la facultățile de informatică, inginerie software sau domenii conexe, precum și programatori care doresc să își extindă cunoștințele și să învețe noi tehnologii.

Pentru a beneficia pe deplin de acest curs, participanții trebuie să aibă deja o înțelegere solidă a conceptelor fundamentale de programare, cum ar fi variabile, tipuri de date, structuri de control (e.g., if-else, bucle), funcții și algoritmi de bază. Familiaritatea cu cel puțin un limbaj de programare orientat pe obiecte, precum Java, C++ sau Python, este de asemenea recomandată, deoarece va facilita înțelegerea conceptelor din C# și tranziția către platforma .NET.

De asemenea, participanții trebuie să fie confortabili cu utilizarea calculatoarelor și să aibă abilități de bază în navigarea sistemelor de operare și gestionarea fișierelor.

Dincolo de abilitățile tehnice, studenții trebuie să fie motivați, curioși și dornici să învețe. Dezvoltarea software este un domeniu vast și în continuă evoluție, iar participanții trebuie să fie pregătiți să investească timp și efort în învățarea și aplicarea conceptelor predate în cadrul cursului. Abilitatea de a rezolva probleme, de a gândi logic și de a persevera în fața provocărilor sunt, de asemenea, calități importante pentru reușita în acest curs și în viitoarea carieră de programator.

În cele din urmă, cursul poate fi atractiv și pentru profesioniștii din alte domenii care doresc să își schimbe cariera sau să își extindă orizonturile. Aceștia pot include, de exemplu, specialiști în IT care doresc să treacă de la roluri de suport tehnic la dezvoltare

software, sau profesioniști din domenii non-tehnice care doresc să dobândească abilități de programare pentru a-și îmbunătăți perspectivele de carieră.

Indiferent de background-ul și experiența anterioară, cursul de programare .NET oferă o oportunitate excelentă pentru oricine dorește să învețe să dezvolte aplicații moderne, robuste și scalabile utilizând una dintre cele mai populare și versatile platforme de dezvoltare software din lume.

### 1.3. Istoric și arhitectura .NET

Platforma .NET a fost dezvoltată de Microsoft ca un succesor al tehnologiilor COM (Component Object Model) și a fost lansată oficial în februarie 2002, odată cu versiunea 1.0 a .NET Framework. Scopul principal al platformei .NET a fost de a oferi un mediu de dezvoltare software unificat, care să permită programatorilor să creeze aplicații robuste, scalabile și ușor de întreținut, indiferent de limbajul de programare utilizat. De-a lungul anilor, platforma .NET a evoluat continuu, cu lansarea de noi versiuni ale .NET Framework, precum și cu introducerea unor variante specializate, cum ar fi .NET Compact Framework pentru dispozitive mobile și .NET Micro Framework pentru dispozitive încorporate. În 2014, Microsoft a lansat .NET Core, o implementare open-source și cross-platform a platformei .NET, care a marcat o schimbare semnificativă în strategia companiei și a deschis noi oportunități pentru dezvoltatorii .NET.

Platforma .NET este construită pe baza a trei componente principale: *Common Language Runtime* (CLR), bibliotecile de clasă .NET și limbajele de programare compatibile .NET.

#### 1.3.1. Common Language Runtime (CLR)

CLR este motorul de execuție al platformei .NET, responsabil pentru gestionarea memoriei, executarea codului, verificarea siguranței tipurilor, gestionarea excepțiilor și alte servicii de nivel scăzut. CLR asigură un mediu de execuție comun pentru toate limbajele de programare compatibile .NET, permițând interoperabilitatea între acestea.

#### 1.3.2. Bibliotecile de clasă .NET

Bibliotecile de clasă .NET oferă un set vast de funcționalități organizate în spații de nume (namespaces), care acoperă o gamă largă de domenii, precum manipularea fișierelor, accesul la baze de date, dezvoltarea de interfețe utilizator, criptografie și multe altele. Aceste biblioteci sunt accesibile din toate limbajele de programare compatibile .NET, asigurând un mod consistent și eficient de a dezvolta aplicații.

#### 1.3.3. Limbajele de programare compatibile .NET

Platforma .NET suportă o varietate de limbaje de programare, printre care se numără C#, F#, Visual Basic .NET și multe altele. Aceste limbaje sunt compilate într-un cod intermediar numit *Microsoft Intermediate Language* (MSIL), care este apoi executat de CLR. Această abordare permite programatorilor să aleagă limbajul de programare preferat, în timp ce beneficiază de avantajele platformei .NET.

### 1.3.4. Arhitecturi hardware suportate

Platforma .NET a fost inițial dezvoltată pentru a rula pe sistemele de operare Windows și arhitectura x86. Cu toate acestea, odată cu lansarea .NET Core, platforma a devenit cross-platform, putând rula pe o varietate de sisteme de operare, inclusiv Windows, macOS și Linux, precum și pe arhitecturi hardware diferite, cum ar fi x64, ARM și ARM64.

Această compatibilitate extinsă a platformei .NET a deschis noi oportunități pentru dezvoltatorii .NET, permițându-le să creeze aplicații care pot fi implementate pe o gamă largă de dispozitive și sisteme, de la servere și computere desktop până la dispozitive mobile și sisteme încorporate.

## 1.4. Limbaje .NET suportate oficial

Platforma .NET suportă oficial două limbaje de programare principale: C# și F#. Aceste limbaje oferă programatorilor opțiuni diverse pentru dezvoltarea aplicațiilor, în funcție de nevoile și preferințele lor. În acest capitol, vom explora originile, particularitățile, paradigmele de programare și cazurile de utilizare pentru fiecare dintre aceste limbaje.

### 1.4.1. C#

C# (pronunțat "C sharp") este un limbaj de programare orientat pe obiecte, dezvoltat de Microsoft ca parte a platformei .NET. Lansat în 2000, C# a fost influențat în mare măsură de limbaje precum C++, Java și Delphi, cu scopul de a combina puterea și flexibilitatea acestora într-un limbaj modern, ușor de învățat și de utilizat.

C# este un limbaj static tipizat, ceea ce înseamnă că tipurile de date ale variabilelor sunt verificate la momentul compilării, reducând astfel erorile la runtime. Limbajul suportă concepte avansate de programare orientată pe obiecte, cum ar fi moștenirea, polimorfismul și încapsularea, precum și caracteristici moderne, precum tipuri nullable, delegați, expresii lambda și programare asincronă cu ajutorul cuvintelor cheie `async` și `await`.

C# este în primul rând un limbaj orientat pe obiecte, dar suportă și alte paradigme de programare, cum ar fi programarea procedurală, programarea funcțională (prin intermediul delegaților și expresiilor lambda) și programarea generică. Această flexibilitate permite programatorilor să aleagă abordarea cea mai potrivită pentru problema pe care o rezolvă.

C# este utilizat pe scară largă pentru dezvoltarea unei game variate de aplicații, inclusiv:

- Aplicații desktop cu interfețe grafice utilizator (folosind tehnologii precum Windows Forms sau WPF)
- Aplicații web (folosind ASP.NET sau ASP.NET Core)
- Servicii web și API-uri RESTful
- Aplicații mobile (folosind .NET MAUI pentru platformele iOS și Android)

- Jocuri (folosind motoare de joc precum Unity sau MonoGame)
- Aplicații IoT și embedded (folosind .NET Core sau .NET Micro Framework)

#### 1.4.2.F#

F# este un limbaj de programare funcțional, dezvoltat de Microsoft Research în 2005. F# este puternic influențat de limbajele funcționale ML (Meta Language) și OCaml, combinând programarea funcțională cu caracteristicile platformei .NET.

F# este un limbaj funcțional static tipizat, cu un sistem de tipuri puternic și inferență de tip. Limbajul pune accentul pe imutabilitate, funcții ca valori de prim rang și programare expresivă. F# oferă, de asemenea, caracteristici precum potrivire de modele (pattern matching), înregistrări (records), discriminated unions și un sistem puternic de manipulare a erorilor bazat pe tipul Result.

F# este în primul rând un limbaj funcțional, încurajând programatorii să scrie cod concis, expresiv și ușor de raționat. Cu toate acestea, F# suportă și programarea orientată pe obiecte și programarea imperativă, permițând integrarea cu bibliotecile .NET existente și utilizarea stilului de programare cel mai potrivit pentru problema în cauză.

F# este utilizat într-o varietate de domenii, cum ar fi:

- Analiză de date și calcul științific
- Machine learning și inteligență artificială
- Dezvoltarea de aplicații web și servicii backend
- Dezvoltarea jocurilor (folosind motorul Unity)
- Dezvoltarea de aplicații financiare și de asigurări

#### 1.4.3.Interoperabilitatea între C# și F#

Unul dintre avantajele cheie ale platformei .NET este abilitatea de a utiliza mai multe limbaje de programare în cadrul aceluiași proiect. C# și F# pot interopera fără probleme, permițând programatorilor să beneficieze de punctele forte ale fiecărui limbaj. De exemplu, un proiect poate utiliza F# pentru componentele care necesită programare funcțională și analiză de date, în timp ce C# poate fi utilizat pentru dezvoltarea interfețelor utilizator și integrarea cu bibliotecile .NET existente.

### 1.5. Limbaje .NET neoficiale

Pe lângă limbajele oficiale suportate de platforma .NET, cum ar fi C#, F# și (mai demult Visual Basic .NET), există și o serie de limbaje neoficiale care pot fi utilizate pentru dezvoltarea aplicațiilor .NET. Aceste limbaje, deși nu sunt suportate în mod direct de Microsoft, beneficiază de ecosistemul .NET și de caracteristicile Common Language Runtime (CLR).

Un exemplu de astfel de limbaj neoficial este **Delta Forth .NET**<sup>1</sup>, realizat chiar de autorul acestui curs. Acesta este o implementare a limbajului de programare Forth, un limbaj de nivel scăzut, bazat pe stivă, cunoscut pentru simplitatea și flexibilitatea sa. Cel mai notabil proiect în care a fost folosit Forth a fost corecția optică a radiotelescopului Hubble în anii 1980.

Delta Forth .NET permite dezvoltatorilor să scrie cod Forth care este compilat în Common Intermediate Language (CIL) și poate fi executat pe platforma .NET. Prin integrarea cu .NET, Delta Forth beneficiază de caracteristici precum gestiunea automată a memoriei (prin intermediul garbage collector), accesul la bibliotecile .NET și interoperabilitatea cu alte limbaje .NET.

Alte limbaje neoficiale care pot fi utilizate pe platforma .NET sunt:

- IronPython: O implementare a limbajului Python pentru .NET, care permite dezvoltatorilor să scrie cod Python și să acceseze bibliotecile .NET.
- IronRuby: Similar cu IronPython, IronRuby este o implementare a limbajului Ruby pentru platforma .NET.
- Boo: Un limbaj de programare orientat pe obiecte, cu o sintaxă similară cu Python, care se compilează în CIL și poate fi utilizat în dezvoltarea aplicațiilor .NET.

Deși limbajele neoficiale pot oferi flexibilitate și pot permite dezvoltatorilor să utilizeze limbajul preferat în dezvoltarea aplicațiilor .NET, este important de luat în considerare faptul că acestea pot avea un suport și o comunitate mai mică comparativ cu limbajele oficiale. De asemenea, compatibilitatea și integrarea cu instrumentele și bibliotecile .NET pot fi limitate în cazul limbajelor neoficiale.

## 1.6. Componentele platformei .NET

### 1.6.1. Common Language Runtime (CLR)

Common Language Runtime (CLR) reprezintă o componentă fundamentală a platformei .NET, care acționează ca un motor de execuție pentru aplicațiile dezvoltate în limbajele suportate de .NET. CLR este responsabil pentru o serie de servicii esențiale în timpul execuției programelor, după cum urmează:

- Gestiunea memoriei: CLR alocă și eliberează automat memoria necesară pentru obiectele create de aplicație, folosind un mecanism numit Garbage Collector. Acest lucru simplifică semnificativ dezvoltarea, eliminând necesitatea de a gestiona manual memoria.
- Securitate: CLR oferă un model de securitate robust, bazat pe permisiuni și domenii de aplicație, care protejează sistemul de cod rău intenționat și asigură izolarea resurselor între aplicații.

---

<sup>1</sup> **Delta Forth .NET** este disponibil la <https://github.com/vbocan/delta-forth>

- Interoperabilitate: Prin intermediul mecanismelor precum P/Invoke și COM Interop, CLR permite interacțiunea între codul managed (administrat de CLR) și cel unmanaged (biblioteci native, scrie în limbaje ca Rust, C, C++, etc.), facilitând integrarea cu sisteme existente.
- Gestionarea excepțiilor: CLR oferă un mecanism unificat de tratare a excepțiilor, permițând prinderea și gestionarea erorilor într-un mod structurat și previzibil.

### 1.6.2. Compilarea Just-in Time (JIT)

Compilarea Just-in-Time (JIT) este o tehnică utilizată de Common Language Runtime (CLR) pentru a îmbunătăți performanța și portabilitatea aplicațiilor .NET. Atunci când codul sursă al unei aplicații .NET este compilat, acesta este transformat într-un limbaj intermediar numit Common Intermediate Language (CIL) sau – anterior denumit – MSIL (Microsoft Intermediate Language). Acest cod CIL este independent de platforma hardware și poate fi executat pe orice sistem care are instalat .NET Framework sau .NET Core.

La momentul execuției, când o metodă este apelată pentru prima dată, compilatorul JIT al CLR compilează codul CIL al metodei respective direct în cod mașină nativ, specific pentru arhitectura procesorului pe care rulează aplicația. Acest proces de compilare are loc "just-in-time" (termen englezesc pentru „la timp”), adică exact în momentul în care este necesar. Codul nativ generat este apoi stocat în memoria cache, astfel încât, la apelurile ulterioare ale aceleiași metode, nu mai este necesară recompilarea, ci se folosește direct codul nativ din cache. Compilarea JIT aduce următoarele beneficii:

- Portabilitate: Codul CIL poate fi executat pe diferite arhitecturi hardware fără a fi necesară recompilarea, deoarece compilarea în cod nativ are loc la runtime (momentul rulării), în funcție de platforma specifică.
- Optimizare: Compilatorul JIT poate aplica optimizări specifice arhitecturii hardware pe care rulează aplicația, rezultând un cod nativ eficient.
- Compilare întârziată (lazy compilation): Prin compilarea metodelor doar atunci când sunt apelate pentru prima dată, se evită compilarea inutilă a codului care nu este executat, reducând timpul de pornire al aplicației și consumul de memorie.

Cu toate acestea, compilarea JIT introduce un overhead la primul apel al fiecărei metode, deoarece compilarea are loc în timpul execuției. Acest overhead este compensat de performanța crescută a codului nativ generat și de faptul că, odată compilată, metoda nu mai necesită recompilare la apelurile ulterioare.

### 1.6.3. Garbage Collector (GC)

Garbage Collector (GC) este un mecanism automat de gestionare a memoriei, implementat în cadrul platformei .NET, care se ocupă de alocarea și eliberarea memoriei pentru obiectele create de aplicații. Rolul principal al GC este de a identifica obiectele care nu mai sunt utilizate de program și de a elibera memoria ocupată de acestea,



prevenind astfel problemele de memorie, cum ar fi scurgerile de memorie (memory leaks) și fragmentarea excesivă.

Garbage Collector funcționează în background, monitorizând constant heap-ul (zona de memorie gestionată de GC) și urmărind referințele între obiecte. Atunci când GC detectează că un obiect nu mai este accesibil din codul aplicației (nu există nicio referință către acesta), el marchează obiectul respectiv ca fiind eligibil pentru colectare. Periodic, GC întrerupe execuția aplicației și efectuează următoarele operații:

- a) Marcarea (Marking): GC traversează toate obiectele accesibile din rădăcinile aplicației (cum ar fi stiva de apeluri și variabilele globale) și le marchează ca fiind "în viață".
- b) Ștergerea (Sweeping): Obiectele care nu au fost marcate în faza anterioară sunt considerate "moarte" și memoria ocupată de acestea este eliberată.
- c) Compactarea (Compacting): Pentru a reduce fragmentarea heap-ului, GC mută obiectele rămase (cele "în viață") într-un bloc contiguu de memorie, actualizând corespunzător referințele către aceste obiecte.

GC folosește o abordare generațională, împărțind obiectele în trei generații (0, 1 și 2) în funcție de vechimea și frecvența de utilizare a acestora. Obiectele nou create sunt plasate în generația 0, iar cele care supraviețuiesc mai multor colectări sunt promovate în generațiile superioare. Această strategie optimizează performanța GC, deoarece majoritatea obiectelor au o durată de viață scurtă și pot fi colectate rapid din generația 0.

## 1.7. Medii de dezvoltare .NET

Platforma .NET oferă dezvoltatorilor o varietate de medii de dezvoltare și instrumente pentru a crea, depana și implementa aplicații. Printre cele mai populare și utilizate medii de dezvoltare se numără Visual Studio, Visual Studio Code și .NET CLI (Command-Line Interface). Fiecare dintre aceste medii are caracteristici și avantaje specifice, permițând dezvoltatorilor să aleagă instrumentul potrivit pentru nevoile și preferințele lor.

### 1.7.1. Visual Studio

Visual Studio este un mediu de dezvoltare integrat (IDE) complet, oferit de Microsoft, care suportă o gamă largă de limbaje de programare și tehnologii, inclusiv C#, F#, VB.NET, C++, Python, JavaScript și multe altele. Visual Studio este disponibil în mai multe ediții, cum ar fi Community (gratuită pentru studenți, dezvoltatori individuali și echipe mici), Professional și Enterprise, cu următoarele caracteristici principale:

- Editor de cod avansat: Visual Studio oferă un editor de cod puternic, cu evidențierea sintaxei, completarea automată a codului (IntelliSense), refactorizare și navigare facilă în cod.



- Proiectare vizuală: Cu ajutorul designerilor vizuali, cum ar fi Windows Forms Designer și WPF Designer, dezvoltatorii pot crea interfețe de utilizator prin simpla glisare și plasare a controalelor.
- Depanare și profilare: Visual Studio include un depanator puternic, care permite setarea punctelor de întrerupere, inspectarea variabilelor și parcurgerea pas cu pas a codului. De asemenea, există instrumente de profilare pentru a analiza performanța aplicațiilor.
- Integrare cu controlul surselor: Visual Studio se integrează cu sisteme populare de control al versiunilor, cum ar fi Git și Azure DevOps, permițând gestionarea facilă a codului sursă direct din IDE.
- Testare și implementare: Visual Studio oferă instrumente pentru crearea și rularea testelor unitare, precum și pentru implementarea aplicațiilor pe diferite platforme, inclusiv Windows, Linux, macOS, web și dispozitive mobile.

### 1.7.2. Visual Studio Code

Visual Studio Code (VS Code) este un editor de cod open-source, ușor și versatil, dezvoltat de Microsoft. Deși nu este la fel de complex ca Visual Studio, VS Code oferă o experiență de dezvoltare rapidă și eficientă pentru o varietate de limbaje de programare, dintre care C#, F#, Go, Rust, JavaScript, TypeScript, Python și multe altele.

Caracteristici cheie ale Visual Studio Code:

- Ușor și extensibil: VS Code are o amprentă redusă și se încarcă rapid, fiind ideal pentru dezvoltarea de proiecte mici și medii. Prin intermediul sistemului său de extensii, VS Code poate fi personalizat cu noi funcționalități și suport pentru diferite limbaje și tehnologii.
- Suport pentru .NET: Cu ajutorul extensiei C# și a extensiei .NET Core, VS Code oferă un suport excelent pentru dezvoltarea aplicațiilor .NET, inclusiv completarea codului, depanarea și acces la instrumente CLI.
- Integrare cu Git: VS Code are suport integrat pentru Git, permițând gestionarea repository-urilor, efectuarea de commit-uri și rezolvarea conflictelor de merge direct din editor.
- Ecosistem vast de extensii: Comunitatea VS Code a dezvoltat mii de extensii care adaugă funcționalități noi, de la teme de culori și suport pentru limbaje suplimentare până la integrarea cu servicii externe și instrumente de productivitate.

### 1.7.3. .NET CLI

.NET Command-Line Interface (CLI) este un set de instrumente în linia de comandă pentru dezvoltarea, construirea, rularea și implementarea aplicațiilor .NET. .NET CLI este inclus în .NET SDK și oferă o modalitate rapidă și flexibilă de a interacționa cu platforma .NET, fără a fi nevoie de un IDE complet.

---

### Caracteristici cheie ale .NET CLI:

- Instrumente de dezvoltare: .NET CLI oferă comenzi pentru crearea de noi proiecte, adăugarea de referințe, gestionarea pachetelor NuGet și compilarea aplicațiilor.
- Rularea și depanarea: Cu ajutorul comenzilor **dotnet run** și **dotnet test**, dezvoltatorii pot rula aplicațiile și testele direct din linia de comandă. De asemenea, este posibilă atașarea unui depanator extern pentru a depana aplicațiile.
- Implementarea: .NET CLI permite publicarea aplicațiilor pentru diferite platforme țintă, cum ar fi Windows, Linux și macOS, precum și pentru medii de găzduire precum Azure și AWS.
- Scriptare și automatizare: Prin utilizarea .NET CLI în scripturi și sisteme de build automate, dezvoltatorii pot automatiza sarcini comune, cum ar fi compilarea, testarea și implementarea aplicațiilor.

Alegerea mediului de dezvoltare potrivit depinde de nevoile specifice ale proiectului, de preferințele personale și de nivelul de experiență al dezvoltatorului. Visual Studio oferă un IDE complet, cu o gamă largă de funcționalități și instrumente vizuale, fiind potrivit pentru proiecte complexe și echipe mari. Visual Studio Code este o opțiune mai ușoară și rapidă, ideală pentru proiecte mai mici și dezvoltare agilă. .NET CLI oferă flexibilitate și control prin utilizarea comenzilor în linia de comandă, fiind util pentru automatizare și integrarea cu alte instrumente.

## 2. Bazele limbajului C#

C# este un limbaj de programare modern, orientat-obiect și tipizat, dezvoltat de Microsoft ca parte a platformei .NET. Acest limbaj a fost conceput pentru a fi ușor de învățat, puternic și versatil, permițând dezvoltatorilor să creeze o gamă largă de aplicații, de la cele desktop și web până la cele mobile și jocuri.

În acest capitol, vom explora fundamentele limbajului C#, acoperind concepte esențiale precum structura unui program C#, tipuri de date, variabile, operatori și expresii, structuri de control și tablouri. De asemenea, vom introduce programarea orientată pe obiecte (OOP) și vom discuta despre modul în care C# implementează principiile de bază ale OOP.

La finalul acestui capitol, studenții vor fi capabili să:

- Înțeleagă structura de bază a unui program C# și rolul fiecărei componente
- Declare și utilizeze variabile și constante de diferite tipuri de date
- Efectueze operații folosind operatorii aritmetici, relaționali, logici și la nivel de bit
- Controleze fluxul de execuție al programului folosind structuri decizionale (if-else, switch) și repetitive (for, while, do-while)
- Creeze și manipuleze tablouri unidimensionale și multidimensionale
- Definească și apeleze metode cu parametri și valori returnate
- Gestioneze erorile folosind mecanismul de tratare a excepțiilor
- Înțeleagă conceptele de bază ale programării orientate pe obiecte, precum clasele, obiectele, câmpurile, proprietățile și metodele.

### 2.1. Fundamente și sintaxă de bază

Înainte de a începe să scriem cod C#, este esențial să înțelegem structura de bază a unui program C# și elementele fundamentale ale sintaxei limbajului. Această secțiune va acoperi concepte precum namespace-uri, directiva using, clasa Program, metoda Main, comentarii și convenții de denumire.

Un program C# este alcătuit dintr-una sau mai multe clase, care sunt definite în interiorul unui namespace. Namespace-urile sunt folosite pentru a organiza și a grupa clase și alte tipuri înrudite, evitând conflictele de nume și oferind un mod clar de a accesa tipurile definite.

Fiecare program C# trebuie să aibă o clasă cu o metodă numită "Main", care reprezintă punctul de intrare al programului. Metoda Main poate fi declarată cu sau fără argumente și poate returna void sau int.

Pe parcursul acestei secțiuni, vom explora aceste concepte în detaliu și vom vedea cum se îmbină pentru a crea un program C# funcțional. Vom discuta, de asemenea, despre

cele mai bune practici și convenții în scrierea codului C#, care vă vor ajuta să creați un cod lizibil, ușor de întreținut și eficient.

### 2.1.1. Structura unui program C#

Un program C# este format din una sau mai multe clase, care conțin membri precum variabile, proprietăți, metode, evenimente și constructori. O structură tipică a unui program C# include următoarele elemente:

- a) Declarația namespace-ului: Namespace-urile sunt folosite pentru a organiza și a grupa clase și alte tipuri înrudite. Declarația namespace începe cu cuvântul cheie "namespace", urmat de numele namespace-ului.
- b) Directivele using: Directivele using permit utilizarea tipurilor din alte namespace-uri fără a specifica numele complet al namespace-ului de fiecare dată. Acestea sunt plasate, de obicei, la începutul fișierului, după declarația namespace-ului.
- c) Declarația clasei: O clasă este un tip de date definit de utilizator, care încapsulează date și comportamente. Declarația unei clase începe cu modificatorul de acces (e.g., public, private), urmat de cuvântul cheie "class" și numele clasei.
- d) Metoda Main: Fiecare program C# trebuie să aibă o metodă Main, care reprezintă punctul de intrare al programului. Metoda Main poate fi declarată static, cu sau fără argumente și poate returna void sau int.
- e) Membrii clasei: În interiorul unei clase, putem defini variabile, proprietăți, metode, evenimente și constructori, care reprezintă starea și comportamentul obiectelor instanțiate din acea clasă.

Iată un exemplu de structură de bază a unui program C#:

```
using System;
namespace MyProgram
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

În acest exemplu, avem un namespace numit "MyProgram", o clasă numită "Program" și metoda Main, care afișează mesajul "Hello, World!" în consolă. Directiva using "System" permite utilizarea tipurilor din namespace-ul System, cum ar fi clasa Console, fără a specifica numele complet al namespace-ului.

### 2.1.2. Namespace-uri și directiva using

Namespace-urile în C# sunt folosite pentru a organiza și a grupa clase și alte tipuri înrudite, evitând conflictele de nume și oferind un mod clar de a accesa tipurile definite. Fiecare namespace poate conține clase, structuri, interfețe, enumărări și alte namespace-uri.

Declararea unui namespace se face folosind cuvântul cheie "namespace", urmat de numele namespace-ului. Numele namespace-ului poate fi format din mai multe părți, separate prin punct, pentru a crea o ierarhie. De exemplu:

```
namespace MyCompany.MyApplication
{
    // clase, structuri, interfețe, etc.
}
```

În acest caz, "MyCompany" și "MyApplication" sunt părți ale aceluiași namespace.

Pentru a utiliza tipurile definite într-un namespace, putem folosi numele complet al tipului, incluzând namespace-ul:

```
MyCompany.MyApplication.MyClass myObject = new
MyCompany.MyApplication.MyClass();
```

Cu toate acestea, folosirea numelor complete poate face codul greu de citit și de întreținut. Pentru a simplifica acest proces, C# oferă directiva using, care permite utilizarea tipurilor dintr-un namespace fără a specifica numele complet al namespace-ului de fiecare dată.

Directiva using este plasată la începutul fișierului, după declarația namespace-ului și înainte de declarația clasei:

```
using System;
using MyCompany.MyApplication;
namespace MyProgram
{
    class Program
    {
        static void Main(string[] args)
        {
            MyClass myObject = new MyClass();
        }
    }
}
```

În acest exemplu, directiva using "System" permite utilizarea tipurilor din namespace-ul System, cum ar fi clasa Console, fără a specifica numele complet al namespace-ului. De asemenea, directiva using "MyCompany.MyApplication" permite utilizarea tipurilor definite în namespace-ul MyCompany.MyApplication, cum ar fi clasa MyClass, fără a specifica numele complet al namespace-ului.

Este important de menționat că directiva using nu importă namespace-ul în sine, ci doar tipurile definite în acesta. Dacă două namespace-uri diferite conțin tipuri cu același nume, putem utiliza numele complet al tipului pentru a evita ambiguitatea.

### 2.1.3. Clasa Program și metoda Main

În C#, fiecare aplicație consolă trebuie să aibă o clasă care conține o metodă numită "Main". Această metodă reprezintă punctul de intrare al programului, adică locul de unde începe execuția codului atunci când aplicația este lansată.

Clasa care conține metoda Main este adesea numită "Program", deși numele poate fi ales de programator. Această clasă poate fi declarată ca public sau poate fi lăsată fără un modificador de acces explicit, caz în care va avea implicit modificadorul de acces **internal**.

Metoda Main trebuie să îndeplinească următoarele cerințe:

- Trebuie să fie declarată ca static, ceea ce înseamnă că poate fi apelată fără a crea o instanță a clasei.
- Trebuie să aibă modificadorul de acces public.
- Poate avea unul dintre următoarele tipuri de returnare: void (nu returnează nimic) sau int (returnează un cod de ieșire).
- Poate accepta un parametru de tip string[], reprezentând argumentele liniei de comandă, sau poate fi declarată fără parametri.

Iată câteva exemple de declarații valide pentru metoda Main:

```
public static void Main() { }  
public static void Main(string[] args) { }  
public static int Main() { return 0; }  
public static int Main(string[] args) { return 0; }
```

Atunci când metoda Main este declarată cu un parametru string[], acesta reprezintă argumentele liniei de comandă pasate programului la lansare. Aceste argumente pot fi accesate și utilizate în cadrul metodei Main.

Dacă metoda Main returnează un int, valoarea returnată reprezintă un cod de ieșire, care poate fi utilizat pentru a indica starea programului la încheierea execuției. Prin convenție, o valoare de retur 0 indică faptul că programul s-a încheiat cu succes, în timp ce o valoare diferită de zero indică o eroare sau o stare de excepție.

Iată un exemplu de clasă Program cu o metodă Main care acceptă argumente din linia de comandă și returnează un cod de ieșire:

```
public class Program  
{  
    public static int Main(string[] args)  
    {  
        if (args.Length == 0)
```

```

    {
        Console.WriteLine("Niciun argument primit.");
        return 0;
    }
    else
    {
        Console.WriteLine($"Ați oferit {args.Length} argumente:");
        foreach (string arg in args)
        {
            Console.WriteLine(arg);
        }
        return 0;
    }
}

```

În acest exemplu, metoda Main verifică dacă au fost pasate argumente din linia de comandă. Dacă nu există argumente, se afișează un mesaj corespunzător. Dacă există argumente, acestea sunt afișate în consolă. În ambele cazuri, metoda Main returnează 0, indicând finalizarea cu succes a programului.

#### 2.1.4. Comentarii și documentație în cod

Comentariile sunt părți din codul sursă care sunt ignorate de compilator și au ca scop principal explicarea sau clarificarea codului pentru programatori. C# oferă două tipuri de comentarii: comentarii pe o singură linie și comentarii pe mai multe linii.

Comentariile pe o singură linie încep cu două slash-uri (//) și se termină la sfârșitul rândului:

```

// Acesta este un comentariu pe o singură linie
int x = 10; // Comentariile pot fi plasate și după codul propriu-zis

```

Comentariile pe mai multe linii încep cu /\* și se termină cu \*/ și pot acoperi mai multe rânduri:

```

/* Acesta este un comentariu
   pe mai multe linii.
   Poate fi folosit pentru descrieri mai detaliate. */

```

Pe lângă comentariile obișnuite, C# oferă și un mecanism pentru documentarea codului, cunoscut sub numele de documentare XML. Comentariile XML încep cu trei slash-uri (///) și permit generarea automată a documentației pentru tipuri și membrii tipurilor (clase, metode, proprietăți etc.).

Comentariile XML sunt plasate imediat deasupra declarației tipului sau a membrului și conțin etichete XML pentru a descrie diferite aspecte, cum ar fi rezumatul, parametrii, valoarea returnată și alte observații:

```

/// <summary>
/// Calculează suma a două numere întregi.
/// </summary>

```

```
/// <param name="a">Primul număr întreg.</param>
/// <param name="b">Al doilea număr întreg.</param>
/// <returns>Suma celor două numere întregi.</returns>
public static int Sum(int a, int b)
{
    return a + b;
}
```

În acest exemplu, comentariul XML descrie metoda `Sum`, oferind un rezumat, descrieri pentru fiecare parametru și o descriere a valorii returnate. Aceste informații pot fi folosite de IDE-uri (Integrated Development Environments) pentru a oferi informații contextuale programatorilor și pot fi, de asemenea, procesate de instrumente specializate pentru a genera documentație separată, cum ar fi fișiere HTML sau PDF.

Când scrieți comentarii și documentație pentru codul dumneavoastră, este important să urmați câteva bune practici:

- Mențineți comentariile concise, clare și relevante pentru codul adiacent.
- Evitați comentarea exagerată a codului auto-explicativ. În schimb, concentrați-vă pe explicarea logicii complexe sau a deciziilor de proiectare.
- Folosiți comentarii XML pentru a documenta tipuri și membri publici, astfel încât alți dezvoltatori să poată înțelege și utiliza codul dumneavoastră mai ușor.
- Actualizați comentariile și documentația atunci când modificați codul, pentru a menține documentația sincronizată cu implementarea.

### 2.1.5. Convenții de denumire în C#

Convențiile de denumire sunt un set de reguli și recomandări pentru denumirea elementelor în codul sursă, cum ar fi namespace-uri, clase, metode, variabile și constante. Urmarea unor convenții de denumire consistente îmbunătățește lizibilitatea, mentenabilitatea și colaborarea într-un proiect software.

În C#, există convenții de denumire bine stabilite, promovate de Microsoft și adoptate pe scară largă de comunitatea de dezvoltatori. Iată câteva dintre cele mai importante convenții de denumire în C#:

- PascalCase pentru namespace-uri, clase, metode și proprietăți publice:
  - Namespace: `System.IO, MyCompany.MyProject`
  - Clasă: `FileStream, MyClass`
  - Metodă: `ReadFile(), CalculateSum()`
  - Proprietate: `FileName, Count`
- camelCase pentru variabile locale, parametri și câmpuri private:
  - Variabilă locală: `int fileSize, string userName;`
  - Parametru: `void ReadFile(string filePath)`
  - Câmp privat: `private int count, private string name;`
- Prefixul `_` pentru câmpurile private (opțional):



- `private int _count;`, `private string _name;`
- Majuscule și underscore pentru constante:
  - `const int MAX_SIZE = 100;`, `const string DEFAULT_NAME = "John";`
- Prefixul "I" pentru interfețe:
  - `IDisposable`, `IEnumerable`, `IComparable`
- Sufixul "Attribute" pentru clasele de atribut:
  - `ObsoleteAttribute`, `SerializableAttribute`
- Sufixul "Exception" pentru clasele de excepții:
  - `InvalidOperationException`, `ArgumentNullException`
- Evitarea numelor ambigue sau prea generale:
  - În loc de `int val;`, folosiți `int numberOfStudents;`
  - În loc de `void Process();`, folosiți `void CalculateAverage();`
- Folosirea numelor semnificative și descriptive pentru variabile și metode:
  - În loc de `int a;`, folosiți `int age;`
  - În loc de `void Calc();`, folosiți `void CalculateTotal();`
- Folosirea acronimelor și a abrevierilor cu precauție:
  - Acronimele consacrate, cum ar fi "HTML", "XML", "JSON", pot fi scrise cu majuscule.
  - Abrevierile formate din mai mult de două litere trebuie scrise în PascalCase sau camelCase, în funcție de context (de exemplu, `HtmlHelper`, `jsonData`).

Deși aceste convenții nu sunt impuse de compilatorul C#, urmarea lor face codul mai ușor de înțeles și de întreținut de către alți dezvoltatori. Consistența este cheia în aplicarea convențiilor de denumire, iar echipele de dezvoltare pot decide să adapteze sau să extindă aceste convenții în funcție de nevoile și preferințele lor specifice.

Visual Studio, oferă refactorizări și sugestii pentru a ajuta dezvoltatorii să urmeze convențiile de denumire și să mențină consistența în cadrul proiectului.

## 2.2. Tipuri de date și variabile

În C#, tipurile de date și variabilele sunt elementele fundamentale pentru stocarea și manipularea datelor în cadrul unui program. C# este un limbaj de programare puternic tipizat, ceea ce înseamnă că fiecare variabilă trebuie să aibă un tip de date specific declarat în momentul definirii sale. Această abordare ajută la prevenirea erorilor în timpul compilării și asigură un comportament previzibil al programului.

Există două categorii principale de tipuri de date în C#: tipuri de valoare și tipuri de referință.

- Tipurile de valoare sunt tipuri de date care conțin direct o valoare și sunt alocate pe stivă. Când o variabilă de tip valoare este atribuită unei alte variabile, valoarea este copiată, astfel încât fiecare variabilă are propria copie a datelor. Tipurile de

valoare includ tipuri numerice întregi (int, long, etc.), tipuri numerice reale (float, double, decimal), tipul boolean (bool) și tipul caracter (char).

- Tipurile de referință, pe de altă parte, conțin o referință (adresă de memorie) la locația datelor în memorie, care este alocată pe heap. Când o variabilă de tip referință este atribuită unei alte variabile, ambele variabile fac referire la aceeași locație de memorie, partajând aceleași date. Tipurile de referință includ clase, interfețe, tablouri și tipul string.

În această secțiune, vom explora în detaliu tipurile de date primitive, cum ar fi int, double, bool și char, precum și tipul string și operațiunile asociate cu șirurile de caractere. Vom învăța cum să declarăm și să inițializăm variabile, cum să efectuăm conversii între tipuri de date și cum să folosim modificatorii const și readonly pentru a defini constante și variabile read-only.

De asemenea, vom discuta despre tipurile nullable, care permit atribuirea valorii null tipurilor de valoare și despre inferența tipurilor folosind cuvântul cheie var, care permite compilatorului să deducă tipul de date al unei variabile pe baza valorii sale de inițializare.

La finalul acestei secțiuni, veți fi capabili să:

- Identificați diferențele dintre tipurile de valoare și tipurile de referință
- Declarați și inițializați variabile folosind diferite tipuri de date primitive
- Efectuați operațiuni cu șiruri de caractere folosind tipul string
- Utilizați tipuri nullable pentru a permite valori null în tipurile de valoare
- Efectuați conversii între tipuri de date folosind casting explicit și implicit
- Declarați constante și variabile read-only folosind modificatorii const și readonly
- Utilizați inferența tipurilor cu ajutorul cuvântului cheie var

### 2.2.1. Tipuri de date primitive (int, double, bool, char)

C# oferă un set de tipuri de date primitive, care sunt tipuri de date predefinite, utilizate pentru a stoca valori simple. Aceste tipuri de date primitive sunt tipuri de valoare și sunt alocate pe stivă. Să analizăm în detaliu câteva dintre cele mai frecvent utilizate tipuri de date primitive în C#:

a) Tipuri numerice întregi:

- i. **int**: Un număr întreg cu semn pe 32 de biți, cu valori între -2,147,483,648 și 2,147,483,647.
- ii. **long**: Un număr întreg cu semn pe 64 de biți, cu valori între -9,223,372,036,854,775,808 și 9,223,372,036,854,775,807.
- iii. **short**: Un număr întreg cu semn pe 16 biți, cu valori între -32,768 și 32,767.
- iv. **byte**: Un număr întreg fără semn pe 8 biți, cu valori între 0 și 255.

b) Tipuri numerice reale:

- i. **double**: Un număr în virgulă mobilă cu precizie dublă pe 64 de biți, cu valori între  $\pm 5.0 \times 10^{-324}$  și  $\pm 1.7 \times 10^{308}$ .
- ii. **float**: Un număr în virgulă mobilă cu precizie simplă pe 32 de biți, cu valori între  $\pm 1.5 \times 10^{-45}$  și  $\pm 3.4 \times 10^{38}$ .
- iii. **decimal**: Un număr zecimal cu 28-29 de cifre semnificative, potrivit pentru calcule financiare și monetare precise.

c) Tipul boolean:

- i. **bool**: Reprezintă o valoare logică care poate fi fie **true** (adevărat), fie **false** (fals).

d) 4. Tipul caracter:

- i. **char**: Reprezintă un singur caracter Unicode pe 16 biți, încadrat între apostrofuri simple (ex.: 'A', '7', '\$').

Iată câteva exemple de declarare și inițializare a variabilelor folosind tipuri de date primitive:

```
int age = 25;
long population = 7800000000;
short dayOfMonth = 23;
byte hour = 18;
double price = 9.99;
float temperature = 23.5f; // Sufixul 'f' indică un literal de tip float
decimal balance = 1234.56m; // Sufixul 'm' indică un literal de tip decimal
bool isStudent = true;
char grade = 'A';
```

Alegerea tipului de date potrivit depinde de natura datelor pe care le stocați și de cerințele specifice ale programului. De exemplu, pentru numere întregi mici, puteți folosi **short** sau **byte** pentru a economisi memorie, în timp ce pentru calcule matematice precise, **double** sau **decimal** ar fi alegeri mai potrivite.

Este important să rețineți intervalele de valori posibile pentru fiecare tip de date și să alegeți tipul adecvat pentru a evita depășirea limitelor sau pierderile de precizie nedorite.

### 2.2.2. Tipul string și operațiuni cu șiruri de caractere

Tipul **string** în C# reprezintă o secvență de caractere Unicode. Spre deosebire de tipul **char**, care poate stoca un singur caracter, tipul **string** poate stoca un șir de caractere de lungime variabilă. Șirurile de caractere sunt înconjurate de ghilimele duble în C# (ex.: "Hello, World!").

Iată câteva operațiuni comune care pot fi efectuate cu șiruri de caractere în C#:

- Concatenarea șirurilor de caractere:

- Operatorul `+` poate fi folosit pentru a concatena două sau mai multe șiruri de caractere.
- Metoda `string.Concat()` poate fi, de asemenea, utilizată pentru a concatena șiruri de caractere.

Exemplu:

```
string firstName = "John";
string lastName = "Doe";
string fullName = firstName + " " + lastName; // "John Doe"
string greeting = string.Concat("Hello, ", fullName, "!"); // "Hello, John Doe!"
```

- Formatarea șirurilor de caractere:
  - Metoda `string.Format()` permite crearea unui șir de caractere formatat, folosind un șablon și argumente corespunzătoare.
  - Interpolarea șirurilor de caractere (disponibilă începând cu C# 6.0) permite includerea expresiilor direct într-un șir de caractere, folosind sintaxa `$"..."`.

Exemplu:

```
int age = 25;
string name = "Alice";
string formattedString = string.Format("{0} is {1} years old.", name, age);
// "Alice is 25 years old."
string interpolatedString = $"{name} is {age} years old."; // "Alice is 25 years old."
```

- Compararea șirurilor de caractere:
  - Operatorii `==` și `!=` pot fi folosiți pentru a verifica egalitatea sau inegalitatea a două șiruri de caractere.
  - Metoda `string.Compare()` permite compararea a două șiruri de caractere și returnează un întreg care indică relația dintre ele (<0, 0, >0).
  - Metoda `string.Equals()` poate fi, de asemenea, utilizată pentru a verifica egalitatea a două șiruri de caractere.

Exemplu:

```
string str1 = "hello";
string str2 = "world";
bool areEqual = (str1 == str2); // false
int comparisonResult = string.Compare(str1, str2); // -1 (str1 precedes str2)
bool ignoreCase = string.Equals(str1, "HELLO",
StringComparison.OrdinalIgnoreCase); // true
```

- Accesarea și manipularea caracterelor dintr-un șir:
  - Caracterele individuale dintr-un șir pot fi accesate folosind indexul lor (începând de la 0).
  - Proprietatea `string.Length` returnează lungimea unui șir de caractere.

- Metode precum `string.Substring()`, `string.ToUpper()`, `string.ToLower()`, `string.Trim()` pot fi folosite pentru a manipula șirurile de caractere.

Exemplu:

```
string text = "Hello, World!";
char firstChar = text[0]; // 'H'
int length = text.Length; // 13
string substring = text.Substring(7, 5); // "World"
string upperCase = text.ToUpper(); // "HELLO, WORLD!"
string trimmed = text.Trim('!', '.'); // "Hello, World"
```

Aceste sunt doar câteva exemple de operațiuni care pot fi efectuate cu șiruri de caractere în C#. Tipul `string` oferă o gamă largă de metode și proprietăți pentru manipularea și procesarea textului, făcându-l un instrument puternic în dezvoltarea aplicațiilor.

Este important de reținut că șirurile de caractere sunt imutabile în C#, ceea ce înseamnă că odată create, conținutul lor nu poate fi modificat. Orice operație care pare să modifice un șir de caractere va returna de fapt un nou șir de caractere, lăsând șirul original nemodificat.

### 2.2.3. Tipuri nullable și operatorul '?'

În C#, tipurile de valoare (cum ar fi `int`, `double`, `bool`, etc.) nu pot avea valoarea `null` în mod implicit. Acest lucru poate fi restrictiv în anumite situații, în special atunci când lucrați cu baze de date sau cu date care pot avea valori lipsă. Pentru a permite atribuirea valorii `null` tipurilor de valoare, C# oferă tipuri nullable și operatorul '?'.

Un tip nullable este creat prin adăugarea operatorului '?' după tipul de date de valoare. Aceasta indică faptul că variabila poate stoca nu numai valorile obișnuite ale tipului de date, ci și valoarea `null`.

Exemplu:

```
int? nullableInt = null;
double? nullableDouble = 3.14;
bool? nullableBool = true;
```

În exemplul de mai sus, `nullableInt` este de tipul `Nullable<int>` (sau `int?`), care poate stoca fie o valoare întreagă, fie `null`. Similar, `nullableDouble` și `nullableBool` pot stoca valorile respective sau `null`.

Pentru a accesa valoarea unui tip nullable, puteți folosi proprietatea `Value`. Cu toate acestea, înainte de a accesa `Value`, este important să verificați dacă variabila conține o valoare sau este `null`, folosind proprietatea `HasValue` sau comparând direct cu `null`.

Exemplu:

```
int? number = 42;
```

```
if (number.HasValue)
{
    int value = number.Value; // 42
}
// sau
if (number != null)
{
    int value = number.Value; // 42
}
```

C# oferă, de asemenea, un operator de propagare null (`?.`), care permite accesarea în siguranță a valorii unui tip nullable fără a provoca o excepție `NullReferenceException`.

Exemplu:

```
int? number = null;
int value = number?.Value ?? 0; // 0
```

În acest caz, dacă `number` este null, expresia `number?.Value` va returna null în loc să arunce o excepție. Operatorul `??` (coalescență null) este apoi folosit pentru a furniza o valoare implicită (în acest caz, 0) dacă rezultatul expresiei `number?.Value` este null.

Tipurile nullable sunt deosebit de utile atunci când lucrați cu date provenite din surse externe, unde valorile pot fi lipsă sau nedefinite. Ele permit reprezentarea explicită a absenței unei valori, evitând astfel erorile și îmbunătățind claritatea codului.

Este important de reținut că performanța poate fi ușor afectată atunci când folosiți tipuri nullable, deoarece ele implică o verificare suplimentară a valorilor null. Cu toate acestea, în majoritatea cazurilor, avantajele în ceea ce privește siguranța și expresivitatea codului depășesc impactul minor asupra performanței.

## 2.2.4. Conversii între tipuri de date (casting)

În C#, uneori este necesar să converțiți o variabilă de la un tip de date la altul. Această operație se numește casting sau conversie de tip. C# oferă două tipuri de casting: casting implicit și casting explicit.

### 2.2.4.1. Casting implicit (conversie implicită)

- Are loc atunci când conversia poate fi realizată în siguranță, fără pierderi de date.
- Nu necesită o sintaxă specială, deoarece compilatorul efectuează automat conversia.

Exemple de conversii implicite:

- De la un tip numeric cu intervalul mai mic la un tip numeric cu intervalul mai mare (de exemplu, de la `int` la `long`, `float` la `double`).
- De la un tip derivat la un tip de bază într-o ierarhie de clase.

```
int intValue = 42;
double doubleValue = intValue; // Conversie implicită de la int la double
```

#### 2.2.4.2. Casting explicit (conversie explicită)

- Este necesar atunci când conversia poate duce la pierderi de date sau când nu există o conversie implicită disponibilă.
- Necesită utilizarea operatorului de casting `()` înaintea valorii sau variabilei care trebuie convertită.

Exemple de conversii explicite:

- De la un tip numeric cu intervalul mai mare la un tip numeric cu intervalul mai mic (de exemplu, de la double la int, long la short).
- De la un tip de bază la un tip derivat într-o ierarhie de clase.

Exemplu:

```
double doubleValue = 42.5;
int intValue = (int)doubleValue; // Conversie explicită de la double la int
(pierdere de date)
```

Când efectuați conversii explicite, trebuie să fiți atenți la potențialele pierderi de date. În exemplul de mai sus, partea fracționară a valorii `doubleValue` este pierdută în timpul conversiei la `int`.

De asemenea, C# oferă metode și clase helper pentru a efectua conversii între diferite tipuri de date:

- Clasa `Convert`: Oferă metode pentru conversia între tipuri de date comune, cum ar fi `Convert.ToInt32()`, `Convert.ToDouble()`, `Convert.ToString()`, etc.

```
string stringValue = "42";
int intValue = Convert.ToInt32(stringValue); // Conversie de la string la int
```

- Metoda `Parse()`: Disponibilă pentru diferite tipuri de date, permite conversia dintr-un șir de caractere în tipul de date corespunzător.

```
string stringValue = "42";
int intValue = int.Parse(stringValue); // Conversie de la string la int
```

- Metoda `TryParse()`: Similar cu `Parse()`, dar în loc să arunce o excepție în caz de eșec, returnează o valoare booleană care indică dacă conversia a reușit.

```
string stringValue = "42";
int intValue;
bool success = int.TryParse(stringValue, out intValue);
```

Este important să alegeți metoda de conversie adecvată în funcție de tipurile de date implicate și de cerințele specifice ale programului dumneavoastră. Conversia implicită este preferată atunci când este posibil, deoarece este sigură și nu necesită o sintaxă

suplimentară. Cu toate acestea, în cazurile în care este necesară o conversie explicită, asigurați-vă că gestionați corect potențialele pierderi de date și excepțiile.

### 2.2.5. Variabile constante (const) și readonly

În C#, puteți declara variabile ale căror valori rămân neschimbate pe parcursul execuției programului, folosind cuvintele cheie `const` și `readonly`. Aceste variabile sunt utile pentru stocarea valorilor fixe, cum ar fi constantele matematice, ratele de impozitare sau configurările aplicației.

#### 2.2.5.1. Variabile constante (const)

- Sunt variabile ale căror valori sunt atribuite în momentul compilării și nu pot fi modificate ulterior.
- Trebuie să fie inițializate în momentul declarării.
- Pot fi declarate în interiorul unei clase, struct sau al unei metode.
- Convențional, numele variabilelor `const` se scriu cu majuscule și se separă cuvintele cu underscore.

Exemplu:

```
public const double PI = 3.14159;
const int MAX_SIZE = 100;
public void PrintCircleArea(double radius)
{
    const double PI = 3.14159;
    double area = PI * radius * radius;
    Console.WriteLine($"Aria cercului: {area}");
}
```

#### 2.2.5.2. Variabile readonly

- Sunt variabile ale căror valori pot fi atribuite fie la declarare, fie în constructorul clasei.
- După atribuirea inițială, valoarea unei variabile `readonly` nu poate fi modificată.
- Pot fi declarate doar la nivelul clasei, nu în interiorul unei metode.
- Spre deosebire de variabilele `const`, valoarea unei variabile `readonly` poate fi determinată în timpul execuției.

Exemplu:

```
public readonly int SerialNumber;
public MyClass(int serialNumber)
{
    SerialNumber = serialNumber;
}
```

Diferențe între `const` și `readonly`:

- Variabilele `const` trebuie să fie inițializate la declarare, în timp ce variabilele `readonly` pot fi inițializate fie la declarare, fie în constructor.



- Variabilele `const` sunt evaluate la momentul compilării, în timp ce variabilele `readonly` pot fi evaluate în timpul execuției.
- Variabilele `const` pot fi declarate în interiorul unei metode, în timp ce variabilele `readonly` pot fi declarate doar la nivelul clasei.

Utilizarea variabilelor `const` și `readonly` aduce câteva beneficii:

- Intenția clară: Prin marcarea unei variabile ca fiind `const` sau `readonly`, indicați explicit că valoarea sa nu ar trebui să fie modificată, ceea ce îmbunătățește lizibilitatea și înțelegerea codului.
- Prevenirea modificărilor accidentale: Declararea variabilelor ca fiind `const` sau `readonly` împiedică modificarea accidentală a valorilor lor, reducând astfel potențialele erori.
- Optimizări ale compilatorului: Compilatorul poate aplica optimizări specifice variabilelor `const`, deoarece valorile lor sunt cunoscute în momentul compilării.

În general, se recomandă utilizarea variabilelor `const` pentru valorile care sunt cunoscute în momentul compilării și care nu se vor modifica niciodată, iar variabilele `readonly` pentru valorile care sunt determinate în timpul execuției (de exemplu, prin intermediul constructorului) și care nu ar trebui să fie modificate ulterior.

### 2.2.6. Inferența tipurilor cu `var`

Începând cu C# 3.0, a fost introdusă o caracteristică numită inferența tipurilor, care permite declararea variabilelor fără a specifica explicit tipul de date, folosind cuvântul cheie `var`. Când folosiți `var`, compilatorul C# deduce (inferă) tipul variabilei din expresia de inițializare.

Exemplu:

```
var name = "John Doe";  
var age = 25;  
var isStudent = true;
```

În acest caz, compilatorul deduce că `name` este de tip `string`, `age` este de tip `int`, iar `isStudent` este de tip `bool`, pe baza valorilor de inițializare.

Avantajele utilizării inferenței tipurilor cu `var` sunt:

- Cod mai concis și mai lizibil: Prin eliminarea duplicării tipului de date în declararea variabilei și în expresia de inițializare, codul poate deveni mai concis și mai ușor de citit.
- Refactorizare ușoară: Dacă tipul de date al expresiei de inițializare se modifică, nu este necesar să actualizați manual tipul variabilei, deoarece acesta va fi dedus automat de compilator.
- Tipuri anonime: Inferența tipurilor permite utilizarea tipurilor anonime, care sunt tipuri generate de compilator fără un nume explicit.

Cu toate acestea, există câteva considerente atunci când folosiți `var`:

- Utilizați `var` doar când tipul de date este evident din expresia de inițializare. Dacă tipul de date nu este clar, este mai bine să specificați explicit tipul pentru a îmbunătăți lizibilitatea.
- Nu folosiți `var` pentru simpla declarare a variabilelor fără inițializare, deoarece compilatorul nu poate deduce tipul de date în acest caz.
- În cod, este posibil să aveți convenții sau linii directoare specifice cu privire la utilizarea `var`. Unele echipe preferă să folosească `var` doar pentru tipuri evidente sau pentru tipuri anonime, în timp ce altele îl folosesc mai extensiv. Respectați convențiile adoptate de echipa dumneavoastră pentru a menține consistența.

Exemplu de utilizare a `var` cu tipuri anonime:

```
var person = new { Name = "John Doe", Age = 25 };  
Console.WriteLine($"Name: {person.Name}, Age: {person.Age}");
```

În acest caz, `person` este un tip anonim cu proprietățile `Name` și `Age`, deduse de compilator din expresia de inițializare.

## 2.3. Operatori și expresii

C# oferă o gamă largă de operatori care permit efectuarea diferitelor operații asupra variabilelor și valorilor. Acești operatori sunt esențiali pentru construirea expresiilor, care reprezintă combinații de variabile, valori și operatori ce pot fi evaluate pentru a produce un rezultat.

În acest paragraf, vom explora principalele categorii de operatori disponibili în C#:

- Operatori aritmetici: utilizați pentru efectuarea operațiilor matematice de bază, cum ar fi adunarea, scăderea, înmulțirea și împărțirea.
- Operatori de comparare: utilizați pentru a compara două valori și a returna un rezultat de tip `bool` (adevărat sau fals).
- Operatori logici: utilizați pentru a combina expresii condiționale și a efectua operații logice, cum ar fi AND, OR și NOT.
- Operatori de atribuire: utilizați pentru a atribui valori variabilelor.
- Operatori la nivel de biți: utilizați pentru a efectua operații asupra reprezentării binare a valorilor numerice întregi.

Pe parcursul acestui paragraf, vom analiza în detaliu fiecare categorie de operatori, sintaxa și semantica acestora, precum și exemple relevante de utilizare. De asemenea, vom discuta despre precedența operatorilor, care determină ordinea în care operatorii sunt evaluați atunci când sunt combinați în expresii complexe.

### 2.3.1. Operatori aritmetici

Operatorii aritmetici sunt utilizați pentru a efectua operații matematice de bază asupra valorilor numerice. C# oferă următorii operatori aritmetici:

- Adunare (+): Adună două valori.

```
int suma = 5 + 3; // suma va fi 8
```

- Scădere (-): Scade o valoare dintr-o alta.

```
int diferenta = 10 - 7; // diferenta va fi 3
```

- Înmulțire (\*): Înmulțește două valori.

```
int produs = 4 * 6; // produs va fi 24
```

- Împărțire (/): Împarte o valoare la alta.

Notă: Când se împart două numere întregi, rezultatul va fi tot un număr întreg, iar partea fracționară va fi eliminată.

```
int catul = 15 / 4; // catul va fi 3
```

- Modulo (%): Returnează restul împărțirii a două valori întregi.

```
int restul = 17 % 5; // restul va fi 2
```

De asemenea, C# oferă variante prescurtate ale operatorilor aritmetici, care permit efectuarea unei operații și atribuirea rezultatului în aceeași variabilă:

- += : Adună valoarea din dreapta la variabila din stânga și atribuie rezultatul variabilei din stânga.

```
int x = 10;  
x += 5; // x va fi 15
```

- -= : Scade valoarea din dreapta din variabila din stânga și atribuie rezultatul variabilei din stânga.

```
int y = 20;  
y -= 3; // y va fi 17
```

- \*= : Înmulțește variabila din stânga cu valoarea din dreapta și atribuie rezultatul variabilei din stânga.

```
int z = 6;  
z *= 4; // z va fi 24
```

- /= : Împarte variabila din stânga la valoarea din dreapta și atribuie rezultatul variabilei din stânga.

```
int w = 25;
```

```
w /= 5; // w va fi 5
```

- **%=** : Calculează restul împărțirii variabilei din stânga la valoarea din dreapta și atribuie rezultatul variabilei din stânga.

```
int v = 18;  
v %= 7; // v va fi 4
```

Operatorii aritmetici pot fi combinați în expresii complexe, iar ordinea evaluării este determinată de precedența operatorilor (discutată în secțiunea 2.3.6).

Exemple de expresii aritmetice:

```
int rezultat1 = 5 + 3 * 4; // rezultat1 va fi 17  
int rezultat2 = (5 + 3) * 4; // rezultat2 va fi 32
```

În primul exemplu, înmulțirea are prioritate, deci **3 \* 4** este evaluat primul, iar apoi rezultatul este adunat cu 5. În cel de-al doilea exemplu, adunarea are prioritate datorită parantezelor, astfel încât **5 + 3** este evaluat primul, iar apoi rezultatul este înmulțit cu 4.

Este important să fiți atenți la tipurile de date ale operanzilor atunci când utilizați operatori aritmetici. Când operați cu numere întregi, rezultatul va fi întotdeauna un număr întreg. Dacă doriți să obțineți un rezultat fracționar, asigurați-vă că cel puțin unul dintre operanzi este de tip float sau double.

```
double rezultat3 = 5 / 2; // rezultat3 va fi 2 (împărțire între întregi)  
double rezultat4 = 5.0 / 2; // rezultat4 va fi 2.5 (împărțire între double și întreg)
```

## 2.3.2. Operatori de comparare

Operatorii de comparare sunt utilizați pentru a compara două valori și a returna un rezultat de tip bool (adevărat sau fals). Acești operatori sunt esențiali pentru controlul fluxului programului și pentru luarea deciziilor bazate pe anumite condiții. C# oferă următorii operatori de comparare:

- Egal (**==**): Verifică dacă două valori sunt egale.

```
int a = 5, b = 5;  
bool rezultat = (a == b); // rezultat va fi true
```

- Diferit (**!=**): Verifică dacă două valori sunt diferite.

```
int x = 10, y = 7;  
bool rezultat = (x != y); // rezultat va fi true
```

- Mai mare (**>**): Verifică dacă prima valoare este mai mare decât a doua.

```
int m = 8, n = 3;  
bool rezultat = (m > n); // rezultat va fi true
```

- Mai mic (<): Verifică dacă prima valoare este mai mică decât a doua.

```
int p = 4, q = 9;  
bool rezultat = (p < q); // rezultat va fi true
```

- Mai mare sau egal (>=): Verifică dacă prima valoare este mai mare sau egală cu a doua.

```
int r = 7, s = 7;  
bool rezultat = (r >= s); // rezultat va fi true
```

- Mai mic sau egal (<=): Verifică dacă prima valoare este mai mică sau egală cu a doua.

```
int t = 5, u = 6;  
bool rezultat = (t <= u); // rezultat va fi true
```

Operatorii de comparare pot fi folosiți cu diverse tipuri de date, cum ar fi int, float, double, char și string. Când se compară șiruri de caractere (string), se utilizează ordinea lexicografică (alfabetică).

```
string cuvant1 = "abc", cuvant2 = "def";  
bool rezultat = (cuvant1 < cuvant2); // rezultat va fi true
```

De asemenea, operatorii de comparare pot fi combinați cu operatorii logici (discuțați în secțiunea 2.3.3) pentru a crea expresii condiționale complexe.

```
int varsta = 25;  
bool eligibil = (varsta >= 18) && (varsta <= 65); // verifică dacă vârsta  
este între 18 și 65 (inclusiv)
```

Un caz special de comparare este compararea obiectelor. Când se folosește operatorul == pentru a compara două obiecte, acesta va verifica dacă obiectele se referă la aceeași instanță în memorie, nu dacă au aceleași valori. Pentru a compara valorile obiectelor, trebuie să suprascrieți metoda Equals a clasei respective.

Folosirea operatorilor de comparare este fundamentală în structurile de control, cum ar fi instrucțiunile if-else și buclele while, pentru a lua decizii bazate pe anumite condiții.

```
int nota = 85;  
if (nota >= 90)  
{  
    Console.WriteLine("A");  
}  
else if (nota >= 80)  
{  
    Console.WriteLine("B");  
}  
else  
{  
    Console.WriteLine("C");  
}
```

```
}
```

În acest exemplu, se folosesc operatorii de comparare `>=` pentru a determina categoria notei în funcție de valoarea acesteia.

### 2.3.3. Operatori logici

Operatorii logici sunt utilizați pentru a combina expresii condiționale și a efectua operații logice. Acești operatori returnează o valoare de tip `bool` (adevărat sau fals), în funcție de valorile expresiilor condiționale. C# oferă următorii operatori logici:

- AND logic (`&&`): Returnează `true` doar dacă ambele expresii condiționale sunt adevărate.

```
bool rezultat = (5 > 3) && (7 < 10); // rezultat va fi true
```

- OR logic (`||`): Returnează `true` dacă cel puțin una dintre expresiile condiționale este adevărată.

```
bool rezultat = (5 > 3) || (7 > 10); // rezultat va fi true
```

- NOT logic (`!`): Inversează valoarea expresiei condiționale.

```
bool rezultat = !(5 > 3); // rezultat va fi false
```

Operatorii logici sunt frecvent utilizați în combinație cu operatorii de comparare pentru a crea expresii condiționale complexe. De exemplu, să presupunem că dorim să verificăm dacă o persoană are vârsta cuprinsă între 18 și 65 de ani:

```
int varsta = 25;  
bool eligibil = (varsta >= 18) && (varsta <= 65);
```

În acest caz, expresia condițională `(varsta >= 18) && (varsta <= 65)` va returna `true` doar dacă ambele condiții sunt îndeplinite, adică vârsta este mai mare sau egală cu 18 și, în același timp, mai mică sau egală cu 65.

Operatorii logici au o ordine de precedență, iar NOT (`!`) are cea mai mare prioritate, urmat de AND (`&&`), iar apoi de OR (`||`). Puteți utiliza paranteze pentru a grupa expresiile condiționale și a controla ordinea de evaluare:

```
bool rezultat = (5 > 3) && ((7 < 10) || (2 < 1)); // rezultat va fi true
```

În acest exemplu, `(7 < 10) || (2 < 1)` este evaluat primul datorită parantezelor, iar apoi rezultatul este combinat cu `(5 > 3)` folosind operatorul AND (`&&`).

O particularitate a operatorilor AND și OR în C# este evaluarea în scurtcircuit (short-circuit evaluation). Aceasta înseamnă că, în anumite situații, a doua expresie condițională poate să nu fie evaluată deloc. De exemplu:

```
bool rezultat = (5 < 3) && (7 / 0 == 0); // rezultat va fi false, fără a  
genera o eroare de împărțire la zero
```

În acest caz, deoarece prima expresie condițională ( $5 < 3$ ) este falsă, iar operatorul este AND (&&), rezultatul final va fi întotdeauna fals, indiferent de valoarea celei de-a doua expresii. Prin urmare, ( $7 / 0 == 0$ ) nu este evaluat deloc, evitându-se astfel o eroare de împărțire la zero.

Evaluarea în scurtcircuit poate fi utilă pentru a evita erorile și pentru a optimiza performanța programului, deoarece expresiile condiționale costisitoare sau potențial periculoase pot fi evitate atunci când nu sunt necesare.

### 2.3.4. Operatori de atribuire

Operatorii de atribuire sunt utilizați pentru a atribui valori variabilelor. Operatorul de bază de atribuire este semnul egal (=), care atribuie valoarea din partea dreaptă a operatorului variabilei din partea stângă:

```
int x = 10;
```

În acest caz, valoarea 10 este atribuită variabilei x.

Pe lângă operatorul de bază de atribuire, C# oferă operatori de atribuire compuși, care combină o operație aritmetică cu atribuirea. Acești operatori efectuează operația specificată între valoarea curentă a variabilei și valoarea din partea dreaptă, iar apoi atribuie rezultatul înapoi variabilei. Iată lista operatorilor de atribuire compuși în C#:

- Adunare și atribuire (+=):

```
int x = 5;  
x += 3; // echivalent cu x = x + 3; x va fi 8
```

- Scădere și atribuire (-=):

```
int y = 10;  
y -= 4; // echivalent cu y = y - 4; y va fi 6
```

- Înmulțire și atribuire (\*=):

```
int z = 6;  
z *= 2; // echivalent cu z = z * 2; z va fi 12
```

- Împărțire și atribuire (/=):

```
int a = 20;  
a /= 4; // echivalent cu a = a / 4; a va fi 5
```

- Modulo și atribuire (%=):

```
int b = 17;  
b %= 5; // echivalent cu b = b % 5; b va fi 2
```

- AND bit cu bit și atribuire (&=):

```
int c = 10; // 1010 în binar
c &= 6;     // 0110 în binar; c va fi 2 (0010 în binar)
```

- OR bit cu bit și atribuire (|=):

```
int d = 5; // 0101 în binar
d |= 3;    // 0011 în binar; d va fi 7 (0111 în binar)
```

- XOR bit cu bit și atribuire (^=):

```
int e = 12; // 1100 în binar
e ^= 6;     // 0110 în binar; e va fi 10 (1010 în binar)
```

- Deplasare la stânga și atribuire (<<=):

```
int f = 5; // 0101 în binar
f <<= 2;   // deplasare la stânga cu 2 biți; f va fi 20 (10100 în binar)
```

- Deplasare la dreapta și atribuire (>>=):

```
int g = 20; // 10100 în binar
g >>= 2;    // deplasare la dreapta cu 2 biți; g va fi 5 (0101 în binar)
```

Operatorii de atribuire compuși pot face codul mai concis și mai ușor de citit, deoarece combină două operații (operația aritmetică sau la nivel de biți și atribuirea) într-una singură.

Este important de menționat că, atunci când se utilizează operatori de atribuire compuși, tipul rezultatului trebuie să fie compatibil cu tipul variabilei căreia i se atribuie valoarea. În caz contrar, poate fi necesară o conversie explicită a tipului.

```
int h = 5;
h += 3.14; // eroare de compilare: nu se poate converti implicit double în int
```

Pentru a rezolva această problemă, putem face o conversie explicită:

```
h += (int)3.14; // h va fi 8
```

### 2.3.5. Operatori la nivel de biți

Operatorii la nivel de biți efectuează operații asupra reprezentării binare a valorilor numerice întregi. Acești operatori tratează operandii ca o secvență de biți și efectuează operații logice sau de deplasare pe acești biți. C# oferă următorii operatori la nivel de biți:

- AND bit cu bit (&): Efectuează operația logică AND pe fiecare pereche de biți corespunzători ai operandilor.

```
int a = 10; // 1010 în binar
```



```
int b = 6; // 0110 în binar
int rezultat = a & b; // rezultat va fi 2 (0010 în binar)
```

- OR bit cu bit (**|**): Efectuează operația logică OR pe fiecare pereche de biți corespunzători ai operanzilor.

```
int a = 5; // 0101 în binar
int b = 3; // 0011 în binar
int rezultat = a | b; // rezultat va fi 7 (0111 în binar)
```

- XOR bit cu bit (**^**): Efectuează operația logică XOR (sau exclusiv) pe fiecare pereche de biți corespunzători ai operanzilor.

```
int a = 12; // 1100 în binar
int b = 6; // 0110 în binar
int rezultat = a ^ b; // rezultat va fi 10 (1010 în binar)
```

- NOT bit cu bit (**~**): Inversează fiecare bit al operandului.

```
int a = 5; // 0101 în binar
int rezultat = ~a; // rezultat va fi -6 (1111...1010 în binar, reprezentare în complement față de 2)
```

- Deplasare la stânga (**<<**): Deplasează biții operandului din stânga spre stânga cu un număr specificat de poziții.

```
int a = 5; // 0101 în binar
int rezultat = a << 2; // rezultat va fi 20 (10100 în binar)
```

- Deplasare la dreapta (**>>**): Deplasează biții operandului din stânga spre dreapta cu un număr specificat de poziții.

```
int a = 20; // 10100 în binar
int rezultat = a >> 2; // rezultat va fi 5 (0101 în binar)
```

Operațiile la nivel de biți sunt adesea utilizate pentru a manipula valori la nivel scăzut, cum ar fi flagurile de stare, permisiuni sau configurații. De asemenea, aceste operații pot fi folosite pentru a optimiza anumite calcule matematice.

Să luăm un exemplu concret de utilizare a operatorilor la nivel de biți. Să presupunem că avem un sistem de permisiuni reprezentate de un număr întreg, unde fiecare bit corespunde unei permisiuni specifice:

- Bitul 0 (cel mai puțin semnificativ): Permisiunea de citire (1 = permis, 0 = interzis)
- Bitul 1: Permisiunea de scriere
- Bitul 2: Permisiunea de execuție

Putem folosi operatorii la nivel de biți pentru a verifica, a acorda sau a revoca permisiuni:

```

int permisiuni = 5; // 0101 în binar (citire și execuție permise)

// Verifică permisiunea de citire
bool poateCiti = (permisiuni & 1) != 0; // true

// Acordă permisiunea de scriere
permisiuni |= 2; // permisiuni va fi 7 (0111 în binar)

// Revocă permisiunea de execuție
permisiuni &= ~4; // permisiuni va fi 3 (0011 în binar)

```

Este important să fim atenți atunci când folosim operatori la nivel de biți, deoarece acestea pot face codul mai puțin lizibil și mai greu de întreținut dacă sunt utilizate excesiv sau în moduri necorespunzătoare.

### 2.3.6. Precedența operatorilor

Precedența operatorilor se referă la ordinea în care operatorii sunt evaluați atunci când există mai mulți operatori într-o expresie. Operatorii cu o precedență mai mare sunt evaluați înaintea operatorilor cu o precedență mai mică. Când operatorii au aceeași precedență, ordinea de evaluare este determinată de asociativitatea lor (de la stânga la dreapta sau de la dreapta la stânga).

Iată o listă parțială a operatorilor în C#, în ordinea descrescătoare a precedenței:

1. Operatori unari postfixați: `x++`, `x--`, `x.y`, `x?.y`, `x?[y]`, `x!`, `x is T`, `x as T`
2. Operatori unari prefixați: `++x`, `--x`, `!x`, `~x`, `+x`, `-x`, `^x`, `&x`, `*x`, `(T)x`, `await x`, `&x`, `*x`, `sizeof(T)`
3. Operatori multiplicativi: `*`, `/`, `%`
4. Operatori aditivi: `+`, `-`
5. Operatori de deplasare: `<<`, `>>`
6. Operatori relațional și de tip: `<`, `>`, `<=`, `>=`, `is`, `as`
7. Operatori de egalitate: `==`, `!=`
8. AND bit cu bit: `&`
9. XOR bit cu bit: `^`
10. OR bit cu bit: `|`
11. AND logic: `&&`
12. OR logic: `||`
13. Operator ternar condițional: `?:`
14. Operatori de atribuire: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `>>=`, `<<=`

Când o expresie conține operatori cu aceeași precedență, ordinea de evaluare este determinată de asociativitatea lor. De exemplu, operatorii aritmetici (`*`, `/`, `%`, `+`, `-`) au asociativitate de la stânga la dreapta, în timp ce operatorii de atribuire (`=`, `+=`, `-=`, etc.) au asociativitate de la dreapta la stânga.

Să analizăm câteva exemple pentru a înțelege mai bine precedența operatorilor:

```
int rezultat = 2 + 3 * 4; // rezultat va fi 14 (3 * 4 este evaluat primul)
```

În acest caz, înmulțirea (\*) are o precedență mai mare decât adunarea (+), deci `3 * 4` este evaluat primul, iar apoi rezultatul este adunat cu `2`.

```
int rezultat = (2 + 3) * 4; // rezultat va fi 20 ((2 + 3) este evaluat primul)
```

Aici, parantezele forțează evaluarea expresiei `(2 + 3)` înainte de înmulțire, astfel încât rezultatul final este diferit.

```
bool rezultat = true || false && false; // rezultat va fi true (false && false este evaluat primul)
```

În acest exemplu, AND logic (&&) are o precedență mai mare decât OR logic (||), deci `false && false` este evaluat primul, iar apoi rezultatul este combinat cu `true` folosind OR logic.

Pentru a evita confuziile și a crește lizibilitatea, este recomandat să folosiți paranteze pentru a grupa explicit operațiile în ordinea dorită, chiar dacă ordinea de evaluare implicită este corectă.

```
int rezultat = 2 + (3 * 4); // mai lizibil, deși rezultatul este același ca și fără paranteze
```

## 2.4. Structuri de control

Structurile de control sunt elemente fundamentale în orice limbaj de programare, permițând dezvoltatorilor să controleze fluxul de execuție al programelor. Acestea dau posibilitatea de a lua decizii, de a repeta anumite părți de cod și de a sări la diferite secțiuni ale programului, în funcție de anumite condiții.

C# oferă o gamă largă de structuri de control, care pot fi împărțite în trei categorii principale:

- **Instrucțiuni condiționale:** Acestea permit executarea selectivă a anumitor părți de cod, pe baza unor condiții specifice. Instrucțiunile condiționale din C# includ `if`, `else`, `if-else` și `switch`.
- **Bucle:** Acestea permit repetarea unei secțiuni de cod de mai multe ori, până când o anumită condiție este îndeplinită. C# oferă trei tipuri de bucle: `for`, `while` și `do-while`, fiecare având propriile caracteristici și cazuri de utilizare.
- **Instrucțiuni de salt:** Acestea permit modificarea fluxului de control al programului, prin sărituri la anumite puncte specifice. Instrucțiunile de salt din C# includ `break`, `continue` și `return`, care pot fi folosite pentru a ieși dintr-o buclă, a trece la următoarea iterație sau a reveni dintr-o metodă.

Pe parcursul acestui paragraf, vom explora fiecare dintre aceste structuri de control, analizând sintaxa, semantica și cazurile de utilizare ale acestora. De asemenea, vom discuta despre bune practici în utilizarea structurilor de control, cum ar fi evitarea codului "spaghetti" și folosirea unor construcții clare și lizibile.

### 2.4.1. Instrucțiuni condiționale

Instrucțiunile condiționale permit executarea selectivă a anumitor părți de cod, în funcție de îndeplinirea unor condiții specifice. C# oferă trei instrucțiuni condiționale principale: `if`, `else` și `switch`, precum și operatorul condițional `?:`.

#### 2.4.1.1. Instrucțiunea `if`

Instrucțiunea `if` este cea mai simplă formă de control al fluxului în C#. Aceasta execută un bloc de cod dacă o condiție specificată este adevărată. Sintaxa de bază este:

```
if (condiție)
{
    // Cod de executat dacă condiția este adevărată
}
```

Exemplu:

```
int varsta = 20;
if (varsta >= 18)
{
    Console.WriteLine("Ești eligibil pentru a vota.");
}
```

#### 2.4.1.2. Instrucțiunea `else`

Instrucțiunea `else` este folosită în combinație cu `if` pentru a executa un bloc de cod alternativ atunci când condiția din `if` este falsă. Sintaxa este:

```
if (condiție)
{
    // Cod de executat dacă condiția este adevărată
}
else
{
    // Cod de executat dacă condiția este falsă
}
```

Exemplu:

```
int varsta = 15;
if (varsta >= 18)
{
    Console.WriteLine("Ești eligibil pentru a vota.");
}
else
{
    Console.WriteLine("Nu ești încă eligibil pentru a vota.");
}
```

#### 2.4.1.3. Instrucțiunea `if-else` în cascadă

Pentru a evalua mai multe condiții, puteți folosi instrucțiuni `if-else` în cascadă. Sintaxa este:

```
if (condiție1)
{
    // Cod de executat dacă condiția1 este adevărată
}
else if (condiție2)
{
    // Cod de executat dacă condiția2 este adevărată
}
else
{
    // Cod de executat dacă nicio condiție de mai sus nu este adevărată
}
```

Exemplu:

```
int nota = 85;
if (nota >= 90)
{
    Console.WriteLine("Calificativ: A");
}
else if (nota >= 80)
{
    Console.WriteLine("Calificativ: B");
}
else if (nota >= 70)
{
    Console.WriteLine("Calificativ: C");
}
else
{
    Console.WriteLine("Calificativ: F");
}
```

#### 2.4.1.4. Instrucțiunea switch

Instrucțiunea `switch` este folosită pentru a selecta una dintre mai multe secvențe de cod de executat, pe baza valorii unei expresii. Este deosebit de utilă când aveți de evaluat multe condiții bazate pe aceeași expresie. Sintaxa este:

```
switch (expresie)
{
    case valoare1:
        // Cod de executat dacă expresia se potrivește cu valoare1
        break;
    case valoare2:
        // Cod de executat dacă expresia se potrivește cu valoare2
        break;
    // ...
    default:
```

```
// Cod de executat dacă expresia nu se potrivește cu niciuna dintre  
valorile de mai sus  
break;  
}
```

Exemplu:

```
string zi = "Luni";  
switch (zi)  
{  
    case "Luni":  
        Console.WriteLine("Este prima zi a săptămânii.");  
        break;  
    case "Sâmbătă":  
    case "Duminică":  
        Console.WriteLine("Este weekend!");  
        break;  
    default:  
        Console.WriteLine("Este o zi obișnuită a săptămânii.");  
        break;  
}
```

Este important de menționat că fiecare **case** trebuie să se încheie cu o instrucțiune **break**, pentru a ieși din blocul **switch**. Dacă nu includeți **break**, execuția va continua la următorul **case**, chiar dacă condiția nu este îndeplinită (comportament cunoscut ca *fall-through*).

#### 2.4.1.5. Operatorul condițional ? :

Operatorul condițional, cunoscut și sub numele de operatorul ternar, este o modalitate concisă de a evalua o condiție și a returna una dintre două valori, în funcție de rezultatul condiției. Sintaxa este:

```
condiție ? expresie1 : expresie2;
```

Dacă **condiție** este adevărată, operatorul condițional returnează valoarea **expresie1**. Dacă **condiție** este falsă, operatorul returnează valoarea **expresie2**.

Exemplu:

```
int a = 5;  
int b = 10;  
string rezultat = a > b ? "a este mai mare" : "b este mai mare sau egal";  
Console.WriteLine(rezultat); // Va afișa "b este mai mare sau egal"
```

Operatorul condițional poate fi folosit ca o alternativă concisă la o instrucțiune `if-else` simplă. De exemplu, codul de mai sus este echivalent cu:

```
int a = 5;  
int b = 10;  
string rezultat;  
if (a > b)  
{  
    rezultat = "a este mai mare";  
}
```

```
}
else
{
    rezultat = "b este mai mare sau egal";
}
Console.WriteLine(rezultat); // Va afișa "b este mai mare sau egal"
```

Operatorul condițional este deosebit de util când doriți să atribuiți o valoare unei variabile în funcție de o condiție, fără a scrie o instrucțiune `if-else` completă.

Cu toate acestea, este important să utilizați operatorul condițional cu moderație și doar pentru expresii simple. Dacă aveți nevoie de mai multă logică sau de cod mai complex, este recomandat să folosiți instrucțiuni `if-else` complete, pentru a menține lizibilitatea și claritatea codului.

De asemenea, rețineți că expresiile `expresie1` și `expresie2` trebuie să aibă același tip sau tipuri compatibile. În caz contrar, veți întâmpina o eroare de compilare.

Exemplu de utilizare incorectă:

```
int a = 5;
int b = 10;
string rezultat = a > b ? "a este mai mare" : 15; // Eroare de compilare:
tipuri incompatibile (string vs. int)
```

## 2.4.2. Bucle

Buclele sunt structuri de control care permit executarea repetată a unui bloc de cod, atât timp cât o anumită condiție este îndeplinită. C# oferă patru tipuri de bucle: `for`, `while`, `do-while` și `foreach`, fiecare având propriile caracteristici și cazuri de utilizare.

### 2.4.2.1. Bucla *for*

Bucla `for` este folosită când știți exact de câte ori doriți să executați un bloc de cod. Sintaxa este:

```
for (inițializare; condiție; incrementare)
{
    // Cod de executat în buclă
}
```

- **inițializare:** O expresie care este executată o singură dată, la începutul buclei. De obicei, aici se inițializează o variabilă de contor.
- **condiție:** Expresia evaluată înainte de fiecare iterație. Bucla continuă atât timp cât această condiție este adevărată.
- **incrementare:** O expresie executată la sfârșitul fiecărei iterații. De obicei, aici se incrementează sau se decrementează variabila de contor.

Exemplu:

```
for (int i = 0; i < 5; i++)  
{  
    Console.WriteLine(i);  
}
```

Acest cod va afișa numerele de la 0 la 4.

#### 2.4.2.2. Bucla while

Bucla **while** execută un bloc de cod atât timp cât o anumită condiție este adevărată. Sintaxa este:

```
while (condiție)  
{  
    // Cod de executat în buclă  
}
```

Exemplu:

```
int i = 0;  
while (i < 5)  
{  
    Console.WriteLine(i);  
    i++;  
}
```

Acest cod va afișa, de asemenea, numerele de la 0 la 4. Bucla **while** este utilă când nu știți exact de câte ori trebuie să se execute bucla, dar aveți o condiție specifică de îndeplinit.

#### 2.4.2.3. Bucla do-while

Bucla **do-while** este similară cu bucla **while**, cu excepția faptului că condiția este evaluată la sfârșitul fiecărei iterații, nu la început. Aceasta înseamnă că bucla va fi executată întotdeauna cel puțin o dată, chiar dacă condiția este falsă de la început. Sintaxa este:

```
do  
{  
    // Cod de executat în buclă  
} while (condiție);
```

Exemplu:

```
int i = 0;  
do  
{  
    Console.WriteLine(i);  
    i++;  
} while (i < 5);
```

Acest cod va afișa numerele de la 0 la 4. Bucla **do-while** este utilă când doriți ca blocul de cod să fie executat cel puțin o dată, indiferent de starea inițială a condiției.



#### 2.4.2.4. Bucla `foreach`

Bucla `foreach` este folosită pentru a parcurge elementele unei colecții (cum ar fi un tablou sau o listă), fără a fi nevoie de un contor explicit. Sintaxa este:

```
foreach (tip_element nume_variabila in colectie)
{
    // Cod de executat pentru fiecare element
}
```

Exemplu:

```
string[] fructe = { "măr", "banană", "portocală" };
foreach (string fruct in fructe)
{
    Console.WriteLine(fruct);
}
```

Acest cod va afișa fiecare element din tabloul `fructe` pe o linie separată. Bucla `foreach` este deosebit de utilă când lucrați cu colecții și doriți să accesați fiecare element individual, fără a vă preocupa de gestionarea explicită a indicilor sau a numărului de elemente.

Când alegeți tipul de buclă pe care să-l folosiți, luați în considerare cazul specific de utilizare:

- Folosiți `for` când știți exact de câte ori doriți să executați bucla.
- Folosiți `while` când aveți o condiție specifică de îndeplinit, dar nu știți exact numărul de iterații necesare.
- Folosiți `do-while` când doriți să executați bucla cel puțin o dată, indiferent de starea inițială a condiției.
- Folosiți `foreach` pentru a parcurge elementele unei colecții într-un mod concis și ușor de citit.

De asemenea, rețineți că puteți utiliza instrucțiunile `break` și `continue` în interiorul oricărui tip de buclă, pentru a controla mai precis fluxul de execuție:

- `break` iese imediat din bucla curentă și continuă cu următoarea instrucțiune după buclă.
- `continue` sare peste restul iterației curente și trece la următoarea iterație a buclei.

Exemplu:

```
for (int i = 0; i < 10; i++)
{
    if (i == 5)
    {
        break;
    }
    Console.WriteLine(i);
}
```

Acest cod va afișa numerele de la 0 la 4, deoarece bucla se va opri când `i` devine 5.

### 2.4.3. Instrucțiuni de salt

Instrucțiunile de salt sunt folosite pentru a modifica fluxul de control al programului, permițând ieșirea dintr-o buclă, săritura la următoarea iterație sau revenirea dintr-o metodă. C# oferă trei instrucțiuni de salt: `break`, `continue` și `return`.

#### 2.4.3.1. Instrucțiunea `break`

Instrucțiunea `break` este folosită pentru a ieși imediat dintr-o buclă (`for`, `while`, `do-while`) sau dintr-un bloc `switch`. Când `break` este întâlnit, execuția programului sare la prima instrucțiune după buclă sau bloc `switch`.

Exemplu:

```
for (int i = 0; i < 10; i++)
{
    if (i == 5)
    {
        break;
    }
    Console.WriteLine(i);
}
```

Acest cod va afișa numerele de la 0 la 4, deoarece bucla se va opri când `i` devine 5.

Instrucțiunea `break` este utilă când doriți să ieșiți dintr-o buclă prematur, pe baza unei anumite condiții.

#### 2.4.3.2. Instrucțiunea `continue`

Instrucțiunea `continue` este folosită pentru a sări peste restul iterației curente a unei bucle și a trece la următoarea iterație. Când `continue` este întâlnit, execuția programului sare la sfârșitul blocului buclei și evaluează din nou condiția buclei.

Exemplu:

```
for (int i = 0; i < 10; i++)
{
    if (i % 2 == 0)
    {
        continue;
    }
    Console.WriteLine(i);
}
```

Acest cod va afișa numerele impare de la 1 la 9, deoarece instrucțiunea `continue` va sări peste iterațiile în care `i` este par.

Instrucțiunea **continue** este utilă când doriți să săriți peste anumite iterații ale buclei, pe baza unei condiții, fără a ieși complet din buclă.

#### 2.4.3.3. Instrucțiunea return

Instrucțiunea **return** este folosită pentru a reveni dintr-o metodă. Când **return** este întâlnit, execuția metodei se încheie, iar controlul este transferat înapoi către codul care a apelat metoda. Opțional, **return** poate fi urmat de o valoare sau o expresie, care reprezintă valoarea de retur a metodei.

Exemplu:

```
public int Aduna(int a, int b)
{
    return a + b;
}
```

În acest caz, metoda **Aduna** primește doi întregi ca parametri și returnează suma lor.

Instrucțiunea **return** poate fi folosită și pentru a ieși prematur dintr-o metodă, pe baza unei anumite condiții:

```
public void VerificaVarsta(int varsta)
{
    if (varsta < 0)
    {
        Console.WriteLine("Vârstă invalidă.");
        return;
    }
    Console.WriteLine("Vârstă validă.");
}
```

În acest exemplu, dacă **varsta** este mai mică decât 0, metoda va afișa un mesaj de eroare și va ieși imediat, fără a executa restul codului din metodă.

Instrucțiunile **break** și **continue** se aplică doar buclei imediate în care sunt folosite. Dacă aveți bucle imbricate și doriți să ieșiți din bucla exterioară, va trebui să folosiți o altă abordare, cum ar fi o variabilă de tip fanion (flag) sau o instrucțiune **goto** (deși aceasta din urmă este în general descurajată).

## 2.5. Tablouri și colecții

Tablourile și colecțiile reprezintă structuri de date esențiale în orice limbaj de programare, inclusiv în C#. Acestea ne permit să stocăm, să organizăm și să manipulăm eficient grupuri de valori de același tip.

Să ne imaginăm că avem o cutie cu compartimente numerotate, în care putem pune diverse obiecte. Fiecare compartiment poate fi accesat folosind numărul său, permițându-ne să adăugăm, să ștergem sau să modificăm cu ușurință conținutul.

Această analogie descrie foarte bine modul în care funcționează tablourile în programare.

Pe de altă parte, colecțiile pot fi comparate cu o servietă cu multe buzunare și compartimente, fiecare putând conține diferite tipuri de obiecte și documente. Spre deosebire de cutia cu compartimente numerotate, servieta ne oferă mai multă flexibilitate în organizarea și accesarea conținutului său.

În acest paragraf, vom explora în detaliu aceste două concepte fundamentale:

- Vom începe cu tablourile și vom învăța cum să le declarăm, să le inițializăm și să le folosim în cod. Vom discuta despre tablouri unidimensionale și multidimensionale, precum și despre tablouri zimțate (jagged arrays).
- Ne vom concentra apoi pe colecții și vom introduce câteva tipuri generice des întâlnite, precum `List<T>` și `Dictionary<K, V>`. Vom vedea cum acestea ne pot ajuta să gestionăm grupuri de obiecte în moduri flexibile și eficiente.

### 2.5.1. Tablouri unidimensionale și multidimensionale

În C#, un tablou este o structură de date care stochează o colecție de elemente de același tip, accesibile printr-un index. Tablourile pot fi unidimensionale (o singură dimensiune) sau multidimensionale (două sau mai multe dimensiuni).

Tablourile unidimensionale, cunoscute și sub numele de vectori, sunt cele mai simple și mai frecvent utilizate. Acestea conțin o secvență liniară de elemente, iar fiecare element poate fi accesat folosind un singur index, care reprezintă poziția sa în tablou. Indexarea elementelor începe de la 0.

Iată câteva exemple de declarare a tablourilor unidimensionale în C#:

```
int[] numere = new int[5];  
string[] cuvinte = { "hello", "world", "csharp" };  
bool[] flags = new bool[] { true, false, true };
```

În primul exemplu, `numere` este un tablou de 5 elemente de tip `int`, inițializate cu valori implicite (0 pentru tipul `int`). Al doilea exemplu, `cuvinte`, utilizează o sintaxă prescurtată pentru a declara și inițializa un tablou de șiruri de caractere. În al treilea exemplu, `flags` este un tablou de valori boolean, inițializat explicit cu valorile specificate.

Tablourile multidimensionale au două sau mai multe dimensiuni, permițând stocarea și accesarea elementelor folosind mai mulți indici. În C#, există două tipuri de tablouri multidimensionale: tablouri rectangulare și tablouri zimțate (jagged).

Tablourile rectangulare au un număr fix de rânduri și coloane, iar fiecare element este accesat folosind doi sau mai mulți indici. Iată un exemplu de declarare a unui tablou bidimensional rectangular:

```
int[,] matrice = new int[3, 4];
```

În acest caz, `matrice` este un tablou bidimensional cu 3 rânduri și 4 coloane.

Tablourile zimțate sunt tablouri de tablouri, unde fiecare element este în sine un tablou unidimensional. Acestea oferă o flexibilitate mai mare, deoarece fiecare "rând" poate avea o lungime diferită. Iată un exemplu de declarare a unui tablou zimțat:

```
int[][] tablouZimtat = new int[3][];  
tablouZimtat[0] = new int[2];  
tablouZimtat[1] = new int[4];  
tablouZimtat[2] = new int[3];
```

Aici, `tablouZimtat` este un tablou de 3 elemente, unde fiecare element este un tablou de întregi. Fiecare "rând" este inițializat individual cu lungimi diferite.

Indiferent de dimensiunea tabloului, elementele acestuia pot fi accesate și modificate folosind indici:

```
int valoare = numere[2]; // Obține valoarea celui de-al treilea element din  
tabloul numere  
cuvinte[1] = "C#"; // Modifică al doilea element din tabloul cuvinte  
matrice[1, 2] = 10; // Setează elementul de la rândul 1, coloana 2 în matrice  
int[] rand1 = tablouZimtat[0]; // Obține primul "rând" din tablouZimtat
```

Tablourile reprezintă o modalitate eficientă de a stoca și manipula colecții de date de același tip în C#. Alegerea între tablouri unidimensionale și multidimensionale depinde de natura problemei și de modul în care trebuie organizate datele.

### 2.5.2. Declararea și inițializarea tablourilor

În C#, tablourile trebuie să fie declarate înainte de a fi utilizate, specificând tipul de date pe care îl vor stoca și dimensiunea tabloului. Există mai multe moduri de a declara și inițializa un tablou, în funcție de context și de preferințele programatorului.

Declararea unui tablou se face folosind tipul de date urmat de paranteze pătrate [], apoi numele tabloului. Putem specifica dimensiunea tabloului în momentul declarării sau putem lăsa compilatorul să o deducă din valorile de inițializare.

Iată sintaxa generală pentru declararea unui tablou unidimensional:

```
tip[] numeTablou = new tip[dimensiune];
```

De exemplu:

```
int[] numere = new int[5];
```

Acest cod declară un tablou numit `numere`, care poate stoca 5 elemente de tip `int`. Elementele tabloului sunt inițializate cu valorile implicite ale tipului de date (în cazul lui `int`, valorile vor fi 0).

De asemenea, putem declara și inițializa un tablou într-o singură instrucțiune, folosind o sintaxă prescurtată:

```
tip[] numeTablou = { valoare1, valoare2, ..., valoareN };
```

De exemplu:

```
string[] cuvinte = { "salut", "lume", "C#" };
```

Acest cod creează un tablou numit `cuvinte`, care conține 3 elemente de tip `string`, inițializate cu valorile specificate între acolade.

Pentru tablourile multidimensionale, trebuie să specificăm dimensiunile folosind virgule în parantezele pătrate. Iată sintaxa generală pentru declararea unui tablou bidimensional:

```
tip[,] numeTablou = new tip[dimensiune1, dimensiune2];
```

De exemplu:

```
int[,] matrice = new int[3, 4];
```

Acest cod declară o matrice cu 3 rânduri și 4 coloane, stocând elemente de tip `int`.

Similar cu tablourile unidimensionale, putem folosi sintaxa prescurtată pentru a inițializa un tablou multidimensional:

```
tip[,] numeTablou = { { valori_rând1 }, { valori_rând2 }, ..., { valori_rândN } };
```

De exemplu:

```
int[,] matrice = {  
    { 1, 2, 3 },  
    { 4, 5, 6 },  
    { 7, 8, 9 }  
};
```

Acest cod inițializează o matrice de 3x3 cu valorile specificate.

Pentru tablourile zimțate (jagged arrays), declararea se face folosind mai multe seturi de paranteze pătrate:

```
tip[][] numeTablou = new tip[dimensiuneRânduri][];
```

De exemplu:

```
int[][] tablouZimțat = new int[3][];
```

Aici, `tablouZimțat` este declarat ca un tablou de tablouri, cu 3 elemente (rânduri). Fiecare element al lui `tablouZimțat` trebuie inițializat separat, ca un tablou unidimensional:

```
tablouZimțat[0] = new int[2];  
tablouZimțat[1] = new int[4];
```

```
tablouZimțat[2] = new int[3];
```

După ce un tablou a fost declarat și inițializat, elementele sale pot fi accesate și modificate folosind indici, așa cum am văzut în exemplele din paragraful anterior.

Tablourile în C# au o dimensiune fixă, determinată în momentul inițializării. Odată creat un tablou, dimensiunea sa nu poate fi modificată. Dacă aveți nevoie de o colecție cu dimensiune variabilă, puteți utiliza tipurile de colecție din .NET, cum ar fi `List<T>` sau `Dictionary<K,V>`.

### 2.5.3.Operațiuni cu tablouri

C# oferă o serie de metode și proprietăți utile pentru manipularea și interogarea tablourilor.

#### Accesarea și modificarea elementelor:

- Putem accesa individual elementele unui tablou folosind indicele lor, care începe de la 0. De exemplu, pentru a obține primul element al unui tablou unidimensional `numere`, folosim `numere[0]`.
- Pentru a modifica valoarea unui element, utilizăm atribuirea. De exemplu, `numere[1] = 10;` va seta al doilea element din tabloul `numere` la valoarea 10.

#### Proprietatea `Length`:

- Fiecare tablou în C# are o proprietate `Length`, care returnează numărul de elemente din tablou.
- Putem folosi această proprietate pentru a itera prin elementele unui tablou folosind o buclă `for`.

Exemplu:

```
int[] numere = { 1, 2, 3, 4, 5 };
for (int i = 0; i < numere.Length; i++)
{
    Console.WriteLine(numere[i]);
}
```

Acest cod va afișa fiecare element al tabloului `numere` pe o linie separată.

#### Alte metode utile pentru tablouri:

- `Array.Sort()`: Sortează elementele unui tablou în ordine crescătoare.
- `Array.Reverse()`: Inversează ordinea elementelor dintr-un tablou.
- `Array.BinarySearch()`: Caută un element specific într-un tablou sortat și returnează indicele său.
- `Array.Copy()`: Copiază o parte sau întregul tablou într-un alt tablou.

Exemplu:

```
int[] numere = { 3, 1, 4, 1, 5, 9, 2, 6, 5 };
```

```
Array.Sort(numere);  
foreach (int numar in numere)  
{  
    Console.Write(numar + " ");  
}  
// Rezultat: 1 1 2 3 4 5 5 6 9
```

#### Parcurgerea tablourilor cu `foreach`:

- Instrucțiunea `foreach` oferă un mod convenabil de a itera prin elementele unui tablou, fără a fi nevoie să folosim indici.
- La fiecare iterație, variabila din bucla `foreach` va conține valoarea elementului curent.

Exemplu:

```
string[] cuvinte = { "hello", "world", "C#" };  
foreach (string cuvant in cuvinte)  
{  
    Console.WriteLine(cuvant);  
}
```

Acest cod va afișa fiecare cuvânt din tabloul `cuvinte`` pe o linie separată.

#### Copierea tablourilor:

- Atribuirea directă a unui tablou unei alte variabile de tablou va crea o referință la același tablou, nu o copie.
- Pentru a crea o copie independentă a unui tablou, putem folosi metoda `Array.Copy()` sau metoda `Clone()`.

Exemplu:

```
int[] numere = { 1, 2, 3 };  
int[] copie = new int[numere.Length];  
Array.Copy(numere, copie, numere.Length);
```

Aici, `copie` este o copie independentă a tabloului `numere`, cu aceleași valori.

### 2.5.4.Tablouri zimțate (jagged)

Tablourile zimțate, cunoscute și sub numele de tablouri de tablouri, sunt o variantă specială de tablouri multidimensionale în C#. Spre deosebire de tablourile rectangulare, unde fiecare dimensiune are o lungime fixă, tablourile zimțate permit ca fiecare "rând" să aibă propria lungime, independent de celelalte rânduri.

Declararea unui tablou zimțat se face folosind mai multe seturi de paranteze pătrate `[]`, unul pentru fiecare dimensiune. Iată sintaxa generală pentru declararea unui tablou zimțat cu două dimensiuni:

```
tip[][] numeTablou = new tip[dimensiuneRânduri][];
```



Aici, `dimensiuneRânduri` reprezintă numărul de rânduri din tabloul zimțat. Fiecare rând este în sine un tablou unidimensional, care trebuie inițializat separat.

Exemplu:

```
int[][] tablouZimțat = new int[3][];
tablouZimțat[0] = new int[2];
tablouZimțat[1] = new int[4];
tablouZimțat[2] = new int[3];
```

În acest exemplu, `tablouZimțat` este un tablou zimțat cu 3 rânduri, unde primul rând are 2 elemente, al doilea rând are 4 elemente, iar al treilea rând are 3 elemente.

Accesarea elementelor într-un tablou zimțat se face folosind două seturi de indici: primul pentru rând și al doilea pentru coloana (sau poziția) din cadrul rândului.

Exemplu:

```
tablouZimțat[0][1] = 10;
int valoare = tablouZimțat[2][0];
```

Aici, `tablouZimțat[0][1] = 10;` setează a doua valoare din primul rând la 10, iar `int valoare = tablouZimțat[2][0];` obține prima valoare din al treilea rând.

Inițializarea unui tablou zimțat poate fi făcută și folosind o sintaxă prescurtată, similar cu tablourile rectangulare:

```
int[][] tablouZimțat = new int[][]
{
    new int[] { 1, 2 },
    new int[] { 3, 4, 5, 6 },
    new int[] { 7, 8, 9 }
};
```

Acest cod inițializează un tablou zimțat cu 3 rânduri și lungimi diferite pentru fiecare rând.

Tablourile zimțate oferă flexibilitate în situațiile în care aveți nevoie de o structură de date asemănătoare unui tabel, dar cu rânduri de lungimi variabile. Cu toate acestea, ele pot fi mai puțin eficiente în ceea ce privește utilizarea memoriei și accesul la elemente, comparativ cu tablourile rectangulare.

Câteva scenarii în care tablourile zimțate pot fi utile:

- Reprezentarea unei matrice triunghiulare sau a unei matrice cu rânduri de lungimi diferite.
- Stocarea datelor ierarhice, cum ar fi un arbore genealogic sau o structură de meniu cu submeniuri.
- Implementarea unor algoritmi specifici, cum ar fi triangularea Delaunay sau algoritmul lui Dijkstra pentru grafuri.

### 2.5.5. Introducere în colecții generice (List<T>, Dictionary<K,V>)

În timp ce tablourile sunt structuri de date fundamentale în C#, ele au anumite limitări, cum ar fi dimensiunea fixă și lipsa de flexibilitate în ceea ce privește adăugarea sau eliminarea elementelor. Pentru a depăși aceste limitări, .NET Framework oferă o gamă largă de colecții generice, care sunt clase optimizate pentru stocarea și manipularea grupurilor de obiecte.

Colecțiile generice folosesc tipuri parametrizate, permițând specificarea tipului de date pe care îl vor stoca, îmbunătățind astfel siguranța tipurilor și eliminând necesitatea de a efectua conversii explicite (cast). Două dintre cele mai frecvent utilizate colecții generice în C# sunt `List<T>` și `Dictionary<K,V>`.

#### Lista (`List<T>`):

- `List<T>` reprezintă o colecție de obiecte de tipul `T`, care poate fi redimensionată dinamic.
- Elementele unei liste sunt accesate folosind un index, similar cu tablourile, dar listele oferă metode suplimentare pentru adăugarea, eliminarea și căutarea elementelor.

Exemplu:

```
List<string> cuvinte = new List<string>();  
cuvinte.Add("hello");  
cuvinte.Add("world");  
cuvinte.Insert(1, "C#");  
foreach (string cuvant in cuvinte)  
{  
    Console.WriteLine(cuvant);  
}
```

Acest cod creează o listă de șiruri de caractere (`List<string>`), adaugă elementele "hello" și "world", inserează elementul "C#" la indexul 1 și apoi afișează fiecare element al listei folosind o buclă `foreach`.

#### Dicționar (`Dictionary<K,V>`):

- `Dictionary<K,V>` reprezintă o colecție de perechi cheie-valoare, unde cheile sunt de tipul `K`, iar valorile sunt de tipul `V`.
- Fiecare cheie din dicționar este unică și este folosită pentru a accesa valoarea corespunzătoare.
- Dicționarele sunt utile pentru stocarea datelor asociative, unde aveți nevoie să căutați rapid valori pe baza unei chei.

Exemplu:

```
Dictionary<string, int> note = new Dictionary<string, int>();
```

```
note.Add("Alice", 90);
note.Add("Bob", 85);
note["Charlie"] = 95;
int notaBob = note["Bob"];
foreach (KeyValuePair<string, int> pair in note)
{
    Console.WriteLine($"{pair.Key}: {pair.Value}");
}
```

Acest cod creează un dicționar (`Dictionary<string, int>`) care asociază numele studenților (chei de tip `string`) cu notele lor (valori de tip `int`). Elementele sunt adăugate folosind metoda `Add()` sau prin accesarea directă a cheii. Valoarea asociată cu o cheie specifică poate fi obținută folosind indexarea cu cheie (`note["Bob"]`). Bucla `foreach` iterează prin fiecare pereche cheie-valoare din dicționar.

Colecțiile generice oferă o serie de avantaje față de tablourile standard:

- Dimensiune dinamică: Colecțiile pot crește sau scădea în dimensiune pe măsură ce elemente sunt adăugate sau eliminate.
- Siguranța tipurilor: Colecțiile generice asigură că toate elementele sunt de un anumit tip, reducând erorile de runtime cauzate de incompatibilitatea tipurilor.
- Metode utile: Colecțiile oferă o gamă largă de metode pentru adăugarea, eliminarea, căutarea și sortarea elementelor.
- Performanță: Colecțiile generice sunt optimizate pentru performanță și utilizează eficient memoria.

Pe lângă `List<T>` și `Dictionary<K,V>`, .NET Framework oferă și alte tipuri de colecții generice, cum ar fi `Queue<T>`, `Stack<T>`, `HashSet<T>` și `SortedList<K,V>`, fiecare având propriile caracteristici și cazuri de utilizare.

Alegerea colecției potrivite depinde de natura problemei și de operațiile pe care trebuie să le efectuați. În general, se recomandă utilizarea colecțiilor generice ori de câte ori aveți nevoie de o structură de date flexibilă și ușor de gestionat pentru stocarea și manipularea grupurilor de obiecte.

## 2.6. Metode și parametri

Metodele și parametrii reprezintă elemente fundamentale în proiectarea și implementarea programelor orientate pe obiecte în C#. Metodele sunt blocuri de cod care efectuează o sarcină specifică și pot accepta date de intrare sub formă de parametri, precum și returna valori. Prin utilizarea metodelor, putem organiza codul în unități logice, reutilizabile și ușor de întreținut.

În acest paragraf, vom explora în detaliu conceptele legate de metode și parametri în C#. Vom învăța cum să declarăm și să apelăm metode, cum să folosim parametri pentru a transmite date metodelor și cum să returnăm valori din metode. De asemenea, vom discuta despre concepte avansate, cum ar fi parametrii opționali, parametrii numiți,

supraîncărcarea metodelor și utilizarea cuvintelor cheie `ref`, `out` și `in` pentru a manipula parametrii.

Pe parcursul acestui paragraf, vom folosi exemple de cod pentru a ilustra fiecare concept și pentru a demonstra cum să aplicăm aceste cunoștințe în practică. La finalul paragrafului, veți avea o înțelegere solidă a metodelor și parametrilor în C# și veți fi capabili să le utilizați eficient în propriile proiecte.

### 2.6.1. Declararea și apelarea metodelor

Metodele în C# sunt declarate în interiorul unei clase sau a unei structuri. O metodă are un nume, un tip de retur, și poate avea zero sau mai mulți parametri. Sintaxa de bază pentru declararea unei metode este următoarea:

```
<modificator_acces> <tip_retur> <nume_metodă>(<lista_parametri>)  
{  
    // Corpul metodei  
    // Instrucțiuni și logica metodei  
    // Opțional: return <valoare_retur>;  
}
```

- **<modificator\_acces>**: Specifică nivelul de accesibilitate al metodei (e.g., `public`, `private`, `protected`, `internal`).
- **<tip\_retur>**: Indică tipul de date al valorii returnate de metodă. Dacă metoda nu returnează o valoare, se folosește cuvântul cheie `void`.
- **<nume\_metodă>**: Reprezintă numele unic al metodei, care respectă convențiile de denumire în C#.
- **<lista\_parametri>**: Conține zero sau mai multe declarații de parametri, separate prin virgulă, care specifică tipul și numele parametrilor de intrare.

Exemplu de declarare a unei metode:

```
public int Aduna(int a, int b)  
{  
    int suma = a + b;  
    return suma;  
}
```

În acest exemplu, metoda `Aduna` are modificatorul de acces `public`, tipul de retur `int`, și doi parametri de tip `int` numiți `a` și `b`. Metoda calculează suma celor doi parametri și o returnează folosind instrucțiunea `return`.

Pentru a apela o metodă, se utilizează numele metodei urmat de paranteze rotunde, în care se specifică valorile argumentelor corespunzătoare parametrilor definiți în declarația metodei.

Exemplu de apelare a unei metode:

```
int rezultat = Aduna(5, 3);  
Console.WriteLine(rezultat); // Va afișa 8
```

În acest caz, metoda `Aduna` este apelată cu argumentele `5` și `3`, iar valoarea returnată este atribuită variabilei `rezultat`.

Câteva observații importante despre metode:

- Numele metodei trebuie să fie unic în cadrul clasei sau structurii în care este declarată.
- Tipul de retur al metodei poate fi orice tip de date valid în C#, inclusiv `void` pentru metode care nu returnează o valoare.
- Parametrii metodei sunt opționali și pot fi omiși dacă metoda nu necesită date de intrare.
- Instrucțiunea `return` este folosită pentru a specifica valoarea de retur a metodei și pentru a încheia execuția metodei. În cazul metodelor `void`, instrucțiunea `return` este opțională și poate fi omisă.
- Metodele pot fi apelate de alte metode din aceeași clasă sau din alte clase, în funcție de modificatorul de acces al metodei.

Declararea și apelarea metodelor sunt concepte fundamentale în programarea C#, permițând organizarea și reutilizarea codului într-un mod structurat și modular.

### 2.6.2. Parametri și argumente

Parametrii și argumentele sunt elemente esențiale în comunicarea datelor între metode și apelantul acestora. Parametrii reprezintă variabilele declarate în definiția metodei, care primesc valori de la apelant atunci când metoda este invocată. Argumentele sunt valorile efective transmise metodei în momentul apelării.

Să analizăm mai detaliat parametrii și argumentele:

#### *Declararea parametrilor:*

- Parametrii sunt declarați în parantezele rotunde ale definiției metodei, după numele metodei.
- Fiecare parametru are un tip de date specific și un nume unic.
- Parametrii sunt separați prin virgulă, dacă există mai mulți.

Exemplu de declarare a parametrilor:

```
public void AfiseazaDetalii(string nume, int varsta)
{
    // Corpul metodei
}
```

#### *Apelarea metodelor cu argumente:*

- Când se apelează o metodă, se furnizează argumente corespunzătoare parametrilor declarați.
- Argumentele trebuie să fie în număr și ordine corespunzătoare cu parametrii definiți.

- Tipurile de date ale argumentelor trebuie să fie compatibile cu tipurile de date ale parametrilor.

Exemplu de apelare a metodei cu argumente:

```
AfiseazaDetalii("John", 25);
```

#### *Transmiterea argumentelor:*

- Argumentele pot fi transmise prin valoare sau prin referință.
- Implicit, tipurile de date de valoare (int, float, double, bool, etc.) sunt transmise prin valoare, ceea ce înseamnă că o copie a valorii este pasată metodei.
- Tipurile de date de referință (obiecte, tablouri, clase, etc.) sunt transmise prin referință, ceea ce înseamnă că o referință către obiectul original este pasată metodei.

#### *Modificarea parametrilor în interiorul metodei:*

- Când un parametru este transmis prin valoare, modificările efectuate asupra parametrului în interiorul metodei nu afectează valoarea originală din afara metodei.
- Când un parametru este transmis prin referință, modificările efectuate asupra parametrului în interiorul metodei afectează obiectul original din afara metodei.

Exemplu de modificare a parametrilor în interiorul metodei:

```
public void ModificaValoare(int x)
{
    x = 10; // Modificarea nu afectează valoarea originală
}

public void ModificaReferinta(int[] array)
{
    array[0] = 10; // Modificarea afectează tabloul original
}
```

#### *Parametri opționali și valori implicite:*

- C# permite definirea parametrilor opționali prin atribuirea unei valori implicite în declarația metodei.
- Parametrii opționali trebuie să fie plasați la sfârșitul listei de parametri.
- La apelarea metodei, argumentele pentru parametrii opționali pot fi omise, caz în care se vor utiliza valorile implicite.

Exemplu de parametri opționali:

```
public void AfiseazaDetalii(string nume, int varsta = 18)
{
    // Corpul metodei
}

AfiseazaDetalii("John"); // Utilizează valoarea implicită pentru varsta
```

```
AfiseazaDetalii("Alice", 25); // Furnizează o valoare explicită pentru varsta
```

Parametrii și argumentele facilitează comunicarea și schimbul de date între metode și apelantul acestora. Prin utilizarea corectă a parametrilor, putem crea metode flexibile și reutilizabile, care pot primi și procesa diverse valori de intrare.

### 2.6.3. Parametri opționali și parametri numiți

C# oferă facilități pentru a defini parametri opționali și pentru a utiliza parametri numiți în apelarea metodelor. Aceste caracteristici sporesc flexibilitatea și lizibilitatea codului, permițând omiterea anumitor argumente și specificarea explicită a numelui parametrului în momentul apelării.

#### *Parametri opționali:*

- Parametrii opționali permit omiterea argumentelor corespunzătoare în momentul apelării metodei.
- Pentru a defini un parametru opțional, se atribuie o valoare implicită parametrului în declarația metodei.
- Parametrii opționali trebuie să fie plasați la sfârșitul listei de parametri.
- La apelarea metodei, argumentele pentru parametrii opționali pot fi omise, caz în care se vor utiliza valorile implicite.

Exemplu de parametri opționali:

```
public void AfiseazaDetalii(string nume, int varsta = 18)
{
    Console.WriteLine($"Nume: {nume}, Vârsta: {varsta}");
}

AfiseazaDetalii("John"); // Output: Nume: John, Vârsta: 18
AfiseazaDetalii("Alice", 25); // Output: Nume: Alice, Vârsta: 25
```

#### *Parametri numiți:*

- Parametrii numiți permit specificarea explicită a numelui parametrului în momentul apelării metodei.
- Utilizarea parametrilor numiți îmbunătățește lizibilitatea codului și permite specificarea argumentelor într-o ordine diferită de cea din declarația metodei.
- Pentru a utiliza parametri numiți, se specifică numele parametrului urmat de semnul egal (=) și valoarea argumentului.

Exemplu de parametri numiți:

```
public void AfiseazaDetalii(string nume, int varsta)
{
    Console.WriteLine($"Nume: {nume}, Vârsta: {varsta}");
}

AfiseazaDetalii(varsta: 25, nume: "John"); // Output: Nume: John, Vârsta: 25
```

### Combinarea parametrilor opționali și numiți:

- Parametrii opționali și parametri numiți pot fi combinați pentru a obține o flexibilitate maximă în apelarea metodelor.
- Când se utilizează parametri numiți, se pot omite argumentele pentru parametri opționali.
- Argumentele pentru parametri numiți pot fi specificate în orice ordine.

Exemplu de combinare a parametrilor opționali și numiți:

```
public void AfiseazaDetalii(string nume, int varsta = 18, string ocupatie = "Student")
{
    Console.WriteLine($"Nume: {nume}, Vârsta: {varsta}, Ocupație: {ocupatie}");
}

AfiseazaDetalii("John"); // Output: Nume: John, Vârsta: 18, Ocupație: Student
AfiseazaDetalii("Alice", ocupatie: "Programator"); // Output: Nume: Alice, Vârsta: 18, Ocupație: Programator
```

Parametrii opționali și parametri numiți aduc beneficii precum:

- Flexibilitate în apelarea metodelor, permițând omiterea argumentelor cu valori implicite.
- Îmbunătățirea lizibilității codului prin specificarea explicită a numelor parametrilor.
- Posibilitatea de a specifica argumentele într-o ordine diferită de cea din declarația metodei.

Cu toate acestea, este important să utilizăm aceste caracteristici cu moderație și în situațiile în care aduc beneficii reale codului. Abuzul de parametri opționali și parametri numiți poate duce la cod mai puțin clar și mai greu de înțeles.

### 2.6.4. Supraîncărcarea metodelor

Supraîncărcarea metodelor (method overloading) este o caracteristică a C# care permite definirea mai multor metode cu același nume în cadrul unei clase, atâta timp cât acestea au semnături diferite. Semnătura unei metode este determinată de numărul, tipul și ordinea parametrilor săi, dar nu include tipul returnat.

Iată un exemplu de supraîncărcare a metodelor:

```
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
```



```
{
    return a + b;
}

public int Add(int a, int b, int c)
{
    return a + b + c;
}
}
```

În acest caz, clasa **Calculator** are trei metode **Add**, fiecare cu o semnătură diferită:

- **Add(int a, int b)**: primește doi parametri întregi și returnează suma lor.
- **Add(double a, double b)**: primește doi parametri double și returnează suma lor.
- **Add(int a, int b, int c)**: primește trei parametri întregi și returnează suma lor.

Când apelezi metoda **Add**, compilatorul C# va determina automat care supraîncărcare a metodei să folosească, în funcție de numărul și tipurile argumentelor furnizate:

```
Calculator calc = new Calculator();

int sum1 = calc.Add(3, 5);           // Apelează Add(int, int)
double sum2 = calc.Add(2.5, 7.8);    // Apelează Add(double, double)
int sum3 = calc.Add(1, 2, 3);        // Apelează Add(int, int, int)
```

Supraîncărcarea metodelor este utilă în mai multe scenarii:

- Când doriți să furnizați implementări alternative ale unei metode pentru diferite tipuri de date, ca în exemplul de mai sus.
- Când doriți să oferiți versiuni ale unei metode cu diferite seturi de parametri, pentru a face interfața clasei mai flexibilă și mai ușor de utilizat.
- Când doriți să simulați parametri opționali. Deoarece C# nu a avut suport pentru parametri opționali înainte de C# 4.0, supraîncărcarea metodelor a fost adesea utilizată pentru a oferi metode cu diferite combinații de parametri.

Câteva lucruri de reținut despre supraîncărcarea metodelor:

- Supraîncărcările metodei trebuie să difere prin numărul, tipul sau ordinea parametrilor. Nu puteți supraîncărca o metodă doar pe baza tipului returnat.
- Când apelezi o metodă supraîncărcată, compilatorul C# va alege supraîncărcarea cea mai potrivită pe baza argumentelor furnizate. Dacă nu există o potrivire exactă, compilatorul va încerca să efectueze conversii implicite pentru a găsi o potrivire.
- Supraîncărcarea metodelor este diferită de suprascrierea metodelor (method overriding), care are legătură cu polimorfismul și moștenirea. Suprascrierea implică furnizarea unei noi implementări pentru o metodă virtuală sau abstractă moștenită din clasa de bază.

### 2.6.5. Parametri ref, out și in

Când definim o metodă în C#, putem specifica modul în care argumentele sunt transmise acelei metode folosind cuvintele cheie `ref`, `out` și `in`. Aceste cuvinte cheie permit metodei să modifice valorile variabilelor transmise ca argumente, oferind un mecanism pentru a returna valori suplimentare din metodă sau pentru a evita copierea inutilă a datelor.

#### Parametri ref:

- Când un parametru este marcat cu `ref`, metoda poate modifica valoarea variabilei transmise ca argument, iar modificările vor fi vizibile în codul apelant.
- Variabila trebuie inițializată înainte de a fi trecută ca argument `ref`.
- Folosirea `ref` permite metodei să modifice valoarea parametrului și să returneze efectiv mai multe valori.

Exemplu:

```
void Swap(ref int a, ref int b)
{
    int temp = a;
    a = b;
    b = temp;
}

int x = 10, y = 20;
Swap(ref x, ref y);
// După apel: x = 20, y = 10
```

#### Parametri out:

- Similar cu `ref`, `out` permite metodei să modifice valoarea variabilei transmise ca argument.
- Spre deosebire de `ref`, variabila nu trebuie inițializată înainte de a fi trecută ca argument `out`.
- Metoda trebuie să atribuie o valoare parametrului `out` înainte de a reveni.
- Folosirea `out` este utilă când doriți ca o metodă să returneze mai multe valori sau când doriți să indicați că un parametru este doar pentru ieșire.

```
void Divide(int a, int b, out int result, out int remainder)
{
    result = a / b;
    remainder = a % b;
}

int result, remainder;
Divide(10, 3, out result, out remainder);
// După apel: result = 3, remainder = 1
```

### Parametri in (începând cu C# 7.2):

- **in** este similar cu **ref**, dar indică faptul că argumentul trebuie să fie citit doar de metodă și nu modificat.
- Folosirea **in** permite compilatorului să aplice optimizări, deoarece garantează că valoarea argumentului nu va fi modificată.
- Poate fi util pentru a evita copierea inutilă a structurilor mari sau a tipurilor de valoare.

```
int CalculateSum(in int[] numbers)
{
    int sum = 0;
    foreach (int number in numbers)
    {
        sum += number;
    }
    return sum;
}

int[] arr = { 1, 2, 3, 4, 5 };
int sum = CalculateSum(in arr);
```

Când alegeți între **ref**, **out** și **in**, luați în considerare următoarele:

- Folosiți **ref** când doriți ca metoda să poată modifica argumentul și să returneze valori suplimentare prin parametri.
- Folosiți **out** când doriți ca metoda să returneze valori suplimentare prin parametri, dar nu necesită ca argumentul să fie inițializat înainte de apel.
- Folosiți **in** când doriți să indicați explicit că un argument este doar pentru intrare și nu trebuie modificat de metodă, permițând astfel optimizări.

În general, este recomandabil să utilizați aceste cuvinte cheie cu moderație și să preferați returnarea valorilor prin instrucțiunea **return** atunci când este posibil, deoarece acest lucru duce adesea la un cod mai clar și mai ușor de înțeles. Cu toate acestea, **ref**, **out** și **in** pot fi instrumente puternice când sunt necesare pentru a obține performanță sau pentru a exprima intenții specifice în cod.

### 2.6.6. Recursivitate

Recursivitatea este o tehnică de programare în care o funcție se autoapelează în cadrul propriei definiții. Aceasta este o abordare puternică pentru rezolvarea problemelor care pot fi descompuse în subprobleme mai mici, similare cu problema inițială. Funcțiile recursive au două componente principale:

- **Cazul de bază:** o condiție care definește cel mai simplu caz al problemei, pentru care rezultatul este cunoscut direct, fără a necesita apeluri recursive suplimentare. Cazul de bază servește ca punct de ieșire din recursivitate.
- **Cazul recursiv:** exprimă problema în termeni de una sau mai multe instanțe mai mici ale aceleiași probleme, cu scopul de a se apropia de cazul de bază.

Iată un exemplu de funcție recursivă care calculează factorialul unui număr:

```
int Factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }
    else
    {
        return n * Factorial(n - 1);
    }
}
```

În acest exemplu:

- Cazul de bază este când  $n$  este egal cu 0. Știm că  $0! = 1$ , deci funcția returnează 1 fără a mai face alte apeluri recursive.
- Cazul recursiv se produce atunci când  $n$  este diferit de 0. Funcția se autoapelează cu  $n - 1$ , reducând astfel problema la o instanță mai mică a aceluiași calcul. Acest proces continuă până când se ajunge la cazul de bază.

Când se apelează `Factorial(4)`, de exemplu, execuția va arăta astfel:

```
Factorial(4)
  4 * Factorial(3)
    3 * Factorial(2)
      2 * Factorial(1)
        1 * Factorial(0)
          return 1
        return 1 * 1 = 1
      return 2 * 1 = 2
    return 3 * 2 = 6
  return 4 * 6 = 24
```

Recursivitatea poate fi folosită pentru a rezolva o varietate de probleme, cum ar fi parcurgerea structurilor de date recursive (de exemplu, arbori), împărțirea problemelor în subprobleme mai mici (de exemplu, sortarea prin interclasare) și explorarea unor spații de soluții (de exemplu, rezolvarea puzzle-urilor).

Când se scriu funcții recursive, este necesar să ne asigurăm că:

- Există un caz de bază care oprește recursivitatea.
- Apelurile recursive progresează către cazul de bază.
- Recursivitatea nu este prea adâncă, deoarece fiecare apel recursiv consumă memorie pe stivă, ceea ce poate duce la depășirea stivei pentru intrări mari.

De asemenea, mulți algoritmi recursivi pot fi implementați și iterativ, folosind bucle. Versiunile iterative sunt adesea mai eficiente în ceea ce privește utilizarea memoriei, deoarece evită supraîncărcarea indusă de apelurile recursive. Cu toate acestea,

recursivitatea poate duce adesea la soluții mai clare și mai ușor de înțeles pentru anumite probleme.

## 2.7. Gestionarea erorilor

Gestionarea erorilor este o parte esențială a dezvoltării de software robust și de încredere. Erorile pot apărea din diverse motive, cum ar fi intrări nevalide, erori de logică sau resurse insuficiente, iar tratarea necorespunzătoare a acestor erori poate duce la un comportament imprevizibil al programului, la pierderi de date sau chiar la căderi ale sistemului.

C# oferă un mecanism puternic pentru gestionarea erorilor, bazat pe excepții. Excepțiile sunt evenimente care apar în timpul execuției unui program și perturbă fluxul normal de control. Când apare o eroare, se aruncă o excepție, care poate fi prinsă și tratată de codul din jur. Acest mecanism permite separarea logicii de gestionare a erorilor de logica principală a programului, rezultând un cod mai curat și mai ușor de întreținut.

În acest paragraf, vom explora conceptele cheie ale gestionării excepțiilor în C#, cum ar fi:

- Blocurile `try-catch-finally` pentru prinderea și tratarea excepțiilor
- Tipurile comune de excepții și ierarhia claselor de excepții din .NET
- Aruncarea excepțiilor folosind instrucțiunea `throw`
- Crearea de tipuri de excepții personalizate pentru a reprezenta erori specifice aplicației
- Bune practici și linii directoare pentru gestionarea eficientă a erorilor în aplicațiile C#

Prin înțelegerea și aplicarea acestor concepte, veți putea scrie cod C# mai robust, mai ușor de menținut și mai rezistent la erori.

### 2.7.1. Blocuri `try-catch-finally`

Blocurile `try-catch-finally` sunt elemente fundamentale în gestionarea excepțiilor în C#. Acestea oferă un mecanism structurat pentru prinderea, tratarea și curățarea după excepții. Să analizăm în detaliu fiecare dintre aceste blocuri:

#### *Blocul `try`:*

- Conține codul care poate arunca o excepție.
- Dacă o excepție este aruncată în interiorul blocului `try`, execuția este întreruptă imediat și controlul este transferat către blocul `catch` corespunzător.
- Poate fi urmat de unul sau mai multe blocuri `catch` și/sau un bloc `finally`.

#### *Blocul `catch`:*

- Specifică tipul de excepție pe care îl poate prinde și conține codul pentru tratarea acelei excepții.

- Dacă tipul excepției aruncate se potrivește cu tipul specificat în blocul `catch` (sau este o clasă derivată), codul din interiorul blocului `catch` este executat.
- Poate avea acces la o instanță a excepției prinse, permițând accesul la informații suplimentare, cum ar fi mesajul de eroare.
- Un bloc `try` poate fi urmat de mai multe blocuri `catch` pentru a trata diferite tipuri de excepții.

#### Blocul *finally*:

- Conține codul care trebuie executat indiferent dacă a fost aruncată sau nu o excepție.
- Este folosit pentru a elibera resursele sau pentru a efectua operații de curățare necesare.
- Blocul `finally` este executat chiar și în cazul în care o excepție este aruncată și prinsă într-un bloc `catch`.
- Blocul `finally` este opțional, dar poate fi folosit fără un bloc `catch`.

Iată un exemplu care ilustrează utilizarea blocurilor `try-catch-finally`:

```
try
{
    int a = 10;
    int b = 0;
    int result = a / b; // Aruncă o excepție DivideByZeroException
    Console.WriteLine(result);
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("A apărut o eroare: " + ex.Message);
}
finally
{
    Console.WriteLine("Blocul finally este executat.");
}
```

Rezultatul:

```
A apărut o eroare: Attempted to divide by zero.
Blocul finally este executat.
```

În acest exemplu:

- Codul din blocul `try` încearcă să împartă `a` la `b`, ceea ce aruncă o excepție `DivideByZeroException`.
- Blocul `catch` prinde excepția `DivideByZeroException` și afișează mesajul de eroare.
- Blocul `finally` este executat indiferent dacă a fost aruncată sau nu o excepție, afișând un mesaj.

Câteva aspecte importante de reținut:

- Dacă nu există un bloc `catch` corespunzător pentru o excepție aruncată, excepția se propagă în sus în stiva de apeluri până când este prinsă sau până când ajunge la nivelul superior al programului, moment în care programul se încheie.
- Blocurile `catch` ar trebui să fie ordonate de la cele mai specifice la cele mai generale, pentru a evita prinderea unei excepții de către un bloc `catch` prea general înainte de a ajunge la un bloc `catch` mai specific.
- Blocurile `finally` sunt utile pentru a vă asigura că resursele (cum ar fi fișierele, conexiunile la baze de date etc.) sunt eliberate corespunzător, indiferent de apariția excepțiilor.

### 2.7.2. Tipuri de excepții comune

În .NET, excepțiile sunt organizate într-o ierarhie de clase, cu `System.Exception` ca clasă de bază. Această ierarhie include multe tipuri de excepții predefinite care reprezintă diferite categorii de erori. Iată câteva dintre cele mai comune tipuri de excepții în C#:

- **`System.ArgumentException`:**
  - Aruncată când un argument trecut unei metode este invalid.
  - Clase derivate: `ArgumentNullException`, `ArgumentOutOfRangeException`.
- **`System.NullReferenceException`:**
  - Aruncată când se încearcă accesarea unui membru (proprietate, metodă etc.) printr-o referință `null`.
- **`System.InvalidOperationException`:**
  - Aruncată când o metodă este apelată într-un moment nepotrivit sau când starea obiectului nu permite executarea metodei.
- **`System.IndexOutOfRangeException`:**
  - Aruncată când se încearcă accesarea unui element al unei colecții (cum ar fi un tablou sau o listă) folosind un index invalid.
- **`System.IO.IOException`:**
  - Aruncată când apare o eroare de intrare/ieșire, cum ar fi încercarea de a citi dincolo de sfârșitul unui fișier.
  - Clase derivate: `DirectoryNotFoundException`, `FileNotFoundException`, `EndOfStreamException`.
- **`System.FormatException`:**
  - Aruncată când formatul datelor furnizate nu se potrivește cu formatul așteptat de o metodă.
- **`System.OverflowException`:**
  - Aruncată când o operație aritmetică depășește domeniul tipului de date rezultat.
- **`System.OutOfMemoryException`:**
  - Aruncată când o alocare de memorie a eșuat din cauza memoriei insuficiente.

- **System.NotImplementedException:**
  - Aruncată când o metodă sau o proprietate nu este implementată în clasă.
- **System.DivideByZeroException:**
  - Aruncată când se încearcă împărțirea unui număr întreg la zero.

Aceste tipuri de excepții acoperă multe scenarii comune de eroare, dar ierarhia de excepții .NET include multe alte tipuri specifice. Când prindeți o excepție, puteți alege să o prindeți la un nivel specific (de exemplu, **FileNotFoundException**) sau la un nivel mai general (de exemplu, **IOException** sau chiar **Exception**), în funcție de nevoile dvs. de tratare a erorilor.

Iată un exemplu care ilustrează prinderea diferitelor tipuri de excepții:

```
try
{
    // Cod care poate arunca diferite excepții
    int[] numbers = { 1, 2, 3 };
    Console.WriteLine(numbers[5]); // Aruncă IndexOutOfRangeException

    string str = null;
    Console.WriteLine(str.Length); // Aruncă NullReferenceException

    int zero = 0;
    int result = 10 / zero; // Aruncă DivideByZeroException
}
catch (IndexOutOfRangeException ex)
{
    Console.WriteLine("IndexOutOfRangeException: " + ex.Message);
}
catch (NullReferenceException ex)
{
    Console.WriteLine("NullReferenceException: " + ex.Message);
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("DivideByZeroException: " + ex.Message);
}
catch (Exception ex)
{
    Console.WriteLine("Altă excepție: " + ex.Message);
}
```

În acest exemplu, blocurile catch sunt ordonate de la cele mai specifice la cele mai generale. Dacă este aruncată o excepție **IndexOutOfRangeException**, **NullReferenceException** sau **DivideByZeroException**, va fi prinsă de blocul catch corespunzător. Dacă este aruncată o excepție diferită, va fi prinsă de ultimul bloc catch, care tratează **Exception** în general.

Cunoașterea tipurilor comune de excepții și a ierarhiei de excepții .NET vă ajută să scrieți un cod de tratare a erorilor mai precis și mai informativ. Vă permite să prindeți și să tratați



excepțiile la nivelul potrivit de specificitate și să oferiți un răspuns adecvat pentru fiecare tip de eroare.

### 2.7.3.Aruncarea excepțiilor (throw)

Pe lângă prinderea și tratarea excepțiilor, C# vă permite, de asemenea, să aruncați excepții în mod explicit, folosind instrucțiunea `throw`. Acest lucru este util atunci când doriți să semnalizați o eroare sau o condiție excepțională în codul dumneavoastră.

Iată câteva scenarii în care ați putea dori să aruncați o excepție:

- Când un argument al metodei este invalid sau nu respectă o anumită condiție prealabilă.
- Când o operație nu poate fi efectuată din cauza stării obiectului sau a mediului de execuție.
- Când apare o eroare în timpul executării unei metode și doriți să o propagați la apelant.

Pentru a arunca o excepție, folosiți instrucțiunea `throw`, urmată de o instanță a excepției pe care doriți să o aruncați. De exemplu:

```
throw new ArgumentException("Mesajul de eroare");
```

Acest cod creează o nouă instanță a clasei `ArgumentException` cu un mesaj de eroare specific și o aruncă. Execuția metodei curente este întreruptă imediat, iar excepția este propagată în sus în stiva de apeluri până când este prinsă de un bloc catch sau până când ajunge la nivelul superior al programului.

Puteți arunca orice tip de excepție derivată din clasa `System.Exception`. Este o practică comună să folosiți tipurile de excepții predefinite din .NET atunci când se aplică (de exemplu, `ArgumentException` pentru argumente nevalide, `InvalidOperationException` pentru apeluri de metode necorespunzătoare etc.) și să creați tipuri de excepții personalizate pentru erori specifice domeniului sau aplicației dumneavoastră.

Iată un exemplu mai detaliat care ilustrează aruncarea unei excepții:

```
public void WithdrawMoney(decimal amount)
{
    if (amount <= 0)
    {
        throw new ArgumentException("Suma de retras trebuie să fie mai mare decât zero.", nameof(amount));
    }

    if (amount > Balance)
    {
        throw new InsufficientFundsException("Fonduri insuficiente pentru a efectua retragerea.");
    }
}
```

```
Balance -= amount;  
}
```

În acest exemplu, metoda `WithdrawMoney` verifică două condiții:

- Dacă suma de retras este mai mică sau egală cu zero, se aruncă o excepție `ArgumentException` cu un mesaj de eroare corespunzător.
- Dacă suma de retras este mai mare decât soldul curent, se aruncă o excepție `InsufficientFundsException` (care ar fi un tip de excepție personalizat) cu un mesaj de eroare corespunzător.

Dacă ambele condiții sunt îndeplinite, retragerea are loc și soldul este actualizat.

Câteva sfaturi pentru utilizarea eficientă a instrucțiunii `throw`:

- Folosiți excepții pentru a semnaliza erori și condiții excepționale, nu pentru controlul normal al fluxului programului.
- Furnizați mesaje de eroare clare și informative atunci când aruncați excepții, pentru a ajuta la depanarea și la identificarea cauzei problemei.
- Evitați să prindeți excepțiile generale (de exemplu, `Exception`) și, în schimb, prindeți tipuri specifice de excepții pe care le puteți trata în mod corespunzător.
- Nu ignorați pur și simplu excepțiile fără a le trata sau a le raporta în mod adecvat.
- Luați în considerare performanța atunci când folosiți excepții, deoarece aruncarea și prinderea excepțiilor poate avea un impact asupra performanței.

#### 2.7.4. Crearea excepțiilor personalizate

Deși .NET oferă o gamă largă de tipuri de excepții predefinite, pot exista situații în care aveți nevoie să definiți propriile tipuri de excepții pentru a reprezenta erori sau condiții excepționale specifice domeniului sau aplicației dumneavoastră. Crearea de tipuri de excepții personalizate vă permite să comunicați informații mai bogate și mai specifice despre erori și să structurați mai bine logica de tratare a erorilor.

Pentru a crea un tip de excepție personalizat în C#, definiți o nouă clasă care derivă direct sau indirect din clasa `System.Exception`. De obicei, veți dori să includeți următoarele elemente în tipul dumneavoastră de excepție personalizat:

##### *Constructor:*

- Definiți cel puțin doi constructori: unul fără parametri și unul care acceptă un mesaj de eroare.
- Apelați constructorul clasei de bază (`Exception`) pentru a permite inițializarea corespunzătoare a excepției.

##### *Proprietăți (opțional):*

- Dacă excepția dumneavoastră necesită informații suplimentare pentru a descrie eroarea, definiți proprietăți pentru a stoca și a accesa aceste informații.

### Metode (optional):

- Dacă sunt necesare comportamente sau funcționalități suplimentare, definiți metode în clasă.

Iată un exemplu de tip de excepție personalizat:

```
public class InsufficientFundsException : Exception
{
    public decimal RequiredAmount { get; }
    public decimal AvailableBalance { get; }

    public InsufficientFundsException()
    {
    }

    public InsufficientFundsException(string message)
        : base(message)
    {
    }

    public InsufficientFundsException(string message, decimal requiredAmount,
    decimal availableBalance)
        : base(message)
    {
        RequiredAmount = requiredAmount;
        AvailableBalance = availableBalance;
    }
}
```

În acest exemplu, `InsufficientFundsException` este un tip de excepție personalizat care reprezintă o eroare în care fondurile disponibile nu sunt suficiente pentru o anumită operație. Pe lângă constructorii obișnuiți, acesta include și proprietăți pentru a stoca suma necesară (`RequiredAmount`) și soldul disponibil (`AvailableBalance`), oferind informații suplimentare despre eroare.

Puteți arunca acest tip de excepție personalizat folosind instrucțiunea `throw`, la fel ca orice altă excepție:

```
decimal requiredAmount = 100;
decimal availableBalance = 50;

if (requiredAmount > availableBalance)
{
    throw new InsufficientFundsException("Fonduri insuficiente.",
    requiredAmount, availableBalance);
}
```

Când creați tipuri de excepții personalizate, este important să urmați câteva bune practici:

- Alegeți un nume clar și descriptiv pentru tipul de excepție, care să indice natura erorii sau a condiției excepționale.

- Derivați tipul de excepție dintr-o clasă de excepție de bază adecvată, cum ar fi `Exception`, `ApplicationException` sau o altă excepție specifică domeniului.
- Includeți informații relevante despre eroare în mesajul de excepție și în proprietățile personalizate.
- Respectați convențiile .NET pentru numirea și structurarea tipurilor de excepții.

### 2.7.5. Bune practici în gestionarea erorilor

Gestionarea eficientă a erorilor este esențială pentru a crea aplicații C# robuste, sigure și ușor de întreținut. Iată câteva bune practici și linii directoare pentru tratarea excepțiilor în C#:

- Folosiți excepții pentru condiții excepționale:
  - Utilizați excepțiile pentru a semnaliza erori și situații neobișnuite, nu pentru controlul normal al fluxului programului.
  - Evitați folosirea excepțiilor pentru logica condițională regulată.
- Fiți specifici în prinderea excepțiilor:
  - Prindeți excepții specifice pe care le puteți trata în mod corespunzător, în loc să prindeți excepția de bază `Exception`.
  - Acest lucru vă permite să furnizați un tratament specific pentru diferite tipuri de erori și ajută la evitarea înghițirii accidentale a excepțiilor importante.
- Folosiți blocuri `try-catch` în jurul codului care poate genera excepții:
  - Înconjurați codul care poate arunca excepții cu blocuri `try-catch` pentru a le prinde și a le trata corespunzător.
  - Acest lucru ajută la prevenirea propagării necontrolate a excepțiilor și la menținerea integrității aplicației.
- Folosiți blocuri `finally` pentru curățare:
  - Utilizați blocuri `finally` pentru a vă asigura că resursele (cum ar fi conexiunile la baze de date, fișierele sau alte resurse externe) sunt eliberate corespunzător, indiferent dacă a apărut sau nu o excepție.
  - Acest lucru ajută la evitarea scurgerilor de resurse și a altor probleme legate de gestionarea resurselor.
- Nu ignorați excepțiile:
  - Nu lăsați niciodată blocurile `catch` goale și nu „înghițiți” pur și simplu excepțiile fără a le trata sau a le raporta în mod corespunzător.
  - Ignorarea excepțiilor poate ascunde erori critice și poate face depanarea și întreținerea foarte dificile.
- Furnizați mesaje de eroare semnificative:
  - Când aruncați excepții sau când definiți tipuri de excepții personalizate, includeți mesaje de eroare clare, descriptive și acționabile.
  - Mesajele de eroare ar trebui să explice problema și, dacă este posibil, să sugereze soluții sau acțiuni de remediere.

- Documentează excepțiile aruncate:
  - Folosiți comentarii XML pentru a documenta excepțiile pe care le poate arunca o metodă, inclusiv tipurile de excepții și condițiile în care sunt aruncate.
  - Acest lucru ajută alți dezvoltatori (sau pe tine însuși în viitor) să înțeleagă și să gestioneze corect excepțiile atunci când folosesc codul.
- Gândiți-vă la experiența utilizatorului:
  - Când tratați excepții, luați în considerare impactul asupra utilizatorului final al aplicației.
  - Furnizați mesaje de eroare prietenoase și instrucțiuni utile pentru utilizatori, păstrând în același timp detaliile tehnice în jurnalele de erori pentru depanare.
- Înregistrați și monitorizați excepțiile:
  - Implementați un sistem de înregistrare (logging) pentru a surprinde și a stoca informațiile despre excepțiile care apar în aplicația dvs.
  - Aceasta poate include înregistrarea în fișiere jurnal, baze de date sau servicii externe de monitorizare a erorilor.
  - Analizați și monitorizați jurnalele de excepții pentru a identifica probleme recurente și pentru a îmbunătăți calitatea și robustețea aplicației.
- Testați scenariile de excepție:
  - Includeți teste pentru scenarii de excepție în suita dvs. de teste unitare și de integrare.
  - Verificați dacă aplicația dvs. gestionează corect diferite tipuri de excepții și dacă menține un stare consistentă și corectă.

## 2.8. Introducere în programarea orientată pe obiecte<sup>2</sup>

Programarea orientată pe obiecte (OOP) este o paradigmă de programare esențială care stă la baza dezvoltării de aplicații moderne. OOP se concentrează pe organizarea codului în jurul conceptelor de obiecte și clase, care încapsulează atât date (proprietăți sau attribute), cât și comportamente (metode).

<sup>2</sup> Rolul programării orientate pe obiecte în dezvoltarea software modernă traversează o perioadă de reevaluare critică, după decenii în care a reprezentat paradigma dominantă. În timp ce OOP a oferit un cadru conceptual solid pentru organizarea și structurarea codului, experiența acumulată în proiecte complexe a scos la iveală limitări semnificative ale acestei abordări.

Fundamentele OOP - încapsularea, moștenirea și polimorfismul - deși concepute pentru a simplifica dezvoltarea software, pot paradoxal să conducă la arhitecturi excesiv de complexe în sistemele enterprise. Ierarhiile adânci de clase, cuplajul strâns între componente și starea mutabilă distribuită pot face codul mai greu de înțeles, testat și modificat pe termen lung. Aceasta nu înseamnă abandonarea totală a OOP ci mai degrabă o regândire a modului în care principiile sale pot fi aplicate selectiv și combinate cu alte paradigme pentru a obține rezultate optime.

Adoptarea unei abordări orientate pe obiecte în dezvoltarea software aduce numeroase beneficii, cum ar fi:

- **Modularitate:** OOP promovează împărțirea unui sistem complex în componente mai mici, autoconținute, numite obiecte, care pot fi dezvoltate, testate și întreținute independent.
- **Reutilizare a codului:** Prin crearea de clase generice care pot fi instanțiate în diferite părți ale aplicației, OOP favorizează reutilizarea codului, reducând duplicarea și îmbunătățind eficiența dezvoltării.
- **Abstractizare:** OOP permite dezvoltatorilor să creeze abstracțiuni ale entităților din lumea reală, concentrându-se pe caracteristicile și comportamentele esențiale și ascunzând detaliile interne complexe.
- **Încapsulare:** Prin încapsularea datelor și a comportamentelor conexe în interiorul obiectelor și prin controlul accesului la starea internă a obiectelor, OOP promovează un cod mai sigur, mai ușor de întreținut și mai puțin predispus la erori.
- **Polimorfism:** OOP permite definirea de interfețe comune pentru obiecte cu comportamente similare, permițând unui singur tip de a reprezenta mai multe tipuri concrete. Acest lucru promovează un cod mai flexibil și extensibil.

În acest paragraf, vom explora conceptele fundamentale ale programării orientate pe obiecte în C#, cum ar fi:

- Noțiunile de clasă și obiect
- Definirea și utilizarea membrilor unei clase, cum ar fi câmpuri, proprietăți și metode
- Crearea și utilizarea constructorilor și destructorilor
- Modificatorii de acces și controlul vizibilității membrilor
- Utilizarea proprietăților auto-implementate pentru o sintaxă concisă
- Membrii statici și utilizarea lor pentru a reprezenta concepte la nivel de clasă

### 2.8.1. Conceptul de clasă și obiect

În programarea orientată pe obiecte (OOP), un obiect reprezintă o entitate din lumea reală sau conceptuală, care are o stare (definită prin atributele sale) și un comportament (definit prin metodele sale). Obiectele sunt create pe baza unor șabloane numite clase.

O clasă este o abstractizare a unui concept sau a unei entități, care încapsulează atât datele (atributele), cât și funcționalitatea (metodele) specifice aceluia concept. Putem privi clasa ca un plan sau o schiță pentru crearea obiectelor. Ea definește structura și comportamentul comun pentru un grup de obiecte similare.

Să luăm un exemplu concret: o clasă "Student". Această clasă poate avea atribute precum nume, vârstă, anul de studiu și lista de cursuri la care este înscris. De asemenea,

clasa poate avea metode precum `AdaugaCurs()` sau `AfiseazaDetalii()`. Fiecare obiect creat pe baza acestei clase (adică fiecare instanță a clasei "Student") va avea propriile valori pentru atribute, dar va împărtăși același comportament definit de metode.

Iată un exemplu simplificat de definire a clasei "Student" în C#:

```
public class Student
{
    // Atribute
    public string Nume;
    public int Varsta;
    public int AnStudiu;
    public List<string> Cursuri;

    // Constructor
    public Student(string nume, int varsta, int anStudiu)
    {
        Nume = nume;
        Varsta = varsta;
        AnStudiu = anStudiu;
        Cursuri = new List<string>();
    }

    // Metode
    public void AdaugaCurs(string numeCurs)
    {
        Cursuri.Add(numeCurs);
    }

    public void AfiseazaDetalii()
    {
        Console.WriteLine($"Student: {Nume}, Vârsta: {Varsta}, An: {AnStudiu}");
        Console.WriteLine("Cursuri:");
        foreach (string curs in Cursuri)
        {
            Console.WriteLine(curs);
        }
    }
}
```

Pentru a crea un obiect al acestei clase (o instanță), folosim cuvântul cheie `new` urmat de numele clasei și argumentele necesare constructorului:

```
Student student1 = new Student("Ion Popescu", 20, 2);
student1.AdaugaCurs("Programare C#");
student1.AdaugaCurs("Baze de date");
student1.AfiseazaDetalii();
```

Acest cod va afișa:

```
Student: Ion Popescu, Vârsta: 20, An: 2
Cursuri:
Programare C#
Baze de date
```

### 2.8.2. Membri ai clasei (câmpuri, proprietăți, metode)

Membrii unei clase reprezintă elementele care definesc structura și comportamentul obiectelor create pe baza acelei clase. Există trei categorii principale de membri ai unei clase în C#: câmpuri, proprietăți și metode.

#### *Câmpuri (fields):*

- Câmpurile sunt variabile declarate în interiorul unei clase, care stochează datele asociate fiecărei instanțe a clasei.
- Ele reprezintă starea internă a unui obiect.
- Câmpurile pot avea diferite tipuri de date și modificatori de acces (public, private, protected).

```
public class Student
{
    public string nume;
    private int varsta;
    protected float medie;
}
```

#### *Proprietăți (properties):*

- Proprietățile sunt membri ai clasei care oferă un mecanism de citire și/sau scriere controlată pentru datele asociate unui obiect.
- Ele permit accesul la starea internă a unui obiect prin intermediul unor metode speciale numite accessori (get și set).
- Proprietățile pot avea diferite modificatori de acces și pot include logică suplimentară în accessori.

```
public class Student
{
    private string nume;

    public string Nume
    {
        get { return nume; }
        set { nume = value; }
    }
}
```

#### *Metode:*

- Metodele sunt funcții declarate în interiorul unei clase, care definesc comportamentul și operațiile ce pot fi efectuate de obiectele acelei clase.
- Ele pot primi parametri, pot returna valori și pot avea diferite modificatori de acces.
- Metodele sunt responsabile pentru implementarea logicii și funcționalității specifice clasei.

```
public class Student
```



```
{
    public void AfiseazaDetalii()
    {
        Console.WriteLine("Detalii student:");
        // Logica de afișare a detaliilor studentului
    }

    public double CalculeazaMedie(double[] note)
    {
        // Logica de calculare a mediei notelor
    }
}
```

Câmpurile, proprietățile și metodele lucrează împreună pentru a defini structura și comportamentul obiectelor create pe baza unei clase. Câmpurile stochează datele, proprietățile oferă acces controlat la aceste date, iar metodele implementează operațiile și funcționalitățile specifice clasei.

Este important să alegi cu atenție modificatorii de acces pentru membrii clasei (public, private, protected) în funcție de nivelul de vizibilitate și încapsulare dorit. De asemenea, se recomandă utilizarea proprietăților în locul accesării directe a câmpurilor, pentru a asigura un control mai bun asupra modului în care datele sunt accesate și modificate.

### 2.8.3. Constructori și destructori

În C#, constructorii și destructorii sunt membri speciali ai unei clase, care joacă un rol important în crearea și eliminarea obiectelor.

Constructorii sunt metode speciale ale unei clase, care sunt apelate automat la crearea unei noi instanțe a clasei. Rolul principal al unui constructor este de a inițializa starea obiectului nou creat, adică de a atribui valori inițiale câmpurilor și proprietăților obiectului.

Iată câteva caracteristici importante ale constructorilor în C#:

- Au același nume ca și clasa din care fac parte.
- Nu au un tip de retur specificat (nici măcar `void`).
- Pot avea parametri, permițând inițializarea obiectului cu valori specifice.
- O clasă poate avea mai mulți constructori cu semnături diferite (supraîncărcare).
- Dacă nu este definit explicit niciun constructor, C# oferă un constructor implicit fără parametri.

Exemplu de constructor:

```
public class Student
{
    public string Nume;
    public int Varsta;

    public Student(string nume, int varsta)
    {
```

```
        Nume = nume;  
        Varsta = varsta;  
    }  
}
```

Pentru a crea un obiect folosind constructorul definit, utilizăm operatorul **new**:

```
Student student1 = new Student("Ion Popescu", 20);
```

Destructorii, pe de altă parte, sunt metode speciale care sunt apelate automat atunci când un obiect este distrus sau eliminat din memorie. În C#, destructorii sunt numiți și finalizatori.

Iată câteva caracteristici ale destructorilor în C#:

- Au același nume ca și clasa, precedat de caracterul `~``.
- Nu au modificatori de acces și nu primesc parametri.
- Sunt apelați automat de Garbage Collector (GC) atunci când obiectul nu mai este utilizat.
- Sunt utilizați pentru a elibera resursele negestionate (ex: fișiere, conexiuni la baze de date) deținute de obiect.

Exemplu de destructor:

```
public class Student  
{  
    ~Student()  
    {  
        // Cod de curățare sau eliberare a resurselor  
    }  
}
```

Este important de menționat că, în majoritatea cazurilor, nu este necesar să definim explicit destructori, deoarece garbage collector-ul se ocupă automat de gestionarea memoriei și de eliberarea resurselor pentru obiectele care nu mai sunt utilizate. Destructorii sunt folosiți doar în situații speciale, cum ar fi lucrul cu resurse negestionate.

De asemenea, este recomandat să utilizăm interfața **IDisposable** și metoda **Dispose()** pentru a elibera manual resursele deținute de un obiect, în loc să ne bazăm exclusiv pe destructori.

#### 2.8.4. Modificatori de acces (public, private, protected)

Modificatorii de acces sunt cuvinte cheie în C# care controlează vizibilitatea și accesibilitatea membrilor unei clase (câmpuri, proprietăți, metode, constructori) din exterior sau din alte părți ale programului. Prin specificarea modificatorilor de acces, putem aplica principiul încapsulării, care este un concept fundamental în programarea orientată pe obiecte.

C# oferă trei modificatori de acces principali: public, private și protected.

#### *Modificatorul public:*

- Membrii declarați ca **public** sunt accesibili din orice parte a programului, atât din interiorul clasei, cât și din exteriorul acesteia.
- Pot fi accesați de orice cod care are o referință la obiectul respectiv.

- Exemplu:

```
public class Student
{
    public string Nume;
    public void AfiseazaDetalii() { /* ... */ }
}
```

#### *Modificatorul private:*

- Membrii declarați ca **private** sunt accesibili doar din interiorul clasei în care sunt definiți.
- Nu pot fi accesați din exteriorul clasei, nici măcar de către clase derivate.
- Sunt folosiți pentru a încapsula starea internă a obiectului și a o proteja de acces neautorizat.

- Exemplu:

```
public class Student
{
    private int varsta;
    private void CalculeazaMedie() { /* ... */ }
}
```

#### *Modificatorul protected:*

- Membrii declarați ca **protected** sunt accesibili din interiorul clasei și din clasele derivate (subclase).
- Nu pot fi accesați din exteriorul ierarhiei de clase.
- Sunt folosiți pentru a permite claselor derivate să acceseze și să extindă funcționalitatea clasei de bază.

- Exemplu:

```
public class Person
{
    protected string CNP;
}
public class Student : Person
{
    public void AfiseazaCNP()
    {
        Console.WriteLine(CNP);
    }
}
```

Pe lângă acești modificatori de acces principali, C# oferă și modificatorul `internal`:

- Membrii declarați ca `internal` sunt accesibili doar din interiorul aceleiași assembly (bibliotecă de clase sau proiect).
- Sunt folosiți pentru a limita accesul la anumite părți ale programului, fără a le expune în exterior.

De asemenea, C# permite combinarea modificatorilor `protected` și `internal` folosind sintaxa `protected internal`:

- Membrii declarați ca `protected internal` sunt accesibili din interiorul aceleiași assembly și din clasele derivate din alte assembly-uri.

### 2.8.5. Proprietăți auto-implementate

Proprietățile auto-implementate sunt o caracteristică introdusă în C# 3.0, care oferă o modalitate concisă de a defini proprietăți într-o clasă fără a fi nevoie să scriem explicit logica getter și setter. Aceste proprietăți sunt utile când dorim să expunem public un câmp privat, fără a adăuga o funcționalitate suplimentară în metodele de acces.

Sintaxa pentru a declara o proprietate auto-implementată este următoarea:

```
public tip NumeProprietate { get; set; }
```

Aici, `tip` reprezintă tipul de date al proprietății, iar `NumeProprietate` este numele proprietății. Cuvintele cheie `get` și `set` indică faptul că proprietatea are atât un accesori getter, cât și un accesori setter.

Când declarăm o proprietate auto-implementată, compilatorul C# generează automat un câmp privat în fundal pentru a stoca valoarea proprietății și implementează metodele de acces getter și setter pentru a citi și a scrie valoarea acestui câmp.

Iată un exemplu de utilizare a proprietăților auto-implementate:

```
public class Student
{
    public string Nume { get; set; }
    public int Varsta { get; set; }
}
```

În acest exemplu, clasa `Student` are două proprietăți auto-implementate: `Nume` și `Varsta`. Aceste proprietăți pot fi accesate și modificate direct, fără a fi nevoie să scriem explicit logica getter și setter.

Putem utiliza aceste proprietăți în felul următor:

```
Student student = new Student();
student.Nume = "Ion Popescu";
student.Varsta = 20;

Console.WriteLine($"Nume: {student.Nume}, Vârsta: {student.Varsta}");
```

Proprietățile auto-implementate oferă câteva avantaje:

- Scurtează codul și îl fac mai lizibil, eliminând necesitatea de a scrie explicit logica getter și setter.
- Asigură o sintaxă consistentă și ușor de înțeles pentru definirea proprietăților simple.
- Permit modificarea ușoară a implementării interne a proprietății fără a afecta codul care o folosește.

Cu toate acestea, există și câteva limitări:

- Nu putem adăuga o logică suplimentară în metodele de acces getter și setter (de exemplu, validarea datelor sau notificarea la modificare).
- Nu putem avea un nivel de acces diferit pentru getter și setter (ambele au același nivel de acces ca și proprietatea).

În cazul în care avem nevoie de o logică personalizată în metodele de acces sau de niveluri de acces diferite pentru getter și setter, vom utiliza proprietăți obișnuite cu implementare explicită a metodelor de acces.

### 2.8.6. Membri statici

Membrii statici sunt membri ai unei clase (câmpuri, proprietăți, metode, constructori) care aparțin clasei în sine, și nu unei anumite instanțe a clasei. Aceștia sunt declarați utilizând cuvântul cheie `static` și prezintă câteva caracteristici importante:

#### Accesibilitate:

- Membrii statici pot fi accesați folosind numele clasei, fără a fi nevoie să creăm o instanță a clasei.
- Putem apela metode statice, accesa câmpuri statice sau proprietăți statice direct prin intermediul numelui clasei.

- Exemplu:

```
public class Matematica
{
    public static double PI = 3.14159;
    public static int Aduna(int a, int b) { return a + b; }
}

// Accesarea membrilor statici
double aria = Matematica.PI * raza * raza;
int suma = Matematica.Aduna(10, 20);
```

#### Partajarea datelor:

- Membrii statici sunt partajați de toate instanțele unei clase.
- Orice modificare a unui membru static este vizibilă pentru toate instanțele clasei și pentru orice cod care accesează acel membru static.

- Exemplu:

```
public class Contor
{
    public static int count = 0;
    public void Increment() { count++; }
}

// Partajarea datelor între instanțe
Contor c1 = new Contor();
Contor c2 = new Contor();
c1.Increment();
c2.Increment();
Console.WriteLine(Contor.count); // Afișează 2
```

#### *Restricții:*

- Membrii statici nu pot accesa membri non-statici ai clasei în mod direct.
- Membrii statici nu pot utiliza cuvântul cheie `this` deoarece nu există o instanță a clasei asociată cu ei.
- Constructorii statici nu pot avea parametri și sunt apelați automat înainte de crearea primei instanțe a clasei sau de accesarea oricărui membru static.

#### *Utilizare:*

- Membrii statici sunt utili pentru a reprezenta constante sau valori care sunt comune tuturor instanțelor unei clase.
- Metodele statice sunt folosite adesea pentru a implementa funcționalități generale sau utilitare care nu depind de starea unei instanțe specifice.
- Proprietățile și câmpurile statice pot fi utilizate pentru a partaja date între toate instanțele unei clase.

Membrii statici oferă un mecanism de a asocia date și funcționalități cu o clasă în sine, și nu cu instanțe individuale ale clasei. Aceștia sunt utili pentru a implementa funcționalități comune, a partaja date între instanțe și a accesa membri ai unei clase fără a crea o instanță a acesteia.

## 3. LINQ (Language Integrated Query)

LINQ (Language Integrated Query) reprezintă o caracteristică puternică și expresivă introdusă în .NET Framework 3.5, care permite interogarea și manipularea datelor dintr-o varietate de surse folosind o sintaxă consistentă și intuitivă, direct în limbajul de programare C#. LINQ unifică operațiile de interogare pentru obiecte aflate în memorie, baze de date relaționale, XML, și alte surse de date, oferind un model de programare uniform și intuitiv.

În acest capitol, vom explora în detaliu conceptele fundamentale și avansate ale LINQ, acoperind următoarele subiecte:

- Fundamentele LINQ: Vom analiza arhitectura LINQ, conceptul de furnizori și extensibilitatea LINQ, precum și expresiile și operatorii de bază utilizați în interogări.
- LINQ to Objects: Vom învăța cum să folosim LINQ pentru a interoga și manipula colecții de obiecte în memorie, aplicând operații precum filtrarea, proiecția, ordonarea și gruparea datelor.
- Expresii Lambda și Delegate: Vom explora utilizarea expresiilor lambda și a delegaților în LINQ pentru a defini funcții anonime și a le aplica în operațiile de interogare.
- Operatori LINQ avansați: Vom acoperi operatorii LINQ avansați, cum ar fi Join, GroupJoin, SelectMany și operatorii de mulțimi, pentru a efectua interogări mai complexe și a combina date din surse multiple.
- LINQ to XML și JSON: Vom vedea cum putem utiliza LINQ pentru a interoga și manipula documente XML și JSON, simplificând procesarea datelor semi-structurate.

Nu vom aborda LINQ to SQL din cauză lipsei de spațiu în acest curs, dar dată fiind similitudinea cu conceptele deja prezentate, nu veți avea nicio problemă în a aborda și acest concept când lucrați cu baze de date.

### 3.1. Fundamentele LINQ

În această secțiune, vom explora arhitectura LINQ și componentele sale principale, vom discuta despre furnizorii LINQ și extensibilitatea acestei tehnologii, și vom introduce expresiile și operatorii de bază utilizați în interogările LINQ.

Prin înțelegerea acestor fundamente, veți avea o bază solidă pentru a începe să utilizați LINQ în proiectele proprii și pentru a explora capacitățile sale avansate în secțiunile ulterioare.

### 3.1.1. Arhitectura LINQ

Componentele principale ale arhitecturii LINQ sunt:

- Expresii de interogare LINQ: Acestea sunt construcții sintactice în C# care permit scrierea interogărilor asupra datelor într-un stil declarativ, asemănător cu SQL. Expresiile de interogare LINQ sunt construite folosind operatori de interogare standard, cum ar fi `select`, `from`, `where`, `orderby`, `group by`, și pot fi aplicate asupra oricărei surse de date care implementează interfața `IEnumerable<T>` sau `IQueryable<T>`.
- Operatori de interogare standard: LINQ definește un set de operatori de interogare standard care pot fi utilizați în expresiile de interogare pentru a efectua operații comune precum filtrarea (`where`), proiecția (`select`), ordonarea (`orderby`), gruparea (`group by`), concatenarea (`concat`), agregarea (`aggregate`) și altele. Acești operatori sunt implementați ca metode de extensie ale interfețelor `IEnumerable<T>` și `IQueryable<T>`, permițând astfel aplicarea lor asupra unei game largi de surse de date.
- Furnizori LINQ: Pentru a permite interogarea diferitelor surse de date folosind sintaxa LINQ, există furnizori LINQ specifici pentru fiecare tip de sursă de date. Un furnizor LINQ translatează o expresie de interogare LINQ într-o formă specifică sursei de date subiacente, cum ar fi SQL pentru o bază de date relațională sau XQuery pentru documente XML. Aceasta permite executarea eficientă a interogărilor LINQ pe sursa de date respectivă. .NET Framework include furnizori LINQ pentru obiecte aflate în memorie (LINQ to Objects), baze de date (LINQ to SQL), documente XML (LINQ to XML), și seturi de date (LINQ to DataSet).
- Compilator C#: Compilatorul C# joacă un rol important în arhitectura LINQ prin traducerea expresiilor de interogare LINQ în apeluri de metode echivalente care utilizează operatorii de interogare standard. Acest proces, cunoscut sub numele de "rewriting" sau "desugaring", transformă sintaxa declarativă a expresiilor de interogare LINQ într-un cod C# executabil. De exemplu, o expresie LINQ precum:

```
var result = from p in products
              where p.Category == "Electronics"
              select p.Name;
```

va fi tradusă de compilator în:

```
var result = products
    .Where(p => p.Category == "Electronics")
    .Select(p => p.Name);
```

Această arhitectură permite LINQ să ofere un model de programare consistent și intuitiv pentru interogarea și manipularea datelor din diverse surse, ascunzând complexitatea specifică fiecărei surse de date în spatele unei interfețe uniforme. Dezvoltatorii pot scrie



interogări LINQ într-un stil declarativ, iar compilatorul C# și furnizorii LINQ se ocupă de traducerea acestor interogări în operații specifice surselor de date subiacente.

### 3.1.2. Furnizori și extensibilitate

Una dintre caracteristicile LINQ este capacitatea sa de a interoga diverse surse de date folosind o sintaxă consistentă și intuitivă, direct în limbajul de programare C#. Această flexibilitate și putere sunt posibile datorită conceptului de furnizori LINQ.

Un furnizor LINQ este un set de clase care implementează logica necesară pentru a traduce o expresie de interogare LINQ într-o formă specifică sursei de date subiacente. Cu alte cuvinte, un furnizor LINQ acționează ca un intermediar între sintaxa de interogare LINQ și operațiile specifice necesare pentru a interoga și manipula datele dintr-o anumită sursă de date.

Rolul unui furnizor LINQ este de a implementa interfețele `IQueryable<T>` și `IQueryFurnizor`, care permit construirea și executarea de interogări LINQ asupra sursei de date respective. Atunci când o expresie de interogare LINQ este evaluată, furnizorul LINQ o translatează într-o formă specifică sursei de date, cum ar fi SQL pentru o bază de date relațională, XQuery pentru documente XML sau apeluri de metode pentru obiecte aflate în memorie.

.NET Framework include mai mulți furnizori LINQ predefiniți pentru cele mai comune surse de date:

- LINQ to Objects: Acest furnizor permite interogarea colecțiilor de obiecte cum ar fi `List<T>`, `Array`, `Dictionary<TKey, TValue>` și orice clasă care implementează interfața `IEnumerable<T>`.
- LINQ to SQL: Acest furnizor permite interogarea și manipularea datelor dintr-o bază de date Microsoft SQL Server folosind LINQ. El generează interogări SQL optimizate pe baza expresiilor LINQ și gestionează comunicarea cu baza de date.
- LINQ to XML: Acest furnizor permite interogarea și manipularea documentelor XML folosind LINQ. El oferă o modalitate intuitivă de a naviga, interoga și modifica structura și conținutul unui document XML.
- LINQ to DataSet: Acest furnizor permite interogarea datelor dintr-un obiect `DataSet` folosind LINQ, oferind o alternativă la interogările tradiționale bazate pe `DataTable` și `DataRow`.

Pe lângă furnizorii LINQ incluși în .NET Framework, arhitectura LINQ este extensibilă, permițând dezvoltatorilor să creeze furnizori LINQ personalizați pentru alte surse de date. De exemplu, există furnizori LINQ populari pentru baze de date NoSQL precum MongoDB și Couchbase, pentru servicii web RESTful, pentru fișiere CSV și multe altele.

Extensibilitatea LINQ este posibilă datorită naturii sale generice și a utilizării interfețelor `IQueryable<T>` și `IQueryProvider`. Prin implementarea acestor interfețe și definirea

logicii de traducere a expresiilor LINQ în operații specifice sursei de date, orice sursă de date poate fi integrată în ecosistemul LINQ.

### 3.1.3. Expresii și operatori de bază

Expresiile și operatorii de bază reprezintă elementele fundamentale ale interogărilor LINQ, permițând dezvoltatorilor să scrie interogări declarative și expresive pentru a manipula și a prelucra date din diferite surse. LINQ oferă un set bogat de operatori de interogare standard care pot fi utilizați pentru a filtra, proiecta, ordona, grupa și agrega datele.

Principalii operatori LINQ sunt:

- **where**: Operatorul **where** este utilizat pentru a filtra datele pe baza unei condiții specificate. El primește un predicat (o funcție care returnează o valoare booleană) și returnează un nou set de date care conține doar elementele pentru care predicatul este adevărat.

Exemplu:

```
var filteredProducts = from p in products
                        where p.Category == "Electronics"
                        select p;
```

- **select**: Operatorul **select** este utilizat pentru a proiecta datele, adică pentru a selecta anumite proprietăți sau pentru a calcula valori noi pe baza proprietăților existente. El primește o funcție de proiecție care specifică modul în care fiecare element din setul de date va fi transformat.

Exemplu:

```
var productNames = from p in products
                    select p.Name;
```

- **orderby** și **orderby descending**: Operatorii **orderby** și **orderby descending** sunt utilizați pentru a ordona datele în ordine crescătoare sau descrescătoare pe baza unei chei specificate. Se pot specifica mai multe chei de ordonare pentru a obține o ordonare complexă.

Exemplu:

```
var sortedProducts = from p in products
                      orderby p.Price descending, p.Name
                      select p;
```

- **group by**: Operatorul **group by** este utilizat pentru a grupa datele pe baza unei chei specificate. El returnează un set de grupuri, fiecare grup conținând elementele care au aceeași valoare a cheii.

Exemplu:

```
var productsByCategory = from p in products
                          group p by p.Category;
```

- **join**: Operatorul **join** este utilizat pentru a combina date din două sau mai multe surse pe baza unei chei de potrivire. El realizează o operație de îmbinare (join) între seturi de date, similară cu operația JOIN din SQL.

Exemplu:

```
var productOrders = from p in products
                    join o in orders on p.ProductId equals o.ProductId
                    select new { p.Name, o.OrderDate, o.Quantity };
```

- **aggregate**: Operatorul **aggregate** este utilizat pentru a efectua calcule de agregare asupra datelor, cum ar fi suma, media, minimul sau maximul. El primește o funcție de agregare care specifică operația de agregare dorită.

Exemplu:

```
var totalSales = orders.Sum(o => o.TotalAmount);
```

Pe lângă operatorii menționați mai sus, LINQ oferă și alți operatori utili, cum ar fi **distinct**, **skip**, **take**, **any**, **all**, **count**, **average**, **min**, **max**, etc., care permit efectuarea unor operații comune de interogare și manipulare a datelor.

Acești operatori pot fi combinați și compuși în expresii LINQ complexe pentru a realiza interogări sofisticate asupra datelor. LINQ oferă două sintaxe pentru scrierea interogărilor: sintaxa de interogare (query syntax) și sintaxa metodă (method syntax). Sintaxa de interogare seamănă cu SQL și utilizează cuvinte cheie precum **from**, **where**, **select**, **group by**, în timp ce sintaxa de metode utilizează apeluri directe de metode și expresii lambda.

Indiferent de sintaxa aleasă, expresiile LINQ sunt evaluate tardiv (lazy), ceea ce înseamnă că execuția efectivă a interogării este amânată până când rezultatele sunt necesare. Aceasta permite optimizări și evaluări eficiente ale interogărilor LINQ.

### 3.1.4. Sintaxa de interogare vs. sintaxa metodă

Ambele sintaxe sunt la fel de puternice și expresive, iar alegerea între ele depinde în mare măsură de preferințele personale și de lizibilitatea codului.

*Sintaxa de interogare (query syntax):*

- Sintaxa query seamănă cu SQL și utilizează cuvinte cheie precum **from**, **where**, **select**, **group by**, **orderby** pentru a construi interogări.
- Este o sintaxă declarativă care permite specificarea intenției interogării fără a intra în detaliile de implementare.
- Este ușor de citit și de înțeles, în special pentru dezvoltatorii familiarizați cu SQL.

- Este limitată la un subset de operatori LINQ și nu acceptă toate construcțiile disponibile în sintaxa metodă.

Exemplu:

```
var filteredProducts = from p in products
                        where p.Category == "Electronics"
                        orderby p.Price descending
                        select new { p.Name, p.Price };
```

*Sintaxa metodă (method syntax):*

- Sintaxa metodă utilizează apeluri directe de metode și expresii lambda pentru a construi interogări.
- Este o sintaxă mai generală și mai flexibilă, care permite utilizarea tuturor operatorilor și construcțiilor LINQ.
- Poate fi folosită în situații în care sintaxa de interogare nu este suficientă sau nu este disponibilă.
- Poate fi mai puțin intuitivă pentru dezvoltatorii obișnuiți cu SQL, dar oferă un control mai fin asupra interogărilor.

Exemplu:

```
var filteredProducts = products
    .Where(p => p.Category == "Electronics")
    .OrderByDescending(p => p.Price)
    .Select(p => new { p.Name, p.Price });
```

Ambele sintaxe produc același rezultat și sunt interschimbabile în majoritatea cazurilor. Compilatorul C# convertește sintaxa query în apeluri de metode echivalente în culise, astfel încât performanța și funcționalitatea sunt identice.

Câteva considerente în alegerea între cele două sintaxe disponibile:

- Lizibilitate și familiaritate: Dacă aveți experiență cu SQL și preferați o sintaxă declarativă, sintaxa query poate fi mai ușor de citit și de înțeles. Pe de altă parte, dacă sunteți obișnuit cu programarea funcțională și cu expresiile lambda, sintaxa metodă poate fi mai intuitivă.
- Complexitatea interogărilor: Pentru interogări simple și comune, sintaxa query poate fi mai concisă și mai expresivă. Cu toate acestea, pentru interogări complexe care implică operatori personalizați sau construcții avansate, sintaxa metodă oferă mai multă flexibilitate.
- Consistența stilului de cod: Dacă restul codului dumneavoastră utilizează pe scară largă expresii lambda și apeluri de metode, folosirea sintaxei metodă în interogările LINQ poate menține consistența și uniformitatea stilului de cod.
- Preferințe personale și convenții ale echipei: Alegerea între sintaxa query și sintaxa metodă poate fi influențată și de preferințele personale sau de convențiile

stabilite în cadrul echipei de dezvoltare. Este important să existe consistență și să se urmeze aceleași convenții în întregul cod.

Este posibil și chiar frecvent să se combine ambele sintaxe într-o singură interogare LINQ, în funcție de cerințele și de lizibilitatea codului. De exemplu:

```
var filteredProducts = from p in products
                        where p.Category == "Electronics"
                        select p;

var sortedAndProjectedProducts = filteredProducts
    .OrderByDescending(p => p.Price)
    .Select(p => new { p.Name, p.Price });
```

În acest exemplu, prima parte a interogării utilizează sintaxa query pentru a filtra produsele, iar a doua parte utilizează sintaxa metodă pentru a aplica ordonarea și proiecția asupra rezultatelor filtrate.

## 3.2. LINQ to Objects

LINQ to Objects reprezintă o parte integrantă a LINQ (Language Integrated Query) care permite interogarea și manipularea colecțiilor de obiecte aflate în memorie folosind aceeași sintaxă și aceiași operatori ca și pentru alte surse de date, cum ar fi bazele de date sau documentele XML. Cu LINQ to Objects, puteți aplica operații de filtrare, proiecție, ordonare, grupare și agregare asupra colecțiilor de obiecte, cum ar fi liste, tablouri, dicționare și alte tipuri care implementează interfața `IEnumerable<T>`.

### 3.2.1. Lucrul cu colecții în memorie

LINQ to Objects permite să aplicăm interogări LINQ asupra oricărei colecții care implementează interfața `IEnumerable<T>`, cum ar fi `List<T>`, `Array`, `Dictionary<TKey, TValue>` și multe altele.

Pentru a începe să lucrați cu LINQ to Objects, tot ce trebuie să faceți este să aveți o colecție de obiecte asupra căreia să aplicați interogările. Iată un exemplu simplu:

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
```

În acest caz, avem o listă de numere întregi asupra căreia putem aplica interogări LINQ.

Să presupunem că dorim să filtrăm numerele pare din această listă. Putem face acest lucru folosind operatorul `where` din LINQ:

```
var evenNumbers = from n in numbers
                  where n % 2 == 0
                  select n;
```

Aceasta va returna o nouă colecție care conține doar numerele pare din lista inițială.

De asemenea, putem aplica operații de proiecție asupra colecției folosind operatorul `select`. De exemplu, să presupunem că dorim să înmulțim fiecare număr par cu 2:

```
var doubledEvenNumbers = from n in numbers
                           where n % 2 == 0
                           select n * 2;
```

Rezultatul va fi o nouă colecție în care fiecare număr par din lista inițială este înmulțit cu 2.

### 3.2.2. Proiecția

Proiecția vă permite să transformați elementele dintr-o colecție într-o nouă formă sau să extrageți doar anumite proprietăți ale elementelor. În LINQ, puteți realiza proiecția folosind operatorul `select`. Acesta primește o expresie lambda care definește transformarea aplicată fiecărui element și returnează o nouă colecție care conține elementele transformate.

Iată un exemplu de proiecție care extrage lungimea fiecărui șir dintr-o listă:

```
List<string> words = new List<string> { "apple", "banana", "cherry" };
var lengths = from w in words
              select w.Length;
```

În acest caz, expresia `w.Length` este transformarea aplicată fiecărui element din colecția `words`. Rezultatul este o nouă colecție `lengths` care conține lungimile șirurilor din lista inițială.

De asemenea, se poate utiliza proiecția pentru a crea obiecte noi pe baza elementelor din colecție. De exemplu:

```
var upperLengths = from w in words
                   select new { Word = w.ToUpper(), Length = w.Length };
```

Aici, utilizăm o expresie de tip anonim pentru a crea un nou obiect cu proprietățile `Word` (șirul în majuscule) și `Length` (lungimea șirului) pentru fiecare element din colecție.

Filtrarea și proiecția pot fi combinate într-o singură interogare LINQ pentru a realiza operații complexe de manipulare a datelor. Iată un exemplu:

```
var longWords = from w in words
                where w.Length > 5
                select w.ToUpper();
```

În acest caz, aplicăm mai întâi o filtrare pentru a selecta doar cuvintele cu o lungime mai mare de 5 caractere, apoi aplicăm o proiecție pentru a transforma fiecare cuvânt în majuscule.

### 3.2.3. Ordonare și grupare

#### *Ordonarea datelor cu LINQ:*

Ordonarea vă permite să sortați elementele dintr-o colecție în funcție de una sau mai multe chei. În LINQ, puteți realiza ordonarea folosind operatorii **orderby** și **orderby descending**. Acești operatori primesc o expresie lambda care definește cheia (sau cheile) de sortare și returnează o nouă colecție cu elementele ordonate.

Iată un exemplu de ordonare a unei liste de cuvinte în ordine alfabetică:

```
List<string> words = new List<string> { "apple", "banana", "cherry" };
var sortedWords = from w in words
                  orderby w
                  select w;
```

În acest caz, expresia **orderby w** specifică faptul că dorim să ordonăm cuvintele în funcție de valoarea lor. Rezultatul este o nouă colecție **sortedWords** care conține cuvintele din lista inițială ordonate alfabetic.

De asemenea, puteți specifica o cheie de sortare personalizată folosind o expresie lambda mai complexă. De exemplu, să presupunem că avem o listă de obiecte de tip **Person** și dorim să le ordonăm după vârstă:

```
List<Person> people = new List<Person>
{
    new Person { Name = "Alice", Age = 25 },
    new Person { Name = "Bob", Age = 30 },
    new Person { Name = "Charlie", Age = 20 }
};

var sortedPeople = from p in people
                  orderby p.Age
                  select p;
```

În acest caz, expresia **orderby p.Age** specifică faptul că dorim să ordonăm persoanele în funcție de proprietatea lor **Age**.

Puteți, de asemenea, să specificați multiple chei de sortare și să combinați ordonarea crescătoare și descrescătoare:

```
var sortedPeople = from p in people
                  orderby p.Age descending, p.Name ascending
                  select p;
```

Aici, persoanele sunt ordonate mai întâi descrescător după vârstă, apoi crescător după nume pentru persoanele cu aceeași vârstă.

#### *Gruparea datelor cu LINQ:*

Gruparea vă permite să organizați elementele dintr-o colecție în grupuri pe baza unei chei comune. În LINQ, puteți realiza gruparea folosind operatorul **group by**. Acest operator

primește o expresie lambda care definește cheia de grupare și returnează o nouă colecție de grupuri, unde fiecare grup conține elementele cu aceeași valoare a cheii.

Iată un exemplu de grupare a unei liste de cuvinte în funcție de prima lor literă:

```
List<string> words = new List<string> { "apple", "banana", "cherry", "date",  
"elderberry" };  
  
var groupedWords = from w in words  
                    group w by w[0] into g  
                    select new { FirstLetter = g.Key, Words = g };
```

În acest caz, expresia `group w by w[0]` specifică faptul că dorim să grupăm cuvintele în funcție de prima lor literă. Clauza `into g` creează o variabilă `g` care reprezintă fiecare grup. Rezultatul este o nouă colecție `groupedWords` de grupuri, unde fiecare grup are o proprietate `FirstLetter` (prima literă) și o proprietate `Words` (cuvintele din acel grup).

Puteți accesa grupurile rezultate și elementele lor astfel:

```
foreach (var group in groupedWords)  
{  
    Console.WriteLine($"Words starting with {group.FirstLetter}:");  
    foreach (var word in group.Words)  
    {  
        Console.WriteLine(word);  
    }  
}
```

Acest cod va afișa:

```
Words starting with a:  
apple  
Words starting with b:  
banana  
Words starting with c:  
cherry  
Words starting with d:  
date  
Words starting with e:  
elderberry
```

Gruparea este deosebit de utilă atunci când doriți să efectuați operații de agregare pe fiecare grup, cum ar fi numărarea elementelor sau calcularea sumei valorilor dintr-un grup.

### 3.2.4. Operații de agregare

Operațiile de agregare sunt un aspect important al LINQ to Objects, permițându-vă să efectuați calcule pe seturi de date și să extrageți informații sumare dintr-o colecție. LINQ oferă o serie de metode de agregare predefinite, cum ar fi `Sum`, `Average`, `Min`, `Max` și `Count`, care vă permit să realizați rapid operații comune de agregare.



Să analizăm câteva exemple de operații de agregare folosind LINQ:

### *Suma valorilor dintr-o colecție*

Pentru a calcula suma valorilor dintr-o colecție, puteți utiliza metoda **Sum**. Iată un exemplu:

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
int sum = numbers.Sum();
Console.WriteLine($"The sum is: {sum}");
```

În acest caz, metoda **Sum** este apelată pe colecția **numbers**, iar rezultatul (suma valorilor) este stocat în variabila **sum**. Rezultatul va fi:

```
The sum is: 15
```

De asemenea, puteți calcula suma unei proprietăți specifice a elementelor dintr-o colecție. De exemplu:

```
List<Person> people = new List<Person>
{
    new Person { Name = "Alice", Age = 25 },
    new Person { Name = "Bob", Age = 30 },
    new Person { Name = "Charlie", Age = 20 }
};
int totalAge = people.Sum(p => p.Age);
Console.WriteLine($"The total age is: {totalAge}");
```

Aici, folosim o expresie lambda **p => p.Age** pentru a specifica proprietatea **Age** pe care dorim să o însumăm.

### *Media valorilor dintr-o colecție*

Pentru a calcula media valorilor dintr-o colecție, puteți utiliza metoda **Average**.

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
double average = numbers.Average();
Console.WriteLine($"The average is: {average}");
```

Rezultatul va fi:

```
The average is: 3
```

La fel ca și **Sum**, puteți calcula media unei proprietăți specifice a elementelor folosind o expresie lambda.

### Valoarea minimă și maximă dintr-o colecție

Pentru a găsi valoarea minimă sau maximă dintr-o colecție, puteți utiliza metodele **Min** și **Max**.

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };  
  
int min = numbers.Min();  
int max = numbers.Max();  
  
Console.WriteLine($"The minimum value is: {min}");  
Console.WriteLine($"The maximum value is: {max}");
```

Rezultatul va fi:

```
The minimum value is: 1  
The maximum value is: 5
```

De asemenea, puteți găsi valoarea minimă sau maximă a unei proprietăți specifice a elementelor folosind o expresie lambda.

### Numărul de elemente dintr-o colecție

Pentru a număra elementele dintr-o colecție, puteți utiliza metoda **Count**.

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };  
  
int count = numbers.Count();  
  
Console.WriteLine($"The count is: {count}");
```

Rezultatul va fi:

```
The count is: 5
```

De asemenea, puteți număra elementele care îndeplinesc o anumită condiție folosind o expresie lambda:

```
int evenCount = numbers.Count(n => n % 2 == 0);  
  
Console.WriteLine($"The count of even numbers is: {evenCount}");
```

Aici, expresia lambda `n => n % 2 == 0` specifică condiția (numărul este par) pentru a număra elementele.

În plus față de aceste operații de agregare de bază, LINQ oferă și alte metode de agregare, cum ar fi **Aggregate**, care vă permite să efectuați calcule personalizate pe o colecție folosind o funcție de acumulare.

## 3.3. Expresii Lambda și Delegates

Expresiile lambda și delegates sunt două concepte fundamentale în C# care joacă un rol important în programarea funcțională și în utilizarea LINQ (Language Integrated Query).

Aceste construcții permit definirea și manipularea funcțiilor ca valori de prim rang, oferind un mod concis și expresiv de a reprezenta logica și comportamentul în cod.

### 3.3.1. Conceptul de delegate în .NET

Delegates sunt un concept fundamental în .NET și joacă un rol esențial în programarea C#. Un delegat este un tip de date care reprezintă o referință la o metodă, cu alte cuvinte un delegat este un tip care poate ține o referință la o funcție, permițând invocarea indirectă a acesteia.

Delegates sunt asemănători cu pointerii la funcții din alte limbaje de programare, dar oferă un mecanism mai sigur și mai flexibil. Aceștia permit tratarea funcțiilor ca entități de prim rang, ceea ce înseamnă că funcțiile pot fi atribuite variabilelor, pasate ca argumente și returnate din alte funcții.

În .NET, delegates sunt definiți folosind cuvântul cheie `delegate`. Iată un exemplu de definiție a unui delegat:

```
public delegate int BinaryOperation(int x, int y);
```

În acest caz, `BinaryOperation` este un tip delegat care reprezintă o funcție care primește doi parametri întregi (`x` și `y`) și returnează un întreg. Putem crea o instanță a acestui delegat și o putem atribui unei metode compatibile ca referință:

```
public static int Add(int x, int y)
{
    return x + y;
}

BinaryOperation op = Add;
```

Aici, `op` este o variabilă de tip `BinaryOperation` care ține o referință la metoda `Add`. Putem invoca delegatul `op` ca și cum am invoca direct metoda `Add`:

```
int result = op(3, 4); // result va fi 7
```

Delegates sunt deosebit de utili pentru a permite comunicarea între componente în mod decuplat și flexibil. Aceștia stau la baza evenimentelor în C#, permițând definirea și gestionarea callback-urilor în mod elegant.

De asemenea, delegates sunt esențiali pentru utilizarea LINQ, deoarece multe metode LINQ, cum ar fi `Where`, `Select`, `OrderBy` etc., primesc delegates ca parametri pentru a specifica logica de filtrare, proiecție, sortare, etc.

.NET oferă câteva tipuri predefinite de delegates, cum ar fi `Func<>` și `Action<>`, care acoperă majoritatea scenariilor comune. `Func<>` reprezintă un delegat care returnează o valoare, în timp ce `Action<>` reprezintă un delegat care face o acțiune și nu returnează o valoare (similar cu o funcție void).

Iată un exemplu de utilizare a unui delegat `Func<>`:

```
Func<int, int, int> multiply = (x, y) => x * y;  
int result = multiply(3, 4); // result va fi 12
```

În acest caz, `multiply` este un delegat `Func<int, int, int>` care reprezintă o funcție care primește doi întregi și returnează un întreg. Folosim o expresie lambda `(x, y) => x * y` pentru a defini logica delegatului inline.

### 3.3.2. Sintaxa și utilizarea expresiilor lambda

Expresiile lambda sunt o caracteristică puternică introdusă în C# 3.0 care permite definirea de funcții anonime inline într-un mod concis și expresiv. Acestea oferă o sintaxă simplificată pentru crearea de delegates sau de arbori de expresie, făcând codul mai lizibil și mai ușor de întreținut.

O expresie lambda este formată din două părți: lista de parametri și corpul expresiei. Sintaxa generală a unei expresii lambda este următoarea:

```
(parametri) => expresie
```

sau

```
(parametri) => { instrucțiuni }
```

Lista de parametri specifică intrările funcției și poate conține zero, unul sau mai mulți parametri. Dacă există un singur parametru, parantezele pot fi omise. Operatorul `=>`, cunoscut sub numele de "fat arrow" sau "goes to", separă lista de parametri de corpul expresiei.

Corpul expresiei poate fi o singură expresie sau un bloc de instrucțiuni. Dacă corpul este o singură expresie, valoarea acesteia este returnată implicit. Dacă corpul este un bloc de instrucțiuni, trebuie să folosiți cuvântul cheie `return` pentru a returna o valoare în mod explicit.

Iată câteva exemple de expresii lambda:

```
// Expresie lambda cu un singur parametru  
x => x * x  
  
// Expresie lambda cu doi parametri  
(x, y) => x + y  
  
// Expresie lambda cu un bloc de instrucțiuni  
(x, y) => {  
    int sum = x + y;  
    return sum * 2;  
}
```

Expresiile lambda sunt adesea utilizate în combinație cu delegații sau cu metodele LINQ. De exemplu, putem utiliza o expresie lambda pentru a defini logica unui delegat `Func<>`:

```
Func<int, int> square = x => x * x;
int result = square(5); // result va fi 25
```

De asemenea, putem utiliza expresii lambda cu metode LINQ pentru a specifica logica de filtrare, proiecție, sortare etc.:

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
List<int> evenNumbers = numbers.Where(x => x % 2 == 0).ToList();
```

Aici, expresia lambda `x => x % 2 == 0` este utilizată cu metoda `Where` pentru a filtra numerele pare din lista `numbers`.

Expresiile lambda pot accesa variabile din domeniul lor lexical, permițând capturarea și utilizarea variabilelor din codul înconjurător. Această caracteristică este cunoscută sub numele de "închidere" (closure) și este deosebit de utilă pentru a crea funcții care "memorează" starea.

```
int factor = 2;
Func<int, int> multiplier = x => x * factor;
int result = multiplier(5); // result va fi 10
```

În acest caz, expresia lambda `x => x * factor` accesează variabila `factor` din domeniul lexical în care este definită.

### 3.3.3. Closure și variabile capturate

Closure-urile (închiderile) sunt un concept fundamental în programarea funcțională și sunt strâns legate de utilizarea expresiilor lambda în C#. Un closure este o funcție care captează și reține accesul la variabilele din domeniul lexical în care este definită, chiar și după ce execuția a ieșit din acel domeniu.

Cu alte cuvinte, un closure permite unei funcții să "învăluie" sau să "închidă" variabilele din mediul înconjurător, permițându-i să acceseze și să modifice aceste variabile chiar și atunci când funcția este invocată într-un alt context.

Să analizăm un exemplu pentru a ilustra conceptul de closure:

```
Func<int, int> CreateMultiplier(int factor)
{
    return x => x * factor;
}

Func<int, int> doubleIt = CreateMultiplier(2);
Func<int, int> tripleIt = CreateMultiplier(3);

int result1 = doubleIt(5); // result1 va fi 10
int result2 = tripleIt(5); // result2 va fi 15
```

În acest exemplu, funcția `CreateMultiplier` returnează o expresie lambda care înmulțește un parametru `x` cu `factor`. Variabila `factor` este o variabilă capturată de către closure-ul creat de expresia lambda.

Când apelăm `CreateMultiplier(2)`, se creează un closure care captează valoarea `2` pentru `factor`. Acest closure este atribuit variabilei `doubleIt`. Similar, când apelăm `CreateMultiplier(3)`, se creează un alt closure care captează valoarea `3` pentru `factor`, și acest closure este atribuit variabilei `tripleIt`.

Acum, când invocăm `doubleIt(5)`, expresia lambda capturată de `doubleIt` are acces la valoarea `2` a variabilei `factor`, chiar dacă execuția a ieșit din funcția `CreateMultiplier`. Similar, când invocăm `tripleIt(5)`, expresia lambda capturată de `tripleIt` are acces la valoarea `3` a variabilei `factor`.

Closure-urile sunt create implicit ori de câte ori o expresie lambda captează o variabilă din domeniul lexical înconjurător. Aceasta permite expresiilor lambda să "memorizeze" starea și să o utilizeze ulterior, chiar și atunci când sunt invocate în contexte diferite.

Este important de remarcat că variabilele capturate de un closure sunt partajate între toate instanțele closure-ului create de aceeași funcție. Aceasta înseamnă că modificările aduse variabilelor capturate de un closure vor fi vizibile pentru toate instanțele closure-ului.

```
Func<int> CreateCounter()
{
    int count = 0;
    return () => count++;
}

Func<int> counter1 = CreateCounter();
Func<int> counter2 = CreateCounter();

int value1 = counter1(); // value1 va fi 0
int value2 = counter1(); // value2 va fi 1
int value3 = counter2(); // value3 va fi 0
```

În acest exemplu, funcția `CreateCounter` returnează o expresie lambda care incrementează și returnează valoarea variabilei `count`. Fiecare apel al funcției `CreateCounter` creează un nou closure cu propria instanță a variabilei `count`.

Când invocăm `counter1()` de două ori, acesta accesează și modifică aceeași instanță capturată a variabilei `count`. Cu toate acestea, când invocăm `counter2()`, acesta accesează o instanță diferită a variabilei `count`, care este inițializată cu valoarea `0`.

### 3.3.4. Funcții anonime

Funcțiile anonime sunt un concept strâns legat de expresiile lambda și delegates în C#. O funcție anonimă este o funcție fără un nume explicit, care poate fi definită inline, în locul în care este necesară. Funcțiile anonime sunt adesea utilizate pentru a crea delegați sau pentru a fi pasate ca argumente către alte funcții, cum ar fi metodele LINQ.

Înainte de introducerea expresiilor lambda în C# 3.0, funcțiile anonime erau definite folosind o sintaxă mai detaliată, utilizând cuvântul cheie `delegate`. Iată un exemplu de funcție anonimă utilizând sintaxa pre-C# 3.0:

```
Func<int, int> square = delegate(int x) { return x * x; };
int result = square(5); // result va fi 25
```

În acest caz, funcția anonimă `delegate(int x) { return x * x; }` este atribuită delegatului `square` de tip `Func<int, int>`. Funcția anonimă primește un parametru `x` și returnează pătratul său.

Cu introducerea expresiilor lambda în C# 3.0, definirea funcțiilor anonime a devenit mult mai concisă și mai expresivă. Expresiile lambda sunt acum modul preferat de a crea funcții anonime în C#. Iată același exemplu folosind o expresie lambda:

```
Func<int, int> square = x => x * x;
int result = square(5); // result va fi 25
```

Expresia lambda `x => x * x` este echivalentă cu funcția anonimă `delegate(int x) { return x * x; }`, dar are o sintaxă mai compactă și mai lizibilă.

Funcțiile anonime sunt deosebit de utile în combinație cu metodele LINQ, deoarece acestea așteaptă adesea delegați sau expresii lambda ca argumente. Iată un exemplu de utilizare a unei funcții anonime cu metoda `Where` din LINQ:

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
List<int> evenNumbers = numbers.Where(delegate(int x) { return x % 2 == 0;
}).ToList();
```

În acest caz, funcția anonimă `delegate(int x) { return x % 2 == 0; }` este pasată ca argument metodei `Where` pentru a filtra numerele pare din lista `numbers`.

Cu toate acestea, utilizarea unei expresii lambda în acest caz ar face codul mai concis și mai lizibil:

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
List<int> evenNumbers = numbers.Where(x => x % 2 == 0).ToList();
```

Cu toate că expresiile lambda au devenit modul preferat de a crea funcții anonime în C#, este important să înțelegem și sintaxa pre-C# 3.0 pentru a putea citi și înțelege codul mai vechi și pentru a aprecia evoluția limbajului de-a lungul timpului.

## 3.4. Operatori LINQ avansați

### 3.4.1. Join și GroupJoin

Join și GroupJoin sunt doi operatori puternici în LINQ care ne permit să combinăm date din mai multe surse pe baza unei chei comune.

Operatorul Join efectuează o operație de îmbinare a două secvențe pe baza unei chei de potrivire. Este similar cu operația de îmbinare (JOIN) din bazele de date relaționale. Join returnează o secvență de obiecte care conțin elemente din ambele secvențe de intrare, unde elementele îndeplinesc condiția de potrivire specificată.

Sintaxa generală a operatorului Join este:

```
var rezultat = secventa1.Join(  
    secventa2,  
    element1 => cheie1,  
    element2 => cheie2,  
    (element1, element2) => proiectie  
);
```

- **secventa1** și **secventa2** sunt secvențele de intrare pe care dorim să le îmbinăm.
- **cheie1** și **cheie2** sunt expresiile de selecție a cheii pentru fiecare element din secvențele respective. Aceste expresii specifică proprietatea sau valoarea pe baza căreia se face îmbinarea.
- **proiectie** este o funcție care definește cum sunt combinate elementele potrivite din cele două secvențe pentru a crea elementele rezultate.

Să analizăm un exemplu concret. Să presupunem că avem două liste: una cu studenți și alta cu notele lor:

```
var studenti = new[]  
{  
    new { Id = 1, Nume = "Alice" },  
    new { Id = 2, Nume = "Bob" },  
    new { Id = 3, Nume = "Charlie" }  
};  
  
var note = new[]  
{  
    new { StudentId = 1, Curs = "Matematica", Nota = 8.5 },  
    new { StudentId = 1, Curs = "Informatica", Nota = 9.0 },  
    new { StudentId = 2, Curs = "Matematica", Nota = 7.5 },  
    new { StudentId = 3, Curs = "Informatica", Nota = 8.0 }  
};
```

Putem utiliza operatorul Join pentru a îmbina aceste liste și a obține notele pentru fiecare student:

```
var rezultat = studenti.Join(  
    note,  
    student => student.Id,  
    nota => nota.StudentId,  
    (student, nota) => new { Student = student.Nume, Curs = nota.Curs, Nota =  
        nota.Nota }  
);
```



În acest exemplu, folosim proprietatea `Id` din lista `studenti` și proprietatea `StudentId` din lista `note` drept chei de potrivire. Rezultatul va conține obiecte anonime cu proprietățile `Student` (numele studentului), `Curs` și `Nota` pentru fiecare pereche student-notă potrivită.

Operatorul `GroupJoin` este similar cu `Join`, dar în loc să producă o îmbinare simplă a elementelor potrivite, acesta grupează elementele din a doua secvență în funcție de cheia de potrivire. Rezultatul unui `GroupJoin` este o secvență de obiecte care conțin cheia și o secvență de elemente potrivite pentru acea cheie.

Sintaxa generală a operatorului `GroupJoin` este:

```
var rezultat = secventa1.GroupJoin(
    secventa2,
    element1 => cheie1,
    element2 => cheie2,
    (element1, elemente2) => proiectie
);
```

- `secventa1` și `secventa2` sunt secvențele de intrare.
- `cheie1` și `cheie2` sunt expresiile de selecție a cheii pentru fiecare element din secvențele respective.
- `proiectie` este o funcție care definește cum sunt combinate elementele din prima secvență cu grupurile de elemente potrivite din a doua secvență.

Să vedem un exemplu de utilizare a operatorului `GroupJoin` cu listele de studenți și note de mai sus:

```
var rezultat = studenti.GroupJoin(
    note,
    student => student.Id,
    nota => nota.StudentId,
    (student, noteStudent) => new { Student = student.Nume, Note =
noteStudent }
);
```

În acest caz, rezultatul va conține obiecte anonime cu proprietățile `Student` (numele studentului) și `Note` (o secvență de obiecte notă asociate studentului respectiv).

### 3.4.2. SelectMany și aplatizare

Operatorul `SelectMany` din LINQ este folosit pentru a transforma și a aplatiza (flatten) secvențe de secvențe într-o singură secvență. Acesta este deosebit de util atunci când lucrăm cu colecții ierarhice sau cu surse de date imbricate și dorim să le "aplatizăm" într-o singură secvență.

Să analizăm sintaxa generală a operatorului `SelectMany`:

```
var rezultat = secventa.SelectMany(
    element => selector(element)
);
```

- **secventa** este secvența de intrare pe care dorim să o transformăm și să o aplatizăm.
- **selector** este o funcție care primește fiecare element din secvența de intrare și returnează o nouă secvență. Această funcție determină modul în care fiecare element este transformat și aplatizat.

SelectMany aplică funcția **selector** la fiecare element din secvența de intrare și aplatizează secvențele rezultate într-o singură secvență. Cu alte cuvinte, ia fiecare element din secvența de intrare, aplică **selector** pentru a obține o secvență intermediară și apoi concatenează toate aceste secvențe intermediare într-o singură secvență finală.

Să vedem un exemplu concret. Să presupunem că avem o listă de obiecte **Student**, unde fiecare student are o proprietate **Cursuri** care reprezintă o listă de cursuri la care este înscris:

```
class Student
{
    public string Nume { get; set; }
    public List<string> Cursuri { get; set; }
}

var studenti = new[]
{
    new Student { Nume = "Alice", Cursuri = new List<string> { "Matematica", "Informatica" } },
    new Student { Nume = "Bob", Cursuri = new List<string> { "Fizica", "Chimie" } },
    new Student { Nume = "Charlie", Cursuri = new List<string> { "Istorie", "Geografie" } }
};
```

Dacă dorim să obținem o singură secvență cu toate cursurile la care sunt înscrși studenții, putem folosi **SelectMany** în felul următor:

```
var toateCursurile = studenti.SelectMany(student => student.Cursuri);
```

În acest caz, **selector** este o expresie lambda care primește un obiect **Student** și returnează secvența de cursuri asociată acelui student (**student.Cursuri**). **SelectMany** va aplica această funcție fiecărui student din lista **studenti** și va aplatiza secvențele de cursuri rezultate într-o singură secvență.

Rezultatul va fi o secvență care conține toate cursurile la care sunt înscrși studenții:

```
{ "Matematica", "Informatica", "Fizica", "Chimie", "Istorie", "Geografie" }
```

**SelectMany** este deosebit de util atunci când lucrăm cu surse de date imbricate, cum ar fi XML sau JSON, unde avem elemente sau proprietăți care conțin la rândul lor secvențe. De asemenea, poate fi folosit în combinație cu alți operatori LINQ, precum **Where** sau **Select**, pentru a realiza interogări complexe și transformări ale datelor.

### 3.4.3. Operatori de mulțimi

LINQ oferă o serie de operatori care ne permit să efectuăm operații de mulțimi asupra secvențelor, similare cu operațiile din teoria mulțimilor. Acești operatori sunt utili atunci când dorim să comparăm, să combinăm sau să transformăm secvențe pe baza apartenenței elementelor la diferite mulțimi.

#### *Distinct:*

Operatorul **Distinct** returnează o secvență care conține elementele distincte din secvența de intrare, eliminând duplicatele. Elementele sunt comparate folosind egalitatea implicită a obiectelor (metoda **Equals**).

```
var numere = new[] { 1, 2, 2, 3, 4, 4, 5 };
var distincte = numere.Distinct();
// Rezultat: { 1, 2, 3, 4, 5 }
```

#### *Union:*

Operatorul **Union** returnează o secvență care conține toate elementele distincte din două secvențe, fără a include duplicatele. Elementele sunt comparate folosind egalitatea implicită a obiectelor.

```
var secventa1 = new[] { 1, 2, 3 };
var secventa2 = new[] { 2, 3, 4, 5 };
var reuniune = secventa1.Union(secventa2);
// Rezultat: { 1, 2, 3, 4, 5 }
```

#### *Intersect:*

Operatorul **Intersect** returnează o secvență care conține elementele comune din două secvențe, păstrând doar o singură apariție a fiecărui element comun. Elementele sunt comparate folosind egalitatea implicită a obiectelor.

```
var secventa1 = new[] { 1, 2, 3, 4 };
var secventa2 = new[] { 3, 4, 5, 6 };
var intersectie = secventa1.Intersect(secventa2);
// Rezultat: { 3, 4 }
```

#### *Except:*

Operatorul **Except** returnează o secvență care conține elementele din prima secvență care nu apar în a doua secvență. Cu alte cuvinte, rezultatul este diferența dintre prima secvență și a doua secvență. Elementele sunt comparate folosind egalitatea implicită a obiectelor.

```
var secventa1 = new[] { 1, 2, 3, 4, 5 };
var secventa2 = new[] { 3, 4, 5, 6, 7 };
var diferenta = secventa1.Except(secventa2);
// Rezultat: { 1, 2 }
```

Acești operatori de mulțimi pot fi combinați cu alți operatori LINQ pentru a realiza interogări și transformări complexe ale datelor. De exemplu, putem folosi **Union** pentru a combina rezultatele mai multor interogări, **Intersect** pentru a găsi elementele comune între mai multe secvențe sau **Except** pentru a exclude anumite elemente dintr-o secvență pe baza unei alte secvențe.

### 3.4.4. Operatori de cuantificare

LINQ oferă operatori de cuantificare, care ne permit să verificăm dacă anumite condiții sunt îndeplinite de către elementele unei secvențe. Acești operatori returnează o valoare booleană (true sau false) în funcție de îndeplinirea condițiilor specificate.

#### Any:

Operatorul **Any** verifică dacă există cel puțin un element în secvență care satisface o anumită condiție. Returnează **true** dacă există un astfel de element și **false** în caz contrar.

```
var numere = new[] { 1, 2, 3, 4, 5 };  
bool existaPare = numere.Any(n => n % 2 == 0);  
// Rezultat: true (deoarece există numere pare în secvență)
```

De asemenea, **Any** poate fi folosit fără o condiție pentru a verifica dacă secvența conține cel puțin un element:

```
bool secventaGoala = numere.Any();  
// Rezultat: true (deoarece secvența nu este goală)
```

#### All:

Operatorul **All** verifică dacă toate elementele dintr-o secvență satisfac o anumită condiție. Returnează **true** dacă toate elementele îndeplinesc condiția și **false** în caz contrar.

```
var numere = new[] { 2, 4, 6, 8, 10 };  
bool toatePare = numere.All(n => n % 2 == 0);  
// Rezultat: true (deoarece toate numerele din secvență sunt pare)
```

#### Contains:

Operatorul **Contains** verifică dacă o secvență conține un anumit element. Returnează **true** dacă elementul există în secvență și **false** în caz contrar. Elementele sunt comparate folosind egalitatea implicită a obiectelor.

```
var fructe = new[] { "măr", "banană", "portocală" };  
bool contineMar = fructe.Contains("măr");  
// Rezultat: true (deoarece "măr" există în secvență)
```

#### SequenceEqual:

Operatorul **SequenceEqual** verifică dacă două secvențe sunt egale, adică dacă conțin aceleași elemente în aceeași ordine. Returnează **true** dacă secvențele sunt egale și

`false` în caz contrar. Elementele sunt comparate folosind egalitatea implicită a obiectelor.

```
var secventa1 = new[] { 1, 2, 3 };
var secventa2 = new[] { 1, 2, 3 };
var secventa3 = new[] { 3, 2, 1 };
bool egale1 = secventa1.SequenceEqual(secventa2);
// Rezultat: true (deoarece secvențele conțin aceleași elemente în aceeași ordine)
bool egale2 = secventa1.SequenceEqual(secventa3);
// Rezultat: false (deoarece secvențele conțin aceleași elemente, dar în ordine diferită)
```

Acești operatori de cuantificare ne permit să interogăm secvențele și să verificăm anumite condiții asupra elementelor lor. Ei sunt deosebit de utili în combinație cu alți operatori LINQ pentru a filtra, a valida sau a prelucra datele în funcție de criterii specifice.

De exemplu, putem folosi `Any` pentru a verifica dacă există elemente care satisfac o anumită condiție înainte de a efectua o operație ulterioară, `All` pentru a ne asigura că toate elementele dintr-o secvență respectă o anumită regulă sau `Contains` pentru a verifica prezența unui anumit element într-o secvență.

Operatorii de cuantificare din LINQ ne oferă un mod expresiv și concis de a interoga și a valida secvențele de date, permițându-ne să scriem cod mai clar, mai ușor de înțeles și mai eficient.

## 3.5. LINQ to XML și JSON

### 3.5.1. Manipularea documentelor XML

LINQ to XML oferă un model obiectual intuitiv și expresiv pentru manipularea documentelor XML în C#. Această bibliotecă ne permite să citim, să modificăm și să creăm documente XML într-un mod convenabil și eficient.

#### *Citirea unui document XML:*

Pentru a citi un document XML, putem folosi clasa `XDocument`. Aceasta reprezintă întregul document XML și oferă metode pentru a încărca un document dintr-un fișier sau dintr-un șir de caractere.

```
XDocument document = XDocument.Load("exemplu.xml");
```

sau

```
string xml = "<rădăcină><element>valoare</element></rădăcină>";
XDocument document = XDocument.Parse(xml);
```

#### *Navigarea în documentul XML:*

După încărcarea documentului, putem naviga în structura sa folosind proprietăți și metode ale claselor `XDocument`, `XElement` și `XAttribute`.

```
XElement radacina = document.Root;  
IEnumerable<XElement> elemente = radacina.Elements();  
XElement primul = radacina.Element("element");  
string valoare = primul.Value;
```

#### Modificarea documentului XML:

LINQ to XML oferă metode pentru a modifica documentul XML, cum ar fi adăugarea, ștergerea sau modificarea elementelor și atributelor.

```
XElement elementNou = new XElement("elementNou", "valoare");  
radacina.Add(elementNou);  
primul.SetValue("valoare nouă");  
XAttribute atributNou = new XAttribute("atribut", "valoare");  
primul.Add(atributNou);
```

#### Crearea unui document XML:

Putem crea un nou document XML de la zero folosind constructorii claselor `XDocument` și `XElement`.

```
XDocument documentNou = new XDocument(  
    new XElement("rădăcină",  
        new XElement("element1", "valoare1"),  
        new XElement("element2", "valoare2")  
    )  
);
```

#### Salvarea documentului XML:

După modificarea documentului XML, putem salva rezultatul înapoi într-un fișier sau îl putem obține ca un șir de caractere.

```
document.Save("exemplu_modificat.xml");  
string xmlModificat = document.ToString();
```

## 3.5.2. Interogări pe structuri XML

LINQ to XML ne permite să efectuăm interogări complexe pe documente XML folosind sintaxa LINQ familiară. Putem utiliza operatorii LINQ, cum ar fi `Select`, `Where`, `OrderBy` și alții, pentru a extrage, filtra și proiecta datele din documentele XML.

#### Extragerea elementelor pe baza numelui:

```
XDocument document = XDocument.Load("exemplu.xml");  
IEnumerable<XElement> elemente = from el in document.Descendants("element")  
                                select el;
```

Această interogare extrage toate elementele cu numele "element" din documentul XML, indiferent de nivelul lor de adâncime.

#### Filtrarea elementelor pe baza unei condiții:

```
IEnumerable<XElement> elementeFiltrate = from el in  
document.Descendants("element")
```

```
> 10                                where (int)el.Attribute("atribut")
                                     select el;
```

Această interogare extrage elementele cu numele `element` care au valoarea atributului `atribut` mai mare decât 10.

*Proiectarea datelor într-un nou format:*

```
var rezultat = from el in document.Descendants("element")
                select new
                {
                    Nume = el.Element("nume").Value,
                    Valoare = (int)el.Element("valoare")
                };
```

Această interogare extrage elementele cu numele `element` și proiectează datele lor într-un nou format anonim, cu proprietăți `Nume` și `Valoare`.

*Ordonarea rezultatelor:*

```
var rezultatOrdonat = from el in document.Descendants("element")
                       orderby (string)el.Attribute("atribut")
                       select el;
```

Această interogare extrage elementele cu numele `element` și le ordonează în funcție de valoarea atributului `atribut`.

*Gruparea rezultatelor:*

```
var rezultatGrupat = from el in document.Descendants("element")
                      group el by (string)el.Attribute("categoria") into
grupare
                      select new
                      {
                          Categoria = grupare.Key,
                          Elemente = grupare
                      };
```

Această interogare extrage elementele cu numele `element`, le grupează în funcție de valoarea atributului `categoria` și proiectează rezultatul într-un nou format anonim, cu proprietăți `Categoria` și `Elemente`.

Aceste exemple ilustrează doar câteva dintre posibilitățile de interogare oferite de LINQ to XML. Putem combina operatorii LINQ, putem naviga în ierarhia documentului XML, putem accesa attribute și elemente folosind sintaxa familiară LINQ.

LINQ to XML face interogarea documentelor XML ușoară și expresivă, permițându-ne să extragem și să transformăm datele XML într-un mod concis și puternic, folosind aceeași

sintaxă LINQ pe care o folosim pentru alte surse de date, cum ar fi colecțiile în memorie sau bazele de date.

### 3.5.3. Parsare și generare JSON

Pe lângă manipularea documentelor XML, LINQ to XML oferă, de asemenea, suport pentru parsarea și generarea datelor în format JSON (JavaScript Object Notation). JSON este un format popular de schimb de date, utilizat pe scară largă în aplicațiile web și în comunicarea între sistemele distribuite.

#### *Parsarea JSON:*

Pentru a parsă un șir JSON într-un obiect LINQ to XML, putem folosi metoda `Parse` a clasei `XDocument`, specificând că formatul este JSON.

```
string json = "{ \"nume\": \"John\", \"varsta\": 30, \"oras\": \"New York\" }";  
XDocument document = XDocument.Parse(json);
```

După parsare, putem accesa datele folosind aceleași metode și proprietăți ca și pentru XML.

```
XElement radacina = document.Root;  
string nume = radacina.Element("nume").Value;  
int varsta = (int)radacina.Element("varsta");  
string oras = radacina.Element("oras").Value;
```

#### *Generarea JSON:*

Pentru a genera un șir JSON dintr-un obiect LINQ to XML, putem folosi metoda `ToString` cu parametrul `SaveOptions.None`.

```
XDocument documentNou = new XDocument(  
    new XElement("rădăcină",  
        new XElement("nume", "John"),  
        new XElement("varsta", 30),  
        new XElement("oras", "New York")  
    )  
);  
string jsonGenerat = documentNou.ToString(SaveOptions.None);
```

Rezultatul va fi un șir JSON valid:

```
{"nume": "John", "varsta": 30, "oras": "New York"}
```

### 3.5.4. Transformări între formate

LINQ to XML ne permite să efectuăm transformări între diferite formate de date, cum ar fi XML și JSON, folosind aceeași abordare bazată pe obiecte și interogări LINQ.

Să vedem un exemplu de transformare dintr-un format XML într-un format JSON:

```
// XML-ul inițial
```



```
string xml = @"<persoane>
    <persoana>
        <nume>John Doe</nume>
        <email>john@example.com</email>
    </persoana>
    <persoana>
        <nume>Jane Smith</nume>
        <email>jane@example.com</email>
    </persoana>
</persoane>";

// Parsarea XML-ului
XDocument documentXML = XDocument.Parse(xml);

// Transformarea în JSON
var jsonResult = new XElement("root",
    from pers in documentXML.Descendants("persoana")
    select new XElement("persoana",
        new XElement("nume", pers.Element("nume").Value),
        new XElement("email", pers.Element("email").Value)
    )
);

// Generarea JSON-ului
string json = jsonResult.ToString(SaveOptions.None);

Console.WriteLine(json);
```

În acest exemplu:

- Avem un șir XML care reprezintă o listă de persoane, fiecare persoană având un nume și un email.
- Folosim `XDocument.Parse` pentru a parse șirul XML într-un obiect `XDocument`.
- Folosind o expresie LINQ, transformăm elementele XML în elemente JSON. Pentru fiecare element `persoana` din XML, creăm un nou element `persoana` în JSON, cu sub-elemente `nume` și `email` extrase din XML.
- Rezultatul transformării este stocat într-un obiect `Element` numit `jsonResult`.
- Folosim metoda `ToString` cu `SaveOptions.None` pentru a genera un șir JSON din obiectul `jsonResult`.
- Afișăm JSON-ul rezultat în consolă.

Rezultatul va fi:

```
{"persoana":[{"nume":"John Doe","email":"john@example.com"}, {"nume":"Jane Smith","email":"jane@example.com"}]}
```

Acest exemplu demonstrează cum putem utiliza LINQ to XML pentru a efectua transformări între formate de date. Putem utiliza aceeași abordare pentru a transforma JSON înapoi în XML sau pentru a efectua transformări mai complexe, în funcție de nevoile noastre.

## 4. Bazele limbajului F#

F# este un limbaj de programare funcțional, dezvoltat de Microsoft Research în 2005. F# este puternic influențat de limbajele funcționale ML (Meta Language) și OCaml, combinând programarea funcțională cu caracteristicile platformei .NET.

F# este un limbaj funcțional static tipizat, cu un sistem de tipuri puternic și inferență de tip. Limbajul pune accentul pe imutabilitate, funcții ca valori de prim rang și programare expresivă. F# oferă, de asemenea, caracteristici precum potrivire de modele (pattern matching), înregistrări (records), discriminated unions și un sistem puternic de manipulare a erorilor bazat pe tipul Result.

F# este în primul rând un limbaj funcțional, încurajând programatorii să scrie cod concis, expresiv și ușor de raționat. Cu toate acestea, F# suportă și programarea orientată pe obiecte și programarea imperativă, permițând integrarea cu bibliotecile .NET existente și utilizarea stilului de programare cel mai potrivit pentru problema în cauză.

F# este utilizat într-o varietate de domenii, cum ar fi:

- Analiză de date și calcul științific
- Machine learning și inteligență artificială
- Dezvoltarea de aplicații web și servicii backend
- Dezvoltarea jocurilor (folosind motorul Unity)
- Dezvoltarea de aplicații financiare și de asigurări

În capitolul de față vom explora bazele limbajului F#, acoperind concepte precum programarea funcțională, tipuri de date, funcții, pattern matching, colecții, operatori și tipuri de date algebrice. Veți învăța cum să scrieți cod F# expresiv și concis, cum să utilizați caracteristicile cheie ale limbajului și cum să vă organizați codul în module și namespace-uri. De asemenea, vom discuta despre gestionarea erorilor în stil funcțional și despre interoperabilitatea cu C# și bibliotecile .NET existente.

### 4.1. Introducere în programarea funcțională

Programarea funcțională este o paradigmă de programare care tratează calculul ca evaluare a funcțiilor matematice și evită schimbarea stării și datelor mutabile. Acest stil de programare pune accentul pe aplicarea funcțiilor pe intrări pentru a produce ieșiri, în loc de a modifica starea programului. Limbajul F# este conceput pentru a încuraja și a facilita programarea funcțională, oferind în același timp suport pentru alte paradigme, cum ar fi programarea orientată pe obiecte și cea imperativă.

În această secțiune, vom explora principiile de bază ale programării funcționale, vom compara paradigma funcțională cu cea imperativă și vom discuta despre concepte cheie, cum ar fi imutabilitatea, funcțiile pure, recursivitatea și evaluarea expresiilor.

Înțelegerea acestor concepte vă va ajuta să profitați la maximum de capacitățile limbajului F# și să scrieți cod concis, expresiv și ușor de întreținut.

#### 4.1.1.Paradigma funcțională vs. paradigma imperativă

Programarea funcțională și programarea imperativă sunt două paradigme distincte, cu filosofii și abordări diferite în rezolvarea problemelor.

Programarea imperativă, utilizată în limbaje precum C, C++, C#, Java și Python, se concentrează pe descrierea pașilor secvențiali necesari pentru a atinge un rezultat. Aceasta implică deseori modificarea stării programului prin intermediul variabilelor mutabile și al efectelor secundare. Codul imperativ tinde să fie mai „limbut” și mai puțin expresiv, deoarece programatorul trebuie să specifice explicit fiecare pas al calculului.

Programarea funcțională, exemplificată de limbaje precum F#, Haskell, Lisp și Scheme, se concentrează pe exprimarea calculelor ca evaluări ale funcțiilor matematice. Funcțiile sunt tratate ca entități de prim rang, ceea ce înseamnă că ele pot fi atribuite variabilelor, pasate ca argumente altor funcții și returnate ca rezultate. Programarea funcțională evită starea mutabilă și efectele secundare, favorizând imutabilitatea și funcțiile pure. Aceasta duce la un cod mai declarativ, mai concis și mai ușor de raționat.

Deși F# este în primul rând un limbaj funcțional, el suportă și construcții imperative, permițând programatorilor să aleagă cel mai potrivit stil pentru sarcina de rezolvat.

#### 4.1.2.Principii fundamentale ale programării funcționale

Programarea funcțională se bazează pe câteva principii fundamentale care o disting de alte paradigme de programare:

- **Funcțiile ca entități de prim rang:** În programarea funcțională, funcțiile sunt tratate ca valori ce pot fi atribuite variabilelor, pasate ca argumente și returnate din alte funcții. Această abordare permite compunerea funcțiilor și crearea de abstracții de nivel înalt.
- **Imutabilitatea:** Datele imutabile nu pot fi modificate odată ce au fost create. Aceasta elimină o clasă întreagă de erori legate de starea globală și efectele secundare, făcând codul mai ușor de înțeles și de raționat.
- **Funcții pure:** O funcție pură este o funcție care, pentru aceleași intrări, produce întotdeauna aceleași ieșiri, fără efecte secundare observabile. Funcțiile pure sunt ușor de testat, de compus și de paralelizat.
- **Recursivitatea:** Deoarece programarea funcțională evită starea mutabilă, recursivitatea este adesea folosită pentru a itera sau a procesa colecții de date. Recursivitatea implică definirea unei funcții în termeni de ea însăși, ceea ce permite rezolvarea elegantă a problemelor prin descompunerea lor în subprobleme mai mici.

- **Expresivitatea:** Limbajele funcționale oferă construcții și abstracțiuni puternice care permit exprimarea ideilor într-un mod concis și declarativ. Aceasta duce la un cod mai clar, mai ușor de citit și de întreținut.

Prin adoptarea acestor principii, programarea funcțională promovează crearea de software robust, modular și ușor de raționat.

#### 4.1.3. Imutabilitatea și efectele secundare

Imutabilitatea este un concept fundamental în programarea funcțională. În acest context, datele imutabile sunt date care nu pot fi modificate odată ce au fost create. În loc să modifice datele existente, operațiile asupra datelor imutabile produc întotdeauna valori noi, lăsând datele originale neschimbate.

Avantaje utilizării datelor imutabile sunt:

- Codul este mai ușor de înțeles și de raționat, deoarece valorile nu se modifică în mod neașteptat.
- Eliminarea problemelor legate de accesul concurent la date, deoarece nu există riscul ca datele să fie modificate simultan de mai multe fire de execuție.
- Testarea devine mai simplă, deoarece funcțiile care operează pe date imutabile sunt în general pure, producând rezultate previzibile.
- Imutabilitatea facilitează programarea paralelă și distribuită, deoarece datele pot fi partajate în siguranță între componente fără a fi nevoie de sincronizare.

Strâns legat de conceptul de imutabilitate este noțiunea de efecte secundare. Un efect secundar apare atunci când o funcție modifică o stare globală sau interacționează cu lumea externă (de ex., prin citirea de la tastatură, scrierea în fișiere sau comunicarea prin rețea), în plus față de calcularea valorii sale de retur. Efectele secundare pot face codul mai dificil de înțeles și de testat, deoarece comportamentul unei funcții poate depinde de factori externi.

Programarea funcțională promovează minimizarea efectelor secundare prin utilizarea funcțiilor pure ori de câte ori este posibil. O funcție pură depinde doar de intrările sale și produce întotdeauna același rezultat pentru aceleași intrări, fără a modifica starea globală. Când efectele secundare sunt necesare (de exemplu, pentru I/O), acestea sunt de obicei izolate și tratate explicit folosind tehnici speciale, cum ar fi monada I/O din Haskell sau workflows din F#.

#### 4.1.4. Evaluarea expresiilor

În programarea funcțională, programele sunt construite în principal prin definirea și evaluarea expresiilor. O expresie este o combinație de valori, variabile, operatori și apeluri de funcții care poate fi evaluată pentru a produce o valoare.

Iată câteva exemple de expresii în F#:

```
let x = 10 // o expresie simplă care leagă valoarea 10 de numele x
let y = x + 5 // o expresie care adună 5 la valoarea lui x
let z = max x y // o expresie care apelează funcția max pe x și y
```

Evaluarea unei expresii înseamnă reducerea acesteia la o valoare. Aceasta implică aplicarea operatorilor și funcțiilor pe valorile lor operand până când nu mai sunt posibile reduceri. În exemplul de mai sus, expresia `x + 5` se evaluează la `15`, iar expresia `max x y` se evaluează la `15`, presupunând că `x` este `10` și `y` este `15`.

F# folosește o strategie de evaluare numită evaluare aplicativă (sau evaluare strictă), în care argumentele unei funcții sunt evaluate înainte de a fi pasate funcției. Aceasta înseamnă că expresiile sunt evaluate în ordinea în care apar, de la stânga la dreapta.

De exemplu, să considerăm următoarea expresie:

```
let result = (printfn "Evaluating x"; 10) + (printfn "Evaluating y"; 20)
```

În acest caz, mesajul `"Evaluating x"` va fi afișat primul, urmat de mesajul `"Evaluating y"`, înainte ca adunarea să aibă loc. Rezultatul final va fi `30`.

F# suportă și evaluarea întârziată (lazy) prin intermediul cuvântului cheie `lazy`. Evaluarea lazy permite amânarea evaluării unei expresii până când valoarea sa este efectiv necesară. Aceasta poate fi utilă pentru a îmbunătăți performanța prin evitarea calculului inutile sau pentru a lucra cu structuri de date infinite.

#### 4.1.5. Structura unui program F#

Un program F# este compus dintr-una sau mai multe declarații, care pot include definirea de valori, funcții, tipuri și module. Iată o privire de ansamblu asupra structurii de bază a unui program F#:

```
// Directive de preprocesor și referințe
open System

// Definirea modulelor și namespace-urilor
namespace MyNamespace

module MyModule =
    // Definirea valorilor imutabile
    let x = 10
    let y = 20

    // Definirea funcțiilor
    let addNumbers a b =
        a + b

    // Definirea tipurilor de date
    type Person = { Name: string; Age: int }

    // Funcția de intrare (entry point)
    [<EntryPoint>]
    let main argv =
```

```
let result = addNumbers x y
printfn "Result: %d" result

let person = { Name = "Alice"; Age = 30 }
printfn "Person: %A" person

0 // Codul de ieșire
```

Să analizăm componentele cheie ale acestui program:

- Directivele `open` sunt utilizate pentru a aduce module și namespace-uri în domeniul de vedere curent, permițând utilizarea elementelor lor fără a fi nevoie de calificarea completă. În exemplu, `open System` permite utilizarea funcției `printfn` fără prefixul `System`.
- Namespace-urile și module permit organizarea logică a codului. Namespace-urile oferă un mecanism de nivel înalt pentru a grupa tipuri și module conexe, în timp ce modulele sunt utilizate pentru a încapsula valori, funcții și tipuri legate.
- Valorile imutabile sunt definite folosind cuvântul cheie `let`, urmat de un nume, un semn egal și o expresie. Odată definite, valorile nu pot fi reatribuite.
- Funcțiile sunt, de asemenea, definite folosind `let`, urmat de un nume, parametri și un corp al funcției. Tipurile parametrilor și valoarea de retur sunt deduse de F# pe baza utilizării lor.
- Tipurile de date, cum ar fi înregistrările (record), pot fi definite folosind cuvântul cheie `type`, urmat de numele tipului și membrii săi.
- Funcția de intrare `main` este punctul de pornire al programului. Aceasta este marcată cu atributul [`<EntryPoint>`] și trebuie să aibă tipul de retur `int`. Argumentul `argv` reprezintă un tablou de argumente în linia de comandă.
- Funcția `main` poate apela alte funcții, poate inițializa valori și poate efectua operații I/O, cum ar fi scrierea pe consolă folosind `printfn`.
- Valoarea returnată de `main` este folosită ca cod de ieșire al programului. Prin convenție, `0` indică o execuție reușită.

Această structură de bază poate fi extinsă pentru a include caracteristici mai avansate, cum ar fi tipurile de date algebrice, expresiile de potrivire a modelelor, secvențele și alte construcții specifice F#.

## 4.2. Tipuri de date și valori

Înțelegerea tipurilor de date este fundamentală în orice limbaj de programare, deoarece acestea definesc felul în care datele sunt reprezentate, stocate și manipulate. F#, fiind un limbaj static tipizat, pune un accent puternic pe corectitudinea tipurilor și oferă un sistem de tipuri expresiv și flexibil.

Pe parcursul acestei secțiuni, veți învăța cum să utilizați diversele tipuri de date oferite de F#, cum să beneficiați de inferența tipurilor pentru un cod mai concis și cum să folosiți

tipuri speciale, cum ar fi Option și Tuple, pentru a reprezenta valori opționale și a returna valori multiple dintr-o funcție.

Să începem prin explorarea tipurilor de date primitive în F#.

### 4.2.1. Tipuri de date primitive

F# oferă o gamă largă de tipuri de date primitive pentru a reprezenta valori simple, cum ar fi numere întregi, numere în virgulă mobilă, valori booleene și caractere. Iată o listă a celor mai frecvent utilizate tipuri de date primitive în F#:

#### *Numere întregi:*

- **int**: număr întreg pe 32 de biți, cuprins între  $-2^{31}$  și  $2^{31}-1$
- **int64**: număr întreg pe 64 de biți, cuprins între  $-2^{63}$  și  $2^{63}-1$
- **byte**: număr întreg fără semn pe 8 biți, cuprins între 0 și 255
- **sbyte**: număr întreg cu semn pe 8 biți, cuprins între -128 și 127
- **uint32**: număr întreg fără semn pe 32 de biți, cuprins între 0 și  $2^{32}-1$

#### *Numere în virgulă mobilă:*

- **float**: număr în virgulă mobilă cu precizie dublă pe 64 de biți
- **float32**: număr în virgulă mobilă cu precizie simplă pe 32 de biți
- **decimal**: număr zecimal cu 128 de biți, potrivit pentru calcule financiare și monetare

#### *Valori booleene:*

- **bool**: o valoare adevărat/fals reprezentată ca **true** sau **false**

#### *Caractere:*

- **char**: un singur caracter Unicode pe 16 biți

Iată câteva exemple de utilizare a tipurilor de date primitive în F#:

```
let x: int = 42
let y: float = 3.14
let z: bool = true
let c: char = 'A'
```

În aceste exemple, am declarat explicit tipul fiecărei valori folosind adnotarea de tip (de ex., **: int**). Totuși, în majoritatea cazurilor, F# poate deduce automat tipul unei valori pe baza expresiei sale, datorită inferenței tipurilor, așa cum vom vedea în secțiunea următoare.

### 4.2.2. Inferența tipurilor

Una dintre caracteristicile puternice ale limbajului F# este inferența tipurilor, care permite compilatorului să deducă automat tipul unei expresii pe baza utilizării sale, fără

a fi nevoie de adnotări de tip explicite. Inferența tipurilor face codul F# mai concis și mai ușor de citit, în timp ce menține beneficiile siguranței tipurilor.

Să vedem cum funcționează inferența tipurilor în practică:

```
let x = 42 // tipul lui x este dedus ca int
let y = 3.14 // tipul lui y este dedus ca float
let z = true // tipul lui z este dedus ca bool
let c = 'A' // tipul lui c este dedus ca char
```

În aceste exemple, nu am specificat explicit tipurile variabilelor `x`, `y`, `z` și `c`. În schimb, compilatorul F# a dedus automat tipurile lor pe baza valorilor atribuite. Aceasta face codul mai concis și mai ușor de întreținut, deoarece nu trebuie să ne preocupăm de adnotarea manuală a tipurilor.

Inferența tipurilor funcționează prin analiza expresiilor și a utilizării valorilor în context. Compilatorul utilizează informațiile disponibile, cum ar fi literalii, operatorii și apelurile de funcții, pentru a determina tipul cel mai general care satisface toate constrângerile.

De exemplu, să considerăm următoarea expresie:

```
let add x y = x + y
```

Aici, am definit o funcție `add` care primește doi parametri `x` și `y` și returnează suma lor. F# poate deduce că `x` și `y` trebuie să aibă tipuri compatibile pentru a putea fi adunate și va atribui funcției `add` tipul `int -> int -> int` (o funcție care primește doi întregi și returnează un întreg). Aceasta se datorează faptului că operatorul `+` este utilizat în mod obișnuit cu valori întregi.

Cu toate acestea, este important de reținut că inferența tipurilor are limitări și poate fi ambiguă în anumite situații. În aceste cazuri, poate fi necesară adnotarea explicită a tipurilor pentru a clarifica intenția programatorului sau pentru a rezolva ambiguitatea.

De exemplu:

```
let numbers = [1; 2; 3] // tipul este dedus ca list<int>
let numbers = [1; 2; 3.14] // eroare de compilare: ambiguitate de tip
```

În al doilea exemplu, compilatorul F# nu poate deduce un tip consistent pentru lista `numbers`, deoarece conține atât valori întregi, cât și în virgulă mobilă. În acest caz, ar trebui să adnotăm explicit tipul listei sau să asigurăm omogenitatea elementelor sale.

### 4.2.3. Tipuri parametrizate și generice

F# acceptă tipuri parametrizate și generice, care vă permit să definiți structuri de date și algoritmi care funcționează cu diferite tipuri de date, fără a duplica codul. Tipurile parametrizate sunt definite cu unul sau mai mulți parametri de tip, care sunt specificați între paranteze unghiulare (`<>`).



Să luăm ca exemplu tipul de date listă predefinit în F#. Listele sunt definite ca un tip parametrizat, `List<'T>`, unde `'T` este un parametru de tip ce reprezintă tipul elementelor din listă. Putem crea liste de orice tip, cum ar fi `List<int>`, `List<string>` sau chiar `List<List<int>>`.

Iată câteva exemple de utilizare a listelor parametrizate:

```
let intList = [1; 2; 3] // tipul este dedus ca List<int>
let stringList = ["Hello"; "World"] // tipul este dedus ca List<string>
let nestedList = [[1; 2]; [3; 4]] // tipul este dedus ca List<List<int>>
```

În aceste exemple, F# deduce automat tipul parametrului `'T` pe baza elementelor furnizate în lista literală.

De asemenea, putem defini propriile noastre tipuri parametrizate. De exemplu, să definim un tip simplu de pereche (tuple cu doi membri):

```
type Pair<'T, 'U> = 'T * 'U
```

Aici, `Pair` este un tip parametrizat cu doi parametri de tip, `'T` și `'U`. Putem crea instanțe ale acestui tip cu diferite tipuri concrete:

```
let intStringPair: Pair<int, string> = (42, "Hello")
let boolFloatPair: Pair<bool, float> = (true, 3.14)
```

Tipurile generice sunt o extensie a tipurilor parametrizate, unde funcțiile sau metodele sunt definite pentru a funcționa cu tipuri parametrizate. Iată un exemplu de funcție generică care inversează o pereche:

```
let swap<'T, 'U> (pair: Pair<'T, 'U>): Pair<'U, 'T> =
    let (x, y) = pair
    (y, x)
```

Funcția `swap` are doi parametri de tip, `'T` și `'U`, și primește o pereche de tipul `Pair<'T, 'U>`. Aceasta returnează o nouă pereche cu tipurile inversate, `Pair<'U, 'T>`. Putem apela această funcție cu perechi de orice tip:

```
let swappedIntStringPair = swap (42, "Hello") // tipul este dedus ca
Pair<string, int>
let swappedBoolFloatPair = swap (true, 3.14) // tipul este dedus ca
Pair<float, bool>
```

Tipurile parametrizate și generice sunt instrumente puternice pentru scrierea de cod reutilizabil și polimorfic în F#. Acestea vă permit să captați modele comune și să scrieți algoritmi generici, fără a sacrifica siguranța tipurilor sau performanța.

#### 4.2.4. Tipul unit

În F#, `unit` este un tip special care reprezintă absența unei valori semnificative. Este similar cu tipul `void` din limbaje precum C# sau Java. O valoare de tipul `unit` este scrisă ca `()` și este adesea folosită când o funcție nu returnează nimic semnificativ sau când

dorim să subliniem faptul că o anumită expresie este evaluată doar pentru efectele sale secundare.

Iată un exemplu de funcție care nu returnează nimic semnificativ:

```
let printMessage message =  
    printfn "%s" message
```

Funcția `printMessage` primește un singur argument de tip `string` și îl afișează pe consolă folosind `printfn`. Deoarece `printfn` returnează `unit`, funcția `printMessage` va avea, de asemenea, tipul de retur `unit`. Putem apela această funcție astfel:

```
printMessage "Hello, World!" // tipărim mesajul și returnăm ()
```

Deși `unit` este utilizat în mod obișnuit pentru funcțiile care nu returnează nimic semnificativ, acesta poate fi, de asemenea, folosit ca tip pentru argumentele funcțiilor sau ca tip de element în colecții. De exemplu:

```
let unitList = [(); (); ()] // o listă de valori unit
```

În acest caz, `unitList` este o listă care conține trei elemente de tip `unit`.

Un aspect important de reținut este că, în timp ce `unit` este utilizat pentru a reprezenta lipsa unei valori semnificative, acesta este încă o valoare în sine și diferă de conceptul de "null" sau "nil" din alte limbaje de programare. În F#, `null` nu este o valoare validă pentru majoritatea tipurilor și este în general considerat un anti-pattern. În schimb, pentru a reprezenta absența unei valori, F# folosește tipul opțional `Option`, pe care îl vom discuta în secțiunea următoare.

#### 4.2.5. Tipul Option

În programare, este frecvent necesar să exprimăm ideea de absență sau prezență unei valori. În multe limbaje, acest lucru se realizează prin folosirea valorilor `null` sau `nil`. Cu toate acestea, utilizarea valorilor `null` poate duce la erori greu de depistat, deoarece pot apărea excepții de tipul "null reference" în timpul execuției dacă încercăm să accesăm un membru al unei valori `null`.

Pentru a aborda acest problema într-un mod mai sigur și expresiv, F# oferă tipul `Option`, care este un tip de date predefinit care reprezintă fie prezența unei valori de un anumit tip, fie absența unei valori. Este definit după cum urmează:

```
type Option<'T> =  
    | Some of 'T  
    | None
```

`Option<'T>` este un tip discriminat uniune (TDU) cu două cazuri:

- `Some`: reprezintă prezența unei valori de tipul `'T`
- `None`: reprezintă absența unei valori

Să vedem cum putem utiliza `Option` în practică:

```
let divide x y =
    if y = 0 then
        None
    else
        Some (x / y)
```

Aici, am definit o funcție `divide` care primește doi întregi `x` și `y` și returnează rezultatul împărțirii lui `x` la `y`, înfășurat în tipul `Option`. Dacă `y` este zero (ceea ce ar duce la o eroare de împărțire la zero), funcția returnează `None`, indicând absența unei valori valide. Altfel, rezultatul este înfășurat în constructorul `Some`.

Putem utiliza pattern matching pentru a accesa valoarea dintr-un `Option` în mod sigur:

```
let result = divide 10 2
match result with
| Some value -> printfn "Rezultatul este %d" value
| None -> printfn "Împărțire la zero"
```

În acest exemplu, folosim pattern matching pentru a verifica dacă `result` este `Some` (are o valoare) sau `None`. În cazul `Some`, legăm valoarea la numele `value` și o afișăm. Dacă `result` este `None`, tratăm cazul corespunzător.

Utilizarea tipului `Option` ne obligă să gestionăm în mod explicit cazurile în care o valoare poate fi absentă, evitând astfel erorile de tip "null reference" și făcând codul mai robust și mai ușor de înțeles.

Pe lângă pattern matching, F# oferă și funcții utilitare pentru a lucra cu valori `Option`, cum ar fi `Option.map`, `Option.bind` și `Option.defaultValue`, care permit transformarea și extragerea valorilor dintr-un `Option` într-un mod concis și sigur.

Prin folosirea tipului `Option` în locul valorilor `null`, putem scrie cod mai expresiv, mai sigur și mai ușor de înțeles în F#.

#### 4.2.6. Tipul Tuple și deconstrucție

În F#, un tuple este o structură de date care grupează mai multe valori de tipuri posibil diferite într-o singură entitate. Tuple-urile sunt utile pentru returnarea mai multor valori dintr-o funcție, reprezentarea unui grup de valori înrudite sau ca o alternativă la crearea unei noi clase sau a unui nou tip de înregistrare pentru un scop specific.

Tuple-urile sunt create folosind paranteze și virgule pentru a separa valorile individuale. Iată câteva exemple:

```
let pair = (42, "Hello") // tuple de (int, string)
let triplet = (true, 3.14, 'A') // tuple de (bool, float, char)
let nestedTuple = ((1, 2), (3, 4)) // tuple de ((int, int), (int, int))
```

În aceste exemple, `pair` este un tuple care conține un întreg și un șir de caractere, `triplet` este un tuple de trei elemente cu un boolean, un număr în virgulă mobilă și un caracter, iar `nestedTuple` este un tuple de tuple-uri.

Putem accesa elementele individuale dintr-un tuple folosind pattern matching:

```
let (x, y) = pair // deconstrucție  
printfn "x = %d, y = %s" x y
```

Aici, folosim deconstrucția pentru a desface tuple-ul `pair` în componentele sale individuale `x` și `y`, pe care le putem apoi utiliza în mod normal.

De asemenea, putem accesa elementele dintr-un tuple folosind funcțiile predefinite `fst` și `snd` pentru tuple-uri cu două elemente:

```
let x = fst pair // accesează primul element  
let y = snd pair // accesează al doilea element
```

Tuple-urile sunt deosebit de utile în funcțiile care necesită returnarea mai multor valori. De exemplu:

```
let divmod x y =  
    let quotient = x / y  
    let remainder = x % y  
    (quotient, remainder)  
  
let (q, r) = divmod 10 3  
printfn "Quotient = %d, Remainder = %d" q r
```

Funcția `divmod` primește doi întregi `x` și `y` și returnează un tuple cu câtul și restul împărțirii. Rezultatul poate fi apoi deconstruit folosind pattern matching.

Un aspect important al tuple-urilor în F# este că tipul unui tuple este determinat de numărul și tipurile elementelor sale. Astfel, un tuple de `(int, string)` are un tip diferit de un tuple de `(bool, float)`. Aceasta înseamnă că tuple-urile cu diferite semnături de tip nu pot fi amestecate direct fără conversie explicită.

Spre deosebire de alte tipuri în F#, tuple-urile nu au un nume de tip explicit. În schimb, semnătura de tip a unui tuple este scrisă ca o secvență de tipuri între paranteze. De exemplu:

```
let pair: (int, string) = (42, "Hello")
```

Aici, adnotăm explicit tipul lui `pair` ca fiind `(int, string)`.

Tuple-urile sunt o caracteristică puternică în F# pentru gruparea valorilor ad-hoc și returnarea mai multor valori dintr-o funcție. Cu toate acestea, pentru date mai complexe cu multiple câmpuri și comportament asociat, se recomandă utilizarea tipurilor de înregistrare sau a claselor în locul tuple-urilor mari.

### 4.3. Funcții și aplicarea funcțiilor

Funcțiile reprezintă elementele fundamentale în programarea funcțională și sunt tratate ca entități de prim rang în F#. Acest lucru înseamnă că funcțiile pot fi atribuite variabilelor, pasate ca argumente altor funcții și returnate ca valori de la alte funcții. Definirea și utilizarea funcțiilor în mod eficient este esențială pentru a profita de puterea și expresivitatea programării funcționale.

În această secțiune, vom explora diferitele aspecte ale funcțiilor în F#, începând cu sintaxa de bază pentru definirea funcțiilor și aplicarea lor. Vom discuta despre funcțiile anonime (lambda) și cum pot fi ele folosite pentru a crea funcții inline concise. De asemenea, vom aborda concepte avansate, cum ar fi aplicarea parțială și currying, care permit o mai mare flexibilitate și reutilizare a codului.

Pe parcursul acestei secțiuni, veți învăța cum să compuneți funcții pentru a crea pipeline-uri de prelucrare a datelor și cum să utilizați funcții de ordin superior, care acceptă alte funcții ca argumente sau returnează funcții ca rezultate. În plus, vom explora recursivitatea și recursivitatea coadă, care sunt tehnici puternice pentru a rezolva probleme într-un stil funcțional.

La finalul acestei secțiuni, veți fi capabili să:

- Definiți și să apelați funcții în F# folosind sintaxa corectă
- Utilizați funcții anonime (lambda) pentru a crea funcții inline
- Aplicați parțial funcții și să folosiți currying pentru a crea funcții specializate
- Compuneți funcții pentru a crea pipeline-uri de prelucrare a datelor
- Înțelegeți și să aplicați conceptele de funcții de ordin superior
- Implementați soluții recursive și să utilizați recursivitatea coadă pentru a optimiza performanța

#### 4.3.1. Definirea funcțiilor

În F#, funcțiile sunt definite folosind cuvântul cheie `let`, urmat de numele funcției, parametrii de intrare și corpul funcției. Iată sintaxa de bază pentru definirea unei funcții:

```
let numeFuncție (parametru1: tip1) (parametru2: tip2) ... : tipReturn =
    // corpul funcției
    expresie
```

Să analizăm fiecare componentă în detaliu:

- `let` este cuvântul cheie folosit pentru a defini o funcție.
- `numeFuncție` este numele pe care îl atribuiți funcției. Acesta trebuie să fie un identificator valid în F#.
- `(parametru1: tip1) (parametru2: tip2) ...` reprezintă lista de parametri de intrare ai funcției. Fiecare parametru este definit prin numele său, urmat de două puncte și tipul său. Parametrii sunt separați prin spații.

- `tipReturn` specifică tipul de retur al funcției. Acesta este opțional; dacă nu este specificat, F# va deduce automat tipul de retur pe baza expresiei din corpul funcției.
- `=` este folosit pentru a separa antetul funcției de corpul acesteia.
- `expresie` reprezintă corpul funcției, care conține logica și calculele pe care funcția le efectuează. În F#, corpul funcției este o singură expresie, nu un bloc de instrucțiuni ca în limbaje imperative.

Iată un exemplu de funcție simplă care adună două numere întregi:

```
let aduna (a: int) (b: int) : int =  
    a + b
```

În acest caz, funcția `aduna` primește doi parametri de tip `int`, `a` și `b`, și returnează suma lor, care este tot un `int`.

Un aspect important al funcțiilor în F# este că tipurile de intrare și ieșire sunt deduse automat de către compilator. Deci, putem omite tipul de retur și scrie funcția astfel:

```
let aduna (a: int) (b: int) =  
    a + b
```

De asemenea, putem lăsa compilatorul să deducă și tipurile parametrilor, dacă acestea pot fi determinate din context:

```
let aduna a b =  
    a + b
```

Acum, să vedem cum putem apela funcția `aduna`:

```
let suma = aduna 5 3  
printfn "Suma este: %d" suma
```

Observați că, pentru a apela o funcție în F#, scriem numele funcției urmat de argumentele sale, separate prin spații. În acest exemplu, `aduna 5 3` apelează funcția `aduna` cu argumentele 5 și 3 și atribuie rezultatul variabilei `suma`.

Definirea și apelarea funcțiilor în F# este simplă și concisă. Datorită deducerii automate a tipurilor și a sintaxei expresive, putem scrie funcții clare și ușor de înțeles. Pe parcursul acestei secțiuni, vom explora concepte mai avansate legate de funcții, cum ar fi aplicarea parțială și compunerea funcțiilor.

#### 4.3.2. Funcții anonime (lambda)

Pe lângă funcțiile numite, F# oferă suport pentru funcții anonime, cunoscute și sub numele de expresii lambda sau funcții lambda. Funcțiile anonime sunt funcții fără un nume explicit și sunt definite inline, acolo unde sunt necesare. Acestea sunt utile pentru a crea funcții mici și concise, care sunt folosite pe loc și nu necesită un nume distinct.

Sintaxa pentru definirea unei funcții anonime este următoarea:

```
fun (parametru1: tip1) (parametru2: tip2) ... -> expresie
```

Să analizăm componentele unei funcții anonime:

- Cuvântul cheie `fun` este folosit pentru a indica începutul unei funcții anonime.
- `(parametru1: tip1) (parametru2: tip2) ...` reprezintă lista de parametri de intrare, similar cu funcțiile numite.
- `->` separă parametrii de intrare de corpul funcției.
- `expresie` este corpul funcției, care poate fi orice expresie validă în F#.

Iată un exemplu de funcție anonimă care calculează pătratul unui număr:

```
let patrat = fun (x: int) -> x * x
```

În acest caz, definim o funcție anonimă care primește un parametru `x` de tip `int` și returnează pătratul său. Funcția anonimă este atribuită unei variabile numite `patrat`, permițându-ne să o apelăm ulterior.

Putem apela funcția anonimă în același mod în care apelăm o funcție numită:

```
let rezultat = patrat 5
printfn "Pătratul lui 5 este: %d" rezultat
```

Funcțiile anonime sunt deosebit de utile în combinație cu funcțiile de ordin superior, cum ar fi `List.map`, `List.filter` și `List.fold`, deoarece acestea așteaptă alte funcții ca argumente. Să vedem un exemplu:

```
let numere = [1; 2; 3; 4; 5]
let patrate = List.map (fun x -> x * x) numere
printfn "Pătratele numerelor sunt: %A" patrate
```

În acest exemplu, folosim funcția `List.map` pentru a aplica o funcție anonimă fiecărui element din lista `numere`. Funcția anonimă `fun x -> x * x` primește un element `x` și returnează pătratul său. Rezultatul este o nouă listă `patrate`, care conține pătratele numerelor din lista originală.

F# oferă și o sintaxă prescurtată pentru funcțiile anonime, folosind operatorul `|>`:

```
let patrate = numere |> List.map (fun x -> x * x)
```

Această sintaxă este cunoscută sub numele de "forward pipe" și permite compunerea funcțiilor într-un mod mai lizibil, similar cu un pipeline de prelucrare a datelor.

#### 4.3.3. Aplicare parțială și currying

Aplicarea parțială (partial application) și currying sunt două concepte puternice în programarea funcțională, care permit o mai mare flexibilitate și reutilizare a codului.

Aceste tehnici se bazează pe abilitatea de a trata funcțiile ca valori de prim rang și de a crea funcții noi, specializate, din funcții existente.

Să începem cu aplicarea parțială. Aplicarea parțială se referă la procesul de a fixa unul sau mai mulți parametri ai unei funcții, rezultând o nouă funcție cu un număr redus de parametri. Cu alte cuvinte, aplicăm parțial o funcție prin a furniza doar o parte din argumentele necesare, obținând o funcție care așteaptă restul argumentelor.

Iată un exemplu de aplicare parțială în F#:

```
let aduna (a: int) (b: int) = a + b
let adunaCu5 = aduna 5
```

În acest caz, definim o funcție `aduna` care primește doi parametri de tip `int` și returnează suma lor. Apoi, aplicăm parțial funcția `aduna` prin a furniza doar primul argument, 5, și atribuim rezultatul unei noi funcții numite `adunaCu5`. Această nouă funcție așteaptă un singur argument și va aduna 5 la orice valoare îi este furnizată.

Putem apela funcția `adunaCu5` astfel:

```
let rezultat = adunaCu5 3
printfn "Rezultatul este: %d" rezultat
```

În acest exemplu, apelăm `adunaCu5` cu argumentul 3, iar rezultatul va fi 8 (5 + 3).

Currying, numit după logicianul Haskell Curry, este un proces strâns legat de aplicarea parțială. În F#, toate funcțiile sunt curryficate automat. Aceasta înseamnă că o funcție care primește mai mulți parametri este transformată într-o serie de funcții, fiecare primind un singur parametru și returnând o nouă funcție care așteaptă parametrul următor.

Să vedem cum arată o funcție curryficate în F#:

```
let aduna (a: int) (b: int) = a + b
```

Deși această funcție pare să primească doi parametri, ea este de fapt curryficate. Putem să o aplicăm parțial astfel:

```
let adunaCu5 = aduna 5
```

Aici, `adunaCu5` este o funcție care așteaptă un singur parametru, deoarece `aduna 5` returnează o funcție care așteaptă al doilea parametru.

Avantajul currying-ului este că permite o mai mare flexibilitate în aplicarea parțială a funcțiilor și în crearea de funcții specializate. De exemplu, putem crea funcții precum `adunaCu5`, `adunaCu10` sau `adunaCu42` fără a defini explicit aceste funcții, ci doar prin aplicarea parțială a funcției `aduna`.

De asemenea, currying-ul permite o sintaxă mai concisă și mai expresivă atunci când compunem funcții. De exemplu:



```
let numerePare = [1..10] |> List.filter (fun x -> x % 2 = 0)
```

În acest caz, folosim o funcție anonimă curryficată (`fun x -> x % 2 = 0`) ca argument pentru `List.filter`. Această funcție primește un singur parametru `x` și returnează `true` dacă `x` este par.

#### 4.3.4. Compunerea funcțiilor

Compunerea funcțiilor este o tehnică fundamentală în programarea funcțională, care permite combinarea a două sau mai multe funcții pentru a crea o nouă funcție. Ideea de bază este de a aplica o funcție la rezultatul altei funcții, formând astfel un "pipeline" de prelucrare a datelor. Compunerea funcțiilor duce la un cod mai modular, mai expresiv și mai ușor de întreținut. În cadrul acestui paragraf vom expune pe scurt modalitatea de compunere a funcțiilor, iar în paragraful 4.6 vom detalia alți operatori de compunere ai limbajului F#.

Compunerea funcțiilor se realizează prin intermediul operatorului `>>`, cunoscut sub numele de "forward composition". Iată cum arată sintaxa pentru compunerea a două funcții:

```
let compusa = functia1 >> functia2
```

În acest caz, `compusa` este o nouă funcție care rezultă din compunerea `functia1` cu `functia2`. Când apelăm `compusa` cu un argument `x`, acesta este mai întâi prelucrat de `functia1`, iar rezultatul este apoi pasat ca argument pentru `functia2`.

Să vedem un exemplu concret:

```
let aduna5 x = x + 5
let inmultesteCu3 x = x * 3

let adunaSiinmulteste = aduna5 >> inmultesteCu3
```

În acest exemplu, definim două funcții simple: `aduna5`, care adună 5 la argumentul său, și `inmultesteCu3`, care înmulțește argumentul cu 3. Apoi, folosim operatorul `>>` pentru a compune aceste funcții într-o nouă funcție numită `adunaSiinmulteste`.

Putem apela funcția compusă astfel:

```
let rezultat = adunaSiinmulteste 2
printfn "Rezultatul este: %d" rezultat
```

În acest caz, valoarea 2 este mai întâi procesată de `aduna5`, rezultând 7 (2 + 5). Apoi, acest rezultat este pasat ca argument pentru `inmultesteCu3`, care înmulțește 7 cu 3, obținând rezultatul final 21.

Compunerea funcțiilor nu se limitează doar la două funcții. Putem compune oricâte funcții dorim, creând astfel pipeline-uri complexe de prelucrare a datelor. De exemplu:

```
let prelucreaza = functia1 >> functia2 >> functia3 >> functia4
```

În acest caz, `prelucreaza` este o funcție compusă care aplică succesiv `functia1`, `functia2`, `functia3` și `functia4` asupra argumentului său.

Un avantaj major al compunerii funcțiilor este că permite scrierea unui cod mai expresiv și mai lizibil. În loc să încapsulăm apelurile funcțiilor una într-alta, putem utiliza operatorul `>>` pentru a crea un flux de prelucrare a datelor ușor de urmărit.

De exemplu, să presupunem că avem o listă de numere și dorim să le filtrăm pe cele pare, să le ridicăm la pătrat și apoi să le însumăm. Folosind compunerea funcțiilor, putem realiza acest lucru într-un mod elegant:

```
let numere = [1..10]
let rezultat =
    numere
    |> List.filter (fun x -> x % 2 = 0)
    |> List.map (fun x -> x * x)
    |> List.sum
```

În acest exemplu, folosim operatorul "pipeline" (`|>`) în combinație cu compunerea funcțiilor pentru a crea un flux de prelucrare a datelor. Lista `numere` este mai întâi filtrată pentru a păstra doar numerele pare, apoi fiecare număr este ridicat la pătrat folosind `List.map` și, în final, numerele rezultate sunt însumate folosind `List.sum`.

#### 4.3.5. Funcții de ordin superior

Funcțiile de ordin superior sunt o caracteristică esențială a programării funcționale și reprezintă un concept fundamental în limbajul F#. O funcție de ordin superior este o funcție care îndeplinește cel puțin una dintre următoarele condiții:

- Primește una sau mai multe funcții ca argumente.
- Returnează o funcție ca rezultat.

Cu alte cuvinte, funcțiile de ordin superior tratează funcțiile ca valori de prim rang, permițând manipularea și compunerea funcțiilor în moduri flexibile și expresive.

Să analizăm un exemplu de funcție de ordin superior care primește o funcție ca argument:

```
let aplicaFunctie (f: int -> int) (x: int) = f x
```

În acest caz, `aplicaFunctie` este o funcție de ordin superior care primește două argumente: o funcție `f` de tipul `int -> int` (o funcție care primește un `int` și returnează un `int`) și un întreg `x`. Corpul funcției aplică pur și simplu funcția `f` asupra lui `x` și returnează rezultatul.

Putem apela `aplicaFunctie` astfel:

```
let aduna5 x = x + 5
let rezultat = aplicaFunctie aduna5 3
printfn "Rezultatul este: %d" rezultat
```

În acest exemplu, definim o funcție `aduna5` care adună 5 la argumentul său. Apoi, apelăm `aplicaFunctie` cu `aduna5` și valoarea 3. Funcția `aplicaFunctie` va aplica `aduna5` asupra lui 3, returnând rezultatul 8.

Acum, să vedem un exemplu de funcție de ordin superior care returnează o funcție:

```
let creareFunctieAdunare (n: int) =  
    fun (x: int) -> x + n
```

Această funcție, `creareFunctieAdunare`, primește un întreg `n` și returnează o nouă funcție care adună `n` la argumentul său. Cu alte cuvinte, ea creează o funcție de adunare personalizată, în funcție de valoarea lui `n`.

Putem utiliza `creareFunctieAdunare` astfel:

```
let adunaCu10 = creareFunctieAdunare 10  
let rezultat = adunaCu10 5  
printfn "Rezultatul este: %d" rezultat
```

În acest caz, apelăm `creareFunctieAdunare` cu valoarea 10, obținând o nouă funcție `adunaCu10`. Apoi, aplicăm `adunaCu10` asupra valorii 5, obținând rezultatul 15.

Funcțiile de ordin superior sunt deosebit de utile în combinație cu funcțiile predefinite din F#, cum ar fi `List.map`, `List.filter` și `List.fold`. Aceste funcții primesc alte funcții ca argumente și le aplică asupra unor colecții de date.

De exemplu, să presupunem că avem o listă de numere și dorim să le înmulțim pe toate cu 2:

```
let numere = [1; 2; 3; 4; 5]  
let numereDublate = List.map (fun x -> x * 2) numere  
printfn "Numerele dublate sunt: %A" numereDublate
```

Aici, folosim funcția `List.map` împreună cu o funcție anonimă `(fun x -> x * 2)` pentru a aplica operația de dublare fiecărui element din lista `numere`. Rezultatul este o nouă listă `numereDublate`, care conține numerele originale înmulțite cu 2.

#### 4.3.6. Recursivitate și recursivitate coadă

Recursivitatea este o tehnică fundamentală în programarea funcțională, care implică definirea unei funcții în termenii săi proprii. Cu alte cuvinte, o funcție recursivă se auto-apellează cu o versiune mai mică a problemei inițiale, până când se ajunge la un caz de bază care nu mai necesită apeluri recursive.

În F#, funcțiile recursive sunt definite folosind cuvântul cheie `rec`. Iată un exemplu clasic de funcție recursivă care calculează factorialul unui număr:

```
let rec factorial n =  
    if n <= 1 then 1
```

```
else n * factorial (n - 1)
```

În această definiție, funcția `factorial` se auto-apelează cu `n - 1` până când `n` ajunge la 1 (cazul de bază). La fiecare apel recursiv, rezultatul este înmulțit cu `n`, obținându-se în final factorialul numărului inițial.

Să analizăm pașii pentru calcularea lui `factorial 5`:

```
factorial 5
= 5 * factorial 4
= 5 * (4 * factorial 3)
= 5 * (4 * (3 * factorial 2))
= 5 * (4 * (3 * (2 * factorial 1)))
= 5 * (4 * (3 * (2 * 1)))
= 5 * (4 * (3 * 2))
= 5 * (4 * 6)
= 5 * 24
= 120
```

Recursivitatea este o tehnică puternică și expresivă, care permite rezolvarea problemelor într-un mod elegant și concis. Cu toate acestea, în anumite situații, recursivitatea poate duce la un consum excesiv de memorie și la depășirea capacității stivei de apeluri.

Pentru a evita aceste probleme, F# oferă suport pentru recursivitatea coadă (*tail recursion*). O funcție este *tail-recursive* dacă apelul recursiv este ultima operație efectuată în funcție și rezultatul apelului recursiv este returnat direct, fără alte calcule.

Iată cum putem rescrie funcția "factorial" folosind recursivitatea coadă:

```
let rec factorialTailRec n acc =
    if n <= 1 then acc
    else factorialTailRec (n - 1) (n * acc)

let factorial n = factorialTailRec n 1
```

În această variantă, introducem un parametru acumulator `acc` care ține evidența rezultatului parțial în timpul apelurilor recursive. La fiecare apel recursiv, `n` este decrementat, iar `acc` este înmulțit cu `n`. Când `n` ajunge la 1, rezultatul final este deja calculat în `acc` și este returnat direct.

Avantajul recursivității coadă este că compilatorul F# poate optimiza codul și evita acumularea apelurilor recursive pe stivă. În schimb, compilatorul transformă recursivitatea coadă într-o buclă iterativă eficientă, eliminând riscul de depășire a capacității stivei.

Recursivitatea coadă este deosebit de utilă când lucrați cu structuri de date recursive, cum ar fi liste sau arbori, deoarece permite prelucrarea eficientă a elementelor fără a consuma memorie suplimentară pentru fiecare apel recursiv.

Este important să rețineți că nu toate problemele pot fi rezolvate folosind recursivitatea coadă și uneori este necesară recursivitatea obișnuită. Cu toate acestea, atunci când

este posibil, se recomandă să utilizați recursivitatea coadă pentru a evita problemele de performanță și a asigura scalabilitatea codului.

#### 4.4. Pattern matching și expresii

Pattern matching-ul și expresiile sunt caracteristici fundamentale ale limbajului F#, care oferă un mod puternic și expresiv de a controla fluxul de execuție al programului pe baza structurii datelor. Aceste construcții vă permit să inspectați și să descompuneți valori în părțile lor componente, permițând un cod concis, lizibil și ușor de întreținut.

În această secțiune, vom explora pattern matching-ul în detaliu, începând cu sintaxa de bază și progresând către tehnici mai avansate. Veți învăța cum să utilizați pattern matching-ul pentru a verifica dacă o valoare se potrivește cu un anumit șablon (pattern) și cum să extrageți informații din structuri de date complexe.

De asemenea, vom acoperi construcții precum expresiile match, care vă permit să evaluați o valoare în raport cu o serie de pattern-uri și să executați codul corespunzător primei potriviri. Veți descoperi cum să utilizați guard-uri pentru a adăuga condiții suplimentare în expresiile match și cum să creați pattern-uri active (active patterns) pentru a încapsula logica de match într-o construcție reutilizabilă.

Pe parcurs, vom discuta și despre best practices în utilizarea pattern matching-ului, precum și despre conceptul de exhaustivitate, care asigură că toate cazurile posibile sunt acoperite de pattern-urile definite.

La finalul acestei secțiuni, veți fi capabili să:

- Utilizați pattern matching-ul pentru a inspecta și a descompune valori
- Scrieți expresii match concise și expresive
- Folosiți guard-uri pentru a rafina condițiile de match
- Creați și să utilizați pattern-uri active (active patterns)
- Aplicați principiile de exhaustivitate în pattern matching

Pattern matching-ul și expresiile match sunt unelte esențiale în programarea funcțională F#, oferind un mod elegant și expresiv de a scrie cod concis și ușor de înțeles. Stăpânirea acestor concepte vă va permite să profitați din plin de puterea și flexibilitatea limbajului F#.

##### 4.4.1. Sintaxa de bază a pattern matching

Pattern matching-ul în F# permite inspectarea structurii unei valori și extragerea informațiilor relevante din ea. Sintaxa de bază pentru pattern matching implică utilizarea cuvântului cheie `match`, urmat de expresia de evaluat și de o serie de clauze `| pattern -> expresie`.

Iată un exemplu simplu de pattern matching pe un tip de date algebric (discriminated union):

```
type Culoare =  
    | Rosu  
    | Verde  
    | Albastru  
  
let descrie culoare =  
    match culoare with  
    | Rosu -> "Culoarea pasiunii"  
    | Verde -> "Culoarea naturii"  
    | Albastru -> "Culoarea cerului"
```

În acest exemplu, definim un tip de date `Culoare` cu trei cazuri posibile: `Rosu`, `Verde` și `Albastru`. Funcția `descrie` primește o valoare de tip `Culoare` și folosește pattern matching-ul pentru a o evalua. Fiecare clauză `| pattern -> expresie` reprezintă un caz posibil. Când valoarea `culoare` se potrivește cu un pattern, expresia corespunzătoare este evaluată și returnată.

Putem apela funcția `descrie` astfel:

```
let result1 = descrie Rosu // "Culoarea pasiunii"  
let result2 = descrie Verde // "Culoarea naturii"
```

Pattern matching-ul poate fi utilizat și pe tipuri de date mai complexe, cum ar fi înregistrările (records) și tuplurile. Iată un exemplu de pattern matching pe un record:

```
type Persoana = { Nume: string; Varsta: int }  
let descriePersoana persoana =  
    match persoana with  
    | { Nume = "Alice"; Varsta = 30 } -> "Alice are 30 de ani"  
    | { Nume = "Bob"; Varsta = _ } -> "Bob are o vârstă necunoscută"  
    | { Nume = nume; Varsta = varsta } -> $"{nume} are {varsta} ani"
```

În acest caz, pattern-urile sunt definite folosind sintaxa de record, cu numele câmpurilor și valorile corespunzătoare. Caracterul underscore `_` este folosit ca wildcard pentru a se potrivi cu orice valoare. De asemenea, putem extrage valorile câmpurilor în variabile (`nume` și `varsta`) pentru a le utiliza în expresia corespunzătoare.

Pattern matching-ul poate fi utilizat și pe tuple-uri:

```
let describeTuple tuple =  
    match tuple with  
    | (1, "A") -> "Primul caz"  
    | (_, "B") -> "Al doilea caz"  
    | (x, y) -> $"Alt caz: {x}, {y}"
```

Aici, pattern-urile sunt definite folosind sintaxa de tuple-uri, cu valorile corespunzătoare pentru fiecare element al tuple-ului.

Este important de menționat că ordinea clauzelor de pattern matching contează. Prima clauză care se potrivește cu valoarea evaluată va fi executată, iar restul clauzelor vor fi ignorate. De aceea, este recomandabil să plasați cazurile mai specifice înaintea celor mai generale.

#### 4.4.2. Pattern matching pe tipuri de date

Pattern matching-ul în F# este deosebit de util când lucrăm cu tipuri de date algebrice, cum ar fi tipurile înregistrare (record types), tipurile discriminate (discriminated unions) și tipurile opționale (option types). Să analizăm câteva exemple pentru a ilustra utilizarea pattern matching-ului pe aceste tipuri de date.

##### *Pattern matching pe tipuri înregistrare (record types):*

Să presupunem că avem un tip înregistrare `Person` definit astfel:

```
type Person = { Name: string; Age: int }
```

Putem utiliza pattern matching-ul pentru a extrage valorile câmpurilor din înregistrare:

```
let printPersonInfo person =  
    match person with  
    | { Name = name; Age = age } -> printfn "Name: %s, Age: %d" name age
```

În acest exemplu, pattern-ul `{ Name = name; Age = age }` se potrivește cu o înregistrare `Person` și extrage valorile câmpurilor `Name` și `Age` în variabilele `name` și `age`, respectiv. Aceste variabile pot fi apoi utilizate în expresia corespunzătoare.

##### *Pattern matching pe tipuri discriminate (discriminated unions):*

Să considerăm următorul tip discriminat `Shape`:

```
type Shape =  
    | Rectangle of width: float * length: float  
    | Circle of radius: float  
    | Triangle of base': float * height: float
```

Putem utiliza pattern matching-ul pentru a determina tipul formei geometrice și a extrage valorile asociate:

```
let getShapeArea shape =  
    match shape with  
    | Rectangle (width, length) -> width * length  
    | Circle radius -> System.Math.PI * radius ** 2.0  
    | Triangle (base', height) -> 0.5 * base' * height
```

În acest caz, pattern-urile `Rectangle (width, length)`, `Circle radius` și `Triangle (base', height)` se potrivesc cu diferitele cazuri ale tipului discriminat `Shape` și extrag valorile corespunzătoare. Pattern matching-ul ne permite să tratăm fiecare caz în mod individual și să efectuăm calculele specifice pentru fiecare tip de formă geometrică.

##### *Pattern matching pe tipuri opționale (option types):*

Tipul opțional în F# este definit astfel:

```
type Option<'a> =  
    | Some of 'a  
    | None
```

Putem utiliza pattern matching-ul pentru a verifica dacă o valoare este prezentă (Some) sau nu (None) și a extrage valoarea în caz afirmativ:

```
let printValue optionalValue =  
    match optionalValue with  
    | Some value -> printfn "Value: %A" value  
    | None -> printfn "No value"
```

În acest exemplu, pattern-ul `Some value` se potrivește cu cazul în care valoarea opțională conține o valoare, iar pattern-ul `None` se potrivește cu cazul în care valoarea opțională este vidă. Pattern matching-ul ne permite să tratăm ambele cazuri în mod elegant și sigur din punct de vedere al tipurilor.

Pattern matching-ul pe tipuri de date este o tehnică puternică în F#, care ne permite să inspectăm și să extragem informații din structuri de date complexe într-un mod concis și expresiv. Prin utilizarea pattern matching-ului, putem evita construcțiile condiționale verbale și putem modela logica programului într-un stil funcțional curat.

#### 4.4.3. Guards în pattern matching

Guards sunt condiții suplimentare care pot fi adăugate într-o expresie match, pentru a rafina procesul de pattern matching. Acestea permit filtrarea suplimentară a valorilor care se potrivesc cu un anumit pattern, pe baza unor criterii specifice.

Sintaxa pentru guards este următoarea:

```
| pattern when condiție -> expresie
```

În această sintaxă, `when` este urmat de o condiție booleană, care este evaluată doar dacă valoarea se potrivește cu pattern-ul specificat. Dacă atât pattern-ul, cât și condiția sunt îndeplinite, expresia corespunzătoare este evaluată și returnată.

Să analizăm un exemplu pentru a înțelege mai bine utilizarea guards în pattern matching. Să presupunem că avem o funcție care primește un întreg și returnează o descriere a acestuia:

```
let describeNumber n =  
    match n with  
    | x when x < 0 -> "Număr negativ"  
    | x when x = 0 -> "Zero"  
    | x when x > 0 && x <= 10 -> "Număr pozitiv mic"  
    | x when x > 10 && x <= 100 -> "Număr pozitiv mare"  
    | _ -> "Număr pozitiv foarte mare"
```

În acest exemplu, funcția `describeNumber` folosește pattern matching cu guards pentru a clasifica numărul primit. Fiecare clauză match are un pattern `x` (care se potrivește cu orice valoare) și o condiție specifică. Guards sunt evaluate în ordine, de sus în jos, până când se găsește o potrivire:

- Dacă numărul este negativ ( $x < 0$ ), se returnează "Număr negativ".



- Dacă numărul este zero ( $x = 0$ ), se returnează "Zero".
- Dacă numărul este pozitiv și mai mic sau egal cu 10 ( $x > 0 \ \&\& \ x \leq 10$ ), se returnează "Număr pozitiv mic".
- Dacă numărul este pozitiv și mai mare decât 10, dar mai mic sau egal cu 100 ( $x > 10 \ \&\& \ x \leq 100$ ), se returnează "Număr pozitiv mare".
- Dacă niciuna dintre condițiile de mai sus nu este îndeplinită (adică numărul este mai mare decât 100), se folosește pattern-ul wildcard `_` și se returnează "Număr pozitiv foarte mare".

Să testăm funcția cu câteva valori:

```
describeNumber -5 // returnează "Număr negativ"
describeNumber 0 // returnează "Zero"
describeNumber 7 // returnează "Număr pozitiv mic"
describeNumber 42 // returnează "Număr pozitiv mare"
describeNumber 123 // returnează "Număr pozitiv foarte mare"
```

Guards oferă o modalitate concisă și expresivă de a adăuga condiții suplimentare în pattern matching, permițând o selecție mai precisă a cazurilor și o manipulare mai flexibilă a valorilor. Acestea pot fi folosite în combinație cu diverse tipuri de pattern-uri (e.g., constant patterns, variable patterns, tuple patterns) pentru a crea construcții match puternice și adaptate nevoilor specifice ale programului.

Este important de reținut că guards sunt evaluate în ordine și procesul de pattern matching se oprește la prima potrivire completă (pattern și condiție îndeplinite). Prin urmare, ordinea clauzelor match și a condițiilor în guards este semnificativă și poate influența rezultatul final.

#### 4.4.4.Active patterns

Active patterns reprezintă o funcționalitate avansată a limbajului F#, care permite definirea de pattern-uri personalizate. Acestea oferă o modalitate de a încapsula logica de potrivire a unui pattern și de a o reutiliza în diverse contexte.

Un active pattern este definit ca o funcție care primește o valoare de input și returnează o opțiune de tuple. Această funcție este responsabilă pentru procesarea valorii de intrare și determinarea dacă aceasta se potrivește cu un anumit pattern.

Există trei tipuri principale de active patterns:

*Single-case active patterns (active patterns cu un singur caz):*

- Sunt definite folosind cuvântul cheie `let` urmat de `(| și |)`, cu numele pattern-ului între paranteze.
- Returnează o valoare de tip `Option<'T>`, unde `'T` este tipul rezultatului în caz de potrivire.

Exemplu:

```
let (|Integer|_|) str =
    match System.Int32.TryParse(str) with
    | true, value -> Some(value)
    | false, _ -> None
```

#### *Multi-case active patterns (active patterns cu mai multe cazuri):*

- Sunt definite folosind cuvântul cheie `let` urmat de `(|` și `|)`, cu numele cazurilor separate prin `|`.
- Returnează o valoare de tip `'T`, unde `'T` este tipul rezultatului corespunzător cazului de potrivire.

Exemplu:

```
let (|Even|Odd|) n =
    if n % 2 = 0 then Even else Odd
```

#### *Partial active patterns (active patterns parțiale):*

- Sunt similare cu multi-case active patterns, dar permit cazuri incomplete.
- Sunt definite folosind cuvântul cheie `let` urmat de `(|` și `|_|)`, cu numele cazurilor separate prin `|`.
- Pot returna o valoare de tip `Option<'T>` pentru cazurile definite și `None` pentru cazul default.

Exemplu:

```
let (|Square|_|) n =
    if float n = sqrt (float n) ** 2.0 then Some(n) else None
```

Odată definit un active pattern, acesta poate fi utilizat în expresiile match pentru a testa și extrage valori din datele de intrare. Iată câteva exemple de utilizare a active patterns:

```
// Utilizarea single-case active pattern
let parseInteger str =
    match str with
    | Integer i -> printfn "Șirul poate fi parsat ca întregul %d" i
    | _ -> printfn "Șirul nu poate fi parsat ca întreg"

parseInteger "42" // Output: Șirul poate fi parsat ca întregul 42
parseInteger "abc" // Output: Șirul nu poate fi parsat ca întreg
```

```
// Utilizarea multi-case active pattern
let printParity n =
    match n with
    | Even -> printfn "%d este par" n
    | Odd -> printfn "%d este impar" n

printParity 4 // Output: 4 este par
printParity 7 // Output: 7 este impar
```

```
// Utilizarea partial active pattern
let isSquare n =
    match n with
    | Square x -> printfn "%d este un pătrat perfect" n
    | _ -> printfn "%d nu este un pătrat perfect" n

isSquare 16 // Output: 16 este un pătrat perfect
isSquare 10 // Output: 10 nu este un pătrat perfect
```

#### 4.4.5.Match expressions în funcții

Match expressions reprezintă un construct fundamental în limbajul F# și sunt utilizate în mod obișnuit în definirea funcțiilor. Acestea oferă un mecanism puternic pentru a controla fluxul de execuție al unei funcții pe baza valorilor de intrare și a pattern-urilor definite.

Sintaxa de bază a unei match expression într-o funcție este următoarea:

```
let numeFuncție parametri =
    match expresie with
    | pattern1 -> rezultat1
    | pattern2 -> rezultat2
    | _ -> rezultatDefault
```

În această sintaxă:

- `numeFuncție` este numele funcției pe care o definiți.
- `parametri` reprezintă parametrii de intrare ai funcției.
- `expresie` este valoarea sau expresia pe care se aplică pattern matching-ul.
- `pattern1`, `pattern2` etc. sunt pattern-urile pe care le testați.
- `rezultat1`, `rezultat2` etc. sunt expresiile corespunzătoare fiecărui pattern, care vor fi evaluate și returnate când pattern-ul respectiv se potrivește.
- `_` este pattern-ul wildcard, care se potrivește cu orice valoare și este utilizat pentru a defini un caz default.

Să analizăm un exemplu concret pentru a ilustra utilizarea match expressions în funcții:

```
let describeNumber n =
    match n with
    | 0 -> "Zero"
    | 1 -> "Unu"
    | 2 -> "Doi"
    | _ when n < 0 -> "Număr negativ"
    | _ -> "Număr mai mare decât 2"
```

În acest exemplu, funcția `describeNumber` primește un parametru `n` de tip întreg și returnează o descriere textuală a acestuia, utilizând o match expression:

- Dacă "`n`" este 0, funcția returnează "Zero".

- Dacă "n" este 1, funcția returnează "Unu".
- Dacă "n" este 2, funcția returnează "Doi".
- Dacă "n" este mai mic decât 0 (utilizând un guard), funcția returnează "Număr negativ".
- Pentru orice altă valoare a lui "n", funcția returnează "Număr mai mare decât 2".

Să testăm funcția cu câteva valori:

```
describeNumber 0 // returnează "Zero"
describeNumber 1 // returnează "Unu"
describeNumber 2 // returnează "Doi"
describeNumber -5 // returnează "Număr negativ"
describeNumber 42 // returnează "Număr mai mare decât 2"
```

Match expressions în funcții oferă un mod elegant și expresiv de a defini comportamentul unei funcții pe baza valorilor de intrare. Acestea permit separarea clară a diferitelor cazuri și asocierea unui rezultat specific cu fiecare caz.

#### 4.4.6. Pattern matching complet

Pattern matching-ul complet (exhaustive pattern matching) este o caracteristică importantă a limbajului F#, care asigură că toate cazurile posibile sunt acoperite atunci când se utilizează o expresie match. Aceasta înseamnă că fiecare valoare posibilă a tipului de date pe care se face pattern matching trebuie să fie tratată de cel puțin un pattern definit.

Compilatorul F# verifică exhaustivitatea pattern matching-ului la momentul compilării și va genera o avertizare sau o eroare dacă există cazuri neacoperite. Acest lucru ajută la prevenirea erorilor în timpul execuției și asigură că programul este corect și robust.

Să luăm un exemplu pentru a ilustra exhaustive pattern matching:

```
type Shape =
    | Circle of float
    | Rectangle of float * float
    | Triangle of float * float * float

let getArea shape =
    match shape with
    | Circle radius -> System.Math.PI * radius * radius
    | Rectangle (width, height) -> width * height
    | Triangle (a, b, c) ->
        let s = (a + b + c) / 2.0
        sqrt (s * (s - a) * (s - b) * (s - c))
```

În acest exemplu, avem un tip de date discriminat `Shape` cu trei cazuri: `Circle`, `Rectangle` și `Triangle`. Funcția `getArea` primește o valoare de tip `Shape` și calculează aria corespunzătoare folosind pattern matching.

Observați că toate cele trei cazuri posibile ale tipului `Shape` sunt acoperite în expresia `match` din funcția `getArea`. Fiecare caz are un pattern specific care extrage datele necesare (raza pentru `Circle`, lățimea și înălțimea pentru `Rectangle`, și cele trei laturi pentru `Triangle`) și calculează aria folosind formula corespunzătoare.

Dacă am omite unul dintre cazuri sau dacă am adăuga un nou caz în tipul `Shape` fără a-l trata în expresia `match`, compilatorul F# ar genera o avertizare sau o eroare, indicând că pattern matching-ul nu este exhaustiv.

Exhaustive pattern matching aduce următoarele beneficii:

- Verificare la compilare: Compilatorul F# verifică exhaustivitatea pattern matching-ului la momentul compilării, asigurând că toate cazurile sunt tratate. Acest lucru reduce riscul de erori în timpul execuției.
- Siguranța tipurilor: Prin acoperirea tuturor cazurilor posibile, exhaustive pattern matching contribuie la siguranța tipurilor în programele F#. Nu există riscul de a uita un caz sau de a avea valori netratate.
- Expresivitate și claritate: Exhaustive pattern matching încurajează exprimarea clară și completă a logicii programului. Toate cazurile sunt explicate și vizibile în codul sursă, făcând codul mai ușor de înțeles și de întreținut.
- Refactorizare sigură: Atunci când se adaugă noi cazuri într-un tip de date discriminat, compilatorul F# va indica toate locurile în care pattern matching-ul nu mai este exhaustiv. Acest lucru facilitează refactorizarea în siguranță a codului și asigură că toate cazurile sunt tratate corespunzător.

Pentru a beneficia pe deplin de exhaustive pattern matching, este recomandat să utilizați tipuri de date discriminate ori de câte ori este posibil și să acoperiți toate cazurile în expresiile `match`. Dacă există un caz default pe care doriți să îl tratați într-un mod generic, puteți folosi pattern-ul wildcard `_` pentru a acoperi toate celelalte cazuri.

## 4.5. Colecții și prelucrarea datelor

Colecțiile și prelucrarea datelor sunt componente esențiale ale programării funcționale în F#. Acest limbaj oferă o gamă largă de structuri de date imutabile, precum și o serie de funcții și operatori puternici pentru manipularea eficientă a colecțiilor.

În această secțiune, vom explora principalele tipuri de colecții disponibile în F#, inclusiv listele (`List`), tablourile (`Array`), secvențele (`Seq`), precum și tipurile `Map` și `Set`. Vom învăța cum să creăm, să accesăm și să modificăm aceste colecții, folosind funcții și operatori specifici.

De asemenea, vom acoperi o serie de operații comune pe colecții, cum ar fi maparea (`map`), filtrarea (`filter`), reducerea (`fold`), concatenarea, sortarea și gruparea. Aceste operații ne permit să transformăm și să prelucrăm datele într-un mod expresiv și concis, fără a recurge la bucle explicite sau la mutarea stării.

Un concept important pe care îl vom aborda este cel de "list comprehension", o construcție elegantă și puternică pentru generarea de liste pe baza unor expresii și condiții. List comprehension ne permite să definim liste într-un mod declarativ, similar cu noțiunea de "set-builder notation" din matematică.

Pe parcursul acestei secțiuni, vom oferi exemple concrete și exerciții practice pentru a ilustra conceptele prezentate. La finalul secțiunii, veți fi capabili să utilizați colecțiile în F# cu încredere și să aplicați tehnici de programare funcțională pentru a prelucra datele în mod eficient și expresiv.

#### 4.5.1. Liste (List)

Listele sunt una dintre cele mai utilizate structuri de date în F#. O listă este o colecție ordonată de elemente de același tip, care poate fi de lungime variabilă. Listele în F# sunt implementate ca liste simplu înlănțuite, ceea ce înseamnă că fiecare element conține o referință către următorul element din listă.

Pentru a defini o listă în F#, putem folosi constructorul de liste `[ ... ]` și separa elementele prin punct-și-virgulă (`;`). De exemplu:

```
let numere = [1; 2; 3; 4; 5]
let cuvinte = ["alb"; "negru"; "roșu"]
```

De asemenea, putem defini liste goale utilizând `[]` sau funcția `List.empty`:

```
let listaGoala1 = []
let listaGoala2 = List.empty
```

Listele în F# sunt imutabile, ceea ce înseamnă că odată create, conținutul lor nu poate fi modificat. Cu toate acestea, putem crea cu ușurință noi liste pe baza celor existente, utilizând diverse funcții și operatori.

Câteva funcții utile pentru manipularea listelor sunt:

- `List.head`: Returnează primul element al unei liste.
- `List.tail`: Returnează lista fără primul element.
- `List.length`: Returnează numărul de elemente dintr-o listă.
- `List.isEmpty`: Verifică dacă o listă este goală.
- `List.item`: Accesează un element al listei la un anumit index.
- `List.append`: Concatenează două liste.
- `List.concat`: Concatenează o listă de liste.
- `List.rev`: Inversează ordinea elementelor dintr-o listă.

Exemple:

```
let fructe = ["măr"; "banană"; "portocală"]
let primulFruct = List.head fructe // "măr"
let restulFructelor = List.tail fructe // ["banană"; "portocală"]
let numarFructe = List.length fructe // 3
```

```
let esteGoala = List.isEmpty fructe // false
let alDoileaFruct = List.item 1 fructe // "banană"
```

```
let maiMulteFructe = List.append fructe ["kiwi"; "ananas"] // ["măr";
"banană"; "portocală"; "kiwi"; "ananas"]
let fructeInverse = List.rev fructe // ["portocală"; "banană"; "măr"]
```

Listele sunt deosebit de utile în combinație cu pattern matching și recursivitate, permițând procesarea elementelor într-un mod elegant și expresiv. Ele sunt, de asemenea, fundamentale pentru multe alte funcții și operatori din biblioteca standard F#.

#### 4.5.2. Tablouri

Pe lângă liste, o altă structură de date importantă în F# este tabloul (array). Un tablou este o colecție de elemente de același tip, cu o dimensiune fixă, care permite accesul aleator la elemente prin intermediul indexului.

Pentru a defini un tablou în F#, putem folosi constructorul de tablouri `[| ... |]` și separă elementele prin punct-și-virgulă (`;`). De exemplu:

```
let numere = [|1; 2; 3; 4; 5|]
let cuvinte = [|"alb"; "negru"; "roșu"|]
```

Spre deosebire de liste, tablourile sunt mutabile, ceea ce înseamnă că putem modifica elementele existente folosind operatorul `<-`. De exemplu:

```
let fructe = [|"măr"; "banană"; "portocală"|]
fructe.[1] <- "kiwi" // [|"măr"; "kiwi"; "portocală"|]
```

Câteva funcții utile pentru manipularea tablourilor sunt:

- `Array.length`: Returnează numărul de elemente dintr-un tablou.
- `Array.get`: Accesează un element al tabloului la un anumit index.
- `Array.set`: Modifică valoarea unui element al tabloului la un anumit index.
- `Array.append`: Concatenează două tablouri.
- `Array.create`: Creează un tablou cu o anumită dimensiune și valoare inițială.
- `Array.copy`: Creează o copie a unui tablou.
- `Array.sub`: Extrage o secțiune a unui tablou.

Exemple:

```
let tablou1 = [|1; 2; 3|]
let tablou2 = [|4; 5; 6|]
```

```
let lungime = Array.length tablou1 // 3
let element = Array.get tablou1 1 // 2
```

```
Array.set tablou1 1 10 // [|1; 10; 3|]
```

```
let tablouConcat = Array.append tablou1 tablou2 // [|1; 10; 3; 4; 5; 6|]  
let tablouNou = Array.create 5 0 // [|0; 0; 0; 0; 0|]  
let copie = Array.copy tablou1 // [|1; 10; 3|]  
let sectiune = Array.sub tablou2 1 2 // [|5; 6|]
```

Deși tablourile sunt mutabile și permit accesul aleator eficient, în programarea funcțională se recomandă utilizarea structurilor de date imutabile, cum ar fi listele, ori de câte ori este posibil. Acest lucru promovează un stil de programare mai sigur și mai ușor de raționat.

Cu toate acestea, tablourile pot fi utile în anumite situații, cum ar fi interoperabilitatea cu biblioteci externe sau atunci când performanța este critică și accesul aleator este necesar.

### 4.5.3. Secvențe (Seq)

Secvențele (Seq) sunt o reprezentare temporizată (lazy) a colecțiilor în F#. Spre deosebire de liste și tablouri, care sunt evaluate în întregime atunci când sunt create, secvențele sunt evaluate la cerere, pe măsură ce elementele sunt accesate. Aceasta permite prelucrarea eficientă a colecțiilor mari sau chiar infinite.

Secvențele sunt definite folosind constructorul `seq { ... }`, care ia o expresie ce generează elementele secvenței. De exemplu:

```
let numere = seq { yield 1; yield 2; yield 3 }  
let patrate = seq { for i in 1..10 do yield i * i }
```

Secvențele pot fi generate folosind diverse construcții, cum ar fi `yield`, `yield!`, `for`, `while` și altele. De asemenea, multe funcții din bibliotecă returnează secvențe, cum ar fi `Seq.initInfinite`, `Seq.unfold`, `Seq.generate`, etc.

Câteva funcții utile pentru manipularea secvențelor sunt:

- `Seq.head`: Returnează primul element al unei secvențe.
- `Seq.tail`: Returnează secvența fără primul element.
- `Seq.length`: Returnează numărul de elemente dintr-o secvență.
- `Seq.item`: Accesează un element al secvenței la un anumit index.
- `Seq.map`: Aplică o funcție fiecărui element al secvenței.
- `Seq.filter`: Filtrează elementele secvenței pe baza unei condiții.
- `Seq.fold`: Reduce secvența la o singură valoare folosind o funcție de acumulare.
- `Seq.iter`: Execută o acțiune pentru fiecare element al secvenței.

Exemple:

```
let secventa = seq { yield "A"; yield "B"; yield "C" }
```



```
let primulElement = Seq.head secventa // "A"
let restulSecventei = Seq.tail secventa // seq ["B"; "C"]
let lungime = Seq.length secventa // 3
let alDoileaElement = Seq.item 1 secventa // "B"

let secventaDublata = Seq.map (fun x -> x + x) secventa // seq ["AA"; "BB"; "CC"]
let secventaFiltrata = Seq.filter (fun x -> x <> "B") secventa // seq ["A"; "C"]
let suma = Seq.fold (fun acc x -> acc + x) 0 [1..10] // 55
Seq.iter (fun x -> printfn "%s" x) secventa // Afișează "A", "B", "C" pe linii separate
```

Secvențele sunt deosebit de utile atunci când lucrăm cu colecții mari sau infinite, deoarece evaluarea lor întârziată ne permite să procesăm elementele pe rând, fără a consuma întreaga colecție în memorie. De asemenea, multe funcții din biblioteca standard F# sunt optimizate pentru a lucra cu secvențe, permițând compunerea facilă a operațiilor.

Este important de menționat că, deoarece secvențele sunt evaluate întârziat, unele operații care necesită evaluarea completă a secvenței (cum ar fi `Seq.length` sau `Seq.item`) pot fi ineficiente sau chiar să nu se termine pentru secvențe infinite.

#### 4.5.4.Map și Set

Pe lângă liste, tablouri și secvențe, F# oferă două structuri de date imutabile specializate: Map și Set.

##### Map

Un Map (sau dicționar) este o colecție de perechi cheie-valoare, unde cheile sunt unice. Map-urile sunt utile pentru a asocia valori cu chei și pentru a accesa valorile în mod eficient pe baza cheilor.

Pentru a crea un Map, putem folosi constructorul `Map.ofList`, care primește o listă de tuple-uri (cheie, valoare):

```
let dictionar = Map.ofList [("A", 1); ("B", 2); ("C", 3)]
```

Câteva funcții utile pentru lucrul cu map-uri sunt:

- `Map.containsKey`: Verifică dacă o cheie există în map.
- `Map.find`: Returnează valoarea asociată unei chei specifice.
- `Map.tryFind`: Încearcă să găsească valoarea asociată unei chei, returnând un `option`.
- `Map.add`: Adaugă o nouă pereche cheie-valoare într-un map.
- `Map.remove`: Elimină o pereche cheie-valoare dintr-un map.
- `Map.map`: Aplică o funcție valorilor dintr-un map.
- `Map.iter`: Execută o acțiune pentru fiecare pereche cheie-valoare dintr-un map.

Exemple:

```
let dictionar = Map.ofList [("A", 1); ("B", 2); ("C", 3)]

let contineA = Map.containsKey "A" dictionar // true
let valoareaB = Map.find "B" dictionar // 2
let valoareaD = Map.tryFind "D" dictionar // None

let dictionarNou = Map.add "D" 4 dictionar // map [("A", 1); ("B", 2); ("C", 3); ("D", 4)]
let dictionarFaraB = Map.remove "B" dictionar // map [("A", 1); ("C", 3)]

let dictionarDublu = Map.map (fun _ v -> v * 2) dictionar // map [("A", 2); ("B", 4); ("C", 6)]
Map.iter (fun k v -> printfn "%s: %d" k v) dictionar // Afișează perechile cheie-valoare
```

## Set

Un Set este o colecție de elemente unice, fără o ordine specifică. Seturile sunt utile pentru a stoca și a testa apartenența elementelor, precum și pentru a efectua operații de mulțime, cum ar fi reuniunea, intersecția și diferența.

Pentru a crea un Set, putem folosi constructorul `Set.ofList`, care primește o listă de elemente:

```
let multime = Set.ofList [1; 2; 3; 4; 5]
```

Câteva funcții utile pentru lucrul cu seturi sunt:

- `Set.contains`: Verifică dacă un element există în set.
- `Set.add`: Adaugă un element nou în set.
- `Set.remove`: Elimină un element din set.
- `Set.union`: Returnează reuniunea a două seturi.
- `Set.intersect`: Returnează intersecția a două seturi.
- `Set.difference`: Returnează diferența între două seturi.
- `Set.isSubset`: Verifică dacă un set este subset al altui set.
- `Set.iter`: Execută o acțiune pentru fiecare element din set.

Exemple:

```
let multime = Set.ofList [1; 2; 3; 4; 5]

let contine3 = Set.contains 3 multime // true
let multimeNoua = Set.add 6 multime // set [1; 2; 3; 4; 5; 6]
let multimeFara4 = Set.remove 4 multime // set [1; 2; 3; 5]

let reuniune = Set.union multime (Set.ofList [4; 5; 6]) // set [1; 2; 3; 4; 5; 6]
let intersecție = Set.intersect multime (Set.ofList [3; 4; 5; 6]) // set [3; 4; 5]
```

```
let diferenta = Set.difference multime (Set.ofList [4; 5; 6]) // set [1; 2; 3]

let esteSubset = Set.isSubset (Set.ofList [1; 2]) multime // true

Set.iter (fun x -> printfn "%d" x) multime // Afișează elementele setului
```

Atât Map, cât și Set sunt structuri de date imutabile, ceea ce înseamnă că operațiile care modifică aceste structuri (cum ar fi `Map.add`, `Map.remove`, `Set.add`, `Set.remove`) returnează o nouă instanță a structurii de date, lăsând neschimbată instanța originală.

Aceste structuri de date sunt eficiente și oferă operații cu o complexitate de timp bună, bazate pe tabele de dispersie (hash tables) și arbori echilibrați. Ele sunt deosebit de utile în situațiile în care avem nevoie de acces rapid pe baza cheilor (în cazul Map) sau de testarea apartenenței și operații de mulțime (în cazul Set).

#### 4.5.5. Operații comune pe colecții

F# oferă o gamă largă de funcții pentru efectuarea operațiilor comune pe colecții, cum ar fi maparea, filtrarea, reducerea, concatenarea, sortarea și gruparea. Aceste funcții sunt disponibile pentru toate tipurile de colecții discutate anterior (liste, tablouri, secvențe, map-uri și set-uri) și sunt esențiale pentru prelucrarea eficientă a datelor în stil funcțional.

Să explorăm câteva dintre cele mai utilizate operații pe colecții:

**map:** Aplică o funcție fiecărui element al colecției și returnează o nouă colecție cu rezultatele.

```
let numere = [1; 2; 3; 4; 5]
let patrate = List.map (fun x -> x * x) numere // [1; 4; 9; 16; 25]
```

**filter:** Filtrează elementele unei colecții pe baza unei condiții și returnează o nouă colecție cu elementele care satisfac condiția.

```
let numere = [1; 2; 3; 4; 5]
let numerePare = List.filter (fun x -> x % 2 = 0) numere // [2; 4]
```

**fold:** Reduce colecția la o singură valoare aplicând o funcție de acumulare elementelor în ordine.

```
let numere = [1; 2; 3; 4; 5]
let suma = List.fold (fun acc x -> acc + x) 0 numere // 15
```

**reduce:** Similar cu **fold**, dar fără o valoare inițială de acumulare. Funcționează doar pe colecții nevide.

```
let numere = [1; 2; 3; 4; 5]
let produs = List.reduce (fun acc x -> acc * x) numere // 120
```

**iter:** Execută o acțiune pentru fiecare element al colecției, fără a returna o nouă colecție.

```
let numere = [1; 2; 3; 4; 5]
List.iter (fun x -> printfn "%d" x) numere // Afișează numerele pe linii separate
```

**collect**: Aplică o funcție fiecărui element al colecției, unde funcția returnează o colecție. Rezultatele sunt concatenate într-o singură colecție.

```
let numere = [1; 2; 3; 4; 5]
let numerePare = List.collect (fun x -> [x; x+1]) numere // [1; 2; 2; 3; 3; 4; 4; 5; 5; 6]
```

**sort**, **sortBy**, **sortBy**: Sortează elementele unei colecții folosind diferite criterii de comparare.

```
let numere = [3; 1; 4; 2; 5]
let numereSortate = List.sort numere // [1; 2; 3; 4; 5]
let cuvinte = ["mere"; "pere"; "banane"; "portocale"]
let cuvinteSortateDupaLungime = List.sortBy (fun s -> s.Length) cuvinte // ["mere"; "pere"; "banane"; "portocale"]
```

**groupBy**: Grupează elementele unei colecții pe baza unei funcții cheie și returnează un map de la cheie la o listă de elemente cu aceeași cheie.

```
let fructe = ["mere"; "pere"; "banane"; "portocale"; "kiwi"; "ananas"]
let grupeDeFructe = List.groupBy (fun s -> s.Length) fructe // map [(4, ["mere"; "pere"; "kiwi"]); (6, ["banane"; "ananas"]); (9, ["portocale"])]
```

Aceste operații sunt extrem de utile în combinație cu pipeline-ul de date (**|>**) și compoziția funcțională, permițând construirea unor transformări complexe și expresive ale colecțiilor într-un mod concis și lizibil.

De exemplu, iată cum am putea combina câteva operații pentru a procesa o listă de numere:

```
let numere = [1..10]
let rezultat =
    numere
    |> List.filter (fun x -> x % 2 = 0)
    |> List.map (fun x -> x * x)
    |> List.fold (fun acc x -> acc + x) 0
// rezultat = 220 (suma pătratelor numerelor pare între 1 și 10)
```

Aceste operații sunt disponibile pentru toate tipurile de colecții din F#, cu mici variații în nume (de exemplu, **Seq.map**, **Array.filter** etc.), permițând o utilizare consecventă și uniformă a acestor funcții puternice indiferent de tipul de colecție.

În rezumat, operațiile comune pe colecții, cum ar fi **map**, **filter**, **fold**, **reduce**, **iter**, **collect**, **sort** și **groupBy**, sunt instrumente esențiale în arsenalul programatorului funcțional F#. Ele permit prelucrarea eficientă și expresivă a datelor, promovând un stil de programare declarativ și compozițional. Prin combinarea acestor operații cu pipeline-

ul de date și compoziția funcțională, putem construi transformări complexe ale colecțiilor într-un mod elegant și ușor de înțeles.

#### 4.5.6. List comprehension

List comprehension este o construcție puternică în F# care permite definirea concisă a listelor pe baza unei expresii și a unor generatori. Aceasta oferă o modalitate elegantă și expresivă de a crea noi liste bazate pe liste existente sau pe alte surse de date.

Sintaxa de bază pentru list comprehension este următoarea:

```
[ expression | generator1 ; ... ; generatorN ; condition ]
```

Unde:

- **expression** este expresia care definește elementele noii liste.
- **generator1**, ..., **generatorN** sunt generatorii care produc valorile pentru **expression**. Un generator are forma **pat in expr**, unde **pat** este un model (pattern) care se potrivește cu elementele din **expr**.
- **condition** este o expresie opțională de filtrare care limitează elementele incluse în lista rezultată.

Iată câteva exemple pentru a ilustra utilizarea list comprehension:

*Generarea unei liste de pătrate ale numerelor de la 1 la 10:*

```
let patrate = [ for i in 1..10 -> i * i ]
// patrate = [1; 4; 9; 16; 25; 36; 49; 64; 81; 100]
```

*Filtrarea numerelor pare dintr-o listă existentă:*

```
let numere = [1..10]
let numerePare = [ for i in numere do if i % 2 = 0 then yield i ]
// numerePare = [2; 4; 6; 8; 10]
Generarea unei liste de perechi (tuple) între două liste:
let lista1 = [1; 2; 3]
let lista2 = ['a'; 'b'; 'c']
let perechi = [ for i in lista1 do for j in lista2 -> (i, j) ]
// perechi = [(1, 'a'); (1, 'b'); (1, 'c'); (2, 'a'); (2, 'b'); (2, 'c'); (3, 'a'); (3, 'b'); (3, 'c')]
```

*Folosirea pattern matching în generator pentru a extrage elementele unui tip de date algebric:*

```
type Persoana = { Nume: string; Varsta: int }

let persoane = [ { Nume = "Alice"; Varsta = 25 }; { Nume = "Bob"; Varsta = 30 }; { Nume = "Carol"; Varsta = 35 } ]
let nume = [ for { Nume = n } in persoane -> n ]
// nume = ["Alice"; "Bob"; "Carol"]
```

List comprehension poate fi, de asemenea, utilizată cu alte tipuri de colecții, cum ar fi secvențe (**seq**) și tablouri (**array**), prin înlocuirea parantezelor pătrate (**[ ]**) cu constructorii corespunzători (**seq { }** pentru secvențe, **[| |]** pentru tablouri).

Un avantaj major al list comprehension este că permite generarea listelor într-un mod declarativ, fără a fi nevoie de a scrie explicit recursivitate sau bucle. De asemenea, expresivitatea și concizia list comprehension pot îmbunătăți lizibilitatea codului, făcând intenția programatorului mai clară.

Cu toate acestea, este important să folosim list comprehension cu moderație și să ne asigurăm că expresiile rămân ușor de înțeles. În cazul unor transformări complexe, ar putea fi mai clar să folosim o combinație de funcții precum `map`, `filter` și `fold`, în loc de a încălca list comprehension cu prea multe condiții și generatori.

## 4.6. Operatori și pipe

În limbajul F#, operatorii și pipe-urile sunt elemente esențiale care facilitează compunerea funcțiilor și crearea unui cod concis și expresiv. Acestea permit combinarea funcțiilor într-un mod natural și lizibil, permițând transformarea datelor prin aplicarea succesivă a operațiilor.

În această secțiune, vom explora în detaliu principalii operatori și pipe-uri disponibile în F#:

- Operatorul pipe-forward (`|>`): Acest operator ia rezultatul expresiei din stânga și îl pasează ca ultimul argument funcției din dreapta sa. Este deosebit de util pentru a crea un flux de prelucrare a datelor, aplicând în mod secvențial mai multe funcții asupra unui set de date.
- Operatorul pipe-backward (`<|`): Similar cu operatorul pipe-forward, operatorul pipe-backward pasează rezultatul expresiei din dreapta sa ca prim argument funcției din stânga sa. Deși este folosit mai rar decât pipe-forward, acesta poate fi util în anumite situații pentru a îmbunătăți lizibilitatea codului.
- Operatorul de compunere (`>>`): Acest operator permite compunerea a două funcții, unde rezultatul primei funcții este pasat automat ca argument celei de-a doua funcții. Este utilizat pentru a crea noi funcții prin combinarea funcțiilor existente.
- Operatorul de compunere inversă (`<<`): Similar cu operatorul de compunere, operatorul de compunere inversă combină două funcții, dar în ordine inversă. Rezultatul celei de-a doua funcții este pasat automat ca argument primei funcții.

Pe parcursul acestei secțiuni, vom analiza în detaliu fiecare dintre acești operatori, sintaxa și semantica lor, precum și exemple relevante de utilizare. De asemenea, vom discuta despre operatorii personalizați și despre modul în care puteți defini proprii operatori în F# pentru a vă adapta nevoilor specifice ale proiectului.

Un alt aspect important pe care îl vom acoperi este precedența operatorilor, care determină ordinea în care operatorii sunt evaluați atunci când sunt combinați în expresii complexe. Înțelegerea precedenței operatorilor este esențială pentru a scrie un cod corect și previzibil.

La finalul acestei secțiuni, veți fi capabili să:

- Utilizați operatorii pipe-forward și pipe-backward pentru a crea fluxuri de prelucrare a datelor
- Compuneți funcții utilizând operatorii de compunere și compunere inversă
- Definiți operatori personalizați pentru a vă extinde limbajul în funcție de nevoile dvs.
- Înțelegeți și să aplicați corect precedența operatorilor în expresii complexe

#### 4.6.1.Operatorul pipe-forward (|>)

Operatorul pipe-forward (|>) este unul dintre cei mai frecvent utilizați operatori în F#. Acesta permite pasarea rezultatului unei expresii ca ultim argument unei funcții, oferind un mod natural de a crea un flux de prelucrare a datelor.

Să analizăm sintaxa operatorului pipe-forward:

```
expresie |> funcție
```

Aici, expresie reprezintă valoarea sau rezultatul unei computații, iar funcție este o funcție care primește un singur argument. Operatorul |> ia rezultatul expresiei și îl pasează ca argument funcției.

Iată un exemplu simplu:

```
let square x = x * x
let result = 5 |> square
```

În acest caz, valoarea 5 este pasată ca argument funcției `square` folosind operatorul |>, iar rezultatul este atribuit variabilei `result`. Acest cod este echivalent cu:

```
let square x = x * x
let result = square 5
```

Deși acest exemplu este trivial, puterea reală a operatorului pipe-forward devine evidentă atunci când este folosit pentru a compune mai multe funcții și a crea un flux de prelucrare a datelor. Să considerăm următorul exemplu:

```
let numbers = [1; 2; 3; 4; 5]
let result =
    numbers
    |> List.map square
    |> List.filter (fun x -> x > 10)
    |> List.sum
```

În acest cod, pornim cu o listă de numere (`numbers`). Folosind operatorul pipe-forward, aplicăm succesiv trei funcții asupra listei:

- `List.map square`: Această funcție aplică funcția `square` asupra fiecărui element din listă, generând o nouă listă cu pătratele numerelor.

- `List.filter (fun x -> x > 10)`: Această funcție filtrează lista rezultată, păstrând doar elementele mai mari decât 10.
- `List.sum`: În final, aplicăm funcția `List.sum` pentru a calcula suma elementelor rămase în listă.

Rezultatul final este atribuit variabilei `result`.

Utilizarea operatorului pipe-forward ne permite să scriem codul într-un mod secvențial și lizibil, unde datele "curg" prin diferite etape de prelucrare. Fiecare funcție primește rezultatul etapei anterioare ca argument, făcând codul ușor de urmărit și de înțeles.

Operatorul pipe-forward este deosebit de util în combinație cu funcțiile din modulele F# standard, cum ar fi `List`, `Array`, `Seq` și altele, permițând aplicarea facilă a operațiilor comune de prelucrare a datelor.

#### 4.6.2.Operatorul pipe-backward (<|)

Operatorul pipe-backward (<|) este similar cu operatorul pipe-forward (|>), dar pasează rezultatul expresiei din dreapta sa ca prim argument funcției din stânga sa. Deși este mai puțin utilizat decât operatorul pipe-forward, acesta poate fi util în anumite situații pentru a îmbunătăți lizibilitatea codului.

Sintaxa operatorului pipe-backward este următoarea:

```
funcție <| expresie
```

Aici, funcție reprezintă o funcție care primește un singur argument, iar expresie este valoarea sau rezultatul unei computații. Operatorul <| ia rezultatul expresiei și îl pasează ca prim argument funcției.

Să considerăm un exemplu simplu:

```
let square x = x * x
let result = square <| 5
```

În acest caz, valoarea 5 este pasată ca argument funcției `square` folosind operatorul <|, iar rezultatul este atribuit variabilei `result`. Acest cod este echivalent cu:

```
let square x = x * x
let result = square 5
```

Deși operatorul pipe-backward poate fi folosit în multe situații în locul apelului obișnuit de funcții, acesta devine util atunci când dorim să combinăm mai multe funcții sau expresii complexe.

Să analizăm un exemplu mai elaborat:

```
let numbers = [1; 2; 3; 4; 5]
let result = List.sum <| List.filter (fun x -> x > 3) <| List.map square
numbers
```



În acest cod, pornim cu o listă de numere (`numbers`) și aplicăm succesiv trei operații:

- `List.map square`: Această funcție aplică funcția `square` asupra fiecărui element din `numbers`, generând o nouă listă cu pătratele numerelor.
- `List.filter (fun x -> x > 3)`: Această funcție filtrează lista rezultată, păstrând doar elementele mai mari decât 3.
- `List.sum`: În final, aplicăm funcția `List.sum` pentru a calcula suma elementelor rămase în listă.

Folosind operatorul pipe-backward, rezultatul fiecărei operații este pasat ca argument funcției următoare. Acest stil de codare poate fi util când dorim să aplicăm o serie de transformări asupra unui set de date, fără a crea variabile intermediare.

### 4.6.3.Operatorul de compunere (>>)

Operatorul de compunere (>>) în F# permite combinarea a două funcții pentru a crea o nouă funcție. Rezultatul primei funcții este pasat automat ca argument celei de-a doua funcții, permițând astfel compunerea funcțiilor într-un mod concis și expresiv.

Să analizăm sintaxa operatorului de compunere:

```
let compusă = funcție1 >> funcție2
```

Aici, `funcție1` și `funcție2` sunt două funcții, iar `compusă` este o nouă funcție care reprezintă compunerea celor două. Atunci când funcția `compusă` este apelată cu un argument, acesta este mai întâi pasat funcției `funcție1`, iar rezultatul acesteia este apoi pasat ca argument funcției `funcție2`.

Iată un exemplu simplu de utilizare a operatorului de compunere:

```
let square x = x * x
let double x = x * 2
let squareThenDouble = square >> double

let result = squareThenDouble 5
```

În acest cod, definim două funcții: `square`, care calculează pătratul unui număr, și `double`, care dublează un număr. Folosind operatorul de compunere (>>), creăm o nouă funcție `squareThenDouble`, care reprezintă compunerea funcțiilor `square` și `double`.

Când apelăm `squareThenDouble` cu argumentul 5, acesta este mai întâi pasat funcției `square`, care returnează 25. Apoi, rezultatul (25) este pasat ca argument funcției `double`, care returnează 50. În final, valoarea 50 este atribuită variabilei `result`.

Operatorul de compunere este deosebit de util atunci când lucrăm cu funcții care primesc un singur argument și returnează un singur rezultat. Prin compunerea funcțiilor, putem crea lanțuri de prelucrare a datelor, aplicând succesiv diferite operații asupra valorilor.

Să considerăm un exemplu mai complex:

```
let numbers = [1; 2; 3; 4; 5]
let transformNumbers = List.map square >> List.filter (fun x -> x > 10) >>
List.sum

let result = transformNumbers numbers
```

În acest exemplu, definim o funcție `transformNumbers` care reprezintă compunerea a trei operații: aplicarea funcției `square` asupra fiecărui element al listei (`List.map square`), filtrarea elementelor mai mari decât 10 (`List.filter (fun x -> x > 10)`) și calcularea sumei elementelor rămase (`List.sum`).

Când apelăm `transformNumbers` cu lista `numbers`, aceasta trece prin întregul lanț de prelucrare definit de funcțiile compuse, rezultând în valoarea finală atribuită variabilei `result`.

Operatorul de compunere ne permite să scriem un cod mai modular și reutilizabil, deoarece putem defini funcții mici și specializate și apoi să le combinăm în diferite moduri pentru a crea operații mai complexe.

De asemenea, este important de menționat că putem compune mai mult de două funcții folosind operatorul `>>`. De exemplu:

```
let compusă = funcție1 >> funcție2 >> funcție3 >> funcție4
```

În acest caz, rezultatul funcției `funcție1` este pasat ca argument funcției `funcție2`, rezultatul acesteia este pasat funcției `funcție3`, și așa mai departe, până la ultima funcție din lanțul de compunere.

Operatorul de compunere (`>>`) este un instrument puternic în arsenalul programatorului F#, permițând crearea de funcții complexe prin combinarea funcțiilor mai simple. Folosirea judicioasă a acestui operator poate duce la un cod mai expresiv, modular și ușor de întreținut.

#### 4.6.4.Operatorul de compunere inversă (<<)

Operatorul de compunere inversă (`<<`) în F# este similar cu operatorul de compunere (`>>`), dar combină funcțiile în ordine inversă. În loc să paseze rezultatul primei funcții ca argument celei de-a doua funcții, operatorul `<<` pasează rezultatul celei de-a doua funcții ca argument primei funcții.

Să analizăm sintaxa operatorului de compunere inversă:

```
let compusăInversă = funcție2 << funcție1
```

Aici, `funcție1` și `funcție2` sunt două funcții, iar `compusăInversă` este o nouă funcție care reprezintă compunerea inversă a celor două. Atunci când funcția `compusăInversă` este

apelată cu un argument, acesta este mai întâi pasat funcției `funcție1`, iar rezultatul acesteia este apoi pasat ca argument funcției `funcție2`.

Iată un exemplu simplu de utilizare a operatorului de compunere inversă:

```
let square x = x * x
let addOne x = x + 1
let addOneThenSquare = square << addOne
let result = addOneThenSquare 5
```

În acest cod, definim două funcții: `square`, care calculează pătratul unui număr, și `addOne`, care adaugă 1 la un număr. Folosind operatorul de compunere inversă (`<<`), creăm o nouă funcție `addOneThenSquare`, care reprezintă compunerea inversă a funcțiilor `square` și `addOne`.

Când apelăm `addOneThenSquare` cu argumentul 5, acesta este mai întâi pasat funcției `addOne`, care returnează 6. Apoi, rezultatul (6) este pasat ca argument funcției `square`, care returnează 36. În final, valoarea 36 este atribuită variabilei `result`.

Deși operatorul de compunere inversă poate părea contra-intuitiv la început, acesta devine util în anumite situații, în special atunci când lucrăm cu funcții care iau ca argument funcții cu un singur parametru.

Să considerăm un exemplu mai complex:

```
let numbers = [1; 2; 3; 4; 5]
let transformNumbers = List.sum << List.filter (fun x -> x > 3) << List.map square
let result = transformNumbers numbers
```

În acest exemplu, definim o funcție `transformNumbers` care reprezintă compunerea inversă a trei operații: aplicarea funcției `square` asupra fiecărui element al listei (`List.map square`), filtrarea elementelor mai mari decât 3 (`List.filter (fun x -> x > 3)`) și calcularea sumei elementelor rămase (`List.sum`).

Ordinea operațiilor în compunerea inversă este de la dreapta la stânga. Astfel, când apelăm `transformNumbers` cu lista `numbers`, aceasta trece mai întâi prin `List.map square`, apoi prin `List.filter (fun x -> x > 3)` și în final prin `List.sum`, rezultând în valoarea finală atribuită variabilei `result`.

#### 4.6.5. Operatori personalizați

F# oferă posibilitatea de a defini operatori personalizați, permițându-vă să extindeți limbajul cu operații specifice domeniului sau problemei pe care o rezolvați. Operatorii personalizați pot face codul mai expresiv și mai ușor de citit, atunci când sunt folosiți în mod corespunzător.

Pentru a defini un operator personalizat în F#, trebuie să urmați câteva reguli:

- Numele operatorului trebuie să fie format din unul sau mai multe caractere din următoarea listă: `!, %, &, *, +, -, ., /, <, =, >, ?, @, ^, |, ~`.

Operatorul trebuie să fie precedat de cuvântul cheie `let` sau `let inline`, urmat de numele operatorului încadrat în paranteze `()`.

- Operatorul trebuie să aibă cel puțin un parametru, specificat după numele operatorului.

Iată un exemplu de definire a unui operator personalizat pentru adunarea a două tuple-uri:

```
let (+) (a, b) (c, d) = (a + c, b + d)
```

În acest caz, definim operatorul `+` care primește două tuple-uri `(a, b)` și `(c, d)` și returnează un tuple cu suma elementelor corespunzătoare.

Putem utiliza acest operator personalizat în felul următor:

```
let tuple1 = (1, 2)
let tuple2 = (3, 4)
let result = tuple1 + tuple2 // result va fi (4, 6)
```

Operatorii personalizați pot fi, de asemenea, metode ale unui tip, permițând utilizarea lor în contextul aceluși tip. De exemplu, putem defini un operator personalizat pentru un tip `Vector2D`:

```
type Vector2D(x: float, y: float) =
    member this.X = x
    member this.Y = y
    static member (+) (v1: Vector2D, v2: Vector2D) =
        Vector2D(v1.X + v2.X, v1.Y + v2.Y)
```

Aici, definim un operator `+` ca metodă statică a tipului `Vector2D`, care primește doi vectori și returnează un nou vector cu suma coordonatelor.

Putem folosi acest operator astfel:

```
let v1 = Vector2D(1.0, 2.0)
let v2 = Vector2D(3.0, 4.0)
let result = v1 + v2 // result va fi un Vector2D cu coordonatele (4.0, 6.0)
```

Operatorii personalizați pot îmbunătăți semnificativ lizibilitatea și expresivitatea codului, atunci când sunt folosiți cu moderație și în contextul potrivit. Cu toate acestea, este important să nu abuzăm de operatorii personalizați și să îi folosim doar atunci când aduc o valoare reală codului și domeniului problemei.

De asemenea, atunci când definiți operatori personalizați, asigurați-vă că respectați convențiile și principiile de design ale limbajului F#, precum și că alegeți nume sugestive și ușor de înțeles pentru operatori.

#### 4.6.6. Precedența operatorilor

Atunci când se evaluează expresii complexe care implică mai mulți operatori, ordinea în care acești operatori sunt aplicați este determinată de regulile de precedență a operatorilor. Fiecare operator în F# are o prioritate asociată, care dictează ordinea în care operațiile sunt efectuate.

Iată o listă parțială a operatorilor în F#, în ordine descrescătoare a precedenței:

1. Operatorul de funcție `|>` și `<|`
2. Operatorii de compunere `>>` și `<<`
3. Operatorul de aplicare a funcției `<|`
4. Operatorii aritmetici: `**`, `*`, `/`, `%`, `+`, `-`
5. Operatorii de comparație: `=`, `<>`, `<`, `>`, `<=`, `>=`, `?=`
6. Operatorii logici: `not`, `&&`, `||`
7. Operatorul de atribuire: `<-`

Când o expresie conține operatori cu aceeași precedență, ordinea de evaluare este determinată de asociativitatea lor. Majoritatea operatorilor în F# au asociativitate la stânga, ceea ce înseamnă că operațiile sunt efectuate de la stânga la dreapta. Cu toate acestea, unii operatori, cum ar fi `|>`, `<|`, `>>`, `<<` și `**`, au asociativitate la dreapta.

Să analizăm câteva exemple pentru a înțelege mai bine precedența operatorilor:

```
let result = 2 + 3 * 4 // result va fi 14
```

În acest caz, operatorul `*` are prioritate mai mare decât operatorul `+`, deci expresia `3 * 4` este evaluată primul, iar apoi rezultatul este adunat cu `2`.

```
let result = 2 ** 3 ** 4 // result va fi 4096
```

Aici, operatorul `**` are asociativitate la dreapta, deci expresia `3 ** 4` este evaluată primul (rezultând 81), iar apoi `2 ** 81` este evaluat, rezultând 4096.

Puteți utiliza paranteze pentru a controla explicit ordinea de evaluare a operatorilor:

```
let result = (2 + 3) * 4 // result va fi 20
```

În acest exemplu, expresia `2 + 3` este evaluată primul, datorită parantezelor, iar apoi rezultatul este înmulțit cu `4`.

Când utilizați operatori personalizați, precedența și asociativitatea lor sunt determinate de prima literă a numelui operatorului. De exemplu, operatorii care încep cu `*` au aceeași precedență ca și operatorul de înmulțire `*`, iar operatorii care încep cu `+` au aceeași

precedență ca și operatorul de adunare `+`. Dacă doriți să definiți o precedență sau o asociativitate diferită pentru un operator personalizat, puteți utiliza directiva `infix`.

## 4.7. Tipuri de date algebrice

În limbajul de programare F#, tipurile de date algebrice (Algebraic Data Types - ADT) reprezintă o modalitate puternică și flexibilă de a modela și a structura datele. Acestea permit definirea de tipuri de date personalizate, care pot fi utilizate pentru a reprezenta informații complexe și ierarhice într-un mod concis și expresiv.

Tipurile de date algebrice se bazează pe conceptele de tipuri înregistrare (record types) și tipuri discriminate (discriminated unions), care pot fi combinate pentru a crea structuri de date bogate și variate. Aceste concepte fundamentale permit modelarea eficientă a scenariilor din lumea reală și facilitează scrierea unui cod clar, modular și ușor de întreținut.

În această secțiune, vom explora în detaliu principalele categorii de tipuri de date algebrice în F#:

- Tipuri înregistrare (record types): Acestea sunt tipuri de date care grupează mai multe câmpuri sub un singur nume, similar cu clasele în programarea orientată pe obiecte. Tipurile înregistrare sunt imutabile și permit accesul facil la câmpurile componente.
- Tipuri discriminate (discriminated unions): Acestea sunt tipuri de date care reprezintă o valoare ce poate avea una dintre mai multe forme posibile. Tipurile sumă sunt deosebit de utile pentru modelarea tipurilor de date recursive și ierarhice, precum și pentru reprezentarea stărilor sau opțiunilor.
- Tipuri discriminate cu un singur caz (single-case discriminated unions): Un caz particular al tipurilor sumă, aceste tipuri de date permit crearea de tipuri "wrapper" în jurul altor tipuri existente, oferind un mod de a adăuga informații suplimentare de tip și de a asigura utilizarea corectă a valorilor.
- Tipuri recursive: Tipurile de date algebrice în F# pot fi definite recursiv, ceea ce înseamnă că un tip poate conține referințe către sine în definiția sa. Această caracteristică este esențială pentru reprezentarea structurilor de date recursive, cum ar fi listele înlanțuite sau arborii.

Pe parcursul acestei secțiuni, vom analiza sintaxa și semantica fiecărui tip de date algebric, împreună cu exemple concrete de utilizare. De asemenea, vom discuta despre avantajele utilizării tipurilor de date algebrice, precum și despre cele mai bune practici în modelarea și structurarea datelor folosind aceste concepte.

La finalul acestei secțiuni, veți fi capabili să:

- Înțelegeți conceptele fundamentale ale tipurilor de date algebrice în F#
- Definiți și utilizați tipuri înregistrare, tipuri discriminate și variante ale acestora

- Modelați scenarii complexe folosind tipuri de date algebrice recursive
- Apreciați avantajele utilizării tipurilor de date algebrice în dezvoltarea de software funcțional

Stăpânirea tipurilor de date algebrice este esențială pentru a profita la maximum de puterea și expresivitatea limbajului F#. Acestea reprezintă un instrument fundamental în arsenalul programatorului funcțional, permițând modelarea precisă și flexibilă a structurilor de date și facilitând scrierea unui cod robust, modular și ușor de înțeles.

#### 4.7.1. Tipuri înregistrare (record)

Tipurile înregistrare în F# sunt o modalitate de a defini tipuri de date personalizate care grupează mai multe câmpuri sub un singur nume. Acestea sunt similare cu clasele în programarea orientată pe obiecte, dar cu câteva diferențe cheie. Tipurile înregistrare sunt imutabile în mod implicit și oferă o sintaxă concisă pentru crearea și accesarea valorilor.

Să analizăm sintaxa de bază pentru definirea unui tip înregistrare:

```
type NumeÎnregistrare = {
    Câmp1: TipCâmp1
    Câmp2: TipCâmp2
    ...
}
```

Aici, `NumeÎnregistrare` este numele tipului înregistrare, iar `Câmp1`, `Câmp2`, etc. sunt numele câmpurilor, fiecare având un tip specificat (`TipCâmp1`, `TipCâmp2`, etc.).

Să vedem un exemplu concret:

```
type Persoană = {
    Nume: string
    Vârstă: int
    Adresă: string
}
```

În acest caz, definim un tip înregistrare numit `Persoană`, cu trei câmpuri: `Nume` de tip `string`, `Vârstă` de tip `int` și `Adresă` de tip `string`.

Pentru a crea o valoare de tip înregistrare, folosim sintaxa `{ Câmp1 = valoare1; Câmp2 = valoare2; ... }`:

```
let persoană1 = { Nume = "John"; Vârstă = 30; Adresă = "Strada Principală 1"
}
```

Accesul la câmpurile unei valori de tip înregistrare se face folosind notația punct:

```
let nume = persoană1.Nume
let vârstă = persoană1.Vârstă
```

Deoarece tipurile înregistrare sunt imutabile, nu putem modifica direct valorile câmpurilor. Cu toate acestea, F# oferă o sintaxă convenabilă pentru a crea o nouă valoare de tip înregistrare cu anumite câmpuri actualizate, folosind constructorul `with`:

```
let persoană2 = { persoană1 with Vârstă = 31 }
```

În acest exemplu, `persoană2` este o nouă valoare de tip `Persoană`, cu aceleași valori pentru câmpurile `Nume` și `Adresă` ca și `persoană1`, dar cu câmpul `Vârstă` actualizat la 31.

Tipurile înregistrare suportă, de asemenea, parametri opționali și impliciți. Puteți specifica valori implicite pentru câmpuri folosind sintaxa `Câmp: Tip = ValoareImplicită`:

```
type Student = {
    Nume: string
    An: int
    Specializare: string
    MedieBac: float option
}

let student1 = { Nume = "Alice"; An = 1; Specializare = "Informatică";
MedieBac = None }
let student2 = { Nume = "Bob"; An = 2; Specializare = "Matematică"; MedieBac
= Some 9.5 }
```

În acest caz, câmpul `MedieBac` este de tip `float option`, permițând specificarea opțională a mediei de la bacalaureat.

#### 4.7.2. Multimi discriminate (discriminated unions)

Multimile discriminate, cunoscute și sub numele de tipuri discriminate, reprezintă un concept fundamental în F# pentru modelarea tipurilor de date care pot avea una dintre mai multe forme posibile. Acestea permit definirea unui tip de date ca o combinație de alte tipuri de date și sunt deosebit de utile pentru reprezentarea structurilor de date recursive, a stărilor sau a opțiunilor.

Să analizăm sintaxa de bază pentru definirea unei mulțimi discriminate:

```
type NumeMultime =
    | Caz1 of TipCaz1
    | Caz2 of TipCaz2
    | ...
    | CazN of TipCazN
```

Aici, `NumeMultime` este numele mulțimii discriminate, iar `Caz1`, `Caz2`, ..., `CazN` sunt cazurile posibile, fiecare fiind asociat cu un anumit tip de date (`TipCaz1`, `TipCaz2`, ..., `TipCazN`).

Să vedem un exemplu concret:

```
type Formă =
    | Cerc of rază: float
    | Dreptunghi of lungime: float * lățime: float
```



| Triunghi of bază: float \* înălțime: float

În acest caz, definim o mulțime discriminată numită **Formă**, cu trei cazuri posibile: **Cerc**, care are un câmp rază de tip **float**, **Dreptunghi**, care are două câmpuri lungime și lățime de tip **float**, și **Triunghi**, care are două câmpuri bază și înălțime de tip **float**.

Pentru a crea o valoare de tip mulțime discriminată, folosim constructorul corespunzător cazului dorit:

```
let cerc = Cerc(rază = 5.0)
let dreptunghi = Dreptunghi(lungime = 4.0, lățime = 3.0)
let triunghi = Triunghi(bază = 6.0, înălțime = 8.0)
```

Prelucrarea valorilor mulțimilor discriminate se face de obicei folosind corespondența de șabloane (pattern matching):

```
let aria formă =
    match formă with
    | Cerc rază -> System.Math.PI * rază ** 2.0
    | Dreptunghi (lungime, lățime) -> lungime * lățime
    | Triunghi (bază, înălțime) -> 0.5 * bază * înălțime
```

În acest exemplu, funcția **aria** primește o valoare de tip **Formă** și folosește pattern matching pentru a determina cazul specific și a calcula aria corespunzătoare.

Mulțimile discriminate pot fi, de asemenea, recursive, ceea ce înseamnă că un caz poate conține referințe către tipul mulțimii în sine:

```
type Expresie =
    | Valoare of int
    | Adunare of Expresie * Expresie
    | Înmulțire of Expresie * Expresie

let expresie = Adunare(Valoare 5, Înmulțire(Valoare 3, Valoare 4))
```

În acest exemplu, definim o mulțime discriminată **Expresie** pentru a reprezenta expresii aritmetice simple. Cazurile **Adunare** și **Înmulțire** conțin referințe recursive către tipul **Expresie**.

#### 4.7.3. Single-case discriminated unions

Single-case discriminated unions sunt un caz particular al mulțimilor discriminate (discriminated unions) care conțin un singur caz. Deși poate părea contraintuitiv la prima vedere, aceste tipuri de date au utilitatea lor în anumite situații.

Un single-case discriminated union este definit în mod similar cu o mulțime discriminată obișnuită, dar cu un singur caz:

```
type NumeTip = Caz of TipDate
```

Aici, **NumeTip** este numele tipului definit, **Caz** este singurul caz al mulțimii, iar **TipDate** reprezintă tipul de date asociat acestui caz.

Să analizăm un exemplu concret:

```
type CustomerId = CustomerId of int
```

În acest caz, definim un tip single-case discriminat numit `CustomerId`, care înfășoară o valoare întreagă (`int`). Putem crea valori de acest tip folosind constructorul `CustomerId`:

```
let customerId = CustomerId 42
```

Utilizarea single-case discriminated unions aduce câteva beneficii:

- Asigură un nivel suplimentar de siguranță a tipurilor. Prin înfășurarea unei valori într-un tip specific, putem preveni utilizarea accidentală a unei valori de un tip diferit în locul celui așteptat.
- Îmbunătățește lizibilitatea și expresivitatea codului. Un nume de tip precum `CustomerId` este mai explicit și mai semnificativ decât simpla utilizare a tipului `int`.
- Permite definirea de funcții specializate pentru tipul respectiv. Putem defini funcții care primesc sau returnează valori de tipul `CustomerId`, asigurând astfel că doar valorile corecte sunt folosite.

De exemplu, putem defini o funcție care primește un `CustomerId` și returnează detaliile clientului asociat:

```
let getCustomerDetails (CustomerId id) =  
    // Logica de obținere a detaliilor clientului pe baza ID-ului  
    // ...
```

În acest caz, funcția `getCustomerDetails` primește o valoare de tip `CustomerId` și extrage valoarea întreagă din interior prin pattern matching.

Single-case discriminated unions sunt deosebit de utile când dorim să creăm tipuri de date semantice, care au o semnificație specifică în domeniul problemei noastre. Acestea ne ajută să exprimăm mai clar intenția codului și să prevenim erorile de utilizare a tipurilor de date greșite.

Cu toate acestea, este important să utilizăm aceste tipuri de date cu moderație și doar atunci când aduc o valoare reală în termeni de siguranță, lizibilitate sau expresivitate. Folosirea excesivă a single-case discriminated unions poate duce la un cod greu de urmărit.

#### 4.7.4. Tipuri recursive

Tipurile recursive sunt acele tipuri ale căror definiție include o referință la ele însele. Cu alte cuvinte, un tip este recursiv dacă un element de acest tip poate conține alte elemente de același tip.

Tipurile recursive sunt deosebit de utile pentru a reprezenta structuri de date care au o natură recursivă, cum ar fi listele înlănțuite, arborii sau expresiile aritmetice. Ele ne permit să modelăm astfel de structuri într-un mod natural și expresiv.

În F#, putem defini cu ușurință tipuri de date recursive folosind constructorul "and". Iată sintaxa generală:

```
type NumeTip1 =
    | Caz1_1 of TipDate1_1
    | Caz1_2 of TipDate1_2
    | ...
and NumeTip2 =
    | Caz2_1 of TipDate2_1
    | Caz2_2 of TipDate2_2
    | ...
    | Caz2_i of NumeTip1
```

Observați cum în definiția lui `NumeTip2` apare o referință către `NumeTip1`, făcând întreaga definiție recursivă.

Să analizăm un exemplu concret. Vom defini un tip de date pentru a reprezenta o listă înlănțuită:

```
type ListNode<'a> =
    | Empty
    | Node of value: 'a * next: ListNode<'a>
```

Acest tip de date reprezintă un nod al unei liste înlănțuite. Fiecare nod poate fi fie `Empty` (lista vidă), fie `Node` (un nod nevid). Un nod nevid conține o valoare de tipul 'a și o referință la următorul nod din listă, care la rândul său este de tipul `ListNode<'a>`.

Iată cum am putea utiliza acest tip de date pentru a crea o listă înlănțuită:

```
let myList =
    Node(1, Node(2, Node(3, Empty)))
```

Această expresie creează o listă înlănțuită cu trei elemente: 1, 2 și 3.

Prelucrarea tipurilor recursive se face de obicei tot folosind funcții recursive. De exemplu, iată cum am putea calcula suma elementelor dintr-o listă înlănțuită:

```
let rec sumList list =
    match list with
    | Empty -> 0
    | Node (value, next) -> value + sumList next
```

Funcția `sumList` primește o listă înlănțuită și folosește pattern matching pentru a diferenția între cele două cazuri posibile:

- Dacă lista este vidă (`Empty`), suma elementelor este 0.

- Dacă lista este nevidă (`Node`), suma elementelor este valoarea din nodul curent (`value`) plus suma elementelor din restul listei (obținută prin apelul recursiv `sumList next`).

Această definiție recursivă a funcției `sumList` urmărește îndeaproape structura recursivă a tipului de date `ListNode`, făcând codul ușor de înțeles și rezonabil.

Să vedem funcția `sumList` în acțiune:

```
let myList = Node(1, Node(2, Node(3, Empty)))
let sum = sumList myList
printfn "Suma elementelor este %d" sum // Va afișa: "Suma elementelor este 6"
```

Un alt exemplu clasic de tip recursiv este reprezentarea expresiilor aritmetice:

```
type Expr =
    | Const of int
    | Add of Expr * Expr
    | Sub of Expr * Expr
    | Mul of Expr * Expr
```

Acest tip de date poate reprezenta expresii aritmetice simple formate din constante întregi și operații de adunare, scădere și înmulțire. Observați cum cazurile `Add`, `Sub` și `Mul` conțin referințe recursive la tipul `Expr`.

Iată cum am putea evalua o expresie de acest tip:

```
let rec eval expr =
    match expr with
    | Const x -> x
    | Add (e1, e2) -> eval e1 + eval e2
    | Sub (e1, e2) -> eval e1 - eval e2
    | Mul (e1, e2) -> eval e1 * eval e2
```

Această funcție primește o expresie și returnează valoarea sa, evaluând recursiv subexpresiile și combinând rezultatele conform operației respective.

Să testăm funcția `eval`:

```
let expr = Add(Const 2, Mul(Const 3, Const 4))
let result = eval expr
printfn "Rezultatul expresiei este %d" result // Va afișa: "Rezultatul expresiei este 14"
```

Tipurile recursive de date și funcțiile recursive merg mână în mână, permițându-ne să exprimăm și să prelucrăm structuri de date cu o complexitate arbitrară. Ele sunt un instrument puternic în programarea funcțională și în special în limbajul F#.

Desigur, atunci când scriem funcții recursive, trebuie să avem grijă să asigurăm că recursivitatea se termină la un moment dat. Altfel, riscăm să intrăm într-o buclă infinită.

În exemplele de mai sus, recursivitatea se termină atunci când ajungem la cazurile de bază (lista vidă, respectiv constanta).

#### 4.7.5. Unități de măsură

Unitățile de măsură ne permit să atașăm informații despre unitatea de măsură (de exemplu, metri, secunde, grame etc.) tipurilor numerice, cum ar fi `int` sau `float`. Acest lucru adaugă un nivel suplimentar de siguranță și expresivitate în codul nostru, ajutând la prevenirea erorilor și făcând intențiile mai clare.

Pentru a defini o unitate de măsură, folosim sintaxa `[<Measure>]` în fața unui tip:

```
[<Measure>] type meter
[<Measure>] type second
[<Measure>] type sqrMeter = meter^2
```

Aici am definit trei unități de măsură: `meter` pentru lungime, `second` pentru timp și `sqrMeter` pentru arie (definit ca `meter` la puterea 2).

Acum putem utiliza aceste unități de măsură pentru a anota tipurile numerice:

```
let length = 5<meter>
let time = 2<second>
let area = 10<sqrMeter>
```

Observați cum atașăm unitățile de măsură la valori folosind sintaxa `<unitate>`.

Avantajul cheie al utilizării unităților de măsură este că F# efectuează verificări la compilare pentru a se asigura că unitățile sunt coerente. De exemplu, dacă încercăm să adunăm o lungime cu un timp, vom primi o eroare de compilare:

```
let invalidAdd = length + time // Eroare de compilare: Unitățile de măsură
nu corespund
```

Pe de altă parte, putem efectua operații între valori cu unități de măsură compatibile:

```
let doubleLength = length * 2<meter>
// Rezultat: 10<meter>

let speed = length / time
// Rezultat: 2.5<meter/second>

let doubleArea = area * 2<sqrMeter>
// Rezultat: 20<sqrMeter>
```

Observați cum F# deduce automat unitățile de măsură rezultate. De exemplu, împărțind o lungime la un timp, obținem o viteză cu unitatea `<meter/second>`.

Unitățile de măsură sunt deosebit de utile în domenii precum fizica, ingineria sau finanțele, unde corectitudinea calculelor este foarte importantă. Ele servesc ca o formă de documentare în cod și pot preveni erori subtile cauzate de utilizarea inconsistentă a unităților.

Este important de reținut că unitățile de măsură sunt pur conceptuale la momentul compilării - ele nu afectează reprezentarea în memorie a valorilor la runtime. De fapt, unitățile de măsură sunt "șterse" la compilare, deci nu introduc niciun overhead la execuție.

F# furnizează, de asemenea, o serie de unități de măsură predefinite în modulul `Microsoft.FSharp.Data.UnitSystems.SI`. Acest modul include unități de măsură pentru Sistemul Internațional (SI), cum ar fi metri, secunde, kilograme etc., precum și operații pentru lucrul cu aceste unități.

```
open Microsoft.FSharp.Data.UnitSystems.SI

let mass = 5<kilogram>
let acceleration = 2<meter/second^2>
let force = mass * acceleration
// Rezultat: 10<kilogram meter/second^2> or 10<newton>
```

Aici am folosit unitățile de măsură predefinite kilogram, meter și second pentru a calcula o forță. Observați cum F# simplifică automat unitatea rezultată la newton, unitatea SI derivată pentru forță.

#### 4.7.6. Abrevieri de tipuri

În ultima parte a discuției noastre despre tipurile de date algebrice în F#, să aruncăm o privire asupra abrevierilor de tipuri (type abbreviations).

Abrevierile de tipuri ne permit să definim alias-uri sau nume alternative pentru tipuri existente. Acestea sunt utile pentru a face codul mai lizibil și mai expresiv, în special când lucrăm cu tipuri complexe.

Pentru a defini o abreviere de tip, folosim sintaxa `type NumeAbreviere = TipExistent`. Iată câteva exemple:

```
type String = System.String
type Int = int
type MyFunction = int -> string
type MyTuple = int * string * bool
```

În primul exemplu, definim `String` ca o abreviere pentru tipul `System.String`. Aceasta ne permite să folosim `String` în loc de numele complet `System.String` în codul nostru.

În al doilea exemplu, definim `Int` ca o abreviere pentru tipul primitiv `int`. Deși acest exemplu nu aduce un beneficiu major, el ilustrează ideea de bază.

În al treilea exemplu, definim `MyFunction` ca o abreviere pentru tipul funcție `int -> string`. Aceasta face tipul mai ușor de citit și de înțeles, în special dacă este folosit în mai multe locuri în cod.

În al patrulea exemplu, definim `MyTuple` ca o abreviere pentru tipul tupe `int * string * bool`. Din nou, aceasta poate îmbunătăți lizibilitatea codului atunci când lucrăm cu tipuri complexe.

Odată definită o abreviere de tip, o putem utiliza în locul tipului original oriunde în codul nostru:

```
let myString: String = "Hello, world!"
let myInt: Int = 42
let myFunction: MyFunction = fun i -> sprintf "Numărul este %d" i
let myTuple: MyTuple = (1, "unu", true)
```

Aici am folosit abrevierile de tipuri definite anterior pentru a adnota tipurile variabilelor.

Este important de reținut că abrevierile de tipuri sunt pur și simplu alias-uri - ele nu definesc tipuri noi. De exemplu, `String` și `System.String` sunt în esență același tip, doar cu nume diferite. Aceasta înseamnă că putem folosi abrevierile de tipuri și tipurile originale interschimbabil:

```
let myString: String = "Hello"
let myOtherString: System.String = myString // Valid, String și
System.String sunt același tip
```

Abrevierile de tipuri sunt utile când lucrăm cu biblioteci externe sau cu cod F# mai vechi, care poate folosi nume de tipuri lungi. Prin definirea de abrevieri cu nume mai concise și mai semnificative, putem face codul nostru mai ușor de citit și de înțeles fără a schimba comportamentul său.

De asemenea, abrevierile de tipuri pot fi folosite pentru a crea nume mai specifice domeniului pentru tipuri generale. De exemplu, în loc să folosim peste tot tipul `string`, am putea defini abrevieri precum `UserName`, `Email` sau `PhoneNumber` pentru a face codul mai expresiv:

```
type UserName = string
type Email = string
type PhoneNumber = string

let validateUser (name: UserName) (email: Email) (phone: PhoneNumber) = //
...
```

În acest caz, deși `UserName`, `Email` și `PhoneNumber` sunt toate de fapt șiruri, folosirea abrevierilor face imediat clar care este rolul fiecărui parametru în funcția `validateUser`.

## 4.8. Module și organizarea codului

Organizarea codului este un aspect esențial al dezvoltării de software, indiferent de limbajul de programare folosit. Un cod bine structurat este mai ușor de înțeles, de modificat și de întreținut pe termen lung. În F#, modulele și namespace-urile sunt uneltele principale pentru organizarea codului și pentru încapsularea funcționalităților conexe.

În această secțiune, vom explora în detaliu conceptele de module și namespace-uri în F#. Vom învăța cum să definim module pentru a grupa funcții, valori și tipuri înrudite, precum și cum să folosim namespace-uri pentru a crea o ierarhie logică a modulelor.

De asemenea, vom discuta despre importarea și exportarea funcționalităților între module și fișiere, precum și despre utilizarea fișierelor de semnătură (signature files) pentru a defini interfețele publice ale modulelor.

Pe parcursul acestei secțiuni, ne vom concentra pe cele mai bune practici de organizare a codului în F#, cum ar fi:

- Crearea de module coezive și cu un singur scop
- Folosirea namespace-urilor pentru a evita conflictele de nume și a crea o structură ierarhică
- Expunerea unei interfețe clare și bine definite pentru module
- Utilizarea fișierelor de semnătură pentru a separa interfața de implementare
- Organizarea fișierelor și a proiectului pentru o navigare și o întreținere ușoară

Prin aplicarea acestor principii și tehnici, veți fi capabili să scrieți un cod F# modular, refolosibil și ușor de întreținut. Indiferent dacă lucrați la un proiect mic sau la o bază de cod mare, o bună organizare a codului este esențială pentru productivitate și succes pe termen lung.

#### 4.8.1. Definirea modulelor

În F#, un modul este o unitate de organizare a codului care grupează la un loc funcții, valori, tipuri și alte definiții legate între ele. Modulele ne ajută să structurăm codul într-un mod logic și să încapsulăm detaliile de implementare, expunând doar interfața dorită utilizatorilor modulului.

Pentru a defini un modul, folosim cuvântul cheie `module`, urmat de numele modulului și corpul acestuia. Iată sintaxa de bază:

```
module NumeModul =  
    // Conținutul modulului  
    // Funcții, valori, tipuri, etc.
```

Să analizăm un exemplu concret:

```
module Matematica =  
    let adunare x y = x + y  
    let scadere x y = x - y  
    let inmultire x y = x * y  
    let impartire x y = x / y
```

În acest exemplu, definim un modul numit `Matematica`, care conține patru funcții: `adunare`, `scadere`, `inmultire` și `impartire`. Aceste funcții sunt legate logic între ele și reprezintă operații matematice de bază.



Pentru a utiliza funcțiile definite într-un modul, folosim notația cu punct, specificând numele modulului, urmat de un punct și numele funcției:

```
let suma = Matematica.adunare 10 5
let diferenta = Matematica.scadere 10 5
let produs = Matematica.inmultire 10 5
let cat = Matematica.impartire 10 5
```

Modulele pot conține, de asemenea, definiții de valori și tipuri:

```
module Geometrie =
    let PI = 3.14159

    type Forma =
        | Cerc of raza: float
        | Dreptunghi of lungime: float * latime: float
        | Triunghi of baza: float * inaltime: float

    let ariaCirc raza = PI * raza * raza
    let ariaDreptunghi lungime latime = lungime * latime
    let ariaTriunghi baza inaltime = 0.5 * baza * inaltime
```

În acest exemplu, modulul `Geometrie` definește o valoare `PI`, un tip discriminat `Forma` și trei funcții pentru calculul ariilor diferitelor forme geometrice.

Un aspect important al modulelor în F# este că ele sunt implicit "deschise" în cadrul fișierului în care sunt definite. Acest lucru înseamnă că putem accesa conținutul unui modul fără a specifica numele modulului, atât timp cât ne aflăm în același fișier:

```
// În același fișier cu modulul Matematica
let suma = aduna 10 5 // Nu este nevoie de prefixul Matematica.
```

Cu toate acestea, pentru a utiliza un modul din afara fișierului în care a fost definit, trebuie să folosim numele complet al modulului sau să îl "deschidem" explicit folosind directiva `open`:

```
open Matematica

let suma = aduna 10 5 // Acum putem folosi direct funcțiile din modulul
Matematica
```

Modulele pot fi de asemenea imbricate, permițându-ne să creăm o ierarhie logică a modulelor:

```
module Stiinta =
    module Fizica =
        let viteza distanta timp = distanta / timp

    module Chimie =
        let nrAvogadro = 6.022e23
```

Pentru a accesa funcțiile și valorile din modulele imbricate, folosim notația cu punct în cascadă:

```
let v = Stiinta.Fizica.viteza 100.0 5.0
```

#### 4.8.2.Namespace-uri în F#

Un namespace este un container pentru module și alte tipuri în F#. Namespace-urile ne permit să grupăm module conexe sub un nume comun, formând o ierarhie logică. Prin utilizarea namespace-urilor, putem evita conflictele de nume între module și putem crea un cod mai modular și mai ușor de navigat.

Pentru a defini un namespace în F#, folosim cuvântul cheie `namespace`, urmat de numele namespace-ului. Modulele și tipurile definite în interiorul unui namespace vor fi accesibile folosind numele complet, format din numele namespace-ului și numele modulului sau tipului.

Iată un exemplu de definire a unui namespace și a unor module în interiorul acestuia:

```
namespace MatemaTech
module Algebră =
    let aduna x y = x + y
    let scade x y = x - y
module Geometrie =
    let PI = 3.14159
    let ariaCerc raza = PI * raza * raza
```

În acest exemplu, definim un namespace numit `MatemaTech`, care conține două module: `Algebră` și `Geometrie`. Pentru a accesa funcțiile și valorile din aceste module din afara namespace-ului, trebuie să folosim numele complet:

```
let suma = MatemaTech.Algebră.aduna 10 5
let aria = MatemaTech.Geometrie.ariaCerc 3.0
```

Namespace-urile pot fi imbricate, permițându-ne să creăm o ierarhie de namespace-uri pentru o organizare și mai bună a codului:

```
namespace MatemaTech
namespace Avansate
    module Calculus =
        let derivata f x h = (f (x + h) - f x) / h
    module AlgebraLiniara =
        let produs (A: float[][]) (B: float[][]) = (* ... *)
module Trigonometrie =
    let sin x = (* ... *)
    let cos x = (* ... *)
```

În acest caz, avem namespace-ul `MatemaTech`, care conține un sub-namespace `Avansate` și un modul `Trigonometrie`. Modulele `Calculus` și `AlgebraLiniara` sunt definite în interiorul namespace-ului `Avansate`.

Pentru a accesa modulele și funcțiile din namespace-uri imbricate, folosim notația cu punct în cascadă:

```
let df = MatemaTech.Avansate.Calculus.derivata (fun x -> x ** 2.0) 1.0 0.001
```

```
let sinPi = MatemaTech.Trigonometrie.sin System.Math.PI
```

Utilizarea namespace-urilor are mai multe beneficii:

- Evită conflictele de nume: Prin încapsularea modulelor în namespace-uri, putem avea module cu același nume în namespace-uri diferite fără a provoca conflicte.
- Îmbunătățește modularitatea: Namespace-urile ne ajută să organizăm codul în unități logice și independente, făcându-l mai modular și mai ușor de întreținut.
- Facilitează navigarea: Prin gruparea modulelor conexe sub un namespace comun, putem crea o structură ierarhică care face codul mai ușor de explorat și de înțeles.
- Promovează încapsularea: Namespace-urile oferă un nivel suplimentar de încapsulare, permițându-ne să ascundem detaliile de implementare și să expunem doar interfața dorită.

Este de menționat că, spre deosebire de alte limbaje de programare, în F# namespace-urile nu afectează vizibilitatea în sine. Controlul accesului se face prin intermediul modificatorilor de acces `public`, `internal` și `private` la nivelul modulelor și al tipurilor.

#### 4.8.3.Importarea și exportarea funcționalității

Într-un proiect software tipic, avem adesea nevoie să folosim funcționalități definite în alte module sau biblioteci. În același timp, poate fi necesar să expunem anumite funcționalități din modulele noastre pentru a fi utilizate de alte componente ale sistemului. F# oferă mai multe mecanisme pentru importarea și exportarea funcționalităților, permițându-ne să controlăm vizibilitatea și accesibilitatea codului nostru.

Importarea funcționalităților se face folosind două construcții principale: instrucțiunea `open` și calificarea completă a numelui.

Instrucțiunea `open` ne permite să importăm toate definițiile publice dintr-un modul sau namespace, astfel încât să le putem utiliza fără a specifica numele complet de fiecare dată:

```
open System
open MatemaTech.Algebră

let suma = aduna 10 5 // Nu este nevoie de prefixul MatemaTech.Algebră
let radacina = sqrt 16.0 // Nu este nevoie de prefixul System
```

În acest exemplu, folosim `open` pentru a importa modulul `MatemaTech.Algebră` și namespace-ul `System`. Astfel, putem utiliza direct funcțiile `aduna` și `sqrt`, fără a specifica numele complete `MatemaTech.Algebră.aduna` și `System.sqrt`.

Calificarea completă a numelui, pe de altă parte, implică utilizarea numelui complet al unui modul sau tip de fiecare dată când îl folosim:

```
let suma = MatemaTech.Algebră.aduna 10 5
let radacina = System.Math.sqrt 16.0
```

Această abordare este utilă când dorim să evităm conflictele de nume sau când folosim o funcționalitate doar într-un singur loc și nu dorim să importăm întregul modul.

În ceea ce privește exportarea funcționalităților, F# folosește modificatori de acces pentru a controla vizibilitatea definițiilor din module. Implicit, toate definițiile dintr-un modul sunt publice și pot fi accesate din afara modului. Cu toate acestea, putem utiliza modificatorul `private` pentru a restricționa accesul la anumite definiții:

```
module Geometrie =
    let private aria baza inaltime = 0.5 * baza * inaltime

    let ariaTriunghi baza inaltime = aria baza inaltime
```

În acest caz, funcția `aria` este marcată ca `private` și poate fi accesată doar din interiorul modului `Geometrie`. Funcția `ariaTriunghi`, în schimb, este publică și poate fi apelată din afara modului.

Pentru un control mai granular asupra exportării funcționalităților, putem utiliza semnături de module (module signatures). O semnătură de modul definește interfața publică a unui modul, specificând ce definiții sunt vizibile și ce tipuri au acestea:

```
module type GeometrieSig =
    val ariaTriunghi : float -> float -> float

module Geometrie : GeometrieSig =
    let private aria baza inaltime = 0.5 * baza * inaltime

    let ariaTriunghi baza inaltime = aria baza inaltime

    let ariaPatrat latura = latura * latura
```

Aici, definim o semnătură de modul `GeometrieSig`, care specifică faptul că modulul `Geometrie` trebuie să exporte o funcție `ariaTriunghi` cu tipul `float -> float -> float`. Implementarea efectivă a modului `Geometrie` poate conține și alte definiții (cum ar fi funcțiile `aria` și `ariaPatrat`), dar doar `ariaTriunghi` va fi vizibilă și accesibilă din afara modului.

Semnăturile de module sunt deosebit de utile pentru crearea de biblioteci și API-uri, deoarece ne permit să definim o interfață clară și stabilă, ascunzând în același timp detaliile de implementare.

Un alt concept legat de importarea și exportarea funcționalităților în F# este noțiunea de "atribute de acces". Atributele de acces, cum ar fi `[<AutoOpen>]` și `[<RequireQualifiedAccess>]`, pot fi aplicate modulelor pentru a modifica comportamentul implicit al instrucțiunii `open`:

```
[<AutoOpen>]
module Helpers =
    let square x = x * x

[<RequireQualifiedAccess>]
module Geometrie =
    let ariaTriunghi baza inaltime = 0.5 * baza * inaltime
```

Atributul [**<AutoOpen>**] face ca definițiile din modulul **Helpers** să fie automat importate când se folosește instrucțiunea **open** pentru namespace-ul sau modulul părinte. Astfel, putem folosi direct funcția **square**, fără să mai fie nevoie de **open Helpers**.

Pe de altă parte, atributul [**<RequireQualifiedAccess>**] impune utilizarea numelui calificat al modulului, chiar dacă am folosit instrucțiunea **open**. În exemplul de mai sus, trebuie să folosim **Geometrie.ariaTriunghi**, chiar dacă am avea **open Geometrie** în cod.

Aceste atribute ne oferă un control mai fin asupra modului în care funcționalitățile sunt importate și accesate, permițându-ne să adaptăm comportamentul modulelor noastre la nevoile specifice ale proiectului.

#### 4.8.4. Signature files

Să discutăm acum despre fișierele de semnătură (signature files) în F# și rolul lor în organizarea și modularizarea codului.

Fișierele de semnătură, cu extensia **.fsi**, sunt un mecanism puternic pentru separarea interfețelor de implementările efective ale modulelor în F#. Un fișier de semnătură descrie interfața publică a unui modul, specificând ce valori, funcții, tipuri și submodule sunt expuse și cum pot fi accesate din exterior.

Iată un exemplu de fișier de semnătură pentru un modul **Matematica**:

```
// Matematica.fsi
module Matematica

val aduna : int -> int -> int
val scade : int -> int -> int

type Forma =
    | Cerc of raza : float
    | Dreptunghi of lungime : float * latime : float

val aria : Forma -> float
```

În acest fișier de semnătură, specificăm că modulul **Matematica** expune două funcții, **aduna** și **scade**, ambele primind doi întregi și returnând un întreg. De asemenea, definim un tip discriminat **Forma** și o funcție **aria** care calculează aria unei forme geometrice.

Apoi, putem defini implementarea efectivă a modulului **Matematica** într-un fișier **.fs** separat:

```
// Matematica.fs
```

```
module Matematica

let aduna x y = x + y
let scade x y = x - y

type Forma =
    | Cerc of raza : float
    | Dreptunghi of lungime : float * latime : float

let aria forma =
    match forma with
    | Cerc raza -> System.Math.PI * raza * raza
    | Dreptunghi (lungime, latime) -> lungime * latime

let perimetru forma =
    match forma with
    | Cerc raza -> 2.0 * System.Math.PI * raza
    | Dreptunghi (lungime, latime) -> 2.0 * (lungime + latime)
```

Observăm că fișierul de implementare `.fs` conține definițiile complete ale funcțiilor și tipurilor menționate în fișierul de semnătură, precum și orice alte definiții private sau auxiliare (cum ar fi funcția `perimetru`).

Avantajele folosirii fișierelor de semnătură sunt multiple:

- Separarea interfețelor de implementări: Prin definirea interfețelor în fișiere de semnătură separate, putem ascunde detaliile de implementare și expune doar funcționalitățile relevante pentru utilizatorii modulului. Aceasta promovează o mai bună modularitate și încapsulare.
- Verificare la momentul compilării: Compilatorul F# verifică dacă implementarea unui modul respectă interfața declarată în fișierul de semnătură. Orice discrepanță va genera erori de compilare, asigurând astfel că modulul se conformează contractului său public.
- Documentare mai clară: Fișierele de semnătură servesc ca o formă de documentație pentru modulele noastre. Prin specificarea tipurilor și a semnăturilor funcțiilor în fișierul `.fsi`, oferim o descriere clară a funcționalităților expuse de modul, fără a fi nevoie să inspectăm codul de implementare.
- Compilare mai rapidă: Atunci când se compilează un proiect F#, fișierele de semnătură sunt compilate înaintea fișierelor de implementare. Aceasta permite detectarea erorilor de interfață mai devreme în procesul de compilare și poate îmbunătăți timpul total de compilare pentru proiecte mari.
- Control mai bun asupra vizibilității: Prin omiterea anumitor definiții din fișierul de semnătură, putem restricționa accesul la funcționalitățile interne sau auxiliare ale modulului. Aceasta ne permite să menținem o interfață publică clară și minimală, ascunzând în același timp complexitatea internă.

Când folosim fișiere de semnătură, este important să menținem sincronizate interfețele și implementările. Orice modificare a interfeței publice a unui modul trebuie reflectată atât în fișierul `.fsi`, cât și în fișierul `.fs` corespunzător.

De asemenea, este o bună practică să folosim fișiere de semnătură pentru bibliotecile și API-urile destinate folosirii de către alte componente sau echipe. Prin definirea unor interfețe clare și stabile în fișierele `.fsi`, putem facilita utilizarea și integrarea modulelor noastre în alte proiecte.

#### 4.8.5. Modularizarea codului

Modularizarea eficientă a codului în F# este de dorit când dimensiunea proiectului crește, astfel încât dezvoltarea să fie scalabilă și ușor de întreținut.

##### *Principii de bază ale modularizării*

- Coeziune funcțională
  - Fiecare modul trebuie să aibă o responsabilitate bine definită și unitară
  - Funcțiile și tipurile dintr-un modul trebuie să fie strâns legate din punct de vedere logic
  - Evitați modulele care combină funcționalități necorelate
- Minimizarea dependențelor
  - Reduceți la minimum dependențele între module
  - Utilizați interfețe clare și bine definite pentru comunicarea între module
  - Evitați dependențele circulare între module
- Strategii de modularizare
  - Organizarea pe domenii funcționale

```
module Domain.Customers
    type Customer = { Id: int; Name: string }
    let createCustomer name = { Id = 0; Name = name }

module Domain.Orders
    type Order = { Id: int; CustomerId: int }
    let createOrder customerId = { Id = 0; CustomerId = customerId }
```

- Separarea preocupărilor

```
module DataAccess
    let saveToDatabase entity = // ...
    let loadFromDatabase<'T> id = // ...

module BusinessLogic
    let processOrder order = // ...
    let validateCustomer customer = // ...

module Presentation
    let displayOrder order = // ...
    let formatCustomerDetails customer = // ...
```

- Tehnici de structurare a codului

- Utilizarea submodulelor

```
module Authentication =  
    module Passwords =  
        let hash password = // ...  
        let verify hash password = // ...  
  
    module Tokens =  
        let generate user = // ...  
        let validate token = // ...
```

- Izolarea implementărilor interne

```
module DataProcessing =  
    // Funcții publice  
    let processData input =  
        input  
        |> validateInput  
        |> transform  
        |> formatOutput  
  
    // Funcții private (interne)  
    let private validateInput data = // ...  
    let private transform data = // ...  
    let private formatOutput data = // ...
```

- Considerații practice
  - Dimensiunea modulelor
    - Mențineți modulele la o dimensiune rezonabilă (în general sub 1000 de linii de cod)
    - Dacă un modul devine prea mare, considerați împărțirea lui în submodule
  - Organizarea fișierelor
    - Păstrați o relație unu-la-unu între module și fișiere
    - Utilizați o structură de directoare care reflectă organizarea logică a modulelor
    - Mențineți convenții consistente de denumire
  - Gestionarea dependențelor
    - Documentați explicit dependențele fiecărui modul
    - Utilizați signature files (.fsi) pentru a defini interfețele publice
    - Minimizați expunerea detaliilor de implementare

#### *Exemplu complet de modularizare*

```
// Domain.fs  
module Domain  
    type Customer = { Id: int; Name: string }  
    type Order = { Id: int; CustomerId: int; Amount: decimal }  
  
// Repository.fs  
module Repository  
    open Domain
```



```

    let saveCustomer customer = // ...
    let loadCustomer id = // ...
    let saveOrder order = // ...
    let loadOrder id = // ...

// BusinessLogic.fs
module BusinessLogic
    open Domain

    let validateOrder order = // ...
    let calculateDiscount order = // ...
    let processPayment order = // ...

// Api.fs
module Api
    open Domain
    open BusinessLogic

    let createOrder customerId amount =
        let order = { Id = 0; CustomerId = customerId; Amount = amount }
        order
        |> validateOrder
        |> processPayment
        |> Repository.saveOrder

```

Această structură modulară facilitează testarea unitară a componentelor individuale, modificarea și extinderea codului, reutilizarea componentelor în alte părți ale aplicației și înțelegerea și navigarea codului.

## 4.9. Gestionarea erorilor în stil funcțional

Gestionarea erorilor este un aspect important în dezvoltarea de software robust și de încredere. În programarea funcțională și în special în F#, gestionarea erorilor este abordată într-un mod diferit față de programarea imperativă tradițională, care se bazează în principal pe excepții.

În acest paragraf, vom explora tehnicile și conceptele cheie pentru gestionarea erorilor în stilul funcțional, folosind mecanismele oferite de F#. Vom începe prin a discuta despre tipul `Result`, o abordare puternică și expresivă pentru reprezentarea rezultatelor care pot fi fie de succes, fie de eșec. Apoi, vom vedea cum tipul `Option` poate fi utilizat pentru a modela valori care pot fi prezente sau absente, evitând astfel problemele îndeobște asociate cu valoarea `NULL`.

De asemenea, vom introduce conceptul de *Railway Oriented Programming (ROP)*, o metodă elegantă de a organiza și a compune funcții care pot produce erori, inspirată de modul în care funcționează macazurile pe calea ferată. ROP ne permite să construim pipeline-uri de prelucrare a datelor care gestionează în mod transparent cazurile de succes și de eșec.

Computation expressions reprezintă un instrument puternic în F# pentru a defini fluxuri de control personalizate. Vom vedea cum putem utiliza computation expressions pentru a crea un mecanism de gestionare a erorilor care se integrează perfect cu paradigma funcțională.

În cele din urmă vom acoperi și tehnicile tradiționale de gestionare a excepțiilor în F#, discutând despre situațiile în care acestea pot fi folosite în mod adecvat și cum să le integrăm în codul funcțional. De asemenea, vom aborda validarea datelor, o practică esențială pentru a asigura corectitudinea și consistența datelor de intrare.

#### 4.9.1. Tipul Result

În programarea funcțională, tipul Result este o abordare comună și expresivă pentru reprezentarea rezultatelor unei operații care poate fi fie un succes, fie un eșec. Acest tip de date este deosebit de util în situațiile în care o funcție poate întâmpina erori sau poate produce rezultate invalide, permițând gestionarea explicită a acestor cazuri fără a recurge la excepții.

În F#, tipul Result este definit în biblioteca standard în felul următor:

```
type Result<'T, 'TError> =  
    | Ok of 'T  
    | Error of 'TError
```

Această definiție arată că Result este o mulțime discriminată (discriminated union) cu două cazuri: Ok și Error. Cazul Ok reprezintă un rezultat de succes și încapsulează o valoare de tipul 'T, în timp ce cazul Error reprezintă un rezultat de eșec și încapsulează o valoare de tipul 'TError, care descrie eroarea sau motivul eșecului.

Să analizăm un exemplu concret pentru a înțelege mai bine utilizarea tipului Result:

```
let impartire (a: int) (b: int) : Result<int, string> =  
    if b = 0 then  
        Error "Împărțire la zero"  
    else  
        Ok (a / b)
```

În acest exemplu, definim o funcție impartire care primește doi întregi a și b și returnează un rezultat de tip Result<int, string>. Dacă b este zero, funcția returnează un rezultat Error cu un mesaj de eroare corespunzător. Altfel, funcția returnează un rezultat Ok cu valoarea calculată a împărțirii.

Utilizarea tipului Result ne obligă să gestionăm explicit atât cazurile de succes, cât și cazurile de eșec atunci când apelăm funcția:

```
let rezultat = impartire 10 2  
match rezultat with  
| Ok valoare -> printfn "Rezultatul este: %d" valoare  
| Error mesaj -> printfn "Eroare: %s" mesaj
```

În acest caz, folosim pattern matching pentru a verifica rezultatul returnat de funcția `impartire`. Dacă rezultatul este `Ok`, afișăm valoarea rezultatului. Dacă rezultatul este `Error`, afișăm mesajul de eroare.

Tipul `Result` ne oferă mai multe avantaje față de gestionarea erorilor bazată pe excepții:

- Rezultatele de eșec sunt modelate explicit în tipul de retur al funcției, făcând clar faptul că funcția poate eșua și încurajând gestionarea adecvată a cazurilor de eroare.
- Erorile pot fi descrise folosind un tip de date adecvat (`TError`), permițând transmiterea de informații suplimentare despre natura erorii.
- Folosirea tipului `Result` încurajează un stil de programare mai expresiv și mai compozițional, în care erorile sunt propagate și gestionate ca valori de prim rang, fără a recurge la construcții imperative precum `try-catch`.
- Tipul `Result` se integrează bine cu alte concepte și tehnici funcționale, cum ar fi `Railway Oriented Programming (ROP)` și `computation expressions`, permițând construirea de pipeline-uri de prelucrare a datelor robuste și expresive.

#### 4.9.2. Tipul `Option` pentru valori absente

În programarea funcțională, tipul `Option` este utilizat pentru a reprezenta valori care pot fi prezente (au o valoare) sau absente. Acesta oferă o alternativă sigură și expresivă la utilizarea valorilor `NULL`, care pot duce adesea la erori greu de depistat și la cod defensiv stufos.

În `F#`, tipul `Option` este definit în felul următor:

```
type Option<'T> =
    | Some of 'T
    | None
```

Această definiție arată că `Option` este o mulțime discriminată (`discriminated union`) cu două cazuri: `Some` și `None`. Cazul `Some` reprezintă prezența unei valori și încapsulează o valoare de tipul `'T`, în timp ce cazul `None` reprezintă absența unei valori.

Să analizăm un exemplu pentru a ilustra utilizarea tipului `Option`:

```
let cautValoare (lista: int list) (valoare: int) : int option =
    let rezultat = List.tryFind (fun x -> x = valoare) lista
    rezultat
```

În acest exemplu, definim o funcție `cautValoare` care primește o listă de întregi și o valoare de căutat. Funcția încearcă să găsească prima apariție a valorii în listă folosind funcția `List.tryFind`, care returnează un rezultat de tip `'T option`. Dacă valoarea este găsită, rezultatul va fi `Some valoare`, iar dacă valoarea nu este găsită, rezultatul va fi `None`.

Utilizarea tipului `Option` ne încurajează să gestionăm explicit cazurile în care o valoare poate fi absentă:

```
let lista = [1; 2; 3; 4; 5]
let rezultat = cautaValoare lista 3

match rezultat with
| Some valoare -> printfn "Valoarea %d a fost găsită în listă" valoare
| None -> printfn "Valoarea nu a fost găsită în listă"
```

În acest caz, folosim pattern matching pentru a verifica rezultatul returnat de funcția `cautaValoare`. Dacă rezultatul este `Some`, afișăm un mesaj corespunzător care include valoarea găsită. Dacă rezultatul este `None`, afișăm un mesaj care indică faptul că valoarea nu a fost găsită în listă.

Tipul Option aduce mai multe beneficii în comparație cu utilizarea valorilor NULL:

- **Expresivitate:** Tipul Option face explicit în semnătura funcției faptul că o valoare poate fi absentă, încurajând gestionarea corectă a acestor cazuri.
- **Siguranță:** Prin eliminarea valorilor NULL, tipul Option previne erorile de tip "*null reference exception*" și reduce nevoia de verificări defensive în cod.
- **Compoziționalitate:** Tipul Option se integrează bine cu alte concepte funcționale, cum ar fi pattern matching, funcțiile de ordin superior și computation expressions, permițând un stil de programare mai expresiv și mai compozițional.
- **Claritate:** Utilizarea tipului Option face codul mai ușor de înțeles și de întreținut, deoarece intențiile sunt exprimate explicit și cazurile de valori absente sunt gestionate în mod sistematic.

#### 4.9.3. Railway-oriented programming

Railway-oriented programming (ROP) este o tehnică de programare funcțională care facilitează gestionarea erorilor și a cazurilor de excepție într-un mod elegant și compozițional. ROP își propune să trateze erorile ca parte integrantă a fluxului de date, permițând propagarea și gestionarea lor într-un mod natural și expresiv.

Conceptul central al ROP este analogia cu o cale ferată. Într-un sistem feroviar, există două șine: o șină pentru cazurile de succes (când totul merge bine) și o șină pentru cazurile de eșec (când apar erori sau excepții). ROP aplică această analogie în programare, reprezentând fluxul de date ca o serie de funcții care pot avea două căi: o cale de succes și o cale de eșec.

În F#, ROP se bazează pe utilizarea tipului `Result<'T, 'TError>` pentru a reprezenta rezultatele unei operații. `Result` este o mulțime discriminată care poate avea două cazuri: `Ok` pentru rezultate de succes și `Error` pentru rezultate de eșec. Prin utilizarea acestui tip, putem modela explicit posibilitatea ca o funcție să eșueze și să returneze o valoare de eroare.

Iată un exemplu de funcție care utilizează tipul `Result` într-un scenariu de validare:

```
let validateAge age =
    if age >= 0 then Ok age
```

```

else Error "Invalid age"

let validateName name =
    if not (String.IsNullOrEmpty(name)) then Ok name
    else Error "Name cannot be empty"

let createPerson name age =
    match validateName name, validateAge age with
    | Ok validName, Ok validAge -> Ok { Name = validName; Age = validAge }
    | Error nameError, _ -> Error nameError
    | _, Error ageError -> Error ageError

```

În acest exemplu, avem două funcții de validare, `validateAge` și `validateName`, care returnează un rezultat de tip `Result<'T, string>`. Funcția `createPerson` primește un nume și o vârstă și utilizează pattern matching pentru a combina rezultatele validărilor. Dacă ambele validări sunt de succes, se creează un obiect de tip `Person`. Dacă una dintre validări eșuează, se returnează eroarea corespunzătoare.

ROP încurajează compunerea funcțiilor care lucrează cu tipul `Result`, permițând construirea unor "pipe-uri" de prelucrare a datelor. Biblioteca `Result` din F# oferă o serie de operatori și funcții utilitare pentru a facilita această compunere. De exemplu, operatorul `>>=` (bind) permite aplicarea unei funcții care returnează un `Result` la un `Result` existent, propagând erorile în mod automat.

Iată cum am putea utiliza operatorul `>>=` pentru a compune funcțiile de mai sus:

```

let createPersonWorkflow name age =
    validateName name >>= fun validName ->
    validateAge age >>= fun validAge ->
    Ok { Name = validName; Age = validAge }

```

În acest caz, folosim operatorul `>>=` pentru a lega rezultatele validărilor într-un flux de prelucrare. Dacă o validare eșuează, eroarea este propagată automat, iar restul fluxului este întrerupt.

ROP aduce următoarele beneficii:

- Gestionare explicită a erorilor: Prin utilizarea tipului `Result`, erorile sunt modelate explicit și devin parte integrantă a semnăturii funcțiilor, încurajând gestionarea lor adecvată.
- Compoziționalitate: ROP permite compunerea ușoară a funcțiilor care pot eșua, facilitând construirea unor fluxuri de prelucrare a datelor robuste și expresive.
- Propagarea automată a erorilor: Prin utilizarea operatorilor și funcțiilor specifice ROP, erorile sunt propagate automat de-a lungul fluxului de prelucrare, eliminând necesitatea unei gestionări manuale repetitive.
- Cod mai clar și mai ușor de înțeles: Prin separarea explicită a căilor de succes și de eșec, codul devine mai ușor de urmărit și de înțeles, facilitând raționamentul despre comportamentul sistemului.

#### 4.9.4. Computation expressions

Computation expressions reprezintă un mecanism puternic în F# pentru definirea și controlul fluxului de execuție al codului într-un mod expresiv și personalizabil. Acestea permit crearea unor contexte de calcul specifice, cum ar fi gestionarea stării, tratarea erorilor, interacțiunea cu mediul extern sau controlul concurenței, într-un stil care se integrează perfect cu paradigma funcțională.

În esență, computation expressions sunt construcții sintactice care permite definirea unui "limbaj specific domeniului" (*DSL – Domain Specific Language*) pentru un anumit scop. Aceste construcții sunt bazate pe o serie de metode cu semnături predefinite, care sunt folosite pentru a controla fluxul de execuție și pentru a combina rezultatele intermediare.

Să analizăm un exemplu simplu de computation expression pentru gestionarea unei resurse:

```
type ResourceManager() =
    member this.Acquire(resource) = printfn "Acquiring resource: %s" resource
    member this.Release(resource) = printfn "Releasing resource: %s" resource

let withResource = new ResourceManager()

let useResource resource = withResource {
    do! withResource.Acquire(resource)
    printfn "Using resource: %s" resource
    do! withResource.Release(resource)
}

useResource "Database connection"
```

În acest exemplu, definim o clasă `ResourceManager` cu două metode: `Acquire` și `Release`, care simulează achiziționarea și eliberarea unei resurse. Apoi, definim un obiect `withResource` de tip `ResourceManager`.

Computation expression-ul `useResource` este definit folosind obiectul `withResource` și un bloc de cod delimitat de acolade `{}`. În interiorul acestui bloc, folosim cuvântul cheie `do!` pentru a apela metodele `Acquire` și `Release` ale obiectului `withResource`, și `printfn` pentru a afișa un mesaj.

Când apelăm `useResource "Database connection"`, se execută computation expression-ul, care achiziționează resursa, o utilizează și apoi o eliberează, totul într-un mod expresiv și concis.

Computation expressions se bazează pe o serie de metode cu semnături predefinite, care trebuie implementate de către un tip de date pentru a defini comportamentul expresiei. Câteva dintre cele mai comune metode sunt:

- **Bind**: Combină două expresii într-un mod secvențial, unde rezultatul primei expresii este pasat ca input celei de-a doua expresii.
- **Return**: Întoarce o valoare ca rezultat al expresiei.
- **Zero**: Reprezintă o valoare implicită sau un caz de bază pentru expresie.
- **Combine**: Combină două expresii într-un mod paralel sau alternativ.
- **Delay**: Întârzie evaluarea unei expresii până când este necesară.

Aceste metode, împreună cu altele, permit definirea unor computation expressions personalizate pentru diverse scenarii, cum ar fi:

- **async**: Pentru programarea asincronă și tratarea operațiilor care nu blochează execuția.
- **seq**: Pentru generarea și prelucrarea secvențelor într-un mod lazy.
- **result**: Pentru propagarea și gestionarea erorilor folosind tipul **Result**.
- **option**: Pentru lucrul cu valori opționale și propagarea valorilor de tip **None**.
- **task**: Pentru lucrul cu task-uri și operații concurente.

Computation expressions aduc numeroase beneficii:

- **Expresivitate**: Permit definirea unor contexte de calcul specifice într-un mod expresiv și natural, folosind un "limbaj" adaptat domeniului problemei.
- **Abstractizare**: Ascund detaliile de implementare și complexitatea, permițând focalizarea pe logica de business și pe fluxul de control al programului.
- **Modularitate**: Pot fi compuse și combinate pentru a crea computation expressions mai complexe, facilitând reutilizarea și organizarea codului.
- **Integrare cu paradigma funcțională**: Se integrează perfect cu conceptele și tehnicile programării funcționale, cum ar fi expresiile, pattern matching-ul și funcțiile de ordin superior.

#### 4.9.5.Exception handling

Deși programarea funcțională pune accentul pe prevenirea și gestionarea erorilor folosind tehnici precum tipurile **Result** și **Option** și Railway-oriented programming, există situații în care utilizarea excepțiilor poate fi adecvată și necesară. F# oferă suport pentru mecanismul tradițional de tratare a excepțiilor, similar cu alte limbaje .NET, cum ar fi C#.

În F#, excepțiile sunt reprezentate prin tipul **Exception** și pot fi create folosind constructorul **Exception(mesaj)** sau prin utilizarea unei excepții predefinite, cum ar fi **ArgumentException**, **InvalidOperationException** sau **IOException**.

Iată un exemplu simplu de aruncare și prindere a unei excepții în F#:

```
let impartire a b =
    if b = 0 then
        raise (DivideByZeroException("Împărțire la zero"))
    else
```

```
    a / b

let printeazaRezultat a b =
    try
        let rezultat = impartire a b
        printfn "Rezultatul este: %d" rezultat
    with
    | :? DivideByZeroException as ex ->
        printfn "Eroare: %s" ex.Message
```

În acest exemplu, funcția `impartire` verifică dacă parametrul `b` este zero și, în caz afirmativ, aruncă o excepție `DivideByZeroException` cu un mesaj specific. Funcția `printeazaRezultat` apelează `impartire` într-un bloc `try` și prinde excepția `DivideByZeroException` folosind construcția `with`.

Fluxul de control al excepțiilor în F# este similar cu alte limbaje .NET:

- Când o excepție este aruncată folosind `raise`, execuția normală a codului este întreruptă și controlul este transferat celui mai apropiat bloc `with` care poate prinde excepția.
- Dacă excepția nu este prinsă în funcția curentă, aceasta se propagă în sus pe stiva de apeluri până când este prinsă sau până când ajunge la nivelul superior al programului, moment în care programul se încheie cu o eroare.
- Blocurile `finally` pot fi utilizate pentru a executa cod de curățare, indiferent dacă a fost aruncată sau nu o excepție.

Cu toate acestea, în programarea funcțională F#, se recomandă utilizarea excepțiilor cu moderație și doar în situații excepționale, cum ar fi erori irecuperabile, condiții de nefuncționare sau interacțiuni cu componente externe care folosesc excepții.

În general, se preferă utilizarea tehnicilor precum tipul `Result` și `Railway-oriented programming` pentru gestionarea erorilor, deoarece acestea oferă beneficii precum:

- Gestionarea explicită a erorilor ca parte a semnăturii funcțiilor, încurajând un tratament adecvat și consecvent al cazurilor de eroare.
- Evitarea utilizării excesive a excepțiilor pentru controlul fluxului normal al programului, ceea ce poate duce la cod mai puțin expresiv și mai greu de urmărit.
- Facilitarea compunerii și combinării funcțiilor care pot genera erori, permițând construirea unor pipeline-uri de prelucrare a datelor robuste și expresive.
- Încurajarea unui stil de programare mai declarativ și mai puțin imperative, în care erorile sunt tratate ca valori de primă clasă și nu ca evenimente excepționale.

#### 4.9.6.Validarea datelor

Validarea datelor este un aspect ce nu poate fi neglijat în dezvoltarea de software robust și fiabil. Aceasta implică verificarea și asigurarea corectitudinii, completitudinii și consistenței datelor de intrare înainte de a le prelucra sau de a le utiliza în alte părți ale



programului. În programarea funcțională F#, validarea datelor poate fi realizată într-un mod elegant și expresiv, folosind tehnici precum tipurile de date algebrice, pattern matching-ul și expresiile de tip Result.

Să analizăm un exemplu de validare a datelor pentru un scenariu simplu de înregistrare a utilizatorilor:

```
type UserName = private UserName of string
type Email = private Email of string
type Password = private Password of string

type UserRegistrationError =
    | InvalidUserName of string
    | InvalidEmail of string
    | InvalidPassword of string

module UserValidation =
    let createUserName (name: string) =
        if String.IsNullOrEmpty(name) then
            Error (InvalidUserName "Username cannot be empty")
        elif name.Length > 50 then
            Error (InvalidUserName "Username cannot exceed 50 characters")
        else
            Ok (UserName name)

    let createEmail (email: string) =
        if String.IsNullOrEmpty(email) then
            Error (InvalidEmail "Email cannot be empty")
        elif not (email.Contains("@")) then
            Error (InvalidEmail "Email must contain '@' symbol")
        else
            Ok (Email email)

    let createPassword (password: string) =
        if String.IsNullOrEmpty(password) then
            Error (InvalidPassword "Password cannot be empty")
        elif password.Length < 8 then
            Error (InvalidPassword "Password must be at least 8 characters
long")
        else
            Ok (Password password)

    let validateUser name email password =
        match UserValidation.createUserName name,
            UserValidation.createEmail email,
            UserValidation.createPassword password with
        | Ok username, Ok emailAddress, Ok pass ->
            Ok { UserName = username; Email = emailAddress; Password = pass }
        | Error (InvalidUserName msg), _, _ -> Error msg
        | _, Error (InvalidEmail msg), _ -> Error msg
        | _, _, Error (InvalidPassword msg) -> Error msg
```

În acest exemplu, definim tipuri de date simple `UserName`, `Email` și `Password` ca wrapper-uri în jurul tipului `string`, pentru a asigura o utilizare corectă și consistentă a acestor valori în întregul program.

De asemenea, definim un tip de date algebrice `UserRegistrationError` pentru a reprezenta diferitele tipuri de erori care pot apărea în procesul de validare a datelor de înregistrare a utilizatorului, cum ar fi nume de utilizator invalid, email invalid sau parolă invalidă.

Modulul `UserValidation` conține funcții pentru validarea fiecărei componente a datelor de înregistrare (nume de utilizator, email, parolă). Aceste funcții primesc o valoare de tip `string` și returnează un rezultat de tip `Result<'T, UserRegistrationError>`, unde `'T` reprezintă tipul wrapper corespunzător (`UserName`, `Email`, `Password`). Dacă datele sunt valide, se returnează `Ok` împreună cu valoarea validată, altfel se returnează `Error` împreună cu un mesaj de eroare specific.

Funcția `validateUser` primește valorile pentru nume, email și parolă și utilizează pattern matching pentru a combina rezultatele validărilor individuale. Dacă toate validările sunt de succes, se returnează `Ok` împreună cu un obiect anonim care conține datele de înregistrare validate. Dacă una dintre validări eșuează, se returnează `Error` cu mesajul de eroare corespunzător.

Această abordare a validării datelor aduce mai multe beneficii:

- Tipurile de date algebrice și wrapper-urile asigură o utilizare corectă și consistentă a valorilor în întregul program, prevenind erori cauzate de date invalide.
- Funcțiile de validare individuale sunt responsabile pentru verificarea și validarea fiecărei componente a datelor de intrare, permițând o separare clară a responsabilităților și o mai bună modularizare a codului.
- Utilizarea tipului `Result` permite o gestionare explicită și elegantă a erorilor de validare, fără a recurge la excepții sau la valori de tip `NULL`.
- Pattern matching-ul și expresiile condiționale sunt utilizate pentru a combina și a propaga rezultatele validărilor individuale, oferind un flux de control clar și expresiv.
- Mesajele de eroare specifice și tipurile de erori distincte permit o tratare adecvată și informativă a cazurilor de date invalide, îmbunătățind experiența utilizatorului și ușurința depanării.

Prin aplicarea acestor tehnici și principii, putem realiza o validare robustă și expresivă a datelor în programele noastre F#, asigurând corectitudinea și consistența datelor de intrare și prevenind erorile cauzate de date invalide.

## 4.10. Interoperabilitatea cu C#

Interoperabilitatea cu C# este o caracteristică esențială a limbajului F#, care permite integrarea și comunicarea facilă între codul scris în F# și cel scris în C#. Această interoperabilitate este posibilă datorită faptului că atât F#, cât și C# sunt limbaje din ecosistemul .NET și împărtășesc același sistem de tipuri și biblioteci de bază.

Abilitatea de a interopera cu C# deschide o gamă largă de posibilități pentru dezvoltatorii F#, permițându-le să beneficieze de ecosistemul vast de biblioteci și framework-uri disponibile în lumea .NET, precum și să integreze codul F# în proiecte existente scrise în C#.

În acest paragraf, vom explora diferite aspecte ale interoperabilității dintre F# și C#, abordând subiecte precum utilizarea tipurilor din C# în codul F#, expunerea tipurilor și funcționalităților F# către C#, utilizarea bibliotecilor .NET comune, precum și cele mai bune practici și considerații privind interoperabilitatea.

Prin înțelegerea și aplicarea corectă a principiilor de interoperabilitate, veți putea valorifica avantajele ambelor limbaje și veți putea construi aplicații robuste și modulare, care combină puterea programării funcționale a F# cu bogăția și flexibilitatea ecosistemului C# și .NET.

### 4.10.1. Utilizarea tipurilor din C#

F# permite utilizarea facilă a tipurilor definite în C# în cadrul codului F#. Putem referenția biblioteci și assembly-uri scrise în C# și putem folosi clasele, interfețele, structurile și tipurile de date definite în acestea în mod natural în codul nostru F#.

Pentru a utiliza un tip din C# în F#, trebuie mai întâi să adăugăm o referință la assembly-ul corespunzător care conține tipul respectiv. Acest lucru se poate face folosind directiva `#r` în scripturile F# sau adăugând o referință în fișierul de proiect (.fsproj) pentru proiectele F#.

Odată ce assembly-ul este referențiat, putem utiliza tipurile din C# folosind numele complet al tipului, incluzând namespace-ul în care este definit.

Iată un exemplu de utilizare a unui tip din C# într-un modul F#:

```
open System.Collections.Generic

let dictionary = new Dictionary<string, int>()
dictionary.Add("one", 1)
dictionary.Add("two", 2)
dictionary.Add("three", 3)

let value = dictionary["two"]
printfn "The value for 'two' is %d" value
```

În acest exemplu, utilizăm clasa `Dictionary<TKey, TValue>` din namespace-ul `System.Collections.Generic` pentru a crea un dicționar care mapează chei de tip `string` la valori de tip `int`. Putem apela metodele clasei `Dictionary`, cum ar fi `Add`, pentru a adăuga elemente în dicționar, și putem accesa valorile folosind sintaxa de indexare cu paranteze pătrate.

De asemenea, putem utiliza tipurile din C# ca parametri și valori de retur pentru funcțiile din F#:

```
open System

let getLength (str: String) = str.Length

let length = getLength "Hello, F#!"
printfn "The length of the string is %d" length
```

Aici, definim o funcție `getLength` care primește un parametru de tip `String` (un tip din C#) și returnează lungimea acestuia folosind proprietatea `Length`.

Când utilizăm tipuri din C# în F#, se aplică aceleași reguli și convenții ca și în C#. Putem accesa proprietăți, apela metode, crea instanțe ale claselor și implementa interfețe în mod similar cu modul în care am face-o în C#.

Este important de menționat că, deși F# are propria sintaxă și convenții, cum ar fi utilizarea *snake\_case* pentru nume de funcții și variabile, când interacționăm cu tipuri din C#, se recomandă să urmărim convențiile de denumire din C# (PascalCase) pentru a menține consistența și lizibilitatea codului.

#### 4.10.2. Expunerea tipurilor F# către C#

Pe lângă utilizarea tipurilor din C# în F#, putem și expune tipurile și funcționalitățile definite în F# către C#. Aceasta permite dezvoltatorilor C# să beneficieze de caracteristicile și avantajele programării funcționale oferite de F#, precum și să integreze codul F# în proiectele lor C# existente.

Pentru a expune un tip F# către C#, trebuie să marcăm tipul respectiv cu atributul `[<CLIMutable>]` și să asigurăm că tipul și membrii săi au modificatorii de acces adecvați (`public`, `internal` etc.).

Iată un exemplu de tip F# expus către C#:

```
[<CLIMutable>]
type Person =
    { FirstName: string
      LastName: string
      Age: int }

module PersonModule =
    let createPerson firstName lastName age =
        { FirstName = firstName
```

```

        LastName = lastName
        Age = age }

let getFullName (person: Person) =
    person.FirstName + " " + person.LastName

```

În acest exemplu, definim un tip înregistrare **Person** cu proprietăți pentru prenume (**FirstName**), nume de familie (**LastName**) și vârstă (**Age**). Marcăm tipul cu atributul [**<CLIMutable>**] pentru a permite modificarea valorilor proprietăților din C#.

De asemenea, definim un modul **PersonModule** care conține funcții pentru crearea unui obiect **Person** (**createPerson**) și pentru obținerea numelui complet al unei persoane (**getFullName**).

Pentru a utiliza tipul **Person** și funcțiile din modulul **PersonModule** în C#, putem referenția assembly-ul F# în proiectul nostru C# și putem accesa tipurile și funcțiile folosind numele lor complet:

```

using FSharpLibrary;

var person = PersonModule.createPerson("John", "Doe", 30);
string fullName = PersonModule.getFullName(person);
Console.WriteLine(fullName); // Output: John Doe

person.Age = 31; // Modificarea valorii proprietății Age
Console.WriteLine(person.Age); // Output: 31

```

Aici, utilizăm funcția **createPerson** din modulul **PersonModule** pentru a crea o instanță a tipului **Person**, apoi apelăm funcția **getFullName** pentru a obține numele complet al persoanei. De asemenea, putem modifica direct valorile proprietăților obiectului **Person** din C# datorită atributului [**<CLIMutable>**].

Când expunem tipuri F# către C#, este important să avem în vedere următoarele aspecte:

- Tipurile F# trebuie să fie marcate cu atributul [**<CLIMutable>**] pentru a permite modificarea valorilor proprietăților din C#.
- Funcțiile și membrii tipurilor F# trebuie să aibă modificatorii de acces adecvați (**public**, **internal** etc.) pentru a fi accesibile din C#.
- Tipurile F# pot avea caracteristici specifice, cum ar fi tipurile opționale, tipurile discriminate unionale etc., care pot necesita un tratament special când sunt utilizate în C#.
- Convențiile de denumire în F# (camelCase pentru funcții și variabile) pot diferi de cele din C# (PascalCase), deci poate fi necesară adaptarea lor pentru a se conforma convențiilor C#.

### 4.10.3. Folosirea bibliotecilor .NET

Unul dintre avantajele majore ale F# este accesul la ecosistemul vast de biblioteci și framework-uri .NET. Putem utiliza cu ușurință bibliotecile .NET existente, indiferent dacă sunt scrise în C#, VB.NET sau orice alt limbaj .NET, direct în codul nostru F#.

Pentru a utiliza o bibliotecă .NET în F#, trebuie să adăugăm o referință la assembly-ul corespunzător care conține biblioteca. Acest lucru se poate face folosind directiva `#r` în scripturile F# sau adăugând o referință în fișierul de proiect (.fsproj) pentru proiectele F#.

Iată câteva exemple de utilizare a bibliotecilor .NET populare în F#:

*Utilizarea bibliotecii System.IO pentru operații de intrare/ieșire:*

```
open System.IO

let writeToFile filePath content =
    use writer = new StreamWriter(filePath)
    writer.Write(content)

let readFromFile filePath =
    use reader = new StreamReader(filePath)
    reader.ReadToEnd()

// Exemplu de utilizare
let filePath = "example.txt"
let content = "Hello, F#!"

writeToFile filePath content
let readContent = readFromFile filePath
printfn "Read from file: %s" readContent
```

În acest exemplu, folosim clasele `StreamWriter` și `StreamReader` din biblioteca `System.IO` pentru a scrie și a citi dintr-un fișier text. Funcția `writeToFile` primește calea fișierului și conținutul de scris, iar funcția `readFromFile` citește conținutul fișierului specificat.

*Utilizarea bibliotecii System.Net.Http pentru efectuarea de cereri HTTP:*

```
open System.Net.Http

let fetchWebPage url =
    use client = new HttpClient()
    let response = client.GetAsync(url).Result
    response.Content.ReadAsStringAsync().Result

// Exemplu de utilizare
let url = "https://api.example.com/data"
let pageContent = fetchWebPage url
printfn "Web page content: %s" pageContent
```

Aici, utilizăm clasa `HttpClient` din biblioteca `System.Net.Http` pentru a efectua o cerere GET către un URL specificat. Funcția `fetchWebPage` primește un URL și returnează conținutul paginii web ca un șir de caractere.

Utilizarea bibliotecii Newtonsoft.Json pentru serializare/deserializare JSON:  
open Newtonsoft.Json

```
type Person =
    { Name: string
      Age: int }

let serializeToJson (person: Person) =
    JsonConvert.SerializeObject(person)

let deserializeFromJson json =
    JsonConvert.DeserializeObject<Person>(json)

// Exemplu de utilizare
let person = { Name = "John"; Age = 30 }
let json = serializeToJson person
printfn "Serialized JSON: %s" json

let deserializedPerson = deserializeFromJson json
printfn "Deserialized Person: Name = %s, Age = %d"
    deserializedPerson.Name deserializedPerson.Age
```

În acest exemplu, folosim biblioteca populară `Newtonsoft.Json` pentru serializarea și deserializarea obiectelor F# în format JSON. Funcția `serializeToJson` primește un obiect și îl serializează într-un șir JSON, iar funcția `deserializeFromJson` primește un șir JSON și îl deserializează înapoi într-un obiect F#.

Aceste exemple ilustrează doar câteva dintre numeroasele biblioteci .NET pe care le putem utiliza în F#. Indiferent dacă avem nevoie de funcționalități legate de lucrul cu fișiere, rețea, baze de date, interfață utilizator sau orice alt domeniu, există biblioteci .NET robuste și bine stabilite pe care le putem integra în codul nostru F#.

#### 4.10.4. Conversii între tipuri F# și C#

```
// Conversii implicite
let intToFloat (x: int) = float x
let floatToInt (x: float) = int x

// Conversii explicite
let intToChar (x: int) = char x
let charToInt (x: char) = int x

// Conversii între tipuri opționale
let optionToNullable (x: int option) =
    match x with
    | Some value -> Nullable(value)
    | None -> Nullable()

let nullableToOption (x: Nullable<int>) =
    if x.HasValue then Some x.Value else None

// Conversii între liste imutabile F# și liste mutabile C#
open System.Collections.Generic
```

```

let listToIList (list: 'a list) =
    let ild = List<'a>()
    for item in list do ild.Add(item)
    ild

let ildToList (ild: IList<'a>) =
    let listBuilder = new System.Collections.Generic.List<'a>()
    for item in ild do listBuilder.Add(item)
    List.ofSeq listBuilder

// Conversii între înregistrări F# și clase C#
[<CLIMutable>]
type PersonRecord =
    { FirstName: string
      LastName: string
      Age: int }

type PersonClass(firstName: string, lastName: string, age: int) =
    member val FirstName = firstName with get, set
    member val LastName = lastName with get, set
    member val Age = age with get, set

let recordToClass (record: PersonRecord) =
    PersonClass(record.FirstName, record.LastName, record.Age)

let classToRecord (pClass: PersonClass) =
    { FirstName = pClass.FirstName
      LastName = pClass.LastName
      Age = pClass.Age }
  
```

Deși F# și C# sunt ambele limbaje .NET, există diferențe între sistemele lor de tipuri, care pot necesita conversii explicite atunci când dorim să utilizăm tipuri F# în C# sau invers.

Iată câteva scenarii comune de conversii între tipuri F# și C#:

#### Conversii implicite și explicite:

- F# acceptă conversii implicite între anumite tipuri, cum ar fi de la `int` la `float`. Putem defini funcții care realizează aceste conversii, precum `intToFloat`.
- Pentru conversii explicite, putem utiliza funcții precum `intToChar` sau `charToInt`, care convertesc în mod explicit între tipurile respective.

#### Conversii între tipuri opționale:

- F# are tipul `Option` pentru a reprezenta valori care pot fi prezente (`Some`) sau absente (`None`), în timp ce C# folosește tipul `Nullable<T>`.
- Putem defini funcții precum `optionToNullable` și `nullableToOption` pentru a converti între aceste tipuri în ambele direcții.

#### Conversii între colecții:

- F# utilizează în mod implicit liste imutabile, în timp ce C# folosește tipuri mutabile precum `List<T>`.



- Putem crea funcții precum `listToIList` și `iListToList` pentru a converti între aceste tipuri de colecții.

#### Conversii între înregistrări F# și clase C#:

- Înregistrările F# sunt tipuri de date imutabile, în timp ce clasele C# sunt mutabile și au constructori.
- Putem defini funcții de conversie, cum ar fi `recordToClass` și `classToRecord`, pentru a mapa între aceste tipuri.
- Pentru a permite modificarea câmpurilor înregistrării F# din C#, putem marca înregistrarea cu atributul [`<CLIMutable>`].

Iată un exemplu concret de utilizare a acestor tehnici de conversie:

```
// În modulul F#
let numbers = [1; 2; 3; 4; 5]
let numbersList = listToIList numbers

let person = { FirstName = "John"; LastName = "Doe"; Age = 30 }
let personClass = recordToClass person

// În codul C#
using FSharpLibrary;

var numbersList = MyFSharpModule.numbersList;
foreach (var number in numbersList)
{
    Console.WriteLine(number);
}

var personClass = MyFSharpModule.personClass;
Console.WriteLine($"Name: {personClass.FirstName} {personClass.LastName},
Age: {personClass.Age}");
```

În acest exemplu, convertim o listă imutabilă F# (`numbers`) într-o listă mutabilă C# (`numbersList`) folosind funcția `listToIList`. Apoi, putem itera peste această listă în C# și afișa elementele.

De asemenea, convertim o înregistrare F# (`person`) într-o instanță a unei clase C# (`personClass`) folosind funcția `recordToClass`. În C#, putem accesa proprietățile acestui obiect și afișa detaliile persoanei.